# ansys-chemkin-core

**ANSYS, Inc. <ansys.support@ansys.com>**

**Oct 22, 2026**

# CONTENTS




PyChemkin provides Pythonic access to the Ansys Chemkin for computational fluid dynamics (CFD) models. It facilitates programmatic customization of Chemkin simulation workflows within the Python ecosystem and permits access to Chemkin property and rate utilities as well as selected reactor. You are viewing version .

Getting started Learn about PyChemkin, its prerequisites, and how to install it.

*Getting started*

User guide Understand key concepts and the main objects of PyChemkin.

*User guide*

API reference A detailed guide describing the PyChemkin modules, classes, and enums.

*API reference*

Examples Learn how to use PyChemkin with examples that demonstrate its capabilities.

*Examples*

Contribute Learn how to contribute to the project and become a part of the PyChemkin community.

*Contribute*

Artifacts Download PyChemkin artifacts such as wheels and source distributions.

*Artifacts*

CHAPTER

ONE

# PREREQUISITES

Before installing PyChemkin, install the following:

- Ansys Chemkin 2025 R2 or later, with a valid license.
- Python 3.10 or later, and earlier than 4.0.

PyChemkin declares its Python dependencies in `pyproject.toml`. They are installed automatically when PyChemkin is installed with `pip`.

**Note**

Using the latest Ansys Chemkin version is recommended.

CHAPTER

TWO

# CONFIGURE ANSYS CHEMKIN

PyChemkin selects the newest supported local Chemkin installation. To select a specific installation, define its `ANSYSxxx_DIR` environment variable, where `xxx` is the Chemkin release number. The value must be the installation's `ANSYS` directory. For example, on Windows:

```
$env:ANSYS261_DIR = "C:\Program Files\ANSYS Inc\v261\ANSYS"
```

When there are multiple `ANSYSxxx_DIR` environment variables defined, PyChemkin selects the newest one by default. Use the `PYCK_CHEMKIN_VER` environment variable to specify the desired local Ansys Chemkin installation. For example, set `PYCK_CHEMKIN_VER="261"` to force PyChemkin to use Ansys Chemkin 2026 R1.

On Linux, define the environment variable in the shell before starting Python. You also need to source the `chemkin_setup.ksh` or `chemkin_setup.csh` script provided in the Chemkin `bin` directory when using Linux.

CHAPTER

THREE

# INSTALL PYCHEMKIN

Install the published package from PyPI with:

```
python -m pip install ansys-chemkin-core
```

To install the current source tree for development, run these commands from the repository root:

```
python -m pip install --upgrade pip
python -m pip install -e .
```

CHAPTER

FOUR

# BUILD AND INSTALL A WHEEL

To build a wheel from the source tree, install the build frontend and run it from the repository root:

```
python -m pip install build
python -m build
```

The wheel and source distribution are written to `dist`. Install the wheel with:

```
python -m pip install dist\ansys_chemkin_core-*.whl
```

CHAPTER

FIVE

# VERIFY THE INSTALLATION

Start Python and import the package:

```
>>> import ansys.chemkin.core
```

The import initializes the native Chemkin library and reports the Chemkin and PyChemkin versions. If initialization fails, verify that the selected Chemkin installation exists, the `ANSYSxxx_DIR` variable is correct, and a valid license is available.

CHAPTER

SIX

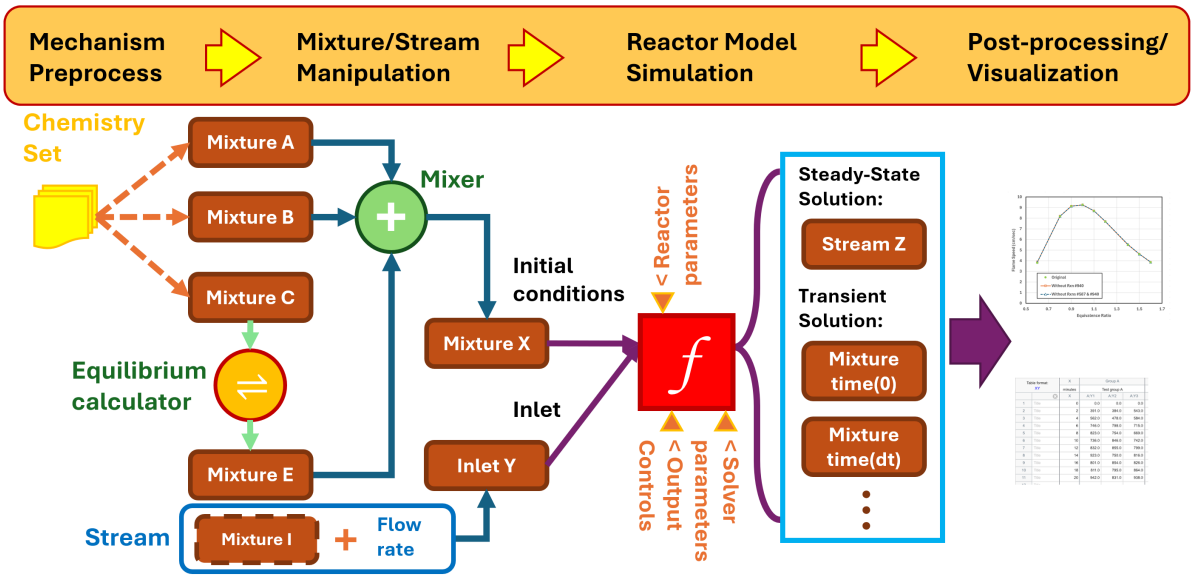
# USER GUIDE

## 6.1 Introduction

PyChemkin inherits all Ansys Chemkin functionalities, providing Pythonic interfaces to Chemkin utilities and reactor models. In addition, PyChemkin introduces a hierarchy of four key objects to enhance user experiences within the Python framework:

- **Chemistry set:** A collection of utilities that handle chemistry and species properties.
- **Mixture:** A basic element representing a gas mixture that can be manipulated and transformed.
- **Stream:** A mixture object with an additional property of mass flow rate.
- **Reactor:** An instance of a Chemkin reactor model that transforms the initial mixture into another mixture.

The following figure shows PyChemkin's mixture-centric concept. It illustrates the basic types of operations applicable to a mixture, which inherits all properties of a chemistry set. A mixture can be combined with a constraint, equilibrium (with constraints), or a process (by a reactor model).




## 6.2 Workflow for a basic reactor model

The workflow for setting up and running a basic reactor model in PyChemkin is the same as in the Ansys Chemkin GUI:

1. Create a chemistry set, which includes the mechanism, thermodynamic data, and/or transport data.

2. Preprocess the chemistry set.
3. Create a mixture or a stream based on the chemistry set.
4. Specify mixture or stream properties, such as temperature, pressure, volume (optional), and species composition.
5. Instantiate the reactor using the created mixture.
6. Set up the simulation:
    - Specify reactor properties not provided by the initial mixture or stream, such as heat loss rate to the surroundings and end time.
    - Specify solver parameters, such as tolerances and solver timestep size.
    - Specify saving controls, such as the solution saving interval and adaptive solution saving.
7. Run the simulation.
8. Process the solution.
9. Customize the output if necessary and generate plots.

## 6.3 Code example

This code example computes the density of a mixture named `air`:

```
import os

# import PyChemkin
import ansys.chemkin.core as chemkin

# create a chemistry set for the GRI 3.0 mechanism in the data directory
mech_dir = os.path.join(chemkin.ansys_dir, "reaction", "data")
# set up mechanism file names
mech_file = os.path.join(mech_dir, "grimech30_chem.inp")
therm_file = os.path.join(mech_dir, "grimech30_thermo.dat")
tran_file = os.path.join(mech_dir, "grimech30_transport.dat")
# instantiate a chemistry set named 'GasMech'
GasMech = chemkin.Chemistry(
    chem=mech_file, therm=therm_file, tran=tran_file, label="GRI 3.0"
)
# preprocess the chemistry set
status = GasMech.preprocess()
# check preprocess status
if status != 0:
    # failed
    print(f"Preprocessing: Error encountered. Code = {status:d}.")
    print(f"See the summary file {GasMech.summaryfile} for details.")
    exit()
# create a mixture named 'air' based on the 'GasMech' chemistry set
air = chemkin.Mixture(GasMech)
# set 'air' condition
# mixture pressure in [dynes/cm2]
air.pressure = 1.0 * chemkin.P_ATM
# mixture temperature in [K]
air.temperature = 300.0
# mixture composition in mole fractions
```

(continues on next page)

(continued from previous page)

```
air.X = [("O2", 0.21), ("N2", 0.79)]
#
print(f"Pressure    = {air.pressure/chemkin.P_ATM} [atm].")
print(f"Temperature = {air.temperature} [K].")
# print the 'air' composition in mass fractions
air.list_composition(mode="mass")
# get 'air' mixture density [g/cm3]
print(f"Mixture density = {air.RHO} [g/cm3].")
```

For more examples, see *Examples*.

## 6.4 Support

### 6.4.1 Pythonic methods

To get help within PyChemkin, use these pythonic methods:

- `ansys.chemkin.core.help()`: Get general information about a topic, such as `ignition`.
- `ansys.chemkin.core.keywordhints()`: Get the description and syntax of a reactor keyword.
- `ansys.chemkin.core.phrasehints()`: Get a list of reactor keywords related to a phrase.

### 6.4.2 Product training and documentation

For product training and documentation, see these Ansys resources:

- Ansys Learning Hub
- Chemkin product documentation

CHAPTER

SEVEN

# API REFERENCE

This section describes ansys-chemkin-core endpoints, their capabilities, and how to interact with them programmatically.

## 7.1 The ansys.chemkin.core library

### 7.1.1 Summary

#### Submodules

| | |
|---|---|
| *chemistry* | Chemkin Chemistry utilities. |
| *chemkin_solution_utility* | Python utilities to post-process Chemkin simulation solutions. |
| *chemkin_wrapper* | Chemkin-CFD-API python interfaces. |
| *color* | Color and print format utilities. |
| *constants* | Constants used by Chemkin utilities and models. |
| *flame* | Chemkin utilities for steady state, 1-D flame models. |
| *grid* | Chemkin grid quality control parameters. |
| *hybridreactornetwork* | Hybrid reactor network comprised of a mix of open reactors such as PSR and PFR. |
| *info* | Chemkin help menu for keywords and key phrases. |
| *inlet* | Chemkin reactor inlet/stream utilities. |
| *keyword* | Chemkin keyword models and helpers. |
| *logger* | Provides the singleton helper class for the logger. |
| *microprocess* | Micro mixing model for the multi-zone engine models. |
| *mixture* | Chemkin Mixture utilities. |
| *particles* | Aerosol size distribution models. |
| *profile* | Chemkin profile keyword data model. |
| *reactormodel* | Chemkin general reactor model utilities. |
| *realgaseos* | Real-gas cubic EOS model. |
| *steadystatesolver* | Chemkin steady-state solver controlling parameters. |
| *surface* | Surface Chemkin Chemistry utilities. |
| *surface_chemistry* | Surface chemistry behavior mixed into the Chemistry class. |
| *surface_chemistry_controlle* | Surface chemistry controller for reactor models. |
| *surface_components* | Surface material and phase data models. |
| *utilities* | pychemkin utilities. |
| *validation* | Reusable validation helpers for core modules. |

### Attributes

| |
|---|
| *__version__* |
| *msg* |
| *this_msg* |
| *msg* |
| *this_msg* |
| *ansys_dir* |
| *ansys_version* |
| *ck_name* |
| *chemkin_dir* |
| *unit_code* |
| *ierror* |
| *frm* |

## The chemistry.py module

### Summary

### Classes

| | |
|---|---|
| *Chemistry* | Define and preprocess Chemkin chemistry set. |

### Functions

| | |
|---|---|
| *verbose* | Get current Pychemkin verbose mode. |
| *set_verbose* | Set Pychemkin verbose mode. |
| *deep_debug* | Enable Python deep debugging mode. |
| *chemkin_version* | Return the Chemkin-CFD-API version number currently in use. |
| *verify_version* | Check the Chemkin-CFD-API vwersion. |
| *chemkin_bin_dir* | Return the local Ansys Chemkin bin directory. |
| *chemkin_data_dir* | Return the local Ansys Chemkin data directory. |
| *ansys_dir* | Return the local Ansys installation. |
| *get_configuration* | Return the Chemkin-CFD-API configuration. |
| *done* | Release Chemkin license and reset the Chemistry sets. |
| *check_chemistryset* | Check whether the Chemistry Set is initialized in Chemkin-CFD-API. |
| *activate_chemistryset* | Switch to (re-activate) the Chemistry Set. |
| *force_activate_chemistryset* | Activate the Chemistry Set automatically and silently. |
| *chemistryset_new* | Create a new Chemistry Set. |
| *chemistryset_initialized* | Set the Chemistry Set Initialization flag to True. |
| *check_active_chemistryset* | Verify if the chemistry set is currently activated. |
| *verify_chemset_surface* | Check the chemistry set has surface chemistry data. |
| *verify_chemset_sizes* | Verify the gas chemistry sizes. |
| *set_temp_array* | Set up the temperature array for Chemkin-CFD-API calls. |
| *get_symbol_length* | Get the length of Chemkin symbols. |

### Attributes

| |
|---|
| *LP_c_char* |
| *chemkin_verbose* |

### Constants

| |
|---|
| *MAX_SPECIES_LENGTH* |
| *COMPLETE* |

#### Chemistry

**class** ansys.chemkin.core.chemistry.**Chemistry**(*chem: str = ''*, *surf: str = ''*, *therm: str = ''*, *tran: str = ''*, *label: str = ''*)

Bases: *ansys.chemkin.core.surface_chemistry.SurfaceChemistryMixin*

Define and preprocess Chemkin chemistry set.

### Overview

### Methods

| | |
|---|---|
| *preprocess_transportdata* | Instruct the preprocessor to include the transport data. |
| *set_file_names* | Assign all input files of the chemistry set. |
| *preprocess* | Run Chemkin preprocessor. |
| *verify_realgas_model* | Verify the availability of real-gas data in the mechanism. |
| *verify_transport_data* | Verify the availability of transport property data in the mechanism. |
| *verify_surface_mechanism* | Verify the availability of surface chemistry data in the mechanism. |
| *get_specindex* | Get global index of the gas species. |
| *species_cp* | Get species specific heat capacity at constant pressure. |
| *species_cv* | Get species specific heat capacity at constant volume. |
| *species_h* | Get species enthalpy. |
| *species_u* | Get species internal energy. |
| *species_visc* | Get species viscosity. |
| *species_cond* | Get species conductivity. |
| *species_diffusioncoeffs* | Get species diffusion coefficients. |
| *species_composition* | Get elemental composition of a species. |
| *use_realgas_cubic_eos* | Turn ON the real-gas cubic EOS. |
| *use_idealgas_law* | Turn on the ideal gas law to compute mixture properties. |
| *get_reaction_parameters* | Get the Arrhenius reaction rate parameters of all gas-phase reactions. |
| *set_reaction_afactor* | Set the Arrhenius A-Factor of the given reaction. |
| *get_reaction_afactor* | Get the Arrhenius A-Factor of the given reaction. |
| *get_gas_reaction_string* | Get the reaction string of the given reaction index. |
| *save* | Store the work spaces of the current Chemistry Set. |
| *activate* | Activate the work spaces of the Chemistry Set. |

### Properties

| | |
|---|---|
| *chemfile* | Get gas-phase mechanism file name of this chemistry set. |
| *thermfile* | Get thermodynamic data filename of this chemistry set. |
| *tranfile* | Get transport data filename of this chemistry set. |
| *summaryfile* | Get the name of the summary file from the preprocessor. |
| *surffile* | Get surface mechanism filename of this chemistry set. |
| *species_symbols* | Get list of gas species symbols. |
| *element_symbols* | Get the list of element symbols. |
| *chemid* | Get chemistry set index. |
| *surfchem* | Get surface chemistry status. |
| *kk* | Get number of gas species. |
| *mm* | Get number of elements in the chemistry set. |
| *ii_gas* | Get number of gas-phase reactions. |
| *awt* | Compute atomic masses. |
| *wt* | Compute gas species molecular masses. |
| *eos* | Get the available real-gas EOS model. |

### Attributes

| |
|---|
| *realgas_cueos* |
| *realgas_mixing_rules* |
| *userealgas* |
| *esymbol* |
| *ksymbol* |
| *label* |
| *material_map* |
| *materials* |
| *matsymbol* |
| *number_species* |
| *number_elements* |
| *number_gas_reactions* |
| *atomic_weight* |
| *species_molar_weight* |

### Special methods

| | |
|---|---|
| *__setattr__* | Prevent Chemistry metadata reassignment after preprocessing. |

### Import detail

```
from ansys.chemkin.core.chemistry import Chemistry
```

### Property detail

**property** Chemistry.**chemfile: str**

Get gas-phase mechanism file name of this chemistry set.

**property** Chemistry.**thermfile: str**

Get thermodynamic data filename of this chemistry set.

**property** Chemistry.**tranfile: str**

Get transport data filename of this chemistry set.

**property** Chemistry.**summaryfile: str**

Get the name of the summary file from the preprocessor.

**property** Chemistry.**surffile: str**

Get surface mechanism filename of this chemistry set.

**property** Chemistry.**species_symbols**

Get list of gas species symbols.

**property** Chemistry.**element_symbols**

Get the list of element symbols.

**property** Chemistry.**chemid: int**

Get chemistry set index.

**property** Chemistry.**surfchem: int**

Get surface chemistry status.

**property** Chemistry.**kk: int**

Get number of gas species.

**property** Chemistry.**mm: int**

Get number of elements in the chemistry set.

**property** Chemistry.**ii_gas: int**

Get number of gas-phase reactions.

**property** Chemistry.**awt: numpy.typing.NDArray[numpy.double]**

Compute atomic masses.

**property** Chemistry.**wt: numpy.typing.NDArray[numpy.double]**

Compute gas species molecular masses.

**property** Chemistry.**eos: int**

Get the available real-gas EOS model.

### Attribute detail

Chemistry.**realgas_cueos = ['ideal gas', 'Van der Waals', 'Redlich-Kwong', 'Soave', 'Aungier', 'Peng-Robinson']**

Chemistry.**realgas_mixing_rules = ['Van der Waals', 'pseudocritical']**

Chemistry.**userealgas = False**

Chemistry.**esymbol: Sequence[str] = []**

Chemistry.**ksymbol: Sequence[str] = []**

Chemistry.**label = ' '**

Chemistry.**material_map: Mapping[str, int]**

Chemistry.**materials: Mapping[str, *ansys.chemkin.core.surface_components.Material*]**

Chemistry.**matsymbol: Sequence[str] = []**

Chemistry.**number_species**

Chemistry.**number_elements**

Chemistry.**number_gas_reactions**

Chemistry.**atomic_weight**

Chemistry.**species_molar_weight**

### Method detail

Chemistry.**__setattr__**(*name: str*, *value*) → None

Prevent Chemistry metadata reassignment after preprocessing.

Chemistry.**preprocess_transportdata**()

Instruct the preprocessor to include the transport data.

Chemistry.**set_file_names**(*chem: str = ''*, *surf: str = ''*, *therm: str = ''*, *tran: str = ''*)

Assign all input files of the chemistry set.

Chemistry.**preprocess**() → int

Run Chemkin preprocessor.

Chemistry.**verify_realgas_model**()

Verify the availability of real-gas data in the mechanism.

Chemistry.**verify_transport_data**() → bool

Verify the availability of transport property data in the mechanism.

Chemistry.**verify_surface_mechanism**() → bool

Verify the availability of surface chemistry data in the mechanism.

Chemistry.**get_specindex**(*specname: str*) → int

Get global index of the gas species.

Chemistry.**species_cp**(*temp: float = 0.0*, *pres: float | None = None*) → numpy.typing.NDArray[numpy.double]

Get species specific heat capacity at constant pressure.

Chemistry.**species_cv**(*temp: float = 0.0*, *pres: float | None = None*) → numpy.typing.NDArray[numpy.double]

Get species specific heat capacity at constant volume.

Chemistry.**species_h**(*temp: float = 0.0*, *pres: float | None = None*) → numpy.typing.NDArray[numpy.double]

Get species enthalpy.

Chemistry.**species_u**(*temp: float = 0.0*, *pres: float | None = None*) → numpy.typing.NDArray[numpy.double]

Get species internal energy.

Chemistry.**species_visc**(*temp: float = 0.0*) → numpy.typing.NDArray[numpy.double]

Get species viscosity.

Chemistry.**species_cond**(*temp: float = 0.0*) → numpy.typing.NDArray[numpy.double]

Get species conductivity.

Chemistry.**species_diffusioncoeffs**(*press: float = 0.0*, *temp: float = 0.0*) → numpy.typing.NDArray[numpy.double]

Get species diffusion coefficients.

Chemistry.**species_composition**(*elemindex: int = -1*, *specindex: int = -1*) → int

Get elemental composition of a species.

Chemistry.**use_realgas_cubic_eos**()

Turn ON the real-gas cubic EOS.

Chemistry.**use_idealgas_law**()

Turn on the ideal gas law to compute mixture properties.

Chemistry.**get_reaction_parameters**() → tuple[numpy.typing.NDArray[numpy.double], numpy.typing.NDArray[numpy.double], numpy.typing.NDArray[numpy.double]]

Get the Arrhenius reaction rate parameters of all gas-phase reactions.

Chemistry.**set_reaction_afactor**(*reaction_index: int*, *a_factor: float*)

Set the Arrhenius A-Factor of the given reaction.

Chemistry.**get_reaction_afactor**(*reaction_index: int*) → float

Get the Arrhenius A-Factor of the given reaction.

Chemistry.**get_gas_reaction_string**(*reaction_index: int*) → str

Get the reaction string of the given reaction index.

Chemistry.**save**()

Store the work spaces of the current Chemistry Set.

Chemistry.**activate**()

Activate the work spaces of the Chemistry Set.

### Description

Chemkin Chemistry utilities.

### Module detail

chemistry.**verbose**() → bool

Get current Pychemkin verbose mode.

chemistry.**set_verbose**(*onoff: bool*)

Set Pychemkin verbose mode.

chemistry.**deep_debug**()

Enable Python deep debugging mode.

chemistry.**chemkin_version**() → int

Return the Chemkin-CFD-API version number currently in use.

chemistry.**verify_version**(*min_version: int*) → bool

Check the Chemkin-CFD-API vwersion.

chemistry.**chemkin_bin_dir**() → str

Return the local Ansys Chemkin bin directory.

chemistry.**chemkin_data_dir**() → str

Return the local Ansys Chemkin data directory.

chemistry.**ansys_dir**() → str

Return the local Ansys installation.

chemistry.**get_configuration**() → tuple[int, str, str, str, list[str]]

Return the Chemkin-CFD-API configuration.

chemistry.**done**()

Release Chemkin license and reset the Chemistry sets.

chemistry.**check_chemistryset**(*chem_index: int*) → bool

Check whether the Chemistry Set is initialized in Chemkin-CFD-API.

chemistry.**activate_chemistryset**(*chem_index: int*) → int

Switch to (re-activate) the Chemistry Set.

chemistry.**force_activate_chemistryset**(*chem_index: int*)

Activate the Chemistry Set automatically and silently.

chemistry.**chemistryset_new**(*chem_index: int*)

Create a new Chemistry Set.

chemistry.**chemistryset_initialized**(*chem_index: int*)

Set the Chemistry Set Initialization flag to True.

chemistry.**check_active_chemistryset**(*chem_index: int*) → bool

Verify if the chemistry set is currently activated.

chemistry.**verify_chemset_surface**(*chem_set_index: int*) → bool

Check the chemistry set has surface chemistry data.

chemistry.**verify_chemset_sizes**(*chem_set_index: int*, *num_elements: int = -1*, *num_gas_species: int = -1*, *num_gas_reactions: int = -1*) → bool

Verify the gas chemistry sizes.

chemistry.**set_temp_array**(*temp: float*, *temp_ele: float | None = None*, *temp_ion: float | None = None*) → numpy.typing.NDArray[numpy.double]

Set up the temperature array for Chemkin-CFD-API calls.

chemistry.**get_symbol_length**() → int

Get the length of Chemkin symbols.

chemistry.**MAX_SPECIES_LENGTH = 17**

chemistry.**COMPLETE = 0**

chemistry.**LP_c_char**

chemistry.**chemkin_verbose = True**

### The `chemkin_solution_utility.py` module

### Summary

### Classes

| | |
|---|---|
| *ChemkinSolutionImporter* | Post-process the regular Chemkin simulation solution. |
| *SolutionData* | Manage the imported solution data for the Chemkin solution utility. |
| *SolutionGroup* | Class representing a single solution group within the Chemkin solution data. |

#### ChemkinSolutionImporter

**class** ansys.chemkin.core.chemkin_solution_utility.**ChemkinSolutionImporter**

Post-process the regular Chemkin simulation solution.

### Overview

### Methods

| | |
|---|---|
| *import_solution* | Get the solution data using the Chemkin post-processor. |
| *process_solution_data* | Import the solution data from the CSV file for further analysis. |
| *process_ckcsv* | Process the CSV file containing the solution data. |
| *get_help* | Get help information from the Chemkin solution utility. |
| *set_working_dir* | Set the working directory for the Chemkin solution utility. |
| *generate_preference_file* | Generate the preference file to customize solution extraction. |
| *set_xml_data_file* | Set the XML data file for the Chemkin solution utility. |
| *use_preference_file* | Enable the use of the preference file for the Chemkin solution utility. |
| *disable_preference_file* | Disable the use of the preference file for the Chemkin solution utility. |
| *enable_mass_fraction* | Enable the use of mass fraction for the Chemkin solution utility. |
| *disable_mass_fraction* | Disable the use of mass fraction for the Chemkin solution utility. |
| *enable_get_everything* | Enable the 'get everything' option for the Chemkin solution utility. |
| *disable_get_everything* | Disable the 'get everything' option for the Chemkin solution utility. |
| *enable_sensitivity* | Enable the sensitivity option for the Chemkin solution utility. |
| *disable_sensitivity* | Disable the sensitivity option for the Chemkin solution utility. |
| *enable_rop* | Select the rate of progress option for the Chemkin solution utility. |
| *disable_rop* | Deselect the rate of progress option for the Chemkin solution utility. |
| *validate_solution_data* | Validate the imported solution data. |
| *get_solution_groups* | Get the list of available solution groups. |
| *get_variable_labels* | Get the list of variable labels for a specific solution group. |
| *get_group_data* | Get the data for a specific solution group. |
| *get_group_object* | Get the solution group object for a specific solution group. |
| *get_solution_variable_array* | Get the array of data for a specific variable within a solution group. |
| *get_number_of_points* | Get the number of solution points in the solution data. |
| *get_number_of_gas_species* | Get the number of gas species in the solution data. |
| *get_gas_species_symbols* | Get the list of gas species symbols in the solution data. |

### Attributes

| |
|---|
| *ckbin* |
| *get_soln* |
| *soln_transpose* |
| *env_file* |
| *get_soln_path* |
| *soln_transpose_path* |
| *get_soln_args* |
| *work_dir_path* |
| *xml_data_file* |
| *use_mass_fraction* |

### Import detail

```
from ansys.chemkin.core.chemkin_solution_utility import ChemkinSolutionImporter
```

### Attribute detail

ChemkinSolutionImporter.**ckbin**

ChemkinSolutionImporter.**get_soln = ''**

ChemkinSolutionImporter.**soln_transpose = ''**

ChemkinSolutionImporter.**env_file = ''**

ChemkinSolutionImporter.**get_soln_path = ''**

ChemkinSolutionImporter.**soln_transpose_path = ''**

ChemkinSolutionImporter.**get_soln_args:  list[str] = []**

ChemkinSolutionImporter.**work_dir_path**

ChemkinSolutionImporter.**xml_data_file = 'XMLdata.zip'**

ChemkinSolutionImporter.**use_mass_fraction = False**

### Method detail

ChemkinSolutionImporter.**import_solution**(*soln_file: str | None = None*)

Get the solution data using the Chemkin post-processor.

ChemkinSolutionImporter.**process_solution_data**()

Import the solution data from the CSV file for further analysis.

ChemkinSolutionImporter.**process_ckcsv**(*ckcsv_file: str | None = None*)

Process the CSV file containing the solution data.

ChemkinSolutionImporter.**get_help**()

Get help information from the Chemkin solution utility.

ChemkinSolutionImporter.**set_working_dir**(*work_dir: str*)

Set the working directory for the Chemkin solution utility.

ChemkinSolutionImporter.**generate_preference_file**(*soln_file: str | None = None*)

Generate the preference file to customize solution extraction.

ChemkinSolutionImporter.**set_xml_data_file**(*xml_file: str*)

Set the XML data file for the Chemkin solution utility.

ChemkinSolutionImporter.**use_preference_file**(*pref_file: str*)

Enable the use of the preference file for the Chemkin solution utility.

ChemkinSolutionImporter.**disable_preference_file**()

Disable the use of the preference file for the Chemkin solution utility.

ChemkinSolutionImporter.**enable_mass_fraction**()

Enable the use of mass fraction for the Chemkin solution utility.

ChemkinSolutionImporter.**disable_mass_fraction**()

Disable the use of mass fraction for the Chemkin solution utility.

ChemkinSolutionImporter.**enable_get_everything**()

Enable the 'get everything' option for the Chemkin solution utility.

ChemkinSolutionImporter.**disable_get_everything**()

Disable the 'get everything' option for the Chemkin solution utility.

ChemkinSolutionImporter.**enable_sensitivity**()

Enable the sensitivity option for the Chemkin solution utility.

ChemkinSolutionImporter.**disable_sensitivity**()

Disable the sensitivity option for the Chemkin solution utility.

ChemkinSolutionImporter.**enable_rop**()

Select the rate of progress option for the Chemkin solution utility.

ChemkinSolutionImporter.**disable_rop**()

Deselect the rate of progress option for the Chemkin solution utility.

ChemkinSolutionImporter.**validate_solution_data**() → bool

Validate the imported solution data.

ChemkinSolutionImporter.**get_solution_groups**() → list[str]

Get the list of available solution groups.

ChemkinSolutionImporter.**get_variable_labels**(*solution_group: str*) → list[str]

Get the list of variable labels for a specific solution group.

ChemkinSolutionImporter.**get_group_data**(*solution_group: str*) → pandas.DataFrame | None

Get the data for a specific solution group.

ChemkinSolutionImporter.**get_group_object**(*solution_group: str*) → *SolutionGroup* | None

Get the solution group object for a specific solution group.

ChemkinSolutionImporter.**get_solution_variable_array**(*solution_group: str*, *variable_label: str*) → pandas.Series[float] | None

Get the array of data for a specific variable within a solution group.

ChemkinSolutionImporter.**get_number_of_points**() → int

Get the number of solution points in the solution data.

ChemkinSolutionImporter.**get_number_of_gas_species**() → int

Get the number of gas species in the solution data.

ChemkinSolutionImporter.**get_gas_species_symbols**() → list[str]

Get the list of gas species symbols in the solution data.

### SolutionData

**class** ansys.chemkin.core.chemkin_solution_utility.**SolutionData**(*ckcsv: str | pathlib.Path*)

Manage the imported solution data for the Chemkin solution utility.

### Overview

### Methods

| | |
|---|---|
| *call_reader* | Read the solution data from the CSV file. |
| *save_current_soln_group* | Convert the collected rows into a wide-format DataFrame. |
| *list_solution_groups* | List all solution groups for the Chemkin solution utility. |
| *list_solution_variables* | List all solution variables for the specified solution group. |
| *get_solution_groups* | Return a list of all solution group names. |
| *get_solution_variables* | Return a list of all solution variables for the specified group. |
| *get_solution_group_data* | Return the raw data for the specified solution group. |
| *get_solution_group_object* | Return the SolutionGroup object for the specified solution group. |
| *get_solution_array* | Return the array for the specified variable in the given solution group. |
| *get_number_of_points* | Return the number of data points in solution data. |
| *get_gas_species_info* | Return the number of gas species and their symbols in solution data. |
| *get_number_of_gas_species* | Return the number of gas species in the solution data. |
| *get_gas_species_symbols* | Return the list of gas species symbols in the solution data. |
| *validate_group* | Validate if the specified solution group exists and has data. |

### Import detail

```
from ansys.chemkin.core.chemkin_solution_utility import SolutionData
```

### Method detail

SolutionData.**call_reader**(*csv_file: str | pathlib.Path*)

Read the solution data from the CSV file.

SolutionData.**save_current_soln_group**()

Convert the collected rows into a wide-format DataFrame.

SolutionData.**list_solution_groups**()

List all solution groups for the Chemkin solution utility.

SolutionData.**list_solution_variables**(*group_name: str*)

List all solution variables for the specified solution group.

SolutionData.**get_solution_groups**() → list[str]

Return a list of all solution group names.

SolutionData.**get_solution_variables**(*group_name: str*) → list[str]

Return a list of all solution variables for the specified group.

SolutionData.**get_solution_group_data**(*group_name: str*) → pandas.DataFrame | None

Return the raw data for the specified solution group.

SolutionData.**get_solution_group_object**(*group_name: str*) → *SolutionGroup* | None

Return the SolutionGroup object for the specified solution group.

SolutionData.**get_solution_array**(*group_name: str*, *var_name: str*) → pandas.Series[float] | None

Return the array for the specified variable in the given solution group.

SolutionData.**get_number_of_points**() → int

Return the number of data points in solution data.

SolutionData.**get_gas_species_info**() → tuple[int, list[str]]

Return the number of gas species and their symbols in solution data.

SolutionData.**get_number_of_gas_species**() → int

Return the number of gas species in the solution data.

SolutionData.**get_gas_species_symbols**() → list[str]

Return the list of gas species symbols in the solution data.

SolutionData.**validate_group**(*group_name: str*) → bool

Validate if the specified solution group exists and has data.

### SolutionGroup

**class** ansys.chemkin.core.chemkin_solution_utility.**SolutionGroup**(*group_name: str*, *raw_group_data: pandas.DataFrame*)

Class representing a single solution group within the Chemkin solution data.

### Overview

### Methods

| | |
|---|---|
| *get_variable_index* | Get the index of a variable in the solution group by its name. |
| *get_species_index* | Get the index of a gas species by its symbol. |
| *get_gas_species_variable_name* | Get the variable name of a gas species by its symbol. |
| *get_gas_species_variable_index* | Get the variable index of a gas species by its symbol. |
| *get_variable_array* | Get the array of values for a variable by its name. |
| *get_variable_array_with_tag* | Get the array of values for a variable by its name and a tag. |
| *get_variable_unit* | Get the unit of a variable by its name. |
| *get_variable_unit_with_tag* | Get the unit of a variable by its name. |

### Properties

| | |
|---|---|
| *number_of_variables* | Return the number of variables in the solution group. |
| *group_name* | Return the name of the solution group. |
| *group_data* | Return the raw data of the solution group. |
| *variable_names* | Return the list of variable names in the solution group. |
| *species_unit* | Return the species units of the solution group. |
| *number_of_points* | Return the number of solution points in the solution group. |
| *has_gas_species* | Return True if the solution group has any gas species, False otherwise. |
| *number_of_gas_species* | Return the number of gas species in the solution group. |
| *gas_species_symbols* | Return the list of gas species symbols in the solution group. |

### Import detail

```
from ansys.chemkin.core.chemkin_solution_utility import SolutionGroup
```

### Property detail

**property** SolutionGroup.**number_of_variables: int**

Return the number of variables in the solution group.

**property** SolutionGroup.**group_name: str**

Return the name of the solution group.

**property** SolutionGroup.**group_data: pandas.DataFrame**

Return the raw data of the solution group.

**property** SolutionGroup.**variable_names: list[str]**

Return the list of variable names in the solution group.

**property** SolutionGroup.**species_unit: str**

Return the species units of the solution group.

**property** SolutionGroup.**number_of_points: int**

Return the number of solution points in the solution group.

**property** SolutionGroup.**has_gas_species: bool**

Return True if the solution group has any gas species, False otherwise.

**property** SolutionGroup.**number_of_gas_species: int**

Return the number of gas species in the solution group.

**property** SolutionGroup.**gas_species_symbols: list[str]**

Return the list of gas species symbols in the solution group.

### Method detail

SolutionGroup.**get_variable_index**(*variable_name: str*) → int

Get the index of a variable in the solution group by its name.

SolutionGroup.**get_species_index**(*species_symbol: str*) → int

Get the index of a gas species by its symbol.

SolutionGroup.**get_gas_species_variable_name**(*species_symbol: str*) → str

Get the variable name of a gas species by its symbol.

SolutionGroup.**get_gas_species_variable_index**(*species_symbol: str*) → int

Get the variable index of a gas species by its symbol.

SolutionGroup.**get_variable_array**(*variable_name: str*) → pandas.Series[float]

Get the array of values for a variable by its name.

SolutionGroup.**get_variable_array_with_tag**(*variable_name: str*, *tag: str*) → pandas.Series[float]

Get the array of values for a variable by its name and a tag.

SolutionGroup.**get_variable_unit**(*variable_name: str*) → str

Get the unit of a variable by its name.

SolutionGroup.**get_variable_unit_with_tag**(*variable_name: str*, *tag: str*) → str

Get the unit of a variable by its name.

### Description

Python utilities to post-process Chemkin simulation solutions.

### The chemkin_wrapper.py module

### Summary

### Functions

| | |
|---|---|
| *get_chemkin_bin_dir* | Return the local Ansys Chemkin bin directory. |

### Attributes

| |
|---|
| *pyck_config* |
| *msg* |
| *this_msg* |
| *msg* |
| *msg* |

### Description

Chemkin-CFD-API python interfaces.

### Module detail

chemkin_wrapper.**get_chemkin_bin_dir**() → str

Return the local Ansys Chemkin bin directory.

chemkin_wrapper.**pyck_config**

chemkin_wrapper.**msg**

chemkin_wrapper.**this_msg = ''**

chemkin_wrapper.**msg**

chemkin_wrapper.**msg**

### The color.py module

### Summary

### Classes

| | |
|---|---|
| *Color* | Define colors used by PyChemkin for printing text messages. |

#### Color

**class** ansys.chemkin.core.color.**Color**

Define colors used by PyChemkin for printing text messages.

### Overview

### Attributes

| |
|---|
| *RED* |
| *MAGENTA* |
| *PURPLE* |
| *CYAN* |
| *GREEN* |
| *BLUE* |
| *YELLOW* |
| *WHITE* |
| *BOLD* |
| *ULINE* |
| *END* |
| *SPACE* |
| *SPACEx6* |
| *msg_modes* |
| *log_modes* |

### Static methods

| | |
|---|---|
| *ckprint* | Customize text messages. |

### Import detail

```
from ansys.chemkin.core.color import Color
```

### Attribute detail

Color.**RED** = '\x1b[91m'

Color.**MAGENTA** = '\x1b[035m'

Color.**PURPLE = '\x1b[95m'**

Color.**CYAN = '\x1b[96m'**

Color.**GREEN = '\x1b[92m'**

Color.**BLUE = '\x1b[94m'**

Color.**YELLOW = '\x1b[93m'**

Color.**WHITE = '\x1b[037m'**

Color.**BOLD = '\x1b[1m'**

Color.**ULINE = '\x1b[4m'**

Color.**END = Multiline-String**

```
"""
 [0m"""
```

Color.**SPACE = ' '**

Color.**SPACEx6 = ' '**

Color.**msg_modes**

Color.**log_modes**

### Method detail

**static** Color.**ckprint**(*mode: str*, *msg: list = []*)

Customize text messages.

### Description

Color and print format utilities.

### The `constants.py` module

### Summary

### Classes

| | |
|---|---|
| *Air* | define the “air” composition in PyChemkin with a fixed mixture “recipe”. |

### Functions

| | |
|---|---|
| *water_heat_vaporization* | Get the heat if vporization of water at the given temperature [K]. |

### Constants

| |
|---|
| *BOLTZMANN* |
| *AVOGADRO* |
| *P_ATM* |
| *P_TORRS* |
| *ERGS_PER_JOULE* |
| *JOULES_PER_CALORIE* |
| *ERGS_PER_CALORIE* |
| *ERGS_PER_EV* |
| *EV_PER_K* |
| *R_GAS* |
| *R_GAS_CAL* |

#### Air

**class** ansys.chemkin.core.constants.**Air**

define the "air" composition in PyChemkin with a fixed mixture "recipe".

### Overview

### Static methods

| | |
|---|---|
| *x* | Return the 'air' composition in mole fractions. |
| *y* | Return the 'air' composition in mass fractions. |

### Import detail

```
from ansys.chemkin.core.constants import Air
```

### Method detail

**static** Air.**x**(*cap: str = 'U'*) → list[tuple[str, float]]

Return the 'air' composition in mole fractions.

**static** Air.**y**(*cap: str = 'U'*) → list[tuple[str, float]]

Return the 'air' composition in mass fractions.

### Description

Constants used by Chemkin utilities and models.

### Module detail

constants.**water_heat_vaporization**(*temperature: float*) → float

Get the heat if vporization of water at the given temperature [K].

constants.**BOLTZMANN = 1.3806504e-16**

constants.**AVOGADRO = 6.02214179e+23**

constants.**P_ATM = 1013250.0**

constants.**P_TORRS = 1333.2236842105262**

constants.**ERGS_PER_JOULE = 10000000.0**

constants.**JOULES_PER_CALORIE = 4.184**

constants.**ERGS_PER_CALORIE = 41840000.0**

constants.**ERGS_PER_EV = 1.602176487e-12**

constants.**EV_PER_K = 11604.505289680863**

constants.**R_GAS = 83144724.71220216**

constants.**R_GAS_CAL = 1.9872066135803572**

### The `flame.py` module

### Summary

### Classes

| | |
|---|---|
| *Flame* | Chemkin steady state, one dimensional flame model. |

#### Flame

**class** ansys.chemkin.core.flame.**Flame**(*fuelstream:* ansys.chemkin.core.inlet.Stream, *label: str*)

Bases: *ansys.chemkin.core.reactormodel.ReactorModel*, *ansys.chemkin.core.steadystatesolver.SteadyStateSolver*, *ansys.chemkin.core.grid.Grid*

Chemkin steady state, one dimensional flame model.

### Overview

### Methods

| | |
|---|---|
| *set_temperature_profile* | Specify temperature profile. |
| *use_temp_profiel_initial_mesh* | Use the temperature profile grid as the initial grid. |
| *set_convection_differencing_type* | Specify the finite differencing scheme. |
| *set_mesh_keywords* | Set mesh related keywords. |
| *set_ss_solver_keywords* | Add steady-state solver parameter keywoprds to the keyword list. |
| *use_mixture_averaged_transport* | Use the mixture-averaged transport properties. |
| *use_multicomponent_transport* | Use the multi-component transport properties. |
| *use_fixed_lewis_number_transport* | Compute the species diffusion coefficient with fixed Lewis number. |
| *use_thermal_diffusion* | Include the thermal diffusion (Doret) effect. |
| *set_species_boundary_types* | Set the species boundary condition type (at inlet and outlet). |

### Attributes

| |
|---|
| *mass_flow_rate* |
| *temp_profile_set* |
| *grid_t_profile* |
| *energytypes* |
| *transport_mode* |

### Import detail

```
from ansys.chemkin.core.flame import Flame
```

### Attribute detail

Flame.**mass_flow_rate**

Flame.**temp_profile_set = False**

Flame.**grid_t_profile = False**

Flame.**energytypes**

Flame.**transport_mode = 0**

### Method detail

Flame.**set_temperature_profile**(*x: numpy.typing.NDArray[numpy.double]*, *temp: numpy.typing.NDArray[numpy.double]*) → int

Specify temperature profile.

Flame.**use_temp_profiel_initial_mesh**(*on: bool = False*)

Use the temperature profile grid as the initial grid.

Flame.**set_convection_differencing_type**(*mode: str*)

Specify the finite differencing scheme.

Flame.**set_mesh_keywords**() → int

Set mesh related keywords.

Flame.**set_ss_solver_keywords**()

Add steady-state solver parameter keywoprds to the keyword list.

Flame.**use_mixture_averaged_transport**()

Use the mixture-averaged transport properties.

Flame.**use_multicomponent_transport**()

Use the multi-component transport properties.

Flame.**use_fixed_lewis_number_transport**(*lewis: float = 1.0*)

Compute the species diffusion coefficient with fixed Lewis number.

Flame.**use_thermal_diffusion**(*mode: bool = True*)

Include the thermal diffusion (Doret) effect.

Flame.**set_species_boundary_types**(*mode: str = 'comp'*)

Set the species boundary condition type (at inlet and outlet).

### Description

Chemkin utilities for steady state, 1-D flame models.

### The grid.py module

### Summary

### Classes

| | |
|---|---|
| *Grid* | Grid quality control parameters for Chemkin 1-D steady-state reactor models. |

### Grid

**class** ansys.chemkin.core.grid.**Grid**

Grid quality control parameters for Chemkin 1-D steady-state reactor models.

### Overview

### Methods

| | |
|---|---|
| *set_numb_grid_points* | Set the number of uniform grid points at the start of simulation. |
| *set_max_grid_points* | Set the max number of grid points allowed during the solution refinement. |
| *set_reaction_zone_center* | Set the coordinate value of the reaction/mixing zone center. |
| *set_reaction_zone_width* | Set the width of the reaction/mixing. |
| *set_max_adaptive_points* | Set the max number of adaptive grid points allowed. |
| *set_solution_quality* | Set the maximum gradient and curvature ratios. |
| *set_grid_profile* | Specify the grid point coordinates of the initial grid points. |

### Properties

| | |
|---|---|
| *start_position* | Get the coordinate value of the first grid point. |
| *end_position* | Get the coordinate value of the last grid point. |

### Attributes

| |
|---|
| *max_numb_grid_points* |
| *max_numb_adapt_points* |
| *gradient* |
| *curvature* |
| *numb_grid_points* |
| *starting_x* |
| *ending_x* |
| *reaction_zone_center_x* |
| *reaction_zone_width* |
| *grid_profile* |
| *numb_grid_profile* |

### Import detail

```
from ansys.chemkin.core.grid import Grid
```

### Property detail

**property** Grid.**start_position: float**

Get the coordinate value of the first grid point.

**property** Grid.**end_position: float**

Get the coordinate value of the last grid point.

### Attribute detail

Grid.**max_numb_grid_points = 250**

Grid.**max_numb_adapt_points = 10**

Grid.**gradient = 0.1**

Grid.**curvature = 0.5**

Grid.**numb_grid_points = 6**

Grid.**starting_x = 0.0**

Grid.**ending_x = 0.0**

Grid.**reaction_zone_center_x = 0.0**

Grid.**reaction_zone_width = 0.0**

Grid.**grid_profile = []**

Grid.**numb_grid_profile = 0**

### Method detail

Grid.**set_numb_grid_points**(*numb_points: int*)

Set the number of uniform grid points at the start of simulation.

Grid.**set_max_grid_points**(*numb_points: int*)

Set the max number of grid points allowed during the solution refinement.

Grid.**set_reaction_zone_center**(*position: float*)

Set the coordinate value of the reaction/mixing zone center.

Grid.**set_reaction_zone_width**(*size: float*)

Set the width of the reaction/mixing.

Grid.**set_max_adaptive_points**(*numb_points: int*)

Set the max number of adaptive grid points allowed.

Grid.**set_solution_quality**(*gradient: float = 0.1*, *curvature: float = 0.5*)

Set the maximum gradient and curvature ratios.

Grid.**set_grid_profile**(*mesh: numpy.typing.NDArray[numpy.double]*) → int

Specify the grid point coordinates of the initial grid points.

### Description

Chemkin grid quality control parameters.

### The `hybridreactornetwork.py` module

### Summary

### Classes

| | |
|---|---|
| *ReactorNetwork* | Reactor network consists of PSRs and PFRs. |

#### ReactorNetwork

**class** ansys.chemkin.core.hybridreactornetwork.**ReactorNetwork**(*chem:* ansys.chemkin.core.chemistry.Chemistry)

Reactor network consists of PSRs and PFRs.

### Overview

### Methods

| | |
|---|---|
| *get_reactor_label* | Get the name/label of the reactor. |
| *add_reactor* | Add a reactor to the network in order. |
| *add_reactor_list* | Add a list of reactors to the network in order. |
| *show_reactors* | Show the reactor labels in the network. |
| *show_internal_outflow_connections* | Show the ouflow connections to other reactors in the network. |
| *show_internal_inflow_connections* | Show the incoming flow connections from other reactors in the network. |
| *add_outflow_connections* | Add outflow connections to other reactors in the network. |
| *clear_connections* | Clear the internal connection configurations. |
| *remove_reactor* | Remove the named reactor from the network. |
| *set_reactor_outflow* | Set up and verify the the outlet flow connections. |
| *set_inflow_connections* | Set up the sources of the internal network inlet. |
| *set_external_outlet* | Add a new network external outlet. |
| *calculate_incoming_streams* | Calculate the net inlet stream from all internal sources. |
| *set_internal_inlet* | Create or update the merged inlet stream. |
| *create_internal_inlet* | Create a new inlet stream from all internal sources. |
| *add_heat_exchange* | Add heat exchange connection. |
| *calculate_heat_exchange* | Calculate the net heat loss rates due to heat exchange. |
| *get_network_run_status* | Get network run status. |
| *run* | Solve the hybrid reactor network. |
| *get_reactor_stream* | Get the solution Stream object. |
| *set_external_streams* | Set up external outlet streams. |
| *get_external_stream* | Get the list of external outlet Stream objects. |
| *run_without_tearstream* | Run the reactor network without using tear stream iteration. |
| *run_with_tearstream* | Run the reactor network using tear stream iteration. |
| *remove_tearpoint* | Remove the tear point from the list. |
| *add_tearingpoint* | Add a new tear point to the list. |
| *set_tear_tolerance* | Set the relative tolerance for tear stream convergence. |
| *set_tear_iteration_limit* | Set the maximum number of tear loop iterations. |
| *check_iteration_count* | Check the iteration count for over the set limit. |
| *set_relaxation_factor* | Set the relaxation factor. |

continues on next page

Table 1 – continued from previous page

| | |
|---|---|
| *check_tearstream_convergence* | Check solution convergency at the tear point. |
| *update_tear_solution* | Update the old/temporary tear point solution. |

### Properties

| | |
|---|---|
| *number_reactors* | Get the number of reactors in the network. |
| *number_external_outlets* | Get the number of external outlets from the network. |

### Attributes

| |
|---|
| *numb_reactors* |
| *last_reactor* |
| *numb_external_outlet* |
| *external_outlets* |
| *external_outlet_streams* |
| *reactor_map* |
| *reactor_objects* |
| *reactor_solutions* |
| *outflow_targets* |
| *outflow_altered* |
| *external_connections* |
| *inflow_sources* |
| *internal_inflow* |
| *internal_inflow_ready* |
| *numb_tearpoints* |
| *tearpoint* |
| *max_tearloop_count* |
| *tolerance* |
| *relaxation_factor* |
| *tear_converged* |
| *numb_heat_exchange_pairs* |
| *heat_exchange_pair* |
| *heat_exchange_coeffcients* |
| *heat_exchange_rate_out* |
| *network_run_status* |

### Import detail

```
from ansys.chemkin.core.hybridreactornetwork import ReactorNetwork
```

### Property detail

**property** ReactorNetwork.**number_reactors: int**

Get the number of reactors in the network.

**property** ReactorNetwork.**number_external_outlets: int**

Get the number of external outlets from the network.

### Attribute detail

ReactorNetwork.**numb_reactors = 0**

ReactorNetwork.**last_reactor = 0**

ReactorNetwork.**numb_external_outlet = 0**

ReactorNetwork.**external_outlets: dict[int, int]**

ReactorNetwork.**external_outlet_streams: dict[int, *ansys.chemkin.core.inlet.Stream*]**

ReactorNetwork.**reactor_map: dict[str, int]**

ReactorNetwork.**reactor_objects: dict[int,**
**ansys.chemkin.core.stirreactors.PSR.PerfectlyStirredReactor |**
**ansys.chemkin.core.flowreactors.PFR.PlugFlowReactor]**

ReactorNetwork.**reactor_solutions: dict[int, *ansys.chemkin.core.inlet.Stream*]**

ReactorNetwork.**outflow_targets: dict[int, list[tuple[int, float]]]**

ReactorNetwork.**outflow_altered = True**

ReactorNetwork.**external_connections: dict[int, int]**

ReactorNetwork.**inflow_sources: dict[int, list[tuple[int, float]]]**

ReactorNetwork.**internal_inflow: dict[int, *ansys.chemkin.core.inlet.Stream*]**

ReactorNetwork.**internal_inflow_ready: dict[int, bool]**

ReactorNetwork.**numb_tearpoints = 0**

ReactorNetwork.**tearpoint: list[int] = []**

ReactorNetwork.**max_tearloop_count = 200**

ReactorNetwork.**tolerance = 1e-06**

ReactorNetwork.**relaxation_factor = 1.0**

ReactorNetwork.**tear_converged = False**

ReactorNetwork.**numb_heat_exchange_pairs = 0**

ReactorNetwork.**heat_exchange_pair: list[tuple[int, int]] = []**

ReactorNetwork.**heat_exchange_coeffcients: list[float] = []**

ReactorNetwork.**heat_exchange_rate_out: dict[int, float]**

ReactorNetwork.**network_run_status = -100**

### Method detail

ReactorNetwork.**get_reactor_label**(*reactor_index: int*) → str

Get the name/label of the reactor.

ReactorNetwork.**add_reactor**(*reactor: ansys.chemkin.core.stirreactors.PSR.PerfectlyStirredReactor | ansys.chemkin.core.flowreactors.PFR.PlugFlowReactor*)

Add a reactor to the network in order.

ReactorNetwork.**add_reactor_list**(*reactor_list: list[ansys.chemkin.core.stirreactors.PSR.PerfectlyStirredReactor | ansys.chemkin.core.flowreactors.PFR.PlugFlowReactor]*)

Add a list of reactors to the network in order.

ReactorNetwork.**show_reactors**()

Show the reactor labels in the network.

ReactorNetwork.**show_internal_outflow_connections**()

Show the ouflow connections to other reactors in the network.

ReactorNetwork.**show_internal_inflow_connections**()

Show the incoming flow connections from other reactors in the network.

ReactorNetwork.**add_outflow_connections**(*source_label: str*, *outflow_split: list[tuple[str, float]]*)

Add outflow connections to other reactors in the network.

ReactorNetwork.**clear_connections**()

Clear the internal connection configurations.

ReactorNetwork.**remove_reactor**(*name: str*)

Remove the named reactor from the network.

ReactorNetwork.**set_reactor_outflow**()

Set up and verify the the outlet flow connections.

ReactorNetwork.**set_inflow_connections**()

Set up the sources of the internal network inlet.

ReactorNetwork.**set_external_outlet**(*reactor_index: int*)

Add a new network external outlet.

ReactorNetwork.**calculate_incoming_streams**(*reactor_index: int*) → *ansys.chemkin.core.inlet.Stream* | None

Calculate the net inlet stream from all internal sources.

ReactorNetwork.**set_internal_inlet**(*reactor_index: int*) → int

Create or update the merged inlet stream.

ReactorNetwork.**create_internal_inlet**(*reactor_index: int*)

Create a new inlet stream from all internal sources.

ReactorNetwork.**add_heat_exchange**(*reactor1: str*, *reactor2: str*, *heat_transfer_coeff: float*, *heat_transfer_area: float*)

Add heat exchange connection.

ReactorNetwork.**calculate_heat_exchange**()

Calculate the net heat loss rates due to heat exchange.

ReactorNetwork.**get_network_run_status**() → int

Get network run status.

ReactorNetwork.**run**() → int

Solve the hybrid reactor network.

ReactorNetwork.**get_reactor_stream**(*reactor_name: str*) → *ansys.chemkin.core.inlet.Stream*

Get the solution Stream object.

ReactorNetwork.**set_external_streams**()

Set up external outlet streams.

ReactorNetwork.**get_external_stream**(*stream_index: int*) → list[*ansys.chemkin.core.inlet.Stream*]

Get the list of external outlet Stream objects.

ReactorNetwork.**run_without_tearstream**() → int

Run the reactor network without using tear stream iteration.

ReactorNetwork.**run_with_tearstream**() → int

Run the reactor network using tear stream iteration.

ReactorNetwork.**remove_tearpoint**(*reactor_name: str*)

Remove the tear point from the list.

ReactorNetwork.**add_tearingpoint**(*reactor_name: str*)

Add a new tear point to the list.

ReactorNetwork.**set_tear_tolerance**(*tol: float = 1e-06*)

Set the relative tolerance for tear stream convergence.

ReactorNetwork.**set_tear_iteration_limit**(*max_count: int*)

Set the maximum number of tear loop iterations.

ReactorNetwork.**check_iteration_count**(*count: int*) → bool

Check the iteration count for over the set limit.

ReactorNetwork.**set_relaxation_factor**(*relax: float*)

Set the relaxation factor.

ReactorNetwork.**check_tearstream_convergence**(*stream_a*, *stream_b*) → tuple[bool, float]

Check solution convergency at the tear point.

ReactorNetwork.**update_tear_solution**(*new_stream:* ansys.chemkin.core.inlet.Stream, *old_stream:* ansys.chemkin.core.inlet.Stream) → *ansys.chemkin.core.inlet.Stream*

Update the old/temporary tear point solution.

### Description

Hybrid reactor network comprised of a mix of open reactors such as PSR and PFR.

### The `info.py` module

### Summary

### Functions

| | |
|---|---|
| *setup_hints* | Set up Chemkin keyword hints. |
| *clear_hints* | Clear the Chemkin keyword data. |
| *keyword_hints* | Get hints about the Chemkin keyword. |
| *phrase_hints* | Get keyword hints by using key phrase in the description. |
| *help* | Provide assistance on finding information about Chemkin keywords. |
| *show_realgas_usage* | Show Chemkin real-gas model usage and options. |
| *show_equilibrium_options* | Show the equilibrium calculation usage and options. |
| *show_ignition_definitions* | Show the ignition definitions available in Chemkin. |
| *manuals* | Access the Chemkin manuals page on the Ansys Help portal. |

### Attributes

| |
|---|
| *ck_dict* |

### Description

Chemkin help menu for keywords and key phrases.

### Module detail

info.**setup_hints**()

Set up Chemkin keyword hints.

info.**clear_hints**()

Clear the Chemkin keyword data.

info.**keyword_hints**(*mykey: str*)

Get hints about the Chemkin keyword.

info.**phrase_hints**(*phrase: str*)

Get keyword hints by using key phrase in the description.

info.**help**(*topic: str | None = None*)

Provide assistance on finding information about Chemkin keywords.

info.**show_realgas_usage**()

Show Chemkin real-gas model usage and options.

info.**show_equilibrium_options**()

Show the equilibrium calculation usage and options.

info.**show_ignition_definitions**()

Show the ignition definitions available in Chemkin.

info.**manuals**()

Access the Chemkin manuals page on the Ansys Help portal.

info.**ck_dict: dict[str, Any]**

### The `inlet.py` module

### Summary

### Classes

| | |
|---|---|
| *Stream* | Generic inlet stream object for Chemkin open reactor models. |

### Functions

| | |
|---|---|
| *clone_stream* | Copy the properties of the source Stream to the target Stream. |
| *compare_streams* | Compare properties of stream B against those of stream A. |
| *adiabatic_mixing_streams* | Create a new Stream object by mixing two streams adiabatically. |
| *create_stream_from_mixture* | Create a new Stream object from the given Mixture object. |

#### Stream

**class** ansys.chemkin.core.inlet.**Stream**(*chem*, *label: str | None = None*)

Bases: *ansys.chemkin.core.mixture.Mixture*

Generic inlet stream object for Chemkin open reactor models.

### Overview

### Methods

| | |
|---|---|
| *convert_to_mass_flowrate* | Convert different types of flow rate value to mass flow rate. |
| *convert_to_vol_flowrate* | Convert different types of flow rate value to volumetric flow rate. |
| *convert_to_sccm* | Convert different types of flow rate value to SCCM. |
| *check_flow_rate_mode* | Get the type of flow rate is provided to this inlet. |
| *notify_inlet_profile* | Display warning that flow rate profile is specified. |
| *profile_out_of_range* | Report value out of the profile data range. |
| *set_mass_flowrate_profile* | Specify inlet mass flow rate profile. |
| *get_mass_flowrate_profile_data* | Get the mass flow rate at the given time from the profile data. |
| *set_vol_flowrate_profile* | Specify inlet volumetric flow rate profile. |
| *get_vol_flowrate_profile_data* | Get the volumetric flow rate at the given time from the profile data. |
| *set_sccm_flowrate_profile* | Specify inlet sccm flow rate profile. |
| *get_sccm_flowrate_profile_data* | Get the sccm flow rate at the given time from the profile data. |

### Properties

| | |
|---|---|
| *flowarea* | Get inlet flow area. |
| *mass_flowrate* | Get inlet mass flow rate. |
| *vol_flowrate* | Get inlet volumetric flow rate. |
| *sccm* | Get inlet SCCM volumetric flow rate. |
| *velocity* | Get inlet gas velocity. |
| *velocity_gradient* | Get inlet gas axial velocity gradient. |
| *label* | Get the label of the Stream. |

### Attributes

| |
|---|
| *flow_rate_profile_keys* |
| *flowrate_profile* |

### Import detail

```
from ansys.chemkin.core.inlet import Stream
```

### Property detail

**property** Stream.**flowarea: float**

Get inlet flow area.

**property** Stream.**mass_flowrate: float**

Get inlet mass flow rate.

**property** Stream.**vol_flowrate: float**

Get inlet volumetric flow rate.

**property** Stream.**sccm: float**

Get inlet SCCM volumetric flow rate.

**property** Stream.**velocity: float**

Get inlet gas velocity.

**property** Stream.**velocity_gradient: float**

Get inlet gas axial velocity gradient.

**property** Stream.**label: str**

Get the label of the Stream.

### Attribute detail

Stream.**flow_rate_profile_keys: dict[int, str]**

Stream.**flowrate_profile: dict[str, tuple[numpy.typing.NDArray[numpy.double], numpy.typing.NDArray[numpy.double]]]**

### Method detail

Stream.**convert_to_mass_flowrate**() → float

Convert different types of flow rate value to mass flow rate.

Stream.**convert_to_vol_flowrate**() → float

Convert different types of flow rate value to volumetric flow rate.

Stream.**convert_to_sccm**() → float

Convert different types of flow rate value to SCCM.

Stream.**check_flow_rate_mode**() → int

Get the type of flow rate is provided to this inlet.

Stream.**notify_inlet_profile**(*mode: str*)

Display warning that flow rate profile is specified.

Stream.**profile_out_of_range**(*time: float*, *max: float*, *min: float*)

Report value out of the profile data range.

Stream.**set_mass_flowrate_profile**(*x: numpy.typing.NDArray[numpy.double]*, *flow_rate: numpy.typing.NDArray[numpy.double]*)

Specify inlet mass flow rate profile.

Stream.**get_mass_flowrate_profile_data**(*time: float = 0.0*) → float

Get the mass flow rate at the given time from the profile data.

Stream.**set_vol_flowrate_profile**(*x: numpy.typing.NDArray[numpy.double]*, *flow_rate: numpy.typing.NDArray[numpy.double]*)

Specify inlet volumetric flow rate profile.

Stream.**get_vol_flowrate_profile_data**(*time: float = 0.0*) → float

Get the volumetric flow rate at the given time from the profile data.

Stream.**set_sccm_flowrate_profile**(*x: numpy.typing.NDArray[numpy.double]*, *flow_rate: numpy.typing.NDArray[numpy.double]*)

Specify inlet sccm flow rate profile.

Stream.**get_sccm_flowrate_profile_data**(*time: float = 0.0*) → float

Get the sccm flow rate at the given time from the profile data.

### Description

Chemkin reactor inlet/stream utilities.

### Module detail

inlet.**clone_stream**(*source:* Stream, *target:* Stream)

Copy the properties of the source Stream to the target Stream.

inlet.**compare_streams**(*stream_a:* Stream, *stream_b:* Stream, *atol: float = 1e-10*, *rtol: float = 0.001*, *mode: str = 'mass'*) → tuple[bool, float, float]

Compare properties of stream B against those of stream A.

inlet.**adiabatic_mixing_streams**(*stream_a:* Stream, *stream_b:* Stream) → *Stream*

Create a new Stream object by mixing two streams adiabatically.

inlet.**create_stream_from_mixture**(*chem:* ansys.chemkin.core.chemistry.Chemistry, *mixture:* ansys.chemkin.core.mixture.Mixture, *flow_rate: float = 0.0*, *mode: str = 'mass'*) → *Stream*

Create a new Stream object from the given Mixture object.

### The `keyword.py` module

### Summary

### Classes

| | |
|---|---|
| *Keyword* | A Chemkin style keyword. |
| *BooleanKeyword* | Chemkin boolean keyword. |
| *IntegerKeyword* | A Chemkin integer keyword. |
| *RealKeyword* | A Chemkin real keyword. |
| *StringKeyword* | A Chemkin string keyword. |

#### Keyword

**class** ansys.chemkin.core.keyword.**Keyword**(*phrase: str*, *value: float | bool | str*, *data_type: str*)

A Chemkin style keyword.

### Overview

### Methods

| | |
|---|---|
| *show* | Display the Chemkin keyword and its parameter value. |
| *resetvalue* | Reset the parameter value of an existing keyword. |
| *parametertype* | Get parameter type of the keyword. |
| *getvalue_as_string* | Create the keyword input line for Chemkin applications. |

### Properties

| | |
|---|---|
| *value* | Get parameter value of the keyword. |
| *keyphrase* | Get the phrase of the keyword. |
| *keyprefix* | Get the status of the keyword. |

### Attributes

| |
|---|
| *gasspecieskeywords* |
| *flowratekeywords* |
| *profilekeywords* |
| *fourspaces* |
| *no_fullkeyword* |

### Static methods

| | |
|---|---|
| *setfullkeywords* | Require all keywords and their parameters must be specified. |

### Import detail

```
from ansys.chemkin.core.keyword import Keyword
```

### Property detail

**property** Keyword.**value: int | float | bool | str**

Get parameter value of the keyword.

**property** Keyword.**keyphrase: str**

Get the phrase of the keyword.

**property** Keyword.**keyprefix: bool**

Get the status of the keyword.

### Attribute detail

Keyword.**gasspecieskeywords = ['REAC', 'XEST', 'FUEL', 'OXID']**

Keyword.**flowratekeywords = ['FLRT', 'VDOT', 'VEL', 'SCCM']**

Keyword.**profilekeywords = ['TPRO', 'PPRO', 'VPRO', 'QPRO', 'AINT', 'AEXT', 'DPRO', 'FPRO', 'SCCMPRO', 'VDOTPRO', 'VELPRO',...**

Keyword.**fourspaces = ' '**

Keyword.**no_fullkeyword = True**

### Method detail

**static** Keyword.**setfullkeywords**(*mode: bool*)

Require all keywords and their parameters must be specified.

Keyword.**show**()

Display the Chemkin keyword and its parameter value.

Keyword.**resetvalue**(*value: float | bool | str*)

Reset the parameter value of an existing keyword.

Keyword.**parametertype**() → type

Get parameter type of the keyword.

Keyword.**getvalue_as_string**() → tuple[int, str]

Create the keyword input line for Chemkin applications.

#### BooleanKeyword

**class** ansys.chemkin.core.keyword.**BooleanKeyword**(*phrase: str*)

Bases: Keyword

Chemkin boolean keyword.

### Import detail

```
from ansys.chemkin.core.keyword import BooleanKeyword
```

#### IntegerKeyword

**class** ansys.chemkin.core.keyword.**IntegerKeyword**(*phrase: str*, *value: int = 0*)

Bases: Keyword

A Chemkin integer keyword.

### Import detail

```
from ansys.chemkin.core.keyword import IntegerKeyword
```

#### RealKeyword

**class** ansys.chemkin.core.keyword.**RealKeyword**(*phrase: str*, *value: float = 0.0*)

Bases: Keyword

A Chemkin real keyword.

### Import detail

```
from ansys.chemkin.core.keyword import RealKeyword
```

#### StringKeyword

**class** ansys.chemkin.core.keyword.**StringKeyword**(*phrase: str*, *value: str = ''*)

Bases: Keyword

A Chemkin string keyword.

### Import detail

```
from ansys.chemkin.core.keyword import StringKeyword
```

### Description

Chemkin keyword models and helpers.

### The logger.py module

### Summary

### Classes

| | |
|---|---|
| *SingletonType* | **no-index** Provides the singleton helper class for the logger. |
| *ChemkinLogger* | Provides the singleton logger for the PyChemkin. |

### Attributes

| |
|---|
| *logger* |

#### SingletonType

**class** ansys.chemkin.core.logger.**SingletonType**

Bases: type

**No-index**
Provides the singleton helper class for the logger.

### Overview

### Special methods

| | |
|---|---|
| *__call__* | Call to redirect new instances to the singleton instance. |

### Import detail

```
from ansys.chemkin.core.logger import SingletonType
```

### Method detail

SingletonType.**__call__**(**args*, ***kwargs*)

Call to redirect new instances to the singleton instance.

#### ChemkinLogger

**class** ansys.chemkin.core.logger.**ChemkinLogger**(*level: int = logging.ERROR*, *logger_name: str = 'PyChemkin'*)

Bases: object

Provides the singleton logger for the PyChemkin.

#### Parameters

**to_file**
[bool, default: False] Whether to include the logs in a file.

### Overview

### Methods

| | |
|---|---|
| *get_logger* | Get the logger. |
| *set_level* | Set the logger output level. |
| *enable_output* | Enable logger output to a given stream. |
| *add_file_handler* | Save logs to a file in addition to printing them to the standard output. |

### Import detail

```
from ansys.chemkin.core.logger import ChemkinLogger
```

### Method detail

ChemkinLogger.**get_logger**() → logging.Logger

Get the logger.

#### Returns

**Logger**
Logger.

ChemkinLogger.**set_level**(*level: int*) → None

Set the logger output level.

#### Parameters

**level**
[int, {0, 10, 20, 30, 40, 50}] Output Level of the logger. 0 = NOTSET 10 = DEBUG 20 = INFO 30 = WARNING 40 = ERROR 50 = CRITICAL

ChemkinLogger.**enable_output**(*stream=None*)

Enable logger output to a given stream.

If a stream is not specified, `sys.stderr` is used.

#### Parameters

**stream: TextIO, default: `sys.stderr`**
Stream to output the log output to.

ChemkinLogger.**add_file_handler**(*logs_dir: str = './.log'*)

Save logs to a file in addition to printing them to the standard output.

#### Parameters

**logs_dir**
[str, default: `"./.log"`] Directory of the logs.

### Description

Provides the singleton helper class for the logger.

### Module detail

logger.**logger**

### The microprocess.py module

### Summary

### Classes

| | |
|---|---|
| *MicroMixing* | Micro mixing process module. |

#### MicroMixing

**class** ansys.chemkin.core.microprocess.**MicroMixing**

Micro mixing process module.

### Overview

### Methods

| | |
|---|---|
| *set_numb_particles* | Set the total number of events/particles. |
| *set_mixing_time_step* | Set the time duration of the micro mixing process. |
| *set_mixing_time_scale* | Set the characteristic scalar mixing time scale. |
| *set_mixing_model_parameter* | Set the micro mixing model parameter. |
| *set_particle_mixtures* | Set up the particle mixtures. |
| *calculate_particles_per_mixture* | Determine the particle count distribution. |
| *map_mixture_particles* | Map particle index to the mixture index. |
| *get_source_integer* | Create an integer list for the random pick process. |
| *update_mixtures* | Update particle mixtures. |
| *modified_curls* | Perform particle mixing with the Modified Curl's mixing model. |
| *particle_mixing_curls* | Select random integers from an integer list. |

### Attributes

| |
|---|
| *numb_particles* |
| *delta_time* |
| *mixing_time_scale* |
| *mixing_model_parameter* |
| *mixing_fraction_limit* |
| *particles_map* |
| *mixture_map* |
| *mixture_particles* |
| *changed_mixtures* |
| *particle_unit_mass* |

### Import detail

```
from ansys.chemkin.core.microprocess import MicroMixing
```

### Attribute detail

MicroMixing.**numb_particles = 0**

MicroMixing.**delta_time = 0.0**

MicroMixing.**mixing_time_scale = 0.0**

MicroMixing.**mixing_model_parameter = 1.0**

MicroMixing.**mixing_fraction_limit = 0.3**

MicroMixing.**particles_map: dict[int, int]**

MicroMixing.**mixture_map: dict[int, *ansys.chemkin.core.inlet.Stream*]**

MicroMixing.**mixture_particles: list[int] = []**

MicroMixing.**changed_mixtures: dict[int, list[*ansys.chemkin.core.inlet.Stream*]]**

MicroMixing.**particle_unit_mass = 0.0**

### Method detail

MicroMixing.**set_numb_particles**(*nparticles: int*)
Set the total number of events/particles.

MicroMixing.**set_mixing_time_step**(*dtime: float*)
Set the time duration of the micro mixing process.

MicroMixing.**set_mixing_time_scale**(*tau: float*)
Set the characteristic scalar mixing time scale.

MicroMixing.**set_mixing_model_parameter**(*cmix: float*)
Set the micro mixing model parameter.

MicroMixing.**set_particle_mixtures**(*particle_mixtures: list[*ansys.chemkin.core.inlet.Stream*]*)
Set up the particle mixtures.

MicroMixing.**calculate_particles_per_mixture**()
Determine the particle count distribution.

MicroMixing.**map_mixture_particles**()
Map particle index to the mixture index.

MicroMixing.**get_source_integer**(*min_integer: int*, *max_integer: int*) → list[int]
Create an integer list for the random pick process.

MicroMixing.**update_mixtures**()
Update particle mixtures.

MicroMixing.**modified_curls**(*delta_time: float*, *tau: float*, *cmix: float = 1.0*) → list[*ansys.chemkin.core.inlet.Stream*]
Perform particle mixing with the Modified Curl's mixing model.

MicroMixing.**particle_mixing_curls**(*numb_picks: int*, *source_list: list[int]*)
Select random integers from an integer list.

### Description

Micro mixing model for the multi-zone engine models.

### The `mixture.py` module

### Summary

### Classes

| | |
|---|---|
| *Mixture* | define a mixture based on the gas species in the given chemistry set. |

### Functions

| | |
|---|---|
| *verify_mixture* | Verify the Mixture is valid and active. |
| *isothermal_mixing* | Mixing multiple gas mixtures at gioven temperature. |
| *adiabatic_mixing* | Mixing multiple gas mixtures adiabatically. |
| *calculate_mixture_temperature_from_ent* | Compute the mixture temperature from the given mixture enthalpy. |
| *cal_mixture_temperature_from_enthalpy* | Compatibility wrapper for calculate_mixture_temperature_from_enthalpy. |
| *create_mixture_recipe_from_fractions* | Build a PyChemkin mixture recipe/formula from a species fraction array. |
| *calculate_stoichiometrics* | Calculate the stoichiometric coefficients. |
| *species_diffusion_velocity* | Compute species diffusive velocities between two gas mixtures. |
| *mixing_by_exchange_with_the_mean* | Mix two mixtures of the same mass using the IEM model. |
| *calculate_mass_weighted_mean_mixture* | Calculate the mass-weighted mean mixture from a list of mixtures. |
| *interpolate_mixtures* | Get Mixture by interpolation. |
| *compare_mixtures* | Compare properties of mixture B against those of mixture A. |
| *calculate_equilibrium* | Get the equilibrium mixture composition. |
| *equilibrium* | Find the equilibrium state mixture corresponding to the given mixture. |
| *detonation* | Find the Chapman-Jouguet state mixture and detonation wave speed. |

#### Mixture

**class** ansys.chemkin.core.mixture.**Mixture**(*chem:* ansys.chemkin.core.chemistry.Chemistry)

define a mixture based on the gas species in the given chemistry set.

### Overview

### Methods

| | |
|---|---|
| *get_specindex* | Get the index of the given gas species symbol. |
| *check_volume* | Check if volume is defined. |
| *list_composition* | List the mixture composition. |
| *find_equilibrium* | Create the equilibrium state mixture corresponding to mixture itself. |
| *hml* | Get enthalpy of the mixture. |
| *cpbl* | Get specific heat capacity of the mixture. |

continues on next page

Table 2 – continued from previous page

| | |
|---|---|
| *set_pressure_by_density* | Set the gas mixture pressure value. |
| *thermicity* | Get the total thermicity of the mixture. |
| *sound_speed* | Get the speed of sound of the mixture. |
| *rop* | Get species rate of productions. |
| *rxn_rates* | Get molar reaction rates. |
| *species_cp* | Get species specific heat capacity at constant pressure. |
| *species_h* | Get species enthalpy. |
| *species_visc* | Get species viscosity. |
| *species_cond* | Get species conductivity. |
| *species_diffusion_coeffs* | Get species diffusion coefficients. |
| *mixture_viscosity* | Get viscosity of the gas mixture. |
| *mixture_conductivity* | Get conductivity of the gas mixture. |
| *mixture_diffusion_coeffs* | Get mixture-averaged species diffusion coefficients of the gas mixture. |
| *mixture_binary_diffusion_coeffs* | Get multi-component species binary diffusion coefficients. |
| *mixture_thermal_diffusion_coeffs* | Get thermal diffusivity of the gas mixture. |
| *vol_hrr* | Get volumetric heat release rate. |
| *rop_mass* | Get species mass rates of production. |
| *list_rop* | List species molar production rate. |
| *list_rop_mass* | List species mass rate of production in descending order. |
| *list_reaction_rates* | List information about reaction rate in descending order. |
| *x_by_equivalence_ratio* | Set mole fractions using equivalence ratio. |
| *y_by_equivalence_ratio* | Set mass fractions using equivalence ratio. |
| *get_egr_mole_fraction* | Compute the EGR composition in mole fraction. |
| *validate* | Check whether the mixture is fully defined. |
| *use_realgas_cubic_eos* | Turn ON the real-gas cubic EOS. |
| *use_idealgas_law* | Turn on the ideal gas law to compute mixture properties. |
| *set_realgas_mixing_rule* | Set the mixing rule for calculating the real-gas mixture properties. |
| *reset_composition* | Reset all species fractions to 0. |
| *get_surf_specindex* | Get the index of the given surface/bulk species symbol. |
| *get_all_surface_temperature* | Get the temperatures of all materials in the mechanism. |
| *rop_surf* | Get species molar rate of production. |
| *rxnrates_surf* | Get molar rates of the surface reactions. |

### Properties

| | |
|---|---|
| *chemid* | Get chemistry set index. |
| *kk* | Get the number of gas species. |
| *pressure* | Get gas mixture pressure. |
| *temperature* | Get gas mixture temperature. |
| *volume* | Get mixture volume. |
| *x* | Get mixture mole fraction. |
| *y* | Get mixture mass fraction. |
| *concentration* | Get mixture molar concentrations. |
| *eos* | Get the available real-gas EOS model that is provided in the mechanism. |
| *wt* | Get species molecular masses. |
| *wtm* | Get mean molar mass of the gas mixture. |
| *rho* | Get mixture mass density. |

### Attributes

| |
|---|
| *transport_data* |
| *userealgas* |
| *mole_fractions* |
| *mass_fractions* |
| *species_molar_weight* |
| *mean_molar_weight* |

### Static methods

| | |
|---|---|
| *normalize* | Normalize the mixture composition. |
| *mean_molar_mass* | Get mean molar mass of the gas mixture. |
| *mole_fraction_to_mass_fraction* | Convert mole fraction to mass fraction. |
| *mass_fraction_to_mole_fraction* | Convert mass fraction to mole fraction. |
| *mass_fraction_to_concentration* | Convert mass fractions to molar concentrations. |
| *mole_fraction_to_concentration* | Convert mole fractions to molar concentrations. |
| *density* | Get mass density from the given mixture condition. |
| *mixture_specific_heat* | Get mixture specific heat capacity at constant pressure. |
| *mixture_enthalpy* | Get mixture enthalpy. |
| *calculate_pressure_from_density* | Calculate the gas mixture pressure. |
| *mixture_thermicity* | Get mixture total thermicity. |
| *mixture_sound_speed* | Get mixture speed of sound. |
| *rate_of_production* | Get species molar rate of production. |
| *reaction_rates* | Get the molar rates of the gas reactions. |

### Special methods

| | |
|---|---|
| *__deepcopy__* | Support copy.deepcopy without copying immutable Chemistry metadata. |

### Import detail

```
from ansys.chemkin.core.mixture import Mixture
```

### Property detail

**property** Mixture.**chemid: int**

Get chemistry set index.

**property** Mixture.**kk: int**

Get the number of gas species.

**property** Mixture.**pressure: float**

Get gas mixture pressure.

**property** Mixture.**temperature: float**

Get gas mixture temperature.

**property** Mixture.**volume: float**

Get mixture volume.

**property** Mixture.**x: numpy.typing.NDArray[numpy.double]**

Get mixture mole fraction.

**property** Mixture.**y: numpy.typing.NDArray[numpy.double]**

Get mixture mass fraction.

**property** Mixture.**concentration: numpy.typing.NDArray[numpy.double]**

Get mixture molar concentrations.

**property** Mixture.**eos: int**

Get the available real-gas EOS model that is provided in the mechanism.

**property** Mixture.**wt: numpy.typing.NDArray[numpy.double]**

Get species molecular masses.

**property** Mixture.**wtm: float**

Get mean molar mass of the gas mixture.

**property** Mixture.**rho: float**

Get mixture mass density.

### Attribute detail

Mixture.**transport_data**

Mixture.**userealgas**

Mixture.**mole_fractions**

Mixture.**mass_fractions**

Mixture.**species_molar_weight**

Mixture.**mean_molar_weight**

### Method detail

Mixture.**__deepcopy__**(*memo: dict[int, object]*) → *Mixture*

Support copy.deepcopy without copying immutable Chemistry metadata.

Mixture.**get_specindex**(*symbol: str*) → int

Get the index of the given gas species symbol.

Mixture.**check_volume**() → bool

Check if volume is defined.

**static** Mixture.**normalize**(*frac: list[float] | numpy.typing.NDArray[numpy.double]*) → tuple[int, numpy.typing.NDArray[numpy.double]]

Normalize the mixture composition.

**static** Mixture.**mean_molar_mass**(*frac: numpy.typing.NDArray[numpy.double]*, *wt: numpy.typing.NDArray[numpy.double]*, *mode: str*) → float

Get mean molar mass of the gas mixture.

**static** Mixture.**mole_fraction_to_mass_fraction**(*molefrac: numpy.typing.NDArray[numpy.double]*, *wt: numpy.typing.NDArray[numpy.double]*) → numpy.typing.NDArray[numpy.double]

Convert mole fraction to mass fraction.

**static** Mixture.**mass_fraction_to_mole_fraction**(*massfrac: numpy.typing.NDArray[numpy.double]*, *wt: numpy.typing.NDArray[numpy.double]*) → numpy.typing.NDArray[numpy.double]

Convert mass fraction to mole fraction.

**static** Mixture.**mass_fraction_to_concentration**(*chemid: int*, *p: float*, *t: float*, *massfrac: numpy.typing.NDArray[numpy.double]*, *wt: numpy.typing.NDArray[numpy.double]*) → numpy.typing.NDArray[numpy.double]

Convert mass fractions to molar concentrations.

**static** Mixture.**mole_fraction_to_concentration**(*chemid: int*, *p: float*, *t: float*, *molefrac: numpy.typing.NDArray[numpy.double]*, *wt: numpy.typing.NDArray[numpy.double]*) → numpy.typing.NDArray[numpy.double]

Convert mole fractions to molar concentrations.

Mixture.**list_composition**(*mode: str*, *option: str = ' '*, *bound: float = 0.0*)

List the mixture composition.

**static** Mixture.**density**(*chemid: int*, *p: float*, *t: float*, *frac: numpy.typing.NDArray[numpy.double]*, *wt: numpy.typing.NDArray[numpy.double]*, *mode: str*) → float

Get mass density from the given mixture condition.

**static** Mixture.**mixture_specific_heat**(*chemid: int*, *p: float*, *t: float*, *frac: numpy.typing.NDArray[numpy.double]*, *wt: numpy.typing.NDArray[numpy.double]*, *mode: str*) → float

Get mixture specific heat capacity at constant pressure.

**static** Mixture.**mixture_enthalpy**(*chemid: int*, *p: float*, *t: float*, *frac: numpy.typing.NDArray[numpy.double]*, *wt: numpy.typing.NDArray[numpy.double]*, *mode: str*) → float

Get mixture enthalpy.

**static** Mixture.**calculate_pressure_from_density**(*chemid: int*, *rho: float*, *temp: float*, *frac: numpy.typing.NDArray[numpy.double]*, *wt: numpy.typing.NDArray[numpy.double]*, *mode: str*) → float

Calculate the gas mixture pressure.

**static** Mixture.**mixture_thermicity**(*chemid: int*, *pres: float*, *temp: float*, *rho: float*, *frac: numpy.typing.NDArray[numpy.double]*, *wt: numpy.typing.NDArray[numpy.double]*, *mode: str*) → float

Get mixture total thermicity.

**static** Mixture.**mixture_sound_speed**(*chemid: int*, *pres: float*, *temp: float*, *frac: numpy.typing.NDArray[numpy.double]*, *wt: numpy.typing.NDArray[numpy.double]*, *mode: str*) → float

Get mixture speed of sound.

**static** Mixture.**rate_of_production**(*chemid: int*, *p: float*, *t: float*, *frac: numpy.typing.NDArray[numpy.double]*, *wt: numpy.typing.NDArray[numpy.double]*, *mode: str*) → numpy.typing.NDArray[numpy.double]

Get species molar rate of production.

**static** Mixture.**reaction_rates**(*chemid: int*, *numbreaction: int*, *p: float*, *t: float*, *frac: numpy.typing.NDArray[numpy.double]*, *wt: numpy.typing.NDArray[numpy.double]*, *mode: str*) → tuple[numpy.typing.NDArray[numpy.double], numpy.typing.NDArray[numpy.double]]

Get the molar rates of the gas reactions.

Mixture.**find_equilibrium**()

Create the equilibrium state mixture corresponding to mixture itself.

Mixture.**hml**() → float

Get enthalpy of the mixture.

Mixture.**cpbl**() → float

Get specific heat capacity of the mixture.

Mixture.**set_pressure_by_density**(*rho: float*)

Set the gas mixture pressure value.

Mixture.**thermicity**() → float

Get the total thermicity of the mixture.

Mixture.**sound_speed**() → float

Get the speed of sound of the mixture.

Mixture.**rop**() → numpy.typing.NDArray[numpy.double]

Get species rate of productions.

Mixture.**rxn_rates**() → tuple[numpy.typing.NDArray[numpy.double], numpy.typing.NDArray[numpy.double]]

Get molar reaction rates.

Mixture.**species_cp**() → numpy.typing.NDArray[numpy.double]

Get species specific heat capacity at constant pressure.

Mixture.**species_h**() → numpy.typing.NDArray[numpy.double]

Get species enthalpy.

Mixture.**species_visc**() → numpy.typing.NDArray[numpy.double]

Get species viscosity.

Mixture.**species_cond**() → numpy.typing.NDArray[numpy.double]

Get species conductivity.

Mixture.**species_diffusion_coeffs**() → numpy.typing.NDArray[numpy.double]

Get species diffusion coefficients.

Mixture.**mixture_viscosity**() → float

Get viscosity of the gas mixture.

Mixture.**mixture_conductivity**() → float

Get conductivity of the gas mixture.

Mixture.**mixture_diffusion_coeffs**() → numpy.typing.NDArray[numpy.double]

Get mixture-averaged species diffusion coefficients of the gas mixture.

Mixture.**mixture_binary_diffusion_coeffs**() → numpy.typing.NDArray[numpy.double]

Get multi-component species binary diffusion coefficients.

Mixture.**mixture_thermal_diffusion_coeffs**() → numpy.typing.NDArray[numpy.double]

Get thermal diffusivity of the gas mixture.

Mixture.**vol_hrr**() → float

Get volumetric heat release rate.

Mixture.**rop_mass**() → numpy.typing.NDArray[numpy.double]

Get species mass rates of production.

Mixture.**list_rop**(*threshold: float = 0.0*) → tuple[numpy.typing.NDArray[numpy.int32], numpy.typing.NDArray[numpy.double]]

List species molar production rate.

Mixture.**list_rop_mass**(*threshold: float = 0.0*) → tuple[numpy.typing.NDArray[numpy.int32], numpy.typing.NDArray[numpy.double]]

List species mass rate of production in descending order.

Mixture.**list_reaction_rates**(*threshold: float = 1e-12*) → tuple[numpy.typing.NDArray[numpy.int32], numpy.typing.NDArray[numpy.double]]

List information about reaction rate in descending order.

Mixture.**x_by_equivalence_ratio**(*chemistryset:* ansys.chemkin.core.chemistry.Chemistry, *fuel_molefrac: numpy.typing.NDArray[numpy.double]*, *oxid_molefrac: numpy.typing.NDArray[numpy.double]*, *add_molefrac: numpy.typing.NDArray[numpy.double]*, *products: list[str]*, *equivalenceratio: float*, *threshold: float = 1e-10*) → int

Set mole fractions using equivalence ratio.

Mixture.**y_by_equivalence_ratio**(*chemistryset:* ansys.chemkin.core.chemistry.Chemistry, *fuel_massfrac: numpy.typing.NDArray[numpy.double]*, *oxid_massfrac: numpy.typing.NDArray[numpy.double]*, *add_massfrac: numpy.typing.NDArray[numpy.double]*, *products: list[str]*, *equivalenceratio: float*, *threshold: float = 1e-10*) → int

Set mass fractions using equivalence ratio.

Mixture.**get_egr_mole_fraction**(*egr_ratio: float*, *threshold: float = 1e-08*) → numpy.typing.NDArray[numpy.double]

Compute the EGR composition in mole fraction.

Mixture.**validate**() → int

Check whether the mixture is fully defined.

Mixture.**use_realgas_cubic_eos**()

Turn ON the real-gas cubic EOS.

Mixture.**use_idealgas_law**()

Turn on the ideal gas law to compute mixture properties.

Mixture.**set_realgas_mixing_rule**(*rule: int = 0*)

Set the mixing rule for calculating the real-gas mixture properties.

Mixture.**reset_composition**()

Reset all species fractions to 0.

Mixture.**get_surf_specindex**(*symbol: str*) → tuple[int, int]

Get the index of the given surface/bulk species symbol.

Mixture.**get_all_surface_temperature**() → numpy.typing.NDArray[numpy.double]

Get the temperatures of all materials in the mechanism.

Mixture.**rop_surf**(*mat_name: str*) → tuple[numpy.typing.NDArray[numpy.double], numpy.typing.NDArray[numpy.double]]

Get species molar rate of production.

Mixture.**rxnrates_surf**(*mat_name: str*) → tuple[numpy.typing.NDArray[numpy.double], numpy.typing.NDArray[numpy.double]]

Get molar rates of the surface reactions.

### Description

Chemkin Mixture utilities.

### Module detail

mixture.**verify_mixture**(*this_mixture:* Mixture) → bool

Verify the Mixture is valid and active.

mixture.**isothermal_mixing**(*recipe: list[tuple[*Mixture*, float]]*, *mode: str*, *finaltemperature: float*) → *Mixture*

Mixing multiple gas mixtures at gioven temperature.

mixture.**adiabatic_mixing**(*recipe: list[tuple[*Mixture*, float]]*, *mode: str*) → *Mixture*

Mixing multiple gas mixtures adiabatically.

mixture.**calculate_mixture_temperature_from_enthalpy**(*mixture:* Mixture, *h_mixture: float*, *guesstemperature: float = 0.0*) → int

Compute the mixture temperature from the given mixture enthalpy.

mixture.**cal_mixture_temperature_from_enthalpy**(*mixture:* Mixture, *h_mixture: float*, *guesstemperature: float = 0.0*) → int

Compatibility wrapper for calculate_mixture_temperature_from_enthalpy.

mixture.**create_mixture_recipe_from_fractions**(*chemistry_set:* ansys.chemkin.core.chemistry.Chemistry, *frac: numpy.typing.NDArray[numpy.double]*) → tuple[int, list[tuple[str, float]]]

Build a PyChemkin mixture recipe/formula from a species fraction array.

mixture.**calculate_stoichiometrics**(*chemistryset:* ansys.chemkin.core.chemistry.Chemistry, *fuel_molefrac: numpy.typing.NDArray[numpy.double]*, *oxid_molefrac: numpy.typing.NDArray[numpy.double]*, *prod_index: numpy.typing.NDArray[numpy.int32]*) → tuple[float, numpy.typing.NDArray[numpy.double]]

Calculate the stoichiometric coefficients.

mixture.**species_diffusion_velocity**(*mixture_a:* Mixture, *mixture_b:* Mixture, *mode: str = 'mix'*, *tdiff: bool = True*) → numpy.typing.NDArray[numpy.double]

Compute species diffusive velocities between two gas mixtures.

mixture.**mixing_by_exchange_with_the_mean**(*mixture_a:* Mixture, *mixture_b:* Mixture, *mix_time: float*, *mix_param: float*, *tau: float*) → tuple[*Mixture*, *Mixture*]

Mix two mixtures of the same mass using the IEM model.

mixture.**calculate_mass_weighted_mean_mixture**(*mixtures: list[*Mixture*]*, *masses: list[float]* | *None* = *None*) → *Mixture*

Calculate the mass-weighted mean mixture from a list of mixtures.

mixture.**interpolate_mixtures**(*mixtureleft:* Mixture, *mixtureright:* Mixture, *ratio: float*) → *Mixture*

Get Mixture by interpolation.

mixture.**compare_mixtures**(*mixture_a:* Mixture, *mixture_b:* Mixture, *atol: float = 1e-10*, *rtol: float = 0.001*, *mode: str = 'mass'*) → tuple[bool, float, float]

Compare properties of mixture B against those of mixture A.

mixture.**calculate_equilibrium**(*chemid: int*, *p: float*, *t: float*, *frac: numpy.typing.NDArray[numpy.double]*, *wt: numpy.typing.NDArray[numpy.double]*, *mode_in: str*, *mode_out: str*, *eq_option: int = 1*, *use_realgas: int = 0*) → tuple[list[float], numpy.typing.NDArray[numpy.double]]

Get the equilibrium mixture composition.

mixture.**equilibrium**(*mixture:* Mixture, *opt: int = 1*) → *Mixture*

Find the equilibrium state mixture corresponding to the given mixture.

mixture.**detonation**(*mixture:* Mixture) → tuple[list[float], *Mixture*]

Find the Chapman-Jouguet state mixture and detonation wave speed.

### The `particles.py` module

### Summary

### Classes

| | |
|---|---|
| *Aerosol* | Aerosol material object. |
| *PDF* | Object representing the population density function (PDF) of aerosols. |

#### Aerosol

**class** ansys.chemkin.core.particles.**Aerosol**(*chemset_index: int*, *material_index: int*, *label: str = ''*)

Bases: *ansys.chemkin.core.surface_components.Material*

Aerosol material object.

### Overview

### Attributes

| |
|---|
| *material_type* |
| *number_nucleation_reactions* |
| *nucleation_reaction_indices* |
| *inception_size* |
| *core_bulk_species_index* |
| *core_bulk_species_symbol* |
| *native_open_site_species_index* |
| *native_open_site_species_symbol* |
| *population_distribution_method* |

### Import detail

```
from ansys.chemkin.core.particles import Aerosol
```

### Attribute detail

Aerosol.**material_type = 'aerosol'**

Aerosol.**number_nucleation_reactions: int = 0**

Aerosol.**nucleation_reaction_indices: list[int] = []**

Aerosol.**inception_size: list[float] = []**

Aerosol.**core_bulk_species_index: int = 0**

Aerosol.**core_bulk_species_symbol: str = ''**

Aerosol.**native_open_site_species_index: int = 0**

Aerosol.**native_open_site_species_symbol: str = ''**

Aerosol.**population_distribution_method: str = ''**

#### PDF

**class** ansys.chemkin.core.particles.**PDF**(*core_wt: float*, *bulk_density: float*)

Object representing the population density function (PDF) of aerosols.

### Overview

### Methods

| | |
|---|---|
| *class_to_volume* | Convert size class to particle volume. |
| *volume_to_class* | Convert particle volume to size class. |
| *class_to_diameter* | Convert particle class to particle diameter. |
| *diameter_to_class* | Convert particle diameter to particle class. |
| *set_moments* | Calculate the key properties of the PDF from the moments values. |
| *get_gamma_pdf* | Reconstruct the PDF by assuming the gamma function distribution. |
| *get_lognormal_pdf* | Reconstruct the PDF by assuming the lognormal distribution. |

### Properties

| | |
|---|---|
| *vol0* | Get volume of the core class unit. |
| *mean_class* | Get the mean size class of the distribution. |
| *mean_volume* | Get the volume corresponding to the mean size class. |

### Attributes

| |
|---|
| *core_molar_mass* |
| *bulk_density* |
| *mass_per_class* |
| *vol_per_class* |
| *m0* |
| *mean* |
| *variance* |
| *std_dev* |
| *mean_shifted* |
| *x_min* |

### Import detail

```
from ansys.chemkin.core.particles import PDF
```

### Property detail

**property** PDF.**vol0: float**

Get volume of the core class unit.

**property** PDF.**mean_class: float**

Get the mean size class of the distribution.

**property** PDF.**mean_volume: float**

Get the volume corresponding to the mean size class.

### Attribute detail

PDF.**core_molar_mass**

PDF.**bulk_density**

PDF.**mass_per_class**

PDF.**vol_per_class**

PDF.**m0 = 0.0**

PDF.**mean = 0.0**

PDF.**variance = 0.0**

PDF.**std_dev = 0.0**

PDF.**mean_shifted = 0.0**

PDF.**x_min = 0.0**

### Method detail

PDF.**class_to_volume**(*x: float*) → float

Convert size class to particle volume.

PDF.**volume_to_class**(*v: float*) → float

Convert particle volume to size class.

PDF.**class_to_diameter**(*x: float*) → float

Convert particle class to particle diameter.

PDF.**diameter_to_class**(*diameter: float*) → float

Convert particle diameter to particle class.

PDF.**set_moments**(*m0: float*, *m1: float*, *m2: float*, *x_min: float*) → None

Calculate the key properties of the PDF from the moments values.

PDF.**get_gamma_pdf**(*samples: int = 1000*, *limit: float = 5.0*, *mode: str = 'class'*) → tuple[numpy.typing.NDArray[numpy.double], numpy.typing.NDArray[numpy.double]]

Reconstruct the PDF by assuming the gamma function distribution.

PDF.**get_lognormal_pdf**(*samples: int = 1000*, *limit: float = 5.0*, *mode: str = 'class'*) → tuple[numpy.typing.NDArray[numpy.double], numpy.typing.NDArray[numpy.double]]

Reconstruct the PDF by assuming the lognormal distribution.

### Description

Aerosol size distribution models.

### The `profile.py` module

### Summary

### Classes

| | |
|---|---|
| *Profile* | Chemkin profile keyword class. |

#### Profile

**class** ansys.chemkin.core.profile.**Profile**(*key: str*, *x: numpy.typing.NDArray[numpy.double]*, *y: numpy.typing.NDArray[numpy.double]*, *label: bool = False*)

Chemkin profile keyword class.

### Overview

### Methods

| | |
|---|---|
| *show* | Show the profile data. |
| *resetprofile* | Reset the profile data. |
| *getprofile_as_string_list* | Create the keyword input lines as a list for Chemkin applications. |

### Properties

| | |
|---|---|
| *size* | Get number of data points in the profile. |
| *status* | Get the validity of the profile object. |
| *pos* | Get position values of profile data. |
| *value* | Get variable values of profile data. |
| *profilekey* | Get profile keyword. |

### Import detail

```
from ansys.chemkin.core.profile import Profile
```

### Property detail

**property** Profile.**size: int**

Get number of data points in the profile.

**property** Profile.**status: int**

Get the validity of the profile object.

**property** Profile.**pos: numpy.typing.NDArray[numpy.double]**

Get position values of profile data.

**property** Profile.**value: numpy.typing.NDArray[numpy.double]**

Get variable values of profile data.

**property** Profile.**profilekey: str**

Get profile keyword.

### Method detail

Profile.**show**()

Show the profile data.

Profile.**resetprofile**(*size: int*, *x: numpy.typing.NDArray[numpy.double]*, *y: numpy.typing.NDArray[numpy.double]*)

Reset the profile data.

Profile.**getprofile_as_string_list**() → tuple[int, list[str]]

Create the keyword input lines as a list for Chemkin applications.

### Description

Chemkin profile keyword data model.

## The `reactormodel.py` module

### Summary

### Classes

| | |
|---|---|
| *ReactorModel* | Serve as a generic Chemkin reactor model framework. |

#### ReactorModel

**class** ansys.chemkin.core.reactormodel.**ReactorModel**(*reactor_condition:* ansys.chemkin.core.inlet.Stream, *label: str*)

Serve as a generic Chemkin reactor model framework.

### Overview

### Methods

| | |
|---|---|
| *usefullkeywords* | Specify all necessary keywords explicitly. |
| *setkeyword* | Set a Chemkin keyword and its parameter. |
| *removekeyword* | Remove an existing Chemkin keyword and its parameter. |
| *showkeywordinputlines* | List all currently-defined keywords and their parameters line by line. |
| *createkeywordinputlines* | Create keyword input lines for Chemkin applications. |
| *showkeywordinputlines_with_tag* | List all currently-defined keywords. |
| *createkeywordinputlines_with_tag* | Create keyword input lines for Chemkin applications. |
| *setprofile* | Set a Chemkin profile and its parameter. |
| *get_profile_value* | Get the y value at given x from the profile data. |
| *createprofileinputlines* | Create profile keyword input lines for Chemkin applications. |
| *createspeciesinputlines* | Create keyword input lines. |
| *createspeciesinputlineswithaddon* | Create keyword input lines. |
| *create_inletspeciesinputlines* | Create keyword input lines. |
| *clear_all_keywords* | Delete all existing keyword components. |
| *chemid* | Get chemistry set index. |
| *set_molefractions* | Set the reactor species mole fractions. |
| *set_massfractions* | Set the reactor species mass fractions. |
| *list_composition* | List the gas mixture composition inside the reactor. |
| *setsensitivityanalysis* | Switch ON/OFF A-factor sensitivity analysis. |
| *set_rop_analysis* | Switch ON/OFF the ROP (Rate Of Production) analysis. |
| *userealgas_eos* | Set the option to turn ON/OFF the real gas model. |
| *setrealgasmixingmodel* | Set the real gas mixing rule/model. |
| *use_rgp_table* | Set the option to turn ON/OFF the use of the RGP table. |
| *disable_rgp_table* | Turn OFF the use of the RGP table. |
| *setrunstatus* | Set the simulation run status. |
| *getrunstatus* | Get the reactor model simulation status. |
| *run* | Serve as a generic Chemkin run reactor model method. |
| *setsolutionspeciesfracmode* | Set the type of species fractions in the solution. |
| *getrawsolutionstatus* | Get the status of the post-process. |
| *getmixturesolutionstatus* | Get the status of the post-process. |
| *get_solution_size* | Get the number of reactors and the number of solution points. |
| *getnumbersolutionpoints* | Get the number of solution points per reactor. |
| *parsespeciessolutiondata* | Parse the 2-D species fraction solution data. |
| *process_solution* | Post-process solution to extract the raw solution variable data. |
| *activate_surface_chemistry* | Activate the surface chemistry. |
| *verify_surface_chemistry* | Verify whether the reactor model is surfac chemistry capable. |
| *no_surface_mechanism_declaration* | Inform users surface chemistry is not available for this reactor. |
| *no_surface_material* | Send surface material not found message. |
| *get_material_names* | Get all surface material names. |
| *get_site_species_names* | Get site species names on the surface materials. |
| *get_bulk_species_names* | Get bulk species names on the surface materials. |
| *get_total_site_species* | Get total number of site species from all materials. |
| *get_total_bulk_species* | Get total number of bulk species from all materials. |
| *check_material_area_fraction* | Verify the surface area fraction is given. |
| *get_material_area_fraction* | Get the surface area fraction. |
| *set_material_area_fraction* | Set the value of the surface area fraction. |
| *check_material_temperature* | Verify the surface temperature is given. |
| *get_material_temperature* | Get the surface temperature. |
| *set_material_temperature* | Set the value of the surface temperature. |

continues on next page

Table 3 – continued from previous page

| | |
|---|---|
| *get_all_surface_temperature* | Get the temperatures of all materials in the mechanism. |
| *get_site_fraction* | Get the surface site fractions. |
| *set_site_fraction* | Set the surface site fractions. |
| *get_all_site_fractions* | Get site fractions of all materials in the mechanism. |
| *get_bulk_activity* | Get the bulk species activities. |
| *set_bulk_activity* | Set the bulk species activities. |
| *get_all_bulk_growth_rates* | Get bulk growth rates of all materials in the mechanism. |
| *set_init_surface_coverage* | Set up the initial surface coverage arrays. |
| *set_surface_chemistry_keywords* | Set up surface chemistry related keywords. |

### Properties

| | |
|---|---|
| *temperature* | Get reactor initial temperature. |
| *pressure* | Get reactor pressure. |
| *massfraction* | Get the gas species mass fractions. |
| *molefraction* | Get the gas species mole fractions. |
| *concentration* | Get the gas species molar concentrations. |
| *gasratemultiplier* | Get the value of the gas-phase reaction rate multiplier. |
| *std_output* | Get text output status. |
| *xml_output* | Get XML solution output status. |
| *realgas* | Get the real gas EOS status. |
| *get_numb_material* | Get the number of surface material. |
| *surface_ratemultiplier* | Get the value of the surface reaction rate multiplier. |

### Attributes

| |
|---|
| *label* |
| *runstatus* |
| *has_surface_chemistry* |
| *numbmaterials* |
| *surface_chemistry* |

### Import detail

```
from ansys.chemkin.core.reactormodel import ReactorModel
```

### Property detail

**property** ReactorModel.**temperature: float**

Get reactor initial temperature.

**property** ReactorModel.**pressure: float**

Get reactor pressure.

**property** ReactorModel.**massfraction: float**

Get the gas species mass fractions.

**property** ReactorModel.**molefraction: numpy.typing.NDArray[numpy.double]**

Get the gas species mole fractions.

**property** ReactorModel.**concentration: numpy.typing.NDArray[numpy.double]**

Get the gas species molar concentrations.

**property** ReactorModel.**gasratemultiplier: float**

Get the value of the gas-phase reaction rate multiplier.

**property** ReactorModel.**std_output: bool**

Get text output status.

**property** ReactorModel.**xml_output: bool**

Get XML solution output status.

**property** ReactorModel.**realgas: bool**

Get the real gas EOS status.

**property** ReactorModel.**get_numb_material: int**

Get the number of surface material.

**property** ReactorModel.**surface_ratemultiplier: float**

Get the value of the surface reaction rate multiplier.

### Attribute detail

ReactorModel.**label**

ReactorModel.**runstatus = -100**

ReactorModel.**has_surface_chemistry**

ReactorModel.**numbmaterials = 0**

ReactorModel.**surface_chemistry: Any = None**

### Method detail

ReactorModel.**usefullkeywords**(*mode: bool*)

Specify all necessary keywords explicitly.

ReactorModel.**setkeyword**(*key: str*, *value: bool | float | str*)

Set a Chemkin keyword and its parameter.

ReactorModel.**removekeyword**(*key: str*)

Remove an existing Chemkin keyword and its parameter.

ReactorModel.**showkeywordinputlines**()

List all currently-defined keywords and their parameters line by line.

ReactorModel.**createkeywordinputlines**() → tuple[int, int]

Create keyword input lines for Chemkin applications.

ReactorModel.**showkeywordinputlines_with_tag**(*tag: str = ''*)

List all currently-defined keywords.

ReactorModel.**createkeywordinputlines_with_tag**(*tag: str = ''*) → tuple[int, int]

Create keyword input lines for Chemkin applications.

ReactorModel.**setprofile**(*key: str*, *x: numpy.typing.NDArray[numpy.double]*, *y: numpy.typing.NDArray[numpy.double]*, *label: bool = False*) → int

Set a Chemkin profile and its parameter.

ReactorModel.**get_profile_value**(*key: str*, *x: float*) → float

Get the y value at given x from the profile data.

ReactorModel.**createprofileinputlines**() → tuple[int, int, list[str]]

Create profile keyword input lines for Chemkin applications.

ReactorModel.**createspeciesinputlines**(*solvertype: int*, *threshold: float = 1e-12*, *molefrac: None | numpy.typing.NDArray[numpy.double] = None*) → tuple[int, list[str]]

Create keyword input lines.

ReactorModel.**createspeciesinputlineswithaddon**(*key: str = 'XEST'*, *threshold: float = 1e-12*, *molefrac: None | numpy.typing.NDArray[numpy.double] = None*, *addon: str = ''*) → tuple[int, list[str]]

Create keyword input lines.

ReactorModel.**create_inletspeciesinputlines**(*inlet_name: str*, *threshold: float = 1e-12*, *molefrac: None | numpy.typing.NDArray[numpy.double] = None*) → tuple[int, list[str]]

Create keyword input lines.

ReactorModel.**clear_all_keywords**()

Delete all existing keyword components.

ReactorModel.**chemid**() → int

Get chemistry set index.

ReactorModel.**set_molefractions**(*molefrac: numpy.typing.NDArray[numpy.double]*)

Set the reactor species mole fractions.

ReactorModel.**set_massfractions**(*massfrac: numpy.typing.NDArray[numpy.double]*)

Set the reactor species mass fractions.

ReactorModel.**list_composition**(*mode: str*, *option: str = ' '*, *bound: float = 0.0*)

List the gas mixture composition inside the reactor.

ReactorModel.**setsensitivityanalysis**(*mode: bool = True*, *absolute_tolerance: float | None = None*, *relative_tolerance: float | None = None*, *temperature_threshold: float | None = None*, *species_threshold: float | None = None*)

Switch ON/OFF A-factor sensitivity analysis.

ReactorModel.**set_rop_analysis**(*mode=True*, *threshold=None*)

Switch ON/OFF the ROP (Rate Of Production) analysis.

ReactorModel.**userealgas_eos**(*mode: bool*)

Set the option to turn ON/OFF the real gas model.

ReactorModel.**setrealgasmixingmodel**(*model: int*)

Set the real gas mixing rule/model.

ReactorModel.**use_rgp_table**(*data_file: str = ''*)

Set the option to turn ON/OFF the use of the RGP table.

ReactorModel.**disable_rgp_table**()

Turn OFF the use of the RGP table.

ReactorModel.**setrunstatus**(*code: int*)

Set the simulation run status.

ReactorModel.**getrunstatus**(*mode: str = 'silent'*) → int

Get the reactor model simulation status.

ReactorModel.**run**() → int

Serve as a generic Chemkin run reactor model method.

ReactorModel.**setsolutionspeciesfracmode**(*mode: str = 'mass'*)

Set the type of species fractions in the solution.

ReactorModel.**getrawsolutionstatus**() → bool

Get the status of the post-process.

ReactorModel.**getmixturesolutionstatus**() → bool

Get the status of the post-process.

ReactorModel.**get_solution_size**() → tuple[int, int]

Get the number of reactors and the number of solution points.

ReactorModel.**getnumbersolutionpoints**() → int

Get the number of solution points per reactor.

ReactorModel.**parsespeciessolutiondata**(*frac: numpy.typing.NDArray[numpy.double]*)

Parse the 2-D species fraction solution data.

ReactorModel.**process_solution**()

Post-process solution to extract the raw solution variable data.

ReactorModel.**activate_surface_chemistry**(*chem:* ansys.chemkin.core.chemistry.Chemistry, *mode: str = 'normal'*)

Activate the surface chemistry.

ReactorModel.**verify_surface_chemistry**() → bool

Verify whether the reactor model is surfac chemistry capable.

ReactorModel.**no_surface_mechanism_declaration**()

Inform users surface chemistry is not available for this reactor.

ReactorModel.**no_surface_material**(*mat_name: str*)

Send surface material not found message.

ReactorModel.**get_material_names**() → list[str]

Get all surface material names.

ReactorModel.**get_site_species_names**() → list[list[str]]

Get site species names on the surface materials.

ReactorModel.**get_bulk_species_names**() → list[list[str]]

Get bulk species names on the surface materials.

ReactorModel.**get_total_site_species**() → int

Get total number of site species from all materials.

ReactorModel.**get_total_bulk_species**() → int

Get total number of bulk species from all materials.

ReactorModel.**check_material_area_fraction**(*mat_name: str*) → int

Verify the surface area fraction is given.

ReactorModel.**get_material_area_fraction**(*mat_name: str*) → float

Get the surface area fraction.

ReactorModel.**set_material_area_fraction**(*mat_name: str*, *fraction: float*)

Set the value of the surface area fraction.

ReactorModel.**check_material_temperature**(*mat_name: str*) → int

Verify the surface temperature is given.

ReactorModel.**get_material_temperature**(*mat_name: str*) → float

Get the surface temperature.

ReactorModel.**set_material_temperature**(*mat_name: str*, *temp: float*)

Set the value of the surface temperature.

ReactorModel.**get_all_surface_temperature**() → numpy.typing.NDArray[numpy.double]

Get the temperatures of all materials in the mechanism.

ReactorModel.**get_site_fraction**(*mat_name: str*) → numpy.typing.NDArray[numpy.double]

Get the surface site fractions.

ReactorModel.**set_site_fraction**(*mat_name: str*, *recipe: list[tuple[str, float]]*)

Set the surface site fractions.

ReactorModel.**get_all_site_fractions**() → numpy.typing.NDArray[numpy.double]

Get site fractions of all materials in the mechanism.

ReactorModel.**get_bulk_activity**(*mat_name: str*) → numpy.typing.NDArray[numpy.double]

Get the bulk species activities.

ReactorModel.**set_bulk_activity**(*mat_name: str*, *recipe: list[tuple[str, float]]*)

Set the bulk species activities.

ReactorModel.**get_all_bulk_growth_rates**() → numpy.typing.NDArray[numpy.double]

Get bulk growth rates of all materials in the mechanism.

ReactorModel.**set_init_surface_coverage**() → tuple[numpy.typing.NDArray[numpy.double], numpy.typing.NDArray[numpy.double]]

Set up the initial surface coverage arrays.

ReactorModel.**set_surface_chemistry_keywords**()

Set up surface chemistry related keywords.

### Description

Chemkin general reactor model utilities.

### The realgaseos.py module

### Summary

### Functions

| | |
|---|---|
| *check_realgas_status* | Check whether the real-gas cubic EOS is active. |
| *set_current_pressure* | Set gas mixture pressure for real-gas EOS calculations. |

### Description

Real-gas cubic EOS model.

### Module detail

realgaseos.**check_realgas_status**(*chem_index: int*) → bool

Check whether the real-gas cubic EOS is active.

realgaseos.**set_current_pressure**(*chem_index: int*, *pressure: float*) → int

Set gas mixture pressure for real-gas EOS calculations.

### The steadystatesolver.py module

### Summary

### Classes

| | |
|---|---|
| *SteadyStateSolver* | Common steady-state solver controlling parameters. |

#### SteadyStateSolver

**class** ansys.chemkin.core.steadystatesolver.**SteadyStateSolver**

Common steady-state solver controlling parameters.

### Overview

### Methods

| | |
|---|---|
| *set_max_pseudo_transient_call* | Set max number of the pseudo transient operation. |
| *set_max_timestep_iteration* | Set max number of iterations per time step. |
| *set_max_search_iteration* | Set the maximum number of iterations. |
| *set_initial_timesteps* | Set the number of pseudo time steps to be performed. |
| *set_species_floor* | Set the minimum species fraction value allowed. |
| *set_temperature_ceiling* | Set the maximum temperature value allowed. |
| *set_species_reset_value* | Set the positive reset value for any negative species fraction. |
| *set_max_pseudo_timestep_size* | Set max time step sizes allowed by the pseudo time stepping. |
| *set_min_pseudo_timestep_size* | Set min time step size of the pseudo time stepping operation. |
| *set_pseudo_timestep_age* | Set min number of time steps before time step size increase. |
| *set_jacobian_age* | Set the number of searches before Jacobian matrix evaluation. |
| *set_pseudo_jacobian_age* | Set the number of time steps before Jacobian matrix evaluation. |
| *set_damping_option* | Turn ON or OFF the damping option of the steady-state solver. |
| *set_legacy_option* | Turn ON or OFF the legacy steady-state solver. |
| *set_print_level* | Set the text output level of the steady-state solver. |
| *set_pseudo_timestepping_parameters* | Set pseudo time stepping parameters. |

### Properties

| | |
|---|---|
| *steady_state_tolerances* | Get tolerance for the steady-state search algorithm. |
| *time_stepping_tolerances* | Get tolerance for the pseudo time stepping solution algorithm. |

### Attributes

| |
|---|
| *ss_absolute_tolerance* |
| *ss_relative_tolerance* |
| *ss_maxiteration* |
| *ss_jacobianage* |
| *maxpseudotransient* |
| *numbinitialpseudosteps* |
| *maxTbound* |
| *speciesfloor* |
| *species_positive* |
| *use_legacy_technique* |
| *ss_damping* |
| *absolute_perturbation* |
| *relative_perturbation* |
| *tr_absolute_tolerance* |
| *tr_relative_tolerance* |
| *tr_maxiteration* |
| *timestepsizeage* |
| *tr_minstepsize* |
| *tr_maxstepsize* |
| *tr_upfactor* |
| *tr_downfactor* |
| *tr_jacobianage* |
| *tr_stride_fixT* |
| *tr_numbsteps_fixT* |
| *tr_stride_ENRG* |
| *tr_numbsteps_ENRG* |
| *print_level* |
| *ss_solverkeywords* |

### Import detail

```
from ansys.chemkin.core.steadystatesolver import SteadyStateSolver
```

### Property detail

**property** SteadyStateSolver.**steady_state_tolerances: tuple[float, float]**

Get tolerance for the steady-state search algorithm.

**property** SteadyStateSolver.**time_stepping_tolerances: tuple[float, float]**

Get tolerance for the pseudo time stepping solution algorithm.

### Attribute detail

SteadyStateSolver.**ss_absolute_tolerance = 1e-09**

SteadyStateSolver.**ss_relative_tolerance = 0.0001**

SteadyStateSolver.**ss_maxiteration = 100**

SteadyStateSolver.**ss_jacobianage = 20**

SteadyStateSolver.**maxpseudotransient = 100**

SteadyStateSolver.**numbinitialpseudosteps = 0**

SteadyStateSolver.**maxTbound = 5000.0**

SteadyStateSolver.**speciesfloor = -1e-14**

SteadyStateSolver.**species_positive = 0.0**

SteadyStateSolver.**use_legacy_technique = False**

SteadyStateSolver.**ss_damping = 1**

SteadyStateSolver.**absolute_perturbation = 0.0**

SteadyStateSolver.**relative_perturbation = 0.0**

SteadyStateSolver.**tr_absolute_tolerance = 1e-09**

SteadyStateSolver.**tr_relative_tolerance = 0.0001**

SteadyStateSolver.**tr_maxiteration = 25**

SteadyStateSolver.**timestepsizeage = 25**

SteadyStateSolver.**tr_minstepsize = 1e-10**

SteadyStateSolver.**tr_maxstepsize = 0.01**

SteadyStateSolver.**tr_upfactor = 2.0**

SteadyStateSolver.**tr_downfactor = 2.2**

SteadyStateSolver.**tr_jacobianage = 20**

SteadyStateSolver.**tr_stride_fixT = 1e-06**

SteadyStateSolver.**tr_numbsteps_fixT = 100**

SteadyStateSolver.**tr_stride_ENRG = 1e-06**

SteadyStateSolver.**tr_numbsteps_ENRG = 100**

SteadyStateSolver.**print_level = 1**

SteadyStateSolver.**ss_solverkeywords: dict[str, int | float | str | bool]**

### Method detail

SteadyStateSolver.**set_max_pseudo_transient_call**(*maxtime: int*)

Set max number of the pseudo transient operation.

SteadyStateSolver.**set_max_timestep_iteration**(*maxiteration: int*)

Set max number of iterations per time step.

SteadyStateSolver.**set_max_search_iteration**(*maxiteration: int*)

Set the maximum number of iterations.

SteadyStateSolver.**set_initial_timesteps**(*initsteps: int*)

Set the number of pseudo time steps to be performed.

SteadyStateSolver.**set_species_floor**(*floor_value: float*)

Set the minimum species fraction value allowed.

SteadyStateSolver.**set_temperature_ceiling**(*ceilingvalue: float*)

Set the maximum temperature value allowed.

SteadyStateSolver.**set_species_reset_value**(*resetvalue: float*)

Set the positive reset value for any negative species fraction.

SteadyStateSolver.**set_max_pseudo_timestep_size**(*dtmax: float*)

Set max time step sizes allowed by the pseudo time stepping.

SteadyStateSolver.**set_min_pseudo_timestep_size**(*dtmin: float*)

Set min time step size of the pseudo time stepping operation.

SteadyStateSolver.**set_pseudo_timestep_age**(*age: int*)

Set min number of time steps before time step size increase.

SteadyStateSolver.**set_jacobian_age**(*age: int*)

Set the number of searches before Jacobian matrix evaluation.

SteadyStateSolver.**set_pseudo_jacobian_age**(*age: int*)

Set the number of time steps before Jacobian matrix evaluation.

SteadyStateSolver.**set_damping_option**(*status: bool*)

Turn ON or OFF the damping option of the steady-state solver.

SteadyStateSolver.**set_legacy_option**(*option: bool*)

Turn ON or OFF the legacy steady-state solver.

SteadyStateSolver.**set_print_level**(*level: int*)

Set the text output level of the steady-state solver.

SteadyStateSolver.**set_pseudo_timestepping_parameters**(*numb_steps: int = 100*, *step_size: float = 1e-06*, *stage: int = 1*)

Set pseudo time stepping parameters.

### Description

Chemkin steady-state solver controlling parameters.

### The surface.py module

### Summary

### Classes

| | |
|---|---|
| *Surface* | Chemkin surface chemistry module. |

#### Surface

**class** ansys.chemkin.core.surface.**Surface**(*chem:* ansys.chemkin.core.chemistry.Chemistry)

Chemkin surface chemistry module.

### Overview

### Methods

| | |
|---|---|
| *check_surface_material* | Verify the the given material name. |
| *require_surface_material* | Return the material index or terminate if not found. |
| *material_not_found* | Return the the material name not found error message. |
| *get_material_names* | Get all surface material symbols. |
| *get_site_frac* | Get surface site species fractions. |
| *set_site_frac* | Set the surface site fractions. |
| *check_site_frac_set* | Check if the surface site species fractions are provided. |
| *get_all_site_frac* | Get site fractions of all materials in the mechanism. |
| *get_bulk_frac* | Get surface bulk species activities/moles. |
| *set_bulk_frac* | Assign the bulk species activities. |
| *check_bulk_act_set* | Check if the bulk species activities are provided. |
| *get_bulk_growth_rates* | Get bulk species linear growth rates. |
| *set_bulk_growth_rates* | Assign bulk species growth rate. |
| *check_bulk_growth_rate_set* | Check if the bulk species growth rates are provided. |
| *get_all_bulk_growth_rates* | Get bulk growth rates of all materials in the mechanism. |
| *get_activity_array* | Get the species activity array. |
| *list_surface_coverage* | List the surface site fractions. |
| *list_bulk_activity* | List the bulk species activity. |
| *get_surface_material* | Get the Material object. |
| *number_surface_reactions* | Get the number of surface reactions. |
| *number_surface_species* | Get the total number of surface species. |
| *total_number_species* | Get the total number of species. |
| *number_phase* | Get the number of phases. |
| *first_site_phase_index* | Get the index of the first site phase. |
| *last_site_phase_index* | Get the index of the last site phase. |
| *first_bulk_phase_index* | Get the index of the first bulk phase. |
| *last_bulk_phase_index* | Get the index of the last bulk phase. |
| *get_phase_names* | Get all phase names. |
| *number_site_species* | Get the total number of site species. |
| *number_bulk_species* | Get the total number of bulk species. |
| *first_site_spec_index* | Get the global index of the first site species. |
| *last_site_spec_index* | Get the global index of the last site species. |
| *first_bulk_spec_index* | Get the global index of the first bulk species. |
| *last_bulk_spec_index* | Get the global index of the last bulk species. |
| *get_site_symbols* | Get all site species symbols. |
| *get_bulk_symbols* | Get all bulk species symbols. |
| *surface_species_symbols* | Get all surface species symbols of the material. |
| *get_surf_specindex* | Get the local and the global species indices. |
| *site_phase_density* | Get the site density of the site phases. |
| *get_site_species_occupany* | Get the occupancy. |
| *get_site_wt* | Get the molar masses of the site species. |
| *get_bulk_wt* | Get the molar masses of the bulk species. |
| *get_surf_wt* | Get the molar masses of all surface species. |
| *get_site_species_h* | Get the site species enthalpy. |
| *get_bulk_species_h* | Get the bulk species enthalpy. |
| *get_surface_species_h* | Get the enthalpy of all surface species. |
| *get_bulk_species_cp* | Get the bulk species specific heat capacity. |
| *get_bulk_species_density* | Get the bulk species density. |
| *check_material_temperature* | Verify the surface temperature is given. |

continues on next page

Table 4 – continued from previous page

| | |
|---|---|
| *get_material_temperature* | Get the surface temperature. |
| *set_material_temperature* | Set the value of the surface temperature. |
| *set_surface_coverage* | Get the surface coverage. |
| *rop_surf* | Get species molar rate of production. |
| *rxnrates_surf* | Get molar rates of the surface reactions. |
| *get_bulk_molar_growth_rates* | Get the molar growth rates of the bulk species. |
| *get_bulk_mass_growth_rates* | Get the mass growth rates of the bulk species. |
| *get_bulk_linear_growth_rates* | Get the linear growth rates of the bulk species. |
| *get_gas_production_rates* | Get the molar production rates of the gas species. |
| *stefan_mass_flux* | Get the Stefan mass flux due to surface reactions. |

### Properties

| | |
|---|---|
| *number_materials* | Get the number of surface materials. |
| *max_number_sites* | Get the max number of site species. |
| *max_number_bulks* | Get the max number of bulk species. |
| *max_number_total_species* | Get the max total number of species. |
| *max_number_surface_reactions* | Get the max number of surface reactions. |
| *max_number_total_reactions* | Get the max number of total reactions. |
| *total_site_species* | Get total number of site species from all materials. |
| *total_bulk_species* | Get total number of bulk species from all materials. |

### Attributes

| |
|---|
| *num_gas_species* |
| *num_gas_reactions* |
| *gas_wt* |
| *num_material* |
| *max_total_species* |
| *max_total_reactions* |
| *material_names* |
| *material_map* |
| *materials* |
| *sitefrac_set* |
| *bulkact_set* |
| *bulk_growth_rates* |
| *bulk_growth_rates_set* |
| *site_density* |
| *siteden_set* |
| *material_temperature* |

### Static methods

| | |
|---|---|
| *set_activity_array* | Set up the activity array. |
| *surface_rate_of_production* | Get species molar rate of production. |
| *surface_reaction_rates* | Get molar reaction rates of the reactive surface. |

### Import detail

```
from ansys.chemkin.core.surface import Surface
```

### Property detail

**property** Surface.**number_materials: int**

Get the number of surface materials.

**property** Surface.**max_number_sites: int**

Get the max number of site species.

**property** Surface.**max_number_bulks: int**

Get the max number of bulk species.

**property** Surface.**max_number_total_species: int**

Get the max total number of species.

**property** Surface.**max_number_surface_reactions: int**

Get the max number of surface reactions.

**property** Surface.**max_number_total_reactions: int**

Get the max number of total reactions.

**property** Surface.**total_site_species: int**

Get total number of site species from all materials.

**property** Surface.**total_bulk_species: int**

Get total number of bulk species from all materials.

### Attribute detail

Surface.**num_gas_species**

Surface.**num_gas_reactions**

Surface.**gas_wt**

Surface.**num_material**

Surface.**max_total_species**

Surface.**max_total_reactions**

Surface.**material_names**

Surface.**material_map**

Surface.**materials**

Surface.**sitefrac_set: Dict[str, bool]**

Surface.**bulkact_set: Dict[str, bool]**

Surface.**bulk_growth_rates: Dict[str, numpy.typing.NDArray[numpy.double]]**

Surface.**bulk_growth_rates_set: Dict[str, bool]**

Surface.**site_density: Dict[str, numpy.typing.NDArray[numpy.double]]**

Surface.**siteden_set: Dict[str, bool]**

Surface.**material_temperature: list[float] = []**

### Method detail

Surface.**check_surface_material**(*matname: str*) → int
Verify the the given material name.

Surface.**require_surface_material**(*matname: str*) → int
Return the material index or terminate if not found.

Surface.**material_not_found**(*matname: str*)
Return the the material name not found error message.

Surface.**get_material_names**() → List[str]
Get all surface material symbols.

Surface.**get_site_frac**(*mat_name: str*) → numpy.typing.NDArray[numpy.double]
Get surface site species fractions.

Surface.**set_site_frac**(*mat_name: str*, *recipe: List[tuple[str, float]] | numpy.typing.NDArray[numpy.double]*)
Set the surface site fractions.

Surface.**check_site_frac_set**(*mat_name: str*) → bool
Check if the surface site species fractions are provided.

Surface.**get_all_site_frac**() → numpy.typing.NDArray[numpy.double]
Get site fractions of all materials in the mechanism.

Surface.**get_bulk_frac**(*mat_name: str*) → numpy.typing.NDArray[numpy.double]
Get surface bulk species activities/moles.

Surface.**set_bulk_frac**(*mat_name: str*, *recipe: List[tuple[str, float]] | numpy.typing.NDArray[numpy.double]*)
Assign the bulk species activities.

Surface.**check_bulk_act_set**(*mat_name: str*) → bool
Check if the bulk species activities are provided.

Surface.**get_bulk_growth_rates**(*mat_name: str*) → numpy.typing.NDArray[numpy.double]
Get bulk species linear growth rates.

Surface.**set_bulk_growth_rates**(*mat_name: str*, *rates: numpy.typing.NDArray[numpy.double]*)
Assign bulk species growth rate.

Surface.**check_bulk_growth_rate_set**(*mat_name: str*) → bool
Check if the bulk species growth rates are provided.

Surface.**get_all_bulk_growth_rates**() → numpy.typing.NDArray[numpy.double]
Get bulk growth rates of all materials in the mechanism.

Surface.**get_activity_array**(*mat_name: str*, *molfrac: numpy.typing.NDArray[numpy.double]*) → numpy.typing.NDArray[numpy.double]
Get the species activity array.

Surface.**list_surface_coverage**(*mat_name: str*, *option: str = ' '*, *bound: float = 0.0*)
List the surface site fractions.

Surface.**list_bulk_activity**(*mat_name: str*, *option: str = ' '*, *bound: float = 0.0*)
List the bulk species activity.

Surface.**get_surface_material**(*mat_name: str*) → *ansys.chemkin.core.surface_components.Material*
Get the Material object.

Surface.**number_surface_reactions**(*mat_name: str*) → int
Get the number of surface reactions.

Surface.**number_surface_species**(*mat_name: str*) → int
Get the total number of surface species.

Surface.**total_number_species**(*mat_name: str*) → int
Get the total number of species.

Surface.**number_phase**(*mat_name: str*) → int
Get the number of phases.

Surface.**first_site_phase_index**(*mat_name: str*) → int
Get the index of the first site phase.

Surface.**last_site_phase_index**(*mat_name: str*) → int
Get the index of the last site phase.

Surface.**first_bulk_phase_index**(*mat_name: str*) → int
Get the index of the first bulk phase.

Surface.**last_bulk_phase_index**(*mat_name: str*) → int
Get the index of the last bulk phase.

Surface.**get_phase_names**(*mat_name: str*) → List[str]
Get all phase names.

Surface.**number_site_species**(*mat_name: str*) → int
Get the total number of site species.

Surface.**number_bulk_species**(*mat_name: str*) → int
Get the total number of bulk species.

Surface.**first_site_spec_index**(*mat_name: str*) → int
Get the global index of the first site species.

Surface.**last_site_spec_index**(*mat_name: str*) → int
Get the global index of the last site species.

Surface.**first_bulk_spec_index**(*mat_name: str*) → int
Get the global index of the first bulk species.

Surface.**last_bulk_spec_index**(*mat_name: str*) → int
Get the global index of the last bulk species.

Surface.**get_site_symbols**(*mat_name: str*) → List[str]
Get all site species symbols.

Surface.**get_bulk_symbols**(*mat_name: str*) → List[str]

Get all bulk species symbols.

Surface.**surface_species_symbols**(*mat_name: str*) → List[str]

Get all surface species symbols of the material.

Surface.**get_surf_specindex**(*specname: str*) → tuple[str, int, int]

Get the local and the global species indices.

Surface.**site_phase_density**(*mat_name: str*) → numpy.typing.NDArray[numpy.double]

Get the site density of the site phases.

Surface.**get_site_species_occupany**(*mat_name: str*) → numpy.typing.NDArray[numpy.double]

Get the occupancy.

Surface.**get_site_wt**(*mat_name: str*) → numpy.typing.NDArray[numpy.double]

Get the molar masses of the site species.

Surface.**get_bulk_wt**(*mat_name: str*) → numpy.typing.NDArray[numpy.double]

Get the molar masses of the bulk species.

Surface.**get_surf_wt**(*mat_name: str*) → numpy.typing.NDArray[numpy.double]

Get the molar masses of all surface species.

Surface.**get_site_species_h**(*mat_name: str*) → numpy.typing.NDArray[numpy.double]

Get the site species enthalpy.

Surface.**get_bulk_species_h**(*mat_name: str*) → numpy.typing.NDArray[numpy.double]

Get the bulk species enthalpy.

Surface.**get_surface_species_h**(*mat_name: str*) → numpy.typing.NDArray[numpy.double]

Get the enthalpy of all surface species.

Surface.**get_bulk_species_cp**(*mat_name: str*) → numpy.typing.NDArray[numpy.double]

Get the bulk species specific heat capacity.

Surface.**get_bulk_species_density**(*mat_name: str*) → numpy.typing.NDArray[numpy.double]

Get the bulk species density.

**static** Surface.**set_activity_array**(*ngas: int*, *nsites: int*, *nbulks: int*, *molfrac: numpy.typing.NDArray[numpy.double]*, *site_frac: numpy.typing.NDArray[numpy.double]*, *bulk_act: numpy.typing.NDArray[numpy.double]*, *min_value: float = 0.0*) → numpy.typing.NDArray[numpy.double]

Set up the activity array.

Surface.**check_material_temperature**(*mat_name: str*) → int

Verify the surface temperature is given.

Surface.**get_material_temperature**(*mat_name: str*) → float

Get the surface temperature.

Surface.**set_material_temperature**(*mat_name: str*, *temp: float*)

Set the value of the surface temperature.

**static** Surface.**surface_rate_of_production**(*chem_id: int*, *mat_id: int*, *numb_gas: int*, *numb_phase: int*, *numb_sites: int*, *numb_bulks: int*, *p: float*, *t: float*, *molfrac: numpy.typing.NDArray[numpy.double]*, *site_frac: numpy.typing.NDArray[numpy.double]*, *bulk_act: numpy.typing.NDArray[numpy.double]*, *site_den: numpy.typing.NDArray[numpy.double]*, *mode: str*) → tuple[numpy.typing.NDArray[numpy.double], numpy.typing.NDArray[numpy.double]]

Get species molar rate of production.

**static** Surface.**surface_reaction_rates**(*chem_id: int*, *mat_id: int*, *numb_gas: int*, *numb_sites: int*, *numb_bulks: int*, *numb_reaction: int*, *p: float*, *t: float*, *molfrac: numpy.typing.NDArray[numpy.double]*, *site_frac: numpy.typing.NDArray[numpy.double]*, *bulk_act: numpy.typing.NDArray[numpy.double]*, *site_den: numpy.typing.NDArray[numpy.double]*, *mode: str*) → tuple[numpy.typing.NDArray[numpy.double], numpy.typing.NDArray[numpy.double]]

Get molar reaction rates of the reactive surface.

Surface.**set_surface_coverage**(*mat_name: str*) → tuple[numpy.typing.NDArray[numpy.double], numpy.typing.NDArray[numpy.double], numpy.typing.NDArray[numpy.double]]

Get the surface coverage.

Surface.**rop_surf**(*mat_name: str*, *pres: float*, *surf_temp: float*, *molfrac: numpy.typing.NDArray[numpy.double]*) → tuple[numpy.typing.NDArray[numpy.double], numpy.typing.NDArray[numpy.double]]

Get species molar rate of production.

Surface.**rxnrates_surf**(*mat_name: str*, *pres: float*, *surf_temp: float*, *molfrac: numpy.typing.NDArray[numpy.double]*) → tuple[numpy.typing.NDArray[numpy.double], numpy.typing.NDArray[numpy.double]]

Get molar rates of the surface reactions.

Surface.**get_bulk_molar_growth_rates**(*mat_name: str*, *surfrop: numpy.typing.NDArray[numpy.double]*) → numpy.typing.NDArray[numpy.double]

Get the molar growth rates of the bulk species.

Surface.**get_bulk_mass_growth_rates**(*mat_name: str*, *surfrop: numpy.typing.NDArray[numpy.double]*) → numpy.typing.NDArray[numpy.double]

Get the mass growth rates of the bulk species.

Surface.**get_bulk_linear_growth_rates**(*mat_name: str*, *surfrop: numpy.typing.NDArray[numpy.double]*) → numpy.typing.NDArray[numpy.double]

Get the linear growth rates of the bulk species.

Surface.**get_gas_production_rates**(*mat_name: str*, *surfrop: numpy.typing.NDArray[numpy.double]*) → numpy.typing.NDArray[numpy.double]

Get the molar production rates of the gas species.

Surface.**stefan_mass_flux**(*mat_name: str*, *surfrop: numpy.typing.NDArray[numpy.double]*) → float

Get the Stefan mass flux due to surface reactions.

### Description

Surface Chemkin Chemistry utilities.

### The surface_chemistry.py module

### Summary

### Classes

| | |
|---|---|
| *SurfaceChemistryMixin* | Surface chemistry related methods for the Chemistry class. |

### Attributes

| |
|---|
| *LP_c_char* |

### Constants

| |
|---|
| *MAX_SPECIES_LENGTH* |

### SurfaceChemistryMixin

**class** ansys.chemkin.core.surface_chemistry.**SurfaceChemistryMixin**

Surface chemistry related methods for the Chemistry class.

### Overview

### Methods

| | |
|---|---|
| *set_surface_chemistry* | Set up the surface chemistry data. |
| *no_surface_mechanism_declaration* | Inform users the Chemistry Set does not have surface chemistry. |
| *get_material_index* | Get the surface material index. |

### Properties

| | |
|---|---|
| *number_materials* | Get the number of surface materials. |
| *max_number_phases* | Get the maximum number of phases. |
| *max_number_sites* | Get the maximum number of surface site species. |
| *max_number_bulks* | Get the maximum number of surface bulk species. |
| *max_total_number_species* | Get the maximum number of all species. |
| *max_number_surface_reactions* | Get the maximum number of surface reactions. |
| *max_total_number_reactions* | Get the maximum number of gas and surface reactions. |
| *material_names* | Get list of surface material names. |

### Import detail

```
from ansys.chemkin.core.surface_chemistry import SurfaceChemistryMixin
```

### Property detail

**property** SurfaceChemistryMixin.**number_materials: int**

Get the number of surface materials.

**property** SurfaceChemistryMixin.**max_number_phases: int**

Get the maximum number of phases.

**property** SurfaceChemistryMixin.**max_number_sites: int**

Get the maximum number of surface site species.

**property** SurfaceChemistryMixin.**max_number_bulks: int**

Get the maximum number of surface bulk species.

**property** SurfaceChemistryMixin.**max_total_number_species: int**

Get the maximum number of all species.

**property** SurfaceChemistryMixin.**max_number_surface_reactions: int**

Get the maximum number of surface reactions.

**property** SurfaceChemistryMixin.**max_total_number_reactions: int**

Get the maximum number of gas and surface reactions.

**property** SurfaceChemistryMixin.**material_names: list[str]**

Get list of surface material names.

### Method detail

SurfaceChemistryMixin.**set_surface_chemistry**()

Set up the surface chemistry data.

SurfaceChemistryMixin.**no_surface_mechanism_declaration**()

Inform users the Chemistry Set does not have surface chemistry.

SurfaceChemistryMixin.**get_material_index**(*mat_name: str*) → int

Get the surface material index.

### Description

Surface chemistry behavior mixed into the Chemistry class.

### Module detail

surface_chemistry.**MAX_SPECIES_LENGTH = 17**

surface_chemistry.**LP_c_char**

### The surface_chemistry_controller.py module

### Summary

### Classes

| | |
|---|---|
| *SurfaceChemistryHost* | Protocol for reactor objects that can host surface chemistry behavior. |
| *SurfaceChemistryController* | Controller that encapsulates reactor surface-chemistry operations. |

### SurfaceChemistryHost

**class** ansys.chemkin.core.surface_chemistry_controller.**SurfaceChemistryHost**

Bases: Protocol

Protocol for reactor objects that can host surface chemistry behavior.

### Overview

### Methods

| | |
|---|---|
| *setkeyword* | Set a Chemkin keyword on the host reactor. |

### Properties

| | |
|---|---|
| *temperature* | Return the current reactor temperature. |

### Attributes

| |
|---|
| *has_surface_chemistry* |
| *numbspecies* |
| *numbmaterials* |
| *surface_chemistry* |

### Import detail

```
from ansys.chemkin.core.surface_chemistry_controller import SurfaceChemistryHost
```

### Property detail

**property** SurfaceChemistryHost.**temperature: float**

Return the current reactor temperature.

### Attribute detail

SurfaceChemistryHost.**has_surface_chemistry: bool**

SurfaceChemistryHost.**numbspecies: int**

SurfaceChemistryHost.**numbmaterials: int**

SurfaceChemistryHost.**surface_chemistry: Any**

### Method detail

SurfaceChemistryHost.**setkeyword**(*key: str*, *value: Any*)

Set a Chemkin keyword on the host reactor.

#### SurfaceChemistryController

**class** ansys.chemkin.core.surface_chemistry_controller.**SurfaceChemistryController**(*reactor:* SurfaceChemistryHost)

Controller that encapsulates reactor surface-chemistry operations.

### Overview

### Methods

| | |
|---|---|
| *activate_surface_chemistry* | Activate surface chemistry on reactor wall materials. |
| *verify_surface_chemistry* | Verify that the host reactor supports surface chemistry. |
| *no_surface_mechanism_declaration* | Inform users that surface chemistry is not available. |
| *require_surface_chemistry* | Return availability and log guidance when surface chemistry is absent. |
| *no_surface_material* | Log an error for an unknown surface material. |
| *get_material_names* | Return all surface material names. |
| *get_site_species_names* | Return site-species names for all surface materials. |
| *get_bulk_species_names* | Return bulk-species names for all surface materials. |
| *get_total_site_species* | Return total number of site species across all materials. |
| *get_total_bulk_species* | Return total number of bulk species across all materials. |
| *check_material_area_fraction* | Return whether a material-specific area fraction is set. |
| *get_material_area_fraction* | Return the area fraction for a given surface material. |
| *set_material_area_fraction* | Set the area fraction for a given surface material. |
| *check_material_temperature* | Return whether a material-specific temperature is set. |
| *get_material_temperature* | Return temperature for a given surface material. |
| *set_material_temperature* | Set temperature for a given surface material. |
| *get_all_surface_temperature* | Return temperatures of all materials in a flat array. |
| *get_site_fraction* | Return site fractions for a surface material. |
| *set_site_fraction* | Set site fractions for a surface material. |
| *get_all_site_fractions* | Return site fractions for all materials as one array. |
| *get_bulk_activity* | Return bulk activities for a surface material. |
| *set_bulk_activity* | Set bulk activities for a surface material. |
| *get_all_bulk_growth_rates* | Return bulk growth rates for all materials as one array. |
| *set_init_surface_coverage* | Build default/initial site-fraction and bulk-activity arrays. |
| *set_surface_chemistry_keywords* | Emit surface-chemistry related keyword lines via host.setkeyword. |

### Properties

| | |
|---|---|
| *get_numb_material* | Return number of materials in the active surface mechanism. |
| *surface_ratemultiplier* | Return the global surface reaction-rate multiplier. |

### Import detail

```
from ansys.chemkin.core.surface_chemistry_controller import SurfaceChemistryController
```

### Property detail

**property** SurfaceChemistryController.**get_numb_material: int**

Return number of materials in the active surface mechanism.

**property** SurfaceChemistryController.**surface_ratemultiplier: float**

Return the global surface reaction-rate multiplier.

### Method detail

SurfaceChemistryController.**activate_surface_chemistry**(*chem:* ansys.chemkin.core.chemistry.Chemistry, *mode: str = 'normal'*)

Activate surface chemistry on reactor wall materials.

SurfaceChemistryController.**verify_surface_chemistry**() → bool

Verify that the host reactor supports surface chemistry.

SurfaceChemistryController.**no_surface_mechanism_declaration**()

Inform users that surface chemistry is not available.

SurfaceChemistryController.**require_surface_chemistry**() → bool

Return availability and log guidance when surface chemistry is absent.

SurfaceChemistryController.**no_surface_material**(*mat_name: str*)

Log an error for an unknown surface material.

SurfaceChemistryController.**get_material_names**() → list[str]

Return all surface material names.

SurfaceChemistryController.**get_site_species_names**() → list[list[str]]

Return site-species names for all surface materials.

SurfaceChemistryController.**get_bulk_species_names**() → list[list[str]]

Return bulk-species names for all surface materials.

SurfaceChemistryController.**get_total_site_species**() → int

Return total number of site species across all materials.

SurfaceChemistryController.**get_total_bulk_species**() → int

Return total number of bulk species across all materials.

SurfaceChemistryController.**check_material_area_fraction**(*mat_name: str*) → int

Return whether a material-specific area fraction is set.

SurfaceChemistryController.**get_material_area_fraction**(*mat_name: str*) → float

Return the area fraction for a given surface material.

SurfaceChemistryController.**set_material_area_fraction**(*mat_name: str*, *fraction: float*)

Set the area fraction for a given surface material.

SurfaceChemistryController.**check_material_temperature**(*mat_name: str*) → int

Return whether a material-specific temperature is set.

SurfaceChemistryController.**get_material_temperature**(*mat_name: str*) → float

Return temperature for a given surface material.

SurfaceChemistryController.**set_material_temperature**(*mat_name: str*, *temp: float*)

Set temperature for a given surface material.

SurfaceChemistryController.**get_all_surface_temperature**() → numpy.typing.NDArray[numpy.double]

Return temperatures of all materials in a flat array.

SurfaceChemistryController.**get_site_fraction**(*mat_name: str*) → numpy.typing.NDArray[numpy.double]

Return site fractions for a surface material.

SurfaceChemistryController.**set_site_fraction**(*mat_name: str*, *recipe: list[tuple[str, float]]*)

Set site fractions for a surface material.

SurfaceChemistryController.**get_all_site_fractions**() → numpy.typing.NDArray[numpy.double]

Return site fractions for all materials as one array.

SurfaceChemistryController.**get_bulk_activity**(*mat_name: str*) → numpy.typing.NDArray[numpy.double]

Return bulk activities for a surface material.

SurfaceChemistryController.**set_bulk_activity**(*mat_name: str*, *recipe: list[tuple[str, float]]*)

Set bulk activities for a surface material.

SurfaceChemistryController.**get_all_bulk_growth_rates**() → numpy.typing.NDArray[numpy.double]

Return bulk growth rates for all materials as one array.

SurfaceChemistryController.**set_init_surface_coverage**() → tuple[numpy.typing.NDArray[numpy.double], numpy.typing.NDArray[numpy.double]]

Build default/initial site-fraction and bulk-activity arrays.

SurfaceChemistryController.**set_surface_chemistry_keywords**()

Emit surface-chemistry related keyword lines via host.setkeyword.

### Description

Surface chemistry controller for reactor models.

This module isolates surface-chemistry related behavior from `ReactorModel`. The controller is composition-based and operates on a host object that exposes reactor state and helper methods.

### The `surface_components.py` module

### Summary

### Classes

| | |
|---|---|
| *Material* | Surface Chemkin material object. |
| *SurfacePhase* | Surface phase object. |

### Functions

| | |
|---|---|
| *set_temp_array* | Set up the temperature array for Chemkin-CFD-API calls. |

### Attributes

| |
|---|
| *LP_c_char* |

### Constants

| |
|---|
| *MAX_SPECIES_LENGTH* |

#### Material

**class** ansys.chemkin.core.surface_components.**Material**(*mat_index: int*, *chem:* ansys.chemkin.core.chemistry.Chemistry, *label: str = ''*)

Surface Chemkin material object.

### Overview

### Methods

| | |
|---|---|
| *information* | List the size information of the material. |
| *get_site_specindex* | Get the local and the global indices of a site species. |
| *get_bulk_specindex* | Get the local and the global indices of a bulk species. |
| *get_surf_specindex* | Get the local and the global indices and type of a surface species. |
| *get_site_density* | Get the site phase densities. |
| *get_bulk_density* | Get bulk species densities. |
| *get_site_molar_weights* | Get the site species molecular weights. |
| *get_bulk_molar_weights* | Get the bulk species molecular weights. |
| *get_site_occupancy* | Get site species occupancy. |
| *get_site_species_h* | Get site species enthalpy. |
| *get_bulk_species_h* | Get bulk species enthalpy. |
| *get_bulk_species_cp* | Get bulk species specific heat capacity. |
| *material_species_composition* | Get elemental composition of a species. |
| *get_surface_reaction_parameters* | Get the Arrhenius reaction rate parameters. |
| *set_surface_reaction_afactor* | Set the Arrhenius A-Factor of the given surface reaction. |
| *get_surface_reaction_afactor* | Get the Arrhenius A-Factor of the given surface reaction. |
| *set_surface_reaction_act_energy* | Set the Arrhenius activation energy of the given surface reaction. |
| *get_surface_reaction_act_energy* | Get the Arrhenius activation energy. |
| *get_surface_reaction_string* | Get the reaction string of the surface reaction. |

### Properties

| | |
|---|---|
| *material_index* | Get the surface material index. |
| *number_phases* | Get the number of phases. |
| *number_site_phases* | Get the number of site phases. |
| *number_bulk_phases* | Get the number of bulk phases. |
| *number_total_species* | Get the number of species in all phases. |
| *number_site_species* | Get the total number of site species. |
| *number_bulk_species* | Get the total number of bulk species. |
| *first_site_phase_index* | Get phase index of the first site phase. |
| *last_site_phase_index* | Get phase index of the last site phase. |
| *first_bulk_phase_index* | Get phase index of the first bulk phase. |
| *last_bulk_phase_index* | Get phase index of the last bulk phase. |
| *number_surface_reactions* | Get the number of surface reactions. |
| *number_total_reactions* | Get the number of all reactions. |
| *phase_names* | Get list of phase names. |
| *site_species_names* | Set list of site species names. |
| *bulk_species_names* | Set list of bulk species names. |
| *first_site_species_index* | Get the global species index of the first site species. |
| *first_bulk_species_index* | Get the global species index of the first bulk species. |
| *last_site_species_index* | Get the global species index of the last site species. |
| *last_bulk_species_index* | Get the global species index of the last bulk species. |
| *surface_temperature* | Get surface material temperature. |
| *surface_area* | Get surface material area. |

### Attributes

| |
|---|
| *num_element* |
| *num_gas_species* |
| *num_gas_reactions* |
| *site_species_map* |
| *bulk_species_map* |
| *ESymbols* |
| *num_site_phase* |
| *num_bulk_phase* |
| *first_site_phase* |
| *last_site_phase* |
| *first_bulk_phase* |
| *last_bulk_phase* |
| *nspecies_of_phase* |
| *first_species_id_of_phase* |
| *last_species_id_of_phase* |
| *PhaseSymbol* |
| *phase_map* |
| *phases* |
| *site_symbols* |
| *bulk_symbols* |
| *sitewt_done* |
| *bulkwt_done* |
| *site_wt* |
| *bulk_wt* |
| *surftemp* |
| *activearea* |

### Import detail

```
from ansys.chemkin.core.surface_components import Material
```

### Property detail

**property** Material.**material_index: int**

Get the surface material index.

**property** Material.**number_phases: int**

Get the number of phases.

**property** Material.**number_site_phases: int**

Get the number of site phases.

**property** Material.**number_bulk_phases: int**

Get the number of bulk phases.

**property** Material.**number_total_species: int**

Get the number of species in all phases.

**property** Material.**number_site_species: int**

Get the total number of site species.

**property** Material.**number_bulk_species: int**

Get the total number of bulk species.

**property** Material.**first_site_phase_index: int**

Get phase index of the first site phase.

**property** Material.**last_site_phase_index: int**

Get phase index of the last site phase.

**property** Material.**first_bulk_phase_index: int**

Get phase index of the first bulk phase.

**property** Material.**last_bulk_phase_index: int**

Get phase index of the last bulk phase.

**property** Material.**number_surface_reactions: int**

Get the number of surface reactions.

**property** Material.**number_total_reactions: int**

Get the number of all reactions.

**property** Material.**phase_names**

Get list of phase names.

**property** Material.**site_species_names**

Set list of site species names.

**property** Material.**bulk_species_names**

Set list of bulk species names.

**property** Material.**first_site_species_index: int**

Get the global species index of the first site species.

**property** Material.**first_bulk_species_index: int**

Get the global species index of the first bulk species.

**property** Material.**last_site_species_index: int**

Get the global species index of the last site species.

**property** Material.**last_bulk_species_index: int**

Get the global species index of the last bulk species.

**property** Material.**surface_temperature: float**

Get surface material temperature.

**property** Material.**surface_area: float**

Get surface material area.

### Attribute detail

Material.**num_element**

Material.**num_gas_species**

Material.**num_gas_reactions**

Material.**site_species_map: dict[str, int]**

Material.**bulk_species_map: dict[str, int]**

Material.**ESymbols**

Material.**num_site_phase**

Material.**num_bulk_phase**

Material.**first_site_phase = 0**

Material.**last_site_phase = 0**

Material.**first_bulk_phase = 0**

Material.**last_bulk_phase = 0**

Material.**nspecies_of_phase**

Material.**first_species_id_of_phase**

Material.**last_species_id_of_phase**

Material.**PhaseSymbol: list[str] = []**

Material.**phase_map: dict[str, int]**

Material.**phases: dict[str, *SurfacePhase*]**

Material.**site_symbols: list[str] = []**

Material.**bulk_symbols: list[str] = []**

Material.**sitewt_done = 0**

Material.**bulkwt_done = 0**

Material.**site_wt**

Material.**bulk_wt**

Material.**surftemp = 300.0**

Material.**activearea = 0.0**

### Method detail

Material.**information**()

List the size information of the material.

Material.**get_site_specindex**(*symbol: str = ''*) → tuple[int, int]

Get the local and the global indices of a site species.

Material.**get_bulk_specindex**(*symbol: str = ''*) → tuple[int, int]

Get the local and the global indices of a bulk species.

Material.**get_surf_specindex**(*symbol: str = ''*, *mode: str = 'normal'*) → tuple[int, int, str]

Get the local and the global indices and type of a surface species.

Material.**get_site_density**() → numpy.typing.NDArray[numpy.double]

Get the site phase densities.

Material.**get_bulk_density**() → numpy.typing.NDArray[numpy.double]

Get bulk species densities.

Material.**get_site_molar_weights**() → numpy.typing.NDArray[numpy.double]

Get the site species molecular weights.

Material.**get_bulk_molar_weights**() → numpy.typing.NDArray[numpy.double]

Get the bulk species molecular weights.

Material.**get_site_occupancy**() → numpy.typing.NDArray[numpy.int32]

Get site species occupancy.

Material.**get_site_species_h**(*temp: float | None = None*) → numpy.typing.NDArray[numpy.double]

Get site species enthalpy.

Material.**get_bulk_species_h**(*temp: float | None = None*, *pres: float | None = None*) → numpy.typing.NDArray[numpy.double]

Get bulk species enthalpy.

Material.**get_bulk_species_cp**(*temp: float | None = None*, *pres: float | None = None*) → numpy.typing.NDArray[numpy.double]

Get bulk species specific heat capacity.

Material.**material_species_composition**(*elemindex: int = -1*, *specindex: int = -1*) → int

Get elemental composition of a species.

Material.**get_surface_reaction_parameters**() → tuple[numpy.typing.NDArray[numpy.double], numpy.typing.NDArray[numpy.double], numpy.typing.NDArray[numpy.double]]

Get the Arrhenius reaction rate parameters.

Material.**set_surface_reaction_afactor**(*reaction_index: int*, *a_factor: float*)

Set the Arrhenius A-Factor of the given surface reaction.

Material.**get_surface_reaction_afactor**(*reaction_index: int*) → float

Get the Arrhenius A-Factor of the given surface reaction.

Material.**set_surface_reaction_act_energy**(*reaction_index: int*, *act_energy: float*)

Set the Arrhenius activation energy of the given surface reaction.

Material.**get_surface_reaction_act_energy**(*reaction_index: int*) → float

Get the Arrhenius activation energy.

Material.**get_surface_reaction_string**(*reaction_index: int*) → str

Get the reaction string of the surface reaction.

### SurfacePhase

**class** ansys.chemkin.core.surface_components.**SurfacePhase**(*phase_index: int*, *phase_type: str*, *label: str = ''*)

Surface phase object.

### Overview

### Properties

| | |
|---|---|
| *number_species* | Get the number of species. |
| *site_density* | Get surface site density. |
| *first_species_index* | Get the global index of the first species. |
| *last_species_index* | Get the global index of the last species. |

### Attributes

| |
|---|
| *phase_index* |
| *first_spec_index* |
| *last_spec_index* |

### Import detail

```
from ansys.chemkin.core.surface_components import SurfacePhase
```

### Property detail

**property** SurfacePhase.**number_species: int**

Get the number of species.

**property** SurfacePhase.**site_density: float**

Get surface site density.

**property** SurfacePhase.**first_species_index: int**

Get the global index of the first species.

**property** SurfacePhase.**last_species_index: int**

Get the global index of the last species.

### Attribute detail

SurfacePhase.**phase_index**

SurfacePhase.**first_spec_index = 0**

SurfacePhase.**last_spec_index = 0**

### Description

Surface material and phase data models.

### Module detail

surface_components.**set_temp_array**(*temp: float*, *temp_ele: float | None = None*, *temp_ion: float | None = None*) → numpy.typing.NDArray[numpy.double]

Set up the temperature array for Chemkin-CFD-API calls.

surface_components.**MAX_SPECIES_LENGTH = 17**

surface_components.**LP_c_char**

### The utilities.py module

### Summary

### Classes

| | |
|---|---|
| *WorkingFolders* | folder/file utilities for multiple runs. |

### Functions

| | |
|---|---|
| *log_critical_error* | Log a critical error message. |
| *log_critical_message* | Log a critical message without additional formatting. |
| *log_error_message* | Log an error message without terminating execution. |
| *log_info_message* | Log an informational message without additional formatting. |
| *error_and_exit* | Log an error message and terminate execution. |
| *critical_and_exit* | Log a critical message and terminate execution. |
| *log_warning_message* | Log a warning message without terminating execution. |
| *warning_and_exit* | Log a warning message and terminate execution. |
| *where_element_in_array_1d* | Find number of occurrence and indices of the target value in an array. |
| *bisect* | Use bisectional method to find the largest index in the array. |
| *find_interpolate_parameters* | Find the indices branket the given value. |
| *interpolate_array* | Find the y-value corresponding to the x-value in data pairs. |
| *random* | Generate a (reproducible) random floating number. |
| *random_pick_integers* | Return a list of random integers from a given integer list. |
| *find_file* | Find the correct version of the given partial file name. |
| *copy_file* | Copy a file from the source folder to the target folder. |
| *delete_files_by_extension* | Delete files with specified extensions. |
| *check_jupyter_notebook* | Check if Pychemkin is running in any Jupyter environment. |
| *interpolate_point* | Interpolate value from a binary data array. |
| *find_string_in_list* | Find the index of a string in a list of strings. |

### Attributes

| |
|---|
| *ck_rng* |

### WorkingFolders

**class** ansys.chemkin.core.utilities.**WorkingFolders**(*dir_name: str*, *root_dir: str*)

folder/file utilities for multiple runs.

### Overview

### Methods

| | |
|---|---|
| *done* | Change back to the root directory. |

### Properties

| | |
|---|---|
| *root* | Get the "root" directory of the current structure. |
| *work* | Get the "work" directory of the current structure. |

### Attributes

| |
|---|
| *root_dir* |
| *work_dir* |

### Import detail

```
from ansys.chemkin.core.utilities import WorkingFolders
```

### Property detail

**property** WorkingFolders.**root: str**

Get the "root" directory of the current structure.

**property** WorkingFolders.**work: str**

Get the "work" directory of the current structure.

### Attribute detail

WorkingFolders.**root_dir**

WorkingFolders.**work_dir**

### Method detail

WorkingFolders.**done**()

Change back to the root directory.

### Description

pychemkin utilities.

### Module detail

utilities.**log_critical_error**(*msg_parts: list[str]*) → None

Log a critical error message.

utilities.**log_critical_message**(*msg_parts: list[str]*) → None

Log a critical message without additional formatting.

utilities.**log_error_message**(*msg_parts: list[str]*) → None

Log an error message without terminating execution.

utilities.**log_info_message**(*msg_parts: list[str]*) → None

Log an informational message without additional formatting.

utilities.**error_and_exit**(*msg_parts: list[str]*) → NoReturn

Log an error message and terminate execution.

utilities.**critical_and_exit**(*msg_parts: list[str]*)

Log a critical message and terminate execution.

utilities.**log_warning_message**(*msg_parts: list[str]*) → None

Log a warning message without terminating execution.

utilities.**warning_and_exit**(*msg_parts: list[str]*) → NoReturn

Log a warning message and terminate execution.

utilities.**where_element_in_array_1d**(*arr: numpy.typing.NDArray[numpy.double] | numpy.typing.NDArray[numpy.int32]*, *target: float*) → tuple[int, numpy.typing.NDArray[numpy.int32]]

Find number of occurrence and indices of the target value in an array.

utilities.**bisect**(*ileft: int*, *iright: int*, *x: float*, *xarray*) → int

Use bisectional method to find the largest index in the array.

utilities.**find_interpolate_parameters**(*x: float*, *xarray: numpy.typing.NDArray[numpy.double]*) → tuple[int, float]

Find the indices branket the given value.

utilities.**interpolate_array**(*x: float*, *x_array: numpy.typing.NDArray[numpy.double]*, *y_array: numpy.typing.NDArray[numpy.double]*) → float

Find the y-value corresponding to the x-value in data pairs.

utilities.**random**(*range: None | tuple[float, float] = None*) → float

Generate a (reproducible) random floating number.

utilities.**random_pick_integers**(*numb_picks: int*, *source_integers: list[int]*) → tuple[list[int], list[int]]

Return a list of random integers from a given integer list.

utilities.**find_file**(*filepath: str*, *partialfilename: str*, *fileext: str*) → str

Find the correct version of the given partial file name.

utilities.**copy_file**(*sourcepath: str*, *targetpath: str*, *filename: str*, *new_file_name: str | None = None*)

Copy a file from the source folder to the target folder.

utilities.**delete_files_by_extension**(*targetpath: str*, *extensions: list[str]*)

Delete files with specified extensions.

utilities.**check_jupyter_notebook**() → bool

Check if Pychemkin is running in any Jupyter environment.

utilities.**interpolate_point**(*x_value: float*, *x_array: list[float] | numpy.typing.NDArray[numpy.double]*, *y_array: list[float] | numpy.typing.NDArray[numpy.double]*) → tuple[int, float]

Interpolate value from a binary data array.

utilities.**find_string_in_list**(*target: str*, *string_list: list[str]*) → int

Find the index of a string in a list of strings.

utilities.**ck_rng = None**

### The validation.py module

### Summary

### Functions

| | |
|---|---|
| validate_chemid | Validate Chemistry set index. |
| validate_temperature | Validate gas temperature. |
| validate_pressure_temperature | Validate pressure and temperature pair. |
| validate_density | Validate density value. |
| validate_fraction_arrays | Validate fraction and molecular-weight arrays are same size. |
| validate_species_array_size | Validate a species-array size against expected species count. |
| validate_minimum_value | Validate that a scalar value is greater than or equal to a minimum. |
| validate_equal_list_lengths | Validate that two list lengths are equal. |
| validate_fraction_array_size | Validate fractions-array size against chemistry species count. |
| validate_file_exists | Validate that a filesystem path points to an existing file. |

### Description

Reusable validation helpers for core modules.

### Module detail

validation.**validate_chemid**(*chemid: int*) → None

Validate Chemistry set index.

validation.**validate_temperature**(*temp: float*) → None

Validate gas temperature.

validation.**validate_pressure_temperature**(*pressure: float*, *temp: float*) → None

Validate pressure and temperature pair.

validation.**validate_density**(*density: float*) → None

Validate density value.

validation.**validate_fraction_arrays**(*frac*, *wt*, *mode: str*) → int

Validate fraction and molecular-weight arrays are same size.

validation.**validate_species_array_size**(*expected: int*, *actual: int*, *context: str*) → None

Validate a species-array size against expected species count.

validation.**validate_minimum_value**(*value: float*, *minimum: float*, *message: str*) → None

Validate that a scalar value is greater than or equal to a minimum.

validation.**validate_equal_list_lengths**(*first_label: str*, *first_len: int*, *second_label: str*, *second_len: int*) → None

Validate that two list lengths are equal.

validation.**validate_fraction_array_size**(*actual: int*, *expected: int*) → None

Validate fractions-array size against chemistry species count.

validation.**validate_file_exists**(*file_path: str*, *context_label: str = 'file'*) → None

Validate that a filesystem path points to an existing file.

### 7.1.2 Description

Chemkin module for Python core.

### 7.1.3 Module detail

core.__version__ = '0.2.0'

core.msg

core.this_msg = ''

core.msg

core.this_msg = ''

core.ansys_dir = ''

core.ansys_version = 251

core.ck_name = 'chemkin.win64'

core.chemkin_dir

core.unit_code

core.ierror

core.frm

CHAPTER

EIGHT

# EXAMPLES

This section shows how to use PyChemkin utilities and reactor models to perform different types of simulations. Early examples demonstrate how to perform essential operations, such as loading and preprocessing the chemistry set. Later examples explain how to tackle more complex simulations, such as running an ignition delay parameter study. Thus, you should go through the examples in the order listed.

**Note**

Some examples require mechanism and data files that are not included through the download links provided with their descriptions. These supplementary files are available in the `examples/data` folder of the *PyChemkin* repository. The corresponding Python source files are available in the `examples` folder. Make sure the full file paths to the mechanism and the data files are correct in the example scripts before running them.

**Note**

When you use the Jupyter Notebook version of the example projects (`*.ipynb`), please remember to *close the project after you finish viewing and/or running the project*. When the project is open, it might hold on to your Ansys license. If you have too many Jupyter Notebook projects active, you might run out of your Ansys licenses. Closing the Jupyter Notebook project will release the license. If the *Jupyter Notebook web interface* is used to load the PyChemkin example projects, you can navigate to the top menu bar and select `File > Close and Shut Down Notebook` to close the project and release the license. If the *VS Code* is used, you can simply close the tab associated with the example.

CHAPTER

NINE

# CHEMISTRY

PyChemkin examples to demonstrate basic operations for instantiating *Chemistry Set* from the gas-phase mechanism and the thermodynamic/transport data files.

The examples walk through the steps about how to pre-process the *Chemistry Set*, retrieve mechanism information and the species properties, as well as how to handle multiple *Chemistry Sets* in a **Python Chemkin** project.

CHAPTER

TEN

# MIXTURE

*Mixture* is the "core" token in **PyChemkin**. It represents a gas-phase mixture by storing its pressure, temperature, and species composition. *Stream* is an extended "Class" of *Mixture* with the additional attribute of "mass/volumetric flow rate". *Mixture* and *Stream* include utilities that can be used to evaluate the mixture properties (density, mixture enthalpy, mixture viscosity, ...) and the reaction rates. There are also methods to manipulate the *Mixture*/*Stream* such as merging two mixtures adiabatically.

The examples demonstrate how to instantiate the *Mixture*/*Stream* objects, how to extract information and properties of a *Mixture*/*Stream*, and how to manipulate *Mixture*/*Stream* objects.

CHAPTER

# ELEVEN

# BATCH REACTOR

The *batch reactor* is an idealized 0-D transient closed reactor model in which the gas inside the reactor is assumed to be homogeneous. When the pressure of the *batch reactor* is constrained, the reactor volume would vary to conserve the total gas mass. Likewise, the reactor pressure might change when the reactor volume is "given". The gas temperature can be "given" by piecewise linear profile data or solved by the energy equation.

The examples consist of projects that utilize the *batch reactor* model to perform simulations of various complexities, from tracking the evolution of the mixture properties to the A-factor sensitivity analysis.

CHAPTER

TWELVE

# 0-D ENGINE MODELS

**PyChemkin** offers two types of 0-D engine models: the *Homogeneous Charged Compression Ignition (HCCI)* engine model and the *Spark Ignition (SI)* engine model. These 0-D engine models simulate the combustion process (or the lack of, in case of a misfire) inside an engine cylinder between the *Intake Valve Close (IVC)* and the *Exhaust Valve Open (EVO)* when the cylinder is considered as a closed system. The Chemkin *Theory* manual has detailed descriptions of these 0-D engine models.

The examples show the steps of setting up simulations with different *Chemkin* engine models.

CHAPTER

# THIRTEEN

# PERFECTLY STIRRED REACTOR

The *perfectly Stirred reactor (PSR)* model is a 0-D steady-state reactor. This ideal reactor model assumes the gas mixture inside is uniform and the properties of the outlet stream are exactly the same as the gas properties inside the reactor. A PSR can have either its pressure or its volume "specified", and the reactor temperature can either be solved from the energy equation or be "given" as an input parameter.

The examples will show the procedures of setting up single PSR simulations with different constraints and with multiple inlet streams.

CHAPTER

# FOURTEEN

# PLUG-FLOW REACTOR

*Plug-Flow Reactor (PFR)* model is an idealized 1-D steady-state flow reactor model with single inlet stream. The reactor pressure is assumed to be given, and the mass (flow rate) conservation is maintained by the velocity change. The gas temperature along the reactor can be either specified or solved by the energy equation.

CHAPTER

FIFTEEN

# SHOCK TUBE REACTOR

The *shock tube reactor* model is a 0-D transient reactor. This ideal reactor model is normally used to reproduce the shock tube experiments for chemical kinetics researches. The *incident shock* model predicts species profiles behind the incident shock front which can be used to compare against corresponding experimental data. Similarly, the *reflected shock* model complements the experiments for kinetics researches at the high temperature conditions behind a reflected shock front.

The examples will show the procedures of setting up an incident shock (with boundary layer correction) simulation and a Zeldovich-von Neumann-Deoring (ZND) analysis of a combustible gas mixture.

CHAPTER

SIXTEEN

# PSR NETWORK

**PyChemkin** can be used to create an *Equivalence Reactor Network (ERN)* as a reduced-order (in geometry) model for complex combustion applications such as gas turbine combustor. Each reactor in the network represents a sub-region (zone) in the actual equipment. Normally these zones are created according to specific criteria such as temperature, equivalence ratio, or location; and the gas flow (mean, turbulent, and diffusive) between the zones becomes the internal streams between the reactors.

There are several ways to build an *ERN* in **PyChemkin**. The first method is to create and solve the reactors one-by-one starting from the most upstream (first) reactor. Adjust/manipulate the external and outlet streams from connected reactors (that is, solutions from the reactors) using the *Mixture*/*Stream* utilities and apply the desired stream as the inlet for the downstream reactors. The second method takes advantage of the *hybridreactornetwork* "model" of **PyChemkin**. Simply defined the reactors with the associated external inlets and add the reactors to the *hybridreactornetwork*. If there are *recycling* streams from the downstream reactor to the upstream ones, the *hybridreactornetwork* will solve the entire *ERN* iteratively. In this case, a *tear point* must be explicitly specified. The last method, the coupled method, is to create the network and its connectivity first, and the entire *ERN* will be solved in a coupled manner. This method is the default method in Chemkin GUI and is available as the *PSRCluster* "model" in **PyChemkin**.

**Chemkin *ERN* has a few limitations:**

- The first reactor of the reactor network must have at least one external inlet.
- when using the *coupled* model, the entire reactor network can have only one outlet to the surroundings, and the outlet must be attached to the last reactor in the network.
- PSR is the only reactor model allowed in the *PSRCluster* and the *hybridreactornetwork* reactor networks.

The *PSRChain_xxx* examples show the two methods to build and run a series of linked PSRs (no stream recycling) can be modeled in **PyChemkin**. The *PSRnetwork* example goes over the steps of creating and running an *ERN* with recycling streams by using the *hybridreactornetwork* method. The *PSRnetwork_coupled* example solves the same *ERN* as the *PSRnetwork* example but utilizes the *coupled* method. Because the sole outlet from the *coupled ERN* must be attached to the last reactor, the order of the downstream zones is different from the *PSRnetwork* example.

CHAPTER

# SEVENTEEN

# PREMIXED FLAME MODELS

The *Premixed Flame* model is a 1-D steady-state application, and it contains two sub-models: *"flame speed calculator"* (or the freely propagating flame model) and *"flat flame"* model. The *flame speed calculator* is commonly used to calculate the laminar flame speed of a premixed fuel-oxidizer mixture. The predicted/computed flame speeds can be used to validate a combustion mechanism or can serve as a prre-processor to construct a "flame speed table" or a "combustion progress table" for other applications. The *flat flame* burner-stablized model is primarily used to study the flame chemistry in conjunction with the flat flame experiments.

The examples show different ways to perform the "premixed flame" calculations.

CHAPTER

EIGHTEEN

# OPPOSED-FLOW FLAME MODEL

The *opposed-flow Flame* model is a 1-D steady-state application to study the structure of stretched flames in the opposed-flow configuration. It can also be used to generate a "flamelet table" for CFD simulations. See the Chemkin *Theory* manual for additional information about this feature.

CHAPTER

# NINETEEN

# SURFACE CHEMISTRY

The examples in this section demonstrate how *surface chemistry* can be included in a *Chemistry Set* and applied to simulate chemical vapor deposition (CVD) and catalytic combustion processes in **PyChemkin**.

CHAPTER

TWENTY

# MULTIPROCESSING PARAMETER STUDY

Chemkin application by itself does not support parallel computing, however, in the context of parameter study, each case can run on its own CPU core to make better use of any available computing power. The examples in this section describe the process of applying Python multi-threading and multi-processing packages to **PyChemkin** parameter study to improve the overall simulation performance.

CHAPTER

TWENTYONE

# ADVANCED APPLICATIONS

Examples in this section are to showcase the use of Pychemkin in more complicated applications. You can make improvements to a Pychemkin reactor model by adding new physical processes. Or, you can have Pychemkin interact with other Python applications to address complex problems.

CHAPTER

# TWENTYTWO

# POST-PROCESSING

PyChemkin has *post-process* utilities that can import the XML solution data file generated by selected Chemkin 1-D and 1.5-D reactor models. The examples in this section demonstrate how to use these utilities to import, visualize, and analyze the solution data from the (1-D steady-state) flame model, the (transient 0-D) multi-zone SI engine model, and the (pseudo 2-D) shear-layer channel flow reactor models.

## 22.1 Chemistry

PyChemkin examples to demonstrate basic operations for instantiating *Chemistry Set* from the gas-phase mechanism and the thermodynamic/transport data files.

The examples walk through the steps about how to pre-process the *Chemistry Set*, retrieve mechanism information and the species properties, as well as how to handle multiple *Chemistry Sets* in a **Python Chemkin** project.

### 22.1.1 Preprocess a gas-phase mechanism

Before you can run a Chemkin simulation, you must load and preprocess the reaction mechanism data. The mechanism data consists of a reaction mechanism input file, a thermodynamic data file, and an optional transport data file.

This example shows how to instantiate and preprocess a chemistry set, which is always the first task in running any PyChemkin simulation.

PyChemkin allows several chemistry sets to coexist in the same Python project. However, only one chemistry set can be active at a time.

#### Import PyChemkin packages and start the logger

```
from pathlib import Path

# import PyChemkin packages
import ansys.chemkin.core as ck
from ansys.chemkin.core import Color
from ansys.chemkin.core.logger import logger

# check the working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)

# set PyChemkin verbose mode
ck.set_verbose(True)
```

### Create a chemistry set

The first mechanism to load is the GRI 3.0 mechanism for methane combustion. This mechanism and its associated data files come with the standard Ansys Chemkin installation in the `/reaction/data` directory.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir

# set mechanism input files
# including the full file path is recommended
# the gas-phase reaction mechanism file (GRI 3.0)
chemfile = str(mechanism_dir / "grimech30_chem.inp")
# the thermodynamic data file
thermfile = str(mechanism_dir / "grimech30_thermo.dat")
# the transport data file
tranfile = str(mechanism_dir / "grimech30_transport.dat")

# create a chemistry set based on the GRI 3.0 methane combustion mechanism
MyGasMech = ck.Chemistry(chem=chemfile, therm=thermfile, tran=tranfile, label="GRI 3.0")
```

### Preprocess the chemistry set

```
# preprocess the mechanism files
ierror = MyGasMech.preprocess()

# display the preprocess status
print()
if ierror != 0:
    # When a non-zero value is returned from the process, check the text output files,
    # "chem.out," "tran.out," and "summary.out," for potential error messages about
    # the mechanism data.
    print(f"Preprocessing error encountered. Code = {ierror:d}.")
    print(f"See the summary file {MyGasMech.summaryfile} for details.")
    exit()
else:
    Color.ckprint("OK", ["Preprocessing succeeded.", "!!!"])
    print("Mechanism information:")
    print(f"Number of elements = {MyGasMech.number_elements:d}.")
    print(f"Number of gas species = {MyGasMech.number_species:d}.")
    print(f"Number of gas reactions = {MyGasMech.number_gas_reactions:d}.")
```

### Display the basic mechanism information

```
print(f"\nElement and species information of mechanism {MyGasMech.label}")
print("=" * 50)

# extract element symbols as a list
elelist = MyGasMech.element_symbols
# get atomic masses as numpy 1D double array
awt = MyGasMech.atomic_weight
# print element information
for k in range(len(elelist)):
```

(continues on next page)

(continued from previous page)

```
    print(f"Element number {k + 1:3d}: {elelist[k]:16}. Mass = {awt[k]:f}.")

print("=" * 50)

# extract gas species symbols as a list
specieslist = MyGasMech.species_symbols
# get species molecular masses as numpy 1D double array
wt = MyGasMech.species_molar_weight
# print gas species information
for k in range(len(specieslist)):
    print(f"Species number {k + 1:3d}: {specieslist[k]:16}. Mass = {wt[k]:f}.")
print("=" * 50)
```

### Create a second chemistry set

The second mechanism to load into the project is the C2-NOx mechanism for the combustion of C1-C2 hydrocarbons. This mechanism differs from the GRI mechanism in the sense that it is self-contained, that is, the thermodynamic and transport data of all species is included in the `C2_NOx_SRK.inp` mechanism input file, which sits in the same reaction data directory as the GRI mechanism input files.

Use the same steps that were used to instantiate the first chemistry set to process any set of reaction mechanism files. Here, use a slightly different procedure to instantiate the second chemistry set. After you have created it, specify the reaction mechanism files one by one. The reaction mechanism file in this case contains all the necessary thermodynamic and transport data. Thus, there is no need to specify thermodynamic and transport data files. However, an additional step is required to instruct the preprocessor to include the transport data.

```
# set the second mechanism directory (the default Chemkin mechanism data directory)
mechanism_dir = data_dir

# create a chemistry set based on C2_NOx using an alternative method
My2ndMech = ck.Chemistry(label="C2 NOx")

# set mechanism input files individually
# this mechanism file contains all the necessary thermodynamic and transport data
# thus, there is no need to specify thermodynamic and transport data files
My2ndMech.chemfile = str(mechanism_dir / "C2_NOx_SRK.inp")

# instruct the preprocessor to include the transport properties
# only when the mechanism file contains all the transport data
My2ndMech.preprocess_transportdata()
```

### Preprocess the second chemistry set

```
# preprocess the second mechanism file
ierror = My2ndMech.preprocess()

# display the preprocess status
print()
if ierror != 0:
    # When a non-zero value is returned from the process, check the text output files,
    # "chem.out," "tran.out," and "summary.out," for potential error messages about
    # the mechanism data.
```

(continues on next page)

(continued from previous page)

```
    print(f"Preprocessing error encountered. Code = {ierror:d}.")
    print(f"See the summary file {My2ndMech.summaryfile} for details.")
    exit()
else:
    Color.ckprint("OK", ["Preprocessing succeeded.", "!!!"])
    print("Mechanism information:")
    print(f"Number of elements = {My2ndMech.mm:d}.")
    print(f"Number of gas species = {My2ndMech.kk:d}.")
    print(f"Number of gas reactions = {My2ndMech.ii_gas:d}.")
```

#### Display the basic mechanism information

```
print(f"\nElement and species information of mechanism {My2ndMech.label}")
print("=" * 50)

# extract element symbols as a list
elelist = My2ndMech.element_symbols
# get atomic masses as numpy 1D double array
awt = My2ndMech.awt
# print element information
for k in range(len(elelist)):
    print(f"Element # {k + 1:3d}: {elelist[k]:16}. Mass = {awt[k]:f}.")

print("=" * 50)

# extract gas species symbols as a list
specieslist = My2ndMech.species_symbols
# get species molecular masses as numpy 1D double array
wt = My2ndMech.wt
# print gas species information
for k in range(len(specieslist)):
    print(f"Species # {k + 1:3d}: {specieslist[k]:16}. Mass = {wt[k]:f}.")

print("=" * 50)

# clean up
ck.done()
```

### 22.1.2 Work with multiple mechanisms

PyChemkin can facilitate multiple mechanisms in one project. However, only one chemistry set can be active at a time. This example shows how to use the `activate()` method to switch between multiple chemistry sets (mechanisms) in the same Python project. You can use this method to compare results from two different mechanisms, such as the *base* and *reduced* mechanisms.

```
```

### Import PyChemkin packages and start the logger

```
from pathlib import Path

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color
from ansys.chemkin.core.logger import logger

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
```

### Create the first chemistry set

The first mechanism to load is the GRI 3.0 mechanism for methane combustion. This mechanism and its associated data files come with the standard Ansys Chemkin installation in the `/reaction/data` directory.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir

# specify the mechanism input files
# including the full file path is recommended
chemfile = str(mechanism_dir / "grimech30_chem.inp")
thermfile = str(mechanism_dir / "grimech30_thermo.dat")
tranfile = str(mechanism_dir / "grimech30_transport.dat")
# create a chemistry set based on GRI 3.0
My1stMech = ck.Chemistry(chem=chemfile, therm=thermfile, tran=tranfile, label="GRI 3.0")
```

### Preprocess the first chemistry set

```
# preprocess the mechanism files
ierror = My1stMech.preprocess()
print()
if ierror != 0:
    # encountered error during preprocessing
    print(f"Preprocessing error encountered. Code = {ierror:d}.")
    print(f"See the summary file {My1stMech.summaryfile} for details.")
    exit()
else:
    # Display the basic mechanism information
    print(Color.GREEN + "Preprocessing succeeded.", end=Color.END)
    print("Mechanism information:")
    print(f"Number of elements = {My1stMech.mm:d}.")
    print(f"Number of gas species = {My1stMech.kk:d}.")
    print(f"Number of gas reactions = {My1stMech.ii_gas:d}.")
```

### Set up a gas mixture

Set up a gas mixture based on the species in this chemistry set. Create a gas mixture named `mymixture1` based on the `My1stMech` chemistry set. The species mole fractions/ratios are given in the `recipe` format. The `X` property is used because the mole fractions are given.

```
mymixture1 = ck.Mixture(My1stMech)
# set mixture temperature [K]
mymixture1.temperature = 1000.0
# set mixture pressure [dynes/cm2]
mymixture1.pressure = ck.P_ATM
# use the "X" property to specify the molar compositions of the mixture
mymixture1.x = [("CH4", 0.1), ("O2", 0.21), ("N2", 0.79)]
```

### Perform an equilibrium calculation

The equilibrium state is stored as a mixture named `equil_mix1_hp`. You can get the equilibrium temperature from the `temperature` property of this mixture. You can learn more about the PyChemkin `equilibrium()` method by either typing `ck.help("equilibrium")` at the Python prompt or uncommenting the following line.

```
# ck.help("equilibrium")

# find the constrained H-P equilibrium state of ``mymixture1``
equil_mix1_hp = ck.equilibrium(mymixture1, opt=5)
# print the equilibrium temperature
print(f"Equilibrium temperature of mymixture1: {equil_mix1_hp.temperature} [K]")
```

### Create the second chemistry set

The second mechanism is the `C2 NOx` mechanism in the `/reaction/data` directory. Here, a different process for setting up a chemistry set is used. The chemistry set instance is created before the mechanism files are specified. You can make changes to the files to include in the chemistry set before running the preprocessing step.

The `C2 NOx` mechanism file, in addition to the reactions, contains the thermodynamic and transport data of all species in the mechanism. Thus, you must only specify the mechanism file, that is, `chemfile`. If your simulation requires transport properties, you must use the `preprocess_transportdata()` method to tell the preprocessor to also include the transport data.

```
# set the second mechanism directory (the default Chemkin mechanism data directory)
mechanism_dir = data_dir
# create a chemistry set based on "C2_NOx" using an alternative method
My2ndMech = ck.Chemistry(label="C2 NOx")
# set mechanism input files individually
# this mechanism file contains all necessary thermodynamic and transport data
# thus, there is no need to specify thermodynamic and transport data files
My2ndMech.chemfile = str(mechanism_dir / "C2_NOx_SRK.inp")

# direct the preprocessor to include the transport properties
# only when the mechanism file contains all the transport data
My2ndMech.preprocess_transportdata()
```

### Preprocess the second chemistry set

The `C2 NOx` mechanism also includes information about the *Soave* cubic Equation of State (EOS) for real-gas applications. The PyChemkin preprocessor indicates the availability of the real-gas model in the chemistry set processed. For example, during preprocessing, this is printed: `real-gas cubic EOS 'Soave' is available`. As soon as the second chemistry set is preprocessed successfully, it becomes the active chemistry set of the project. The first chemistry set, `My1stMech`, is pushed to the background.

```
# preprocess the second mechanism files
ierror = My2ndMech.preprocess()
print()
if ierror != 0:
    # encountered error during preprocessing
    print(f"Preprocessing error encountered. Code = {ierror:d}.")
    print(f"See the summary file {My2ndMech.summaryfile} for details.")
    exit()
else:
    # Display the basic mechanism information
    print(Color.GREEN + "Preprocessing succeeded.", end=Color.END)
    print("Mechanism information:")
    print(f"Number of elements = {My2ndMech.mm:d}.")
    print(f"Number of gas species = {My2ndMech.kk:d}.")
    print(f"Number of gas reactions = {My2ndMech.ii_gas:d}.")
```

### Set up the second gas mixture based on the species in the second chemistry set

Create a gas mixture named `mymixture2` based on the `My2ndMech` chemistry set.

```
mymixture2 = ck.Mixture(My2ndMech)
# set mixture temperature [K]
mymixture2.temperature = 500.0
# set mixture pressure [dynes/cm2]
mymixture2.pressure = 2.0 * ck.P_ATM
# set mixture molar composition
mymixture2.x = [("H2", 0.02), ("O2", 0.2), ("N2", 0.8)]
```

### Run the detonation calculation

You can now compute the detonation wave speed with the `mymixture2` gas mixture. The `CJ_mix2` represents the mixture at the Chapman-Jouguet state. The speed of sound and the detonation wave speed are returned in the `speeds_mix2` tuple.

```
speeds_mix2, cj_mix2 = ck.detonation(mymixture2)
#  print the detonation calculation results
print(f"Detonation mymixture2 temperature: {cj_mix2.temperature} [K]")
print(f"Detonation wave speed = {speeds_mix2[1] / 100.0} [m/sec]")
```

### Switch to the first chemistry set and gas mixture

Use the `activate()` method to reactivate the `My1stMech` chemistry set and the `mymixture1` gas mixture.

```
My1stMech.activate()
```

#### Run the detonation calculation

You can now compute the detonation wave speed with the `mymixture1` gas mixture. The `CJ_mix1` represents the mixture at the Chapman-Jouguet state. The speed of sound and the detonation wave speed are returned in the `speeds_mix1` tuple.

**Note**

The `mymixture1` and `mymixture2` gas mixtures have different initial conditions.

```
speeds_mix1, cj_mix1 = ck.detonation(mymixture1)
# print the detonation calculation results
print(f"Detonation 'mymixture1' temperature: {cj_mix1.temperature} [K]")
print(f"Detonation wave speed = {speeds_mix1[1] / 100.0} [m/sec]")

# clean up
ck.done()
```

### 22.1.3 Set up a PyChemkin project

This example shows how to set up a PyChemkin project and start to use PyChemkin features. You begin by importing the PyChemkin package, which is named `ansys.chemkin.core`, and optionally the PyChemkin logger package, which is named `ansys.chemkin.core.logger`.

Next, you use the `Chemistry()` method to instantiate and preprocess a chemistry set. This requires specifying the file names (with full file paths) of the mechanism file, thermodynamic data file, and optional transport data file.

Once the chemistry set instance is instantiated, you use the `preprocess()` method to preprocess the mechanism data. You can then start to use PyChemkin features to build your simulation.

**Note**

Running PyChemkin requires Ansys 2025 R2 or later.

```

```

#### Import PyChemkin packages

Import the `ansys.chemkin.core` package to start using PyChemkin in the project. Importing the `ansys.chemkin.core.logger` package is optional.

```
from pathlib import Path

import ansys.chemkin.core  # import PyChemkin
from ansys.chemkin.core.logger import logger
```

#### Set up the file paths of the mechanism and data files

The `ansys_dir` variable on your local computer specifies the location of your Ansys installation.

Use `os.path` methods to construct the file names with the full paths.

Assign the files to the corresponding chemistry set arguments:

- `mech_file`: Mechanism input file
- `therm_file`: Thermodynamic data file
- `tran_file`: Transport data file (optional)

```
# create GRI 3.0 mechanism from the data directory
mechanism_dir = Path(ansys.chemkin.core.ansys_dir) / "reaction" / "data"
# set up mechanism file names
mech_file = str(mechanism_dir / "grimech30_chem.inp")
therm_file = str(mechanism_dir / "grimech30_thermo.dat")
tran_file = str(mechanism_dir / "grimech30_transport.dat")
```

### Instantiate the chemistry set

Use the `Chemistry()` method to instantiate a chemistry set named `GasMech`.

```
GasMech = ansys.chemkin.core.Chemistry(
    chem=mech_file, therm=therm_file, tran=tran_file, label="GRI 3.0"
)
```

### Preprocess the chemistry set

Preprocess the GRI 3.0 chemistry set.

```
status = GasMech.preprocess()
```

### Check the preprocessing status

```
if status != 0:
    # failed
    print(f"Preprocessing: Error encountered. Code = {status:d}.")
    print(f"See the summary file {GasMech.summaryfile} for details.")
    logger.error("Preprocessing failed.")
    exit()
```

### Start to use PyChemkin features

Start to use PyChemkin features to build your simulation. For example, using the `Mixture()` method, create an `air` mixture based on the `GasMech` chemistry set.

```
# create 'air' mixture based on 'GasMech' chemistry set
air = ansys.chemkin.core.Mixture(GasMech)
# set 'air' condition
# mixture pressure in [dynes/cm2]
air.pressure = 1.0 * ansys.chemkin.core.P_ATM
# mixture temperature in [K]
air.temperature = 300.0
# mixture composition in mole fractions
air.x = [("O2", 0.21), ("N2", 0.79)]
```

#### Print the properties of the air mixture for verification

**Note**

- The default units of temperature and pressure are [K] and [dynes/cm$^2$], respectively.
- The constant `P_ATM` is a conversion multiplier for pressure.
- Transport property methods such as `mixture_viscosity()` require transport data. You must include the transport data file when creating the chemistry set.

```
# print pressure and temperature of the `air` mixture
print(f"Pressure    = {air.pressure / ansys.chemkin.core.P_ATM} [atm]")
print(f"Temperature = {air.temperature} [K]")
# print the 'air' composition in mass fractions
air.list_composition(mode="mass")
# get 'air' mixture density [g/cm3]
print(f"Mixture density   = {air.rho} [g/cm3]")
# get 'air' mixture viscosity [g/cm-sec] or [poise]
print(f"Mixture viscosity = {air.mixture_viscosity() * 100.0} [cP]")

# clean up
ansys.chemkin.core.done()
```

### 22.1.4 Evaluate gas species properties

PyChemkin inherits many Ansys Chemkin utilities for evaluating species properties in a mechanism. Most tools for calculating the species properties are part of the PyChemkin `Chemistry()` methods. This example shows how use some of these methods to evaluate species thermodynamic and transport properties.

```

```

#### Import PyChemkin packages and start the logger

```
from pathlib import Path

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core.logger import logger

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create a chemistry set

The mechanism to load is the GRI 3.0 mechanism for methane combustion. This mechanism and its associated data files come with the standard Ansys Chemkin installation in the `/reaction/data` directory.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir

# create a chemistry set based on GRI 3.0
MyGasMech = ck.Chemistry(label="GRI 3.0")
# set mechanism input files
# inclusding the full file path is recommended
MyGasMech.chemfile = str(mechanism_dir / "grimech30_chem.inp")
MyGasMech.thermfile = str(mechanism_dir / "grimech30_thermo.dat")
MyGasMech.tranfile = str(mechanism_dir / "grimech30_transport.dat")
```

### Preprocess the chemistry set

```
# preprocess the mechanism files
_ = MyGasMech.preprocess()
```

### Display the basic mechanism information

Display the element and species information from the GRI 3.0 mechanism. You can get the entire list of the elements and the gas species in the mechanism by using the `element_symbols` and `species_symbols` lists, respectively.

Use the `get_specindex()` method to get the species index of a species in the mechanism. Use the `SpeciesComposition()` method to check the elemental composition of a gas species.

**Note**

Both the species name/symbol and element name/symbol are case-sensitive.

```
# extract element symbols as a list
elelist = MyGasMech.element_symbols
# extract gas species symbols as a list
specieslist = MyGasMech.species_symbols
# list of gas species of interest
plotspeclist = ["CH4", "O2", "N2"]
# find elemental compositions of selected species
print(" ")
for s in plotspeclist:
    species_index = MyGasMech.get_specindex(s)
    print("species " + specieslist[species_index])
    print("elemental composition")
    for elem_index in range(MyGasMech.mm):
        num_elem = MyGasMech.species_composition(elem_index, species_index)
        print(f"    {elelist[elem_index]:>4}: {num_elem:2d}")
    print("=" * 10)
print()
```

### Plot selected species properties

Calculate and plot the properties of $CH_4$, $O_2$, and $N_2$ against the temperature. Use these property methods:

- `species_cv`: Species specific heat capacity at constant volume [erg/mol-K]
- `SpeciesCond`: Species thermal conductivity [erg/cm-K-sec]
- `SpeciesDiffusionCoeffs`: Binary diffusion coefficients between species pairs [$cm^2$/sec]

```
# plot Cv and thermal conductivity values at different temperatures for
# selected gas species
#
plt.figure(figsize=(12, 6))
# temperature increment
dtemp = 20.0
# number of property data points
points = 100
# curve attributes
curvelist = ["g", "b--", "r:"]
```

### Prepare the species Cv data for plotting

```
# create arrays
# specify specific heat capacity at constant volume data
cv = np.zeros(points, dtype=np.double)
# temperature data
t = np.zeros(points, dtype=np.double)
# start plotting loop #1
k = 0
# loop over selected gas species
for s in plotspeclist:
    # starting temperature at 300 [K]
    temp = 300.0
    # loop over temperature data points
    for i in range(points):
        heatcapacity = MyGasMech.species_cv(temp)
        index = MyGasMech.get_specindex(s)
        t[i] = temp
        # convert ergs to joules
        cv[i] = heatcapacity[index] / ck.ERGS_PER_JOULE
        temp += dtemp
    plt.subplot(121)
    plt.plot(t, cv, curvelist[k])
    k += 1
# plot Cv versus temperature
plt.xlabel("Temperature [K]")
plt.ylabel("Cv [J/mol-K]")
plt.legend(plotspeclist, loc="upper left")
```

### Prepare the species thermal conductivity data for plotting

```
# create arrays
# specify conductivity
```

(continues on next page)

(continued from previous page)

```
kappa = np.zeros(points, dtype=np.double)
# start plotting loop #2
k = 0
# loop over selected gas species
for s in plotspeclist:
    # starting temperature at 300 [K]
    temp = 300.0
    # loop over temperature data points
    for i in range(points):
        conductivity = MyGasMech.species_cond(temp)
        index = MyGasMech.get_specindex(s)
        t[i] = temp
        # convert ergs to joules
        kappa[i] = conductivity[index] / ck.ERGS_PER_JOULE
        temp += dtemp
    plt.subplot(122)
    plt.plot(t, kappa, curvelist[k])
    k += 1
# plot conductivity versus temperature
plt.xlabel("Temperature [K]")
plt.ylabel("Conductivity [J/cm-K-sec]")
plt.legend(plotspeclist, loc="upper left")
```

### Evaluate the binary diffusion coefficients between different gas species

Use the `SpeciesDiffusionCoeffs()` method to calculate the binary diffusion coefficients between pairs of gas species. Here, the binary diffusion coefficients are evaluated at 2 [atm] and 500 [K]. The binary diffusion coefficient between $CH_4$ and $O_2$ is shown.

```
diffcoef = MyGasMech.species_diffusioncoeffs(2.0 * ck.P_ATM, 500.0)
id1 = MyGasMech.get_specindex(plotspeclist[0])
id2 = MyGasMech.get_specindex(plotspeclist[1])
c = diffcoef[id1][id2]
print(
    f"Diffusion coefficient for {plotspeclist[0]}"
    f"against {plotspeclist[1]} is {c:e} [cm^2/sec]."
)

# clean up
ck.done()

# plot results
if interactive:
    plt.show()
else:
    plt.savefig("plot_species_properties.png", bbox_inches="tight")
```

## 22.2 Mixture

*Mixture* is the "core" token in **PyChemkin**. It represents a gas-phase mixture by storing its pressure, temperature, and species composition. *Stream* is an extended "Class" of *Mixture* with the additional attribute of "mass/volumetric flow rate". *Mixture* and *Stream* include utilities that can be used to evaluate the mixture properties (density, mixture

enthalpy, mixture viscosity, ...) and the reaction rates. There are also methods to manipulate the *Mixture*/*Stream* such as merging two mixtures adiabatically.

The examples demonstrate how to instantiate the *Mixture*/*Stream* objects, how to extract information and properties of a *Mixture*/*Stream*, and how to manipulate *Mixture*/*Stream* objects.

### 22.2.1 Estimate the adiabatic flame temperature of a gas mixture

This example shows how to find the equilibrium state of a mixture. It uses the `equilibrium()` method with the `constant pressure and enthalpy` option to estimate the adiabatic flame temperature of a methane-oxygen mixture. This example also explores the influence of the equivalence ratio on the predicted adiabatic flame temperature.

#### Import PyChemkin packages and start the logger

```
from pathlib import Path

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core.logger import logger

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

#### Create a chemistry set

The first mechanism to load is the GRI 3.0 mechanism for methane combustion. This mechanism and its associated data files come with the standard Ansys Chemkin installation in the `/reaction/data` directory.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir

# create a chemistry set based on the GRI 3.0 methane combustion mechanism
MyGasMech = ck.Chemistry(label="GRI 3.0")
# set mechanism input files
# including the full file path is recommended
MyGasMech.chemfile = str(mechanism_dir / "grimech30_chem.inp")
MyGasMech.thermfile = str(mechanism_dir / "grimech30_thermo.dat")
# skip the transport data file where is not needed by this example
```

### Preprocess the chemistry set

```
# preprocess the mechanism files
_ = MyGasMech.preprocess()
```

### Set up gas mixtures

Set up gas mixtures based on the species in this chemistry set. PyChemkin has a few methods for creating a gas mixture. Here, the equivalence ratio method is used to set up the combustible mixture so that you can easily change the mixture composition by assigning a different equivalence ratio value.

```
# create an "oxid" mixture associated with the 'MyGasMech' chemistry set
oxid = ck.Mixture(MyGasMech)
# use a "recipe" to set the mole fractions of the mixture
# the "oxid" mixture consists of 100% O2
oxid.x = [("O2", 1.0)]
oxid.temperature = 295.15  # [K]
oxid.pressure = ck.P_ATM  # 1 atm

# create the "fuel" mixture
fuel = ck.Mixture(MyGasMech)
# set the "fuel" molar composition to 100% CH4
fuel.x = [("CH4", 1.0)]
fuel.temperature = oxid.temperature
fuel.pressure = oxid.pressure

# create the final fuel-oxidizer mixture
mixture = ck.Mixture(MyGasMech)
mixture.pressure = oxid.pressure
mixture.temperature = oxid.temperature

# the use of equivalence ratio requires the definition of the complete combustion
# products of the fuel-oxidizer pair:
# CH4 + 2O2 => CO2 + 2H2O
products = ["CO2", "H2O"]

# create an array to specify the composition of the additives to
# the fuel-oxidizer mixture For example, a diluent such as argon or
# helium might be added to the fuel-oxidizer mixture Use an all-zero array
# if there is no additive
add_frac = np.zeros(MyGasMech.kk, dtype=np.double)
```

### Set up the parameter study

Set up a parameter study to find out the impact of the equivalence ratio on the adiabatic flame temperature of the fuel-oxidizer mixture. The equivalence ratio varies from 0.5 to 1.6 with an increment of 0.1.

```
points = 12
deq = 0.1
equiv_ini = 0.5

# create the solution arrays as double arrays
t = np.zeros(points, dtype=np.double)
equiv = np.zeros_like(t, dtype=np.double)
```

### Run the parameter study

Use the `equilibrium()` method to estimate the adiabatic flame temperature of the fuel-oxidizer mixture. Choose the option corresponding to constant pressure and constant enthalpy for the equilibrium calculation because you are finding the adiabatic temperature.

The `equilibrium()` method returns a mixture object representing the mixture at the equilibrium state. Use the `temperature()` method to get the equilibrium temperature. To see all available options for this method, use the `ck.help(topic="equilibrium")` method.

This example uses the `x_by_equivalence_ratio()` method to set the fuel-oxidizer composition with the given equivalence ratios because the composition of both the fuel and oxidizer mixtures is specified in mole fractions.

```
for i in range(points):
    # set the current mixture equivalence ratio
    equiv_current = equiv_ini

    # create the fuel-oxidizer mixture with the given equivalence ratio
    ierror = mixture.x_by_equivalence_ratio(
        MyGasMech, fuel.x, oxid.x, add_frac, products, equivalenceratio=equiv_current
    )
    # check fuel-oxidizer mixture creation status
    if ierror != 0:
        print("Error: Failed to create the fuel-oxidizer mixture.")
        print(f"       Equivalence ratio = {equiv_current}.")
        exit()

    # use "equilibrium()" method to calculate the gas mixture at the equilibrium state
    # Option #5 ("opt=5") corresponds to the constant pressure and constant-enthalpy
    # constraints of the equilibrium state
    # "EQ_mixture" is the gas mixture at the equilibrium state
    EQ_mixture = ck.equilibrium(mixture, opt=5)

    # save the results to the solution arrays
    # use "temperature" to obtain the temperature of the equilibrium state
    t[i] = EQ_mixture.temperature
    equiv[i] = equiv_current
    equiv_ini = equiv_ini + deq
```

### Plot the result from the parameter study

When you plot the result from the parameter study, the adiabatic flame temperature should exhibit a peak at around the stoichiometric, that is, equivalence ratio = 1.

```
# plot equilibrium/adiabatic temperatures against mixture equivalence ratios
plt.plot(equiv, t, "bs--")
# set up axis labels
plt.xlabel("Equivalence ratio")
plt.ylabel("Temperature [K]")

# clean up
ck.done()

# display or save the plot
if interactive:
```

(continues on next page)

(continued from previous page)

```
    plt.show()
else:
    plt.savefig("plot_adiabatic_flame_temperature.png", bbox_inches="tight")
```

### 22.2.2 Create a mixture

A *mixture* is a core component of the PyChemkin framework. In addition to getting mixture thermodynamic and transport properties, such as density, heat capacity, and viscosity, you can combine two mixtures, find the equilibrium state of a mixture, or use a mixture to define the initial state of a reactor. A PyChemkin *reactor model* is a black box that transforms a mixture from its initial state to a new one.

The following schematic shows the basic operations available for a mixture in PyChemkin: **create**, **combine/mix**, and **transform** (by a reactor model).

examples/mixture/mixture_concept.png

This example shows different ways to create a mixture in PyChemkin. The use of the composition *recipe* lets you provide just the non-zero species components with a list of species-fraction pairing tuples. Alternatively, the NumPy array lets you use a *full-size* (equal to the number of species) mole/mass fraction array to specify the mixture composition. The `equivalence ratio()` method creates a new mixture from predefined fuel and oxidizer mixtures by assigning an equivalence ratio value.

#### Import PyChemkin packages and start the logger

```
import copy
from pathlib import Path

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core.logger import logger

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create a chemistry set

The mechanism to load is the C2 NOx mechanism. This mechanism file and the associated data files come with the standard Ansys Chemkin installation in the `/reaction/data` directory. The C2 NOx mechanism file, in addition to the reactions, contains the thermodynamic and transport data of all species in the mechanism. In this case, you only need to specify the mechanism file, that is, `chemfile`. If your simulation requires the transport properties, you must use the `preprocess_transportdata()` method to tell the PyChemkin preprocessor to also include the transport data. The preprocessor does not include the transport data by default.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir
# create a chemistry set based on the C2 NOx mechanism
MyGasMech = ck.Chemistry(label="C2 NOx")
# set mechanism input files
# this mechanism file contains all the necessary thermodynamic and transport data
# thus, there is no need to specify thermodynamic and the transport data files
MyGasMech.chemfile = str(mechanism_dir / "C2_NOx_SRK.inp")

# direct the preprocessor to include the transport properties
# (when the tran data file is not provided)
MyGasMech.preprocess_transportdata()
```

### Preprocess the C2 NOx chemistry set

The C2 NOx mechanism includes information about the *Soave* cubic Equation of State (EOS) for real-gas applications. The preprocessor indicates the availability of the real-gas model in the chemistry set processed. For example, during preprocessing, you see this print out: `real-gas cubic EOS 'Soave' is available`.

```
# preprocess the mechanism files
_ = MyGasMech.preprocess()
```

### Set up gas mixtures based on the species in the C2 NOx chemistry set

Create a gas mixture instance named `premixed` based on the `My2ndMech` chemistry set.

```
premixed = ck.Mixture(MyGasMech)
# set mixture pressure [dynes/cm2]
mixpressure = 2.0  # given in atm
# convert to dynes/cm2
premixed.pressure = mixpressure * ck.P_ATM
# set mixture temperature [K]
premixed.temperature = 500.0
# create a recipe for the molar composition of the mixture
mixture_recipe = [("CH4", 0.08), ("N2", 0.6), ("O2", 0.2), ("H2O", 0.12)]
# set mixture mole fractions
premixed.x = mixture_recipe
```

### Find mixture mean molecular mass

Use the `wtm()` method to get the mean molar mass of the gas mixture.

```
print(f"Mean molecular mass = {premixed.wtm:f} gm/mole")
print("=" * 40)
```

### List the mixture composition

Use the `Y()` method to automatically convert the mole fractions to mass fractions and vice versa. Use the `list_composition()` method to display only the non-zero components of the gas mixture.

```
print("mixture mass fractions (raw data):")
print(str(premixed.y))
# switch back to mole fractions
print("\nmixture mole fractions (raw data):")
print(str(premixed.x))
# beautify the composition list
print("\nformatted mixture composition output:")
print("=" * 40)
premixed.list_composition(mode="mole")
print("=" * 40)
```

### Create a hard copy of a mixture

Create a hard copy of the premixed mixture. (Soft copying, that is, using `anotherpremixed = premixed`, also works.)

```
anotherpremixed = copy.deepcopy(premixed)
# display the molar composition of the new mixture
print("\nFormatted mixture composition of the copied mixture")
anotherpremixed.list_composition(mode="mole")
print("=" * 40)
```

### Compute and plot mixture properties

The PyChemkin `Mixture` module offers many basic methods to compute the thermodynamic and transport properties of a species and a gas mixture. The following code plots selected mixture properties as a function of the mixture temperature. It uses the `RHO()` and `HML()` methods to get the mixture density and mixture enthalpy, respectively. Use the `mixture_viscosity()` method to get the mixture transport property, `viscosity`. Use the `mixture_diffusion_coeffs()` method to get the mixture-averaged diffusion coefficient of $CH_4$. The temperature and pressure are required to compute the properties.

```
plt.subplots(2, 2, sharex="col", figsize=(9, 9))
dtemp = 20.0
points = 100
# curve attributes
curvelist = ["g", "b--", "r:"]
# list of pressure values for the plot
press = [1.0, 5.0, 10.0]  # given in atm
# create arrays for the plot
# mixture density
rho = np.zeros(points, dtype=np.double)
# mixture enthalpy
enthalpy = np.zeros_like(rho, dtype=np.double)
# mixture viscosity
visc = np.zeros_like(rho, dtype=np.double)
# mixture averaged diffusion coefficient of CH4
diff_ch4 = np.zeros_like(rho, dtype=np.double)
# species index of CH4
ch4_index = MyGasMech.get_specindex("CH4")
```

(continues on next page)

(continued from previous page)

```
# temperature data
t = np.zeros_like(rho, dtype=np.double)
# start the plotting of loop #1
k = 0
# loop over the pressure values
for j in range(len(press)):
    # starting temperature at 300K
    temp = 300.0
    # loop over temperature data points
    for i in range(points):
        # set mixture pressure [dynes/cm2]
        premixed.pressure = press[j] * ck.P_ATM
        # set mixture temperature [K]
        premixed.temperature = temp
        # get mixture density [gm/cm3]
        rho[i] = premixed.rho
        # get mixture enthalpy [ergs/mol] and convert it to [kJ/mol]
        enthalpy[i] = premixed.hml() * 1.0e-3 / ck.ERGS_PER_JOULE
        # get mixture viscosity [gm/cm-sec]
        visc[i] = premixed.mixture_viscosity()
        # get mixture-averaged diffusion coefficient of CH4 [cm2/sec]
        diffcoeffs = premixed.mixture_diffusion_coeffs()
        diff_ch4[i] = diffcoeffs[ch4_index]
        t[i] = temp
        temp += dtemp

    # create subplots
    # plot mixture density versus temperature
    plt.subplot(221)
    plt.plot(t, rho, curvelist[k])
    plt.ylabel("Density [g/cm3]")
    plt.legend(("1 atm", "5 atm", "10 atm"), loc="upper right")
    # plot mixture enthalpy versus temperature
    plt.subplot(222)
    plt.plot(t, enthalpy, curvelist[k])
    plt.ylabel("Enthalpy [kJ/mole]")
    plt.legend(("1 atm", "5 atm", "10 atm"), loc="upper left")
    # plot mixture viscosity versus temperature
    plt.subplot(223)
    plt.plot(t, visc, curvelist[k])
    plt.xlabel("Temperature [K]")
    plt.ylabel("Viscosity [g/cm-sec]")
    plt.legend(("1 atm", "5 atm", "10 atm"), loc="lower right")
    # plot mixture averaged CH4 diffusion coefficient versus temperature
    plt.subplot(224)
    plt.plot(t, diff_ch4, curvelist[k])
    plt.xlabel("Temperature [K]")
    plt.ylabel(r"$D_{CH_4}$ [cm2/sec]")
    plt.legend(("1 atm", "5 atm", "10 atm"), loc="upper left")
    k += 1

# plot legends
```

(continues on next page)

(continued from previous page)

```
plt.legend(("1 atm", "5 atm", "10 atm"), loc="upper left")

# clean up
ck.done()

# plot results
if interactive:
    plt.show()
else:
    plt.savefig("plot_create_mixture.png", bbox_inches="tight")
```

### 22.2.3 Calculate the detonation wave speed of a real-gas mixture

Use the `detonation()` method on a combustible mixture to compute the Chapman-Jouguet (C-J) state and detonation wave speed. This example shows how to predict the detonation wave speeds of a natural gas-air mixture at various initial pressures and compare the *ideal-gas* and the *real-gas* results against the experimental data.

#### Import PyChemkin packages and start the logger

```
from pathlib import Path

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core.logger import logger

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

#### Create a chemistry set

The `C2 NOx` mechanism comes with the standard Ansys Chemkin installation in the `/reaction/data` directory. This mechanism includes information about the *Soave* cubic Equation of State (EOS) for the real-gas applications. The PyChemkin preprocessor indicates the availability of the real-gas model in the chemistry set processed.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir
# create a chemistry set based on C2 NOx using an alternative method
MyMech = ck.Chemistry(label="C2 NOx")
```

(continues on next page)

(continued from previous page)

```
# set mechanism input files individually
# Because this mechanism file contains all the necessary thermodynamic and
# transport data, you do not need to specify thermodynamic and
# transport data files.
MyMech.chemfile = str(mechanism_dir / "C2_NOx_SRK.inp")
```

### Preprocess the C2 NOx chemistry set

During preprocessing, you should see this printed: `real-gas cubic EOS 'Soave' is available`. Because no transport data file is provided and the `preprocess_transportdata()` method is not used, transport property methods are not available in this project.

```
# preprocess the mechanism files
_ = MyMech.preprocess()
```

### Set up gas mixtures based on the species in the C2 NOx chemistry set

Create gas mixtures named `fuel` (natural gas) and `air` based on `MyMech`. Then, use these two mixtures to form the combustible `premixed` mixture for the detonation calculations. Use the `x_by_equivalence_ratio()` method to set the equivalence ratio of the fuel-air mixture to 1.

```
# create the fuel mixture
fuel = ck.Mixture(MyMech)
# set mole fraction
fuel.x = [("CH4", 0.8), ("C2H6", 0.2)]
fuel.temperature = 290.0
fuel.pressure = 40.0 * ck.P_ATM
# create the air mixture
air = ck.Mixture(MyMech)
# set mass fraction
air.x = [("O2", 0.21), ("N2", 0.79)]
air.temperature = fuel.temperature
air.pressure = fuel.pressure
# create the initial mixture
# create the premixed mixture to be defined by equivalence ratio
premixed = ck.Mixture(MyMech)
# products from the complete combustion of the fuel mixture and air
products = ["CO2", "H2O", "N2"]
# species mole fractions of added/inert mixture.
# Can also create an additives mixture here.
add_frac = np.zeros(MyMech.kk, dtype=np.double)  # no additives: all zeros

ierror = premixed.x_by_equivalence_ratio(
    MyMech, fuel.x, air.x, add_frac, products, equivalenceratio=1.0
)
# check fuel-oxidizer mixture creation status
if ierror != 0:
    print("Error: Failed to create the premixed mixture.")
    exit()
```

### Display the molar composition of the premixed mixture

List the composition of the premixed mixture for verification.

```
premixed.list_composition(mode="mole")
```

### Run the detonation calculation

Use the `detonation()` method to find the C-J state and detonation wave speed of the fuel-air mixture with the initial mixture pressure increasing from 40 to 80 [atm]. The initial mixture temperature is kept at 290 [K].

The `detonation()` method returns two objects: a `speed` tuple containing the speed of sound and the detonation wave speed at the C-J state. The `CJState` mixture contains the mixture properties at the C-J state. For instance, you can use `CJState.pressure` to get the mixture pressure at the C-J state.

**Note**

- By default, Chemkin variables are in cgs units.
- You can enter the `ansys.chemkin.core.help("equilibrium")` command at the Python prompt to see the input and output parameters of the `detonation()` method.

### Run the parameter study

Set up the parameter study of the detonation wave speed with respect to the initial pressure. The predicted detonation wave speed values are saved in the `Det` array. The experimental data is stored in the `Det_data` array. By default, the *ideal-gas law* is assumed. You can use the `use_realgas_cubicEOS()` method to turn on the *real-gas model* if the mechanism contains the real-gas parameters in the EOS block. Use the `use_idealgas_law()` method to reactivate the ideal-gas law assumption.

```
points = 5
dpres = 10.0 * ck.P_ATM
pres = fuel.pressure
p = np.zeros(points, dtype=np.double)
det = np.zeros_like(p, dtype=np.double)
premixed.pressure = pres
premixed.temperature = fuel.temperature

# start of pressure loop
for i in range(points):
    # compute the C-J state corresponding to the initial mixture
    speed, cj_state = ck.detonation(premixed)
    # update plot data
    # convert pressure to atm
    p[i] = pres / ck.P_ATM
    # convert speed to m/sec
    det[i] = speed[1] / 1.0e2
    # update pressure value
    pres += dpres
    premixed.pressure = pres

# create plot for ideal-gas results
plt.plot(p, det, "bo--", label="ideal gas", markersize=5, fillstyle="none")
```

### Switch to the real-gas EOS model

Use the `use_realgas_cubicEOS()` method to turn on the real-gas EOS model. You can enter `ansys.chemkin.core.help("real gas")` to see usage information on real-gas models or `ansys.chemkin.core.help("manuals")` to access the online *Chemkin Theory* manual for descriptions of the real-gas EOS models.

**Note**

By default, the *Van der Waals* mixing rule is applied to evaluate thermodynamic properties of a real-gas mixture. You can use the `set_realgas_mixing_rule()` method to switch to a different mixing rule.

```
# turn on real-gas cubic equation of state
premixed.use_realgas_cubic_eos()
# set mixture mixing rule to Van der Waals (default)
# premixed.set_realgas_mixing_rule(rule=0)
# restart the calculation with real-gas EOS
premixed.pressure = fuel.pressure
pres = fuel.pressure
p[:] = 0.0e0
det[:] = 0.0e0
# set verbose mode to false to turn off extra printouts
ck.set_verbose(False)
# start of pressure loop
for i in range(points):
    # compute the C-J state corresponding to the initial mixture
    speed, cj_state = ck.detonation(premixed)
    # update plot data
    p[i] = pres / ck.P_ATM
    det[i] = speed[1] / 1.0e2
    # update pressure value
    pres += dpres
    premixed.pressure = pres

# stop Chemkin
ck.done()

# create plot for real-gas results
plt.plot(p, det, "r^-", label="real gas", markersize=5, fillstyle="none")
# plot data
p_data = [44.1, 50.6, 67.2, 80.8]
det_data = [1950.0, 1970.0, 2000.0, 2020.0]
plt.plot(p_data, det_data, "gD:", label="data", markersize=4)
```

### Plot the result from the parameter study

You should see that the ideal-gas assumption fails to show any noticeable pressure influence on the detonation wave speeds. Because of the relatively high pressures in this study, you can observe significant differences in the predicted detonation wave speeds between the ideal-gas and real-gas models.

```
plt.legend(loc="upper left")
plt.xlabel("Pressure [atm]")
plt.ylabel("Detonation wave speed [m/sec]")
```

(continues on next page)

(continued from previous page)

```
plt.suptitle("Natural Gas/Air Detonation", fontsize=16)

# plot results
if interactive:
    plt.show()
else:
    plt.savefig("plot_detonation.png", bbox_inches="tight")
```

### 22.2.4 Estimate the steady-state NO emission level of a complete burned fuel-air mixture

This example shows how to quickly estimate the steady-state NO level that is formed by the combustion of a fuel-air mixture at a given temperature. Without needing any reaction, the NO concentration gets to its steady state (or the maximum level) when the product mixture from the fuel-air combustion reaches the equilibrium state at the given temperature. To find the equilibrium state of the fresh fuel-air mixture, the `equilibrium()` method is used with the `fixed pressure` and `fixed temperature` options.

This example explores the influence of temperature on the predicted adiabatic flame temperature. Knowing that nitrogen oxides (NOx) are stable at high temperatures (> 2000 [K]), you can expect that the steady-state NO level should increase sharply when the temperature gets high enough.

```

```

#### Import PyChemkin packages and start the logger

```
from pathlib import Path

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core.logger import logger

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

#### Create a chemistry set

The first mechanism to load is the GRI 3.0 mechanism for methane combustion. This mechanism and its associated data files come with the standard Ansys Chemkin installation in the `/reaction/data` directory.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir
```

(continues on next page)

(continued from previous page)

```
# create a chemistry set based on GRI 3.0
MyGasMech = ck.Chemistry(label="GRI 3.0")
# set mechanism input files
# including the full file path is recommended
MyGasMech.chemfile = str(mechanism_dir / "grimech30_chem.inp")
MyGasMech.thermfile = str(mechanism_dir / "grimech30_thermo.dat")
# transport data is not needed
```

### Preprocess the chemistry set

```
# preprocess the mechanism files
_ = MyGasMech.preprocess()
```

### Set up gas mixtures

Set up gas mixtures based on the species in this chemistry set. PyChemkin provides a few methods for creating a gas mixture. Here, the `isothermal_mixing()` method is used to create a `fuel-air` mixture by mixing the `fuel` and `air` mixtures with a predetermined air/fuel rate.

### Create a fuel mixture

Create a fuel mixture of 80% methane and 20% hydrogen.

```
fuel = ck.Mixture(MyGasMech)
# set mole fraction
fuel.x = [("CH4", 0.8), ("H2", 0.2)]
fuel.temperature = 300.0
fuel.pressure = ck.P_ATM  # 1 atm
```

### Create an air mixture

Create an air mixture of oxygen and nitrogen.

```
air = ck.Mixture(MyGasMech)
# set mass fraction
air.y = [("O2", 0.23), ("N2", 0.77)]
air.temperature = 300.0
air.pressure = ck.P_ATM  # 1 atm
```

### Create a fuel-air mixture by mixing

Use the `isothermal_mixing()` method to mix the `fuel` and `air` mixtures created earlier. Define the mixing formula using `mixture_recipe` with this mass ratio: `fuel:air=1.00:17.19`. Use the `finaltemperature` parameter to set the temperature of the new `premixed` mixture to 300 [K]. Set `mode="mass"` because the ratios given in `mixture_recipe` are mass ratios.

```
mixture_recipe = [(fuel, 1.0), (air, 17.19)]
# create the new mixture (the air-fuel ratio is by mass)
premixed = ck.isothermal_mixing(
    recipe=mixture_recipe, mode="mass", finaltemperature=300.0
)
```

#### Find the equilibrium composition at different temperatures

Perform the equilibrium calculation with fixed pressure and fixed temperature. The NO mole fraction of the equilibrium state at each temperature is stored in an array. The gas temperature is increased from 500 to 2480 [K].

```
# NO species index
no_index = MyGasMech.get_specindex("NO")

# set up plotting temperatures
temp = 500.0
dtemp = 20.0
points = 100
# curve
t = np.zeros(points, dtype=np.double)
no = np.zeros_like(t, dtype=np.double)
# start the temperature loop
for k in range(points):
    # reset mixture temperature
    premixed.temperature = temp
    # find the equilibrium state mixture at the given mixture temperature and pressure
    eqstate = ck.equilibrium(premixed, opt=1)
    #
    no[k] = eqstate.x[no_index] * 1.0e6  # convert to ppm
    t[k] = temp
    temp += dtemp


##################
# Plot the results
# ================
# Plot the equilibrium NO mole fractions versus the temperatures
# of the fuel-air mixtures.

plt.plot(t, no, "bs--", markersize=3, markevery=4)
plt.xlabel("Temperature [K]")
plt.ylabel("NO [ppm]")

# clean up
ck.done()

# plot results
if interactive:
    plt.show()
else:
    plt.savefig("plot_equilibrium_composition.png", bbox_inches="tight")
```

### 22.2.5 Calculate the heating values of fuel mixtures

One of the advantages of PyChemkin is flexibility. You can establish your own workflow with PyChemkin to meet your simulation goals. This tutorial is an example of building a specialized algorithm to calculate the *heating values*, the lower heating value (LHV) and the higher heating value (HHV), of fuel mixtures.

The **heating value** is the net heat release from the complete combustion of a hydrocarbon (HC) in oxygen at the standard state condition (298.15 [K] and 1 [atm]). The combustion products are mainly $CO_2$ and $H_2O$, and are assumed to be cooled back to the standard state condition in the heating value calculation. The difference between the lower heating

value and the higher heating value is the final form of the $H_2O$; the water is in *gas phase* for the *lower* heating value, and in *liquid phase* for the *higher* heating value. You can see that the higher heating value can be obtained by adding the water *heat of vaporization* to the lower heating value. Thus, the workflow for calculating the heating values of any HC fuels consists of two main steps:

1. Calculating the water heat of vaporization at the standard state condition: this can be done by getting the value from a well trusted database or by using the **chemkin** trick described below.

2. Calculating the heat of combustion of the fuel mixture in pure oxygen: here you will use the `find_equilibrium` method with the *fixed pressure* and *fixed temperature* option (the default setting).

The lower heating value is the enthalpy different between the fresh fuel and oxygen mixture and the final product mixture. Adding the heat of vaporization to the lower heating value, and you get the higher heating value.

In this tutorial, you will compute the heating values of some pure fuel species such as methane and n-butane as well as some fuel mixtures such as PRF RON 80 and biodiesel. You can compare the values you get here with the known values from a trusty database.

### Import PyChemkin package and start the logger

```
from pathlib import Path

import numpy as np  # number crunching

import ansys.chemkin.core as ck
from ansys.chemkin.core import Color
from ansys.chemkin.core.logger import logger
from ansys.chemkin.core.utilities import find_file

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
logger.debug("data directory: " + str(data_dir))
# set pressure & temperature condition of the standard state
thispressure = ck.P_ATM
thistemperature = 298.15
```

### Calculate the heat of vaporization

Because the thermodynamic data (mainly the enthalpy) of both the vapor and the liquid water are available in the standard thermodynamic data file `therm.dat` that comes with the Ansys Chemkin in the *reaction/data* directory, you can, in theory, compute the water heat of vaporization at a given temperature by finding the enthalpy difference between the water vapor and its liquid counterpart. In the `getwaterheatofvaporization` method, a mechanism with just the water vapor `H2O` and the liquid water `H2O(L)` is created in situ. Simply find and return the enthalpy difference of the two species by using the `SpecisH` method after preprocessing the `WaterMech Chemistry Set`.

```
def getwaterheatofvaporization(temp: float) -> float:
    """Compute water heat of vaporization [erg/g-water] at the given temperature."""
    """
    Use the enthalpy difference between water vapor and liquid water
    at the temperature. Enthalpy data depend on temperature only
```

(continues on next page)

(continued from previous page)

```
    There are empirical formulas for heat of vaporization, for example,
    DIPPR EQ.

    Parameters
    ----------
        temp: double scalar
            water temperature [K]

    Returns
    -------
        enthalpy: double
            water enthalpy of vaporization [erg/g-water]

    """
    # compute water heat of vaporization
    # create a chemistry set object
    watermech = ck.Chemistry(label="Water Only")
    #
    # create a new mechanism input file
    #
    global current_dir
    waterfile = Path(current_dir) / "water_chem.inp"
    w = Path.open(waterfile, "w")
    # the water mechanism contains two species:
    # water vapor (H2O) and liquid water (H2O(L))
    # declare elements
    w.write("ELEMENT H O END\n")
    w.write("SPECIES\n")
    w.write("H2O   H2O(L)\n")
    w.write("END\n")
    # no reaction is needed for thermodynamic property calculation
    w.write("REACTION\n")
    w.write("END\n")
    # close the mechnaism file
    w.close()
    # set mechanism input files
    # including the full file path is recommended
    watermech.chemfile = str(waterfile)
    watermech.thermfile = str(data_dir / "therm.dat")
    # pre-process
    ierror = watermech.preprocess()
    if ierror != 0:
        return 0.0
    # get species enthalpies [erg/mol] at 298.15 [K]
    waterenthalpies = watermech.species_h(thistemperature)
    # compute heat of vaporization of water [erg/g-water]
    # H_water_vapor - H_liquid_water
    heatvaporization = (waterenthalpies[0] - waterenthalpies[1]) / watermech.wt[0]
    # remove the temporary water mechanism file
    Path(waterfile).unlink()
    return heatvaporization
```

### Create your own fuel library

You can create a fuel "library" mechanism that contains typical fuel species, oxygen, and the complete combustion products o(carbon dioxide and water *vapor*) only. No reaction is needed for this purpose as you will perform equilibrium calculation to get the complete combustion product mixture.

**Warning**

It is recommended **not** to include any intermediate species such as CH3 or OH in the fuel library as these species might interfere with the equilibrium calculation used to determine the complete combustion products.

**Note**

The thermodynamic data file `Gasoline-Diesel-Biodiesel_PAH_NOx_therm_MFL2024.dat` should have the property data of most commonly seen fuel species. This encrypted data file is developed under the *Model Fuel Library* project and is available in the *reaction/data/ModelFuelLibrary*.

### Instantiate the fuel `Chemistry Set`

Create the chemistry set object `MyGasMech`. The mechanism input file `fuel_chem.inp` will be created below.

**Note**

You can keep this file for later uses, for example, by adding more fuel species. (remember to remove the``remove(mymechfile))`` command at the end of this project).

```
MyGasMech = ck.Chemistry(label="EQ")
```

### Create the fuel mechanism file

create a new mechanism input file

```
mymechfile = Path(current_dir) / "fuels_chem.inp"
m = Path.open(mymechfile, "w")
# the mechanism contains only the necessary species
# (fuel, oxygen, and major combustion products)
# declare elements
m.write("ELEMENT c h o END\n")
# declare species
# ch4: Methane                  c4h10: n-Butane
# nc5h12: n-Pentane             nc7h16: n-Heptane
# ic8h18: iso-Octane            nc9h20: n-Nonane
# nc10h22: n-Decane             hmn: Heptamethylnonane
# c6h5ch3: Toluene              c6h5c2h5: Ethylbenzene
# chx: Cyclohexane              mch: Methylcyclohexane
# decalin: Decalin              ch3oh: Methanol
# c2h5oh: Alcohol               ch3och3: Dimethyl ether (DME)
# nc4h9oh: n-Butanol            mb: Mythyl butanoate
# md: Methyl decanoate          mhd: Methyl stearate
m.write("SPECIES\n")
```

(continues on next page)

(continued from previous page)

```
m.write("ch4 c4h10 nc5h12 nc7h16 ic8h18 nc9h20 nc10h22\n")
m.write("hmn c6h5ch3 c6h5c2h5 chx mch decalin\n")
m.write("ch3oh c2h5oh ch3och3 nc4h9oh\n")
m.write("mb md mhd\n")
m.write("o2 co2 h2o\n")
m.write("END\n")
# no reaction is needed for equilibrium calculation
m.write("REACTION\n")
m.write("END\n")
# close the mechnaism file
m.close()
#
# set mechanism input files
# including the full file path is recommended
# note that the "year" in thermodynamic data file name could be different.
# it's MFL2023 for Ansys Chemkin 2025R1, MFL2024 for 2025R2, ...
MyGasMech.chemfile = str(mymechfile)
therm_dir = str(data_dir / "ModelFuelLibrary" / "Full")
MyGasMech.thermfile = find_file(
    therm_dir,
    "Gasoline-Diesel-Biodiesel_PAH_NOx_therm_MFL",
    "dat",
)
```

### Pre-process the fuel `Chemistry Set`

preprocess the fuel mechanism "MyGasMech`` just created.

```
ierror = MyGasMech.preprocess()
if ierror == 0:
    print(Color.GREEN + ">>> preprocess OK", end=Color.END)
else:
    print(Color.RED + ">>> preprocess failed!", end=Color.END)
    exit()
```

### Find the water heat of vaporization

Set the fresh fuel-oygen mixture pressure & temperature to the standard state condition for the heating value calculations. Since the difference between the *lower heating value (LHV)* and the *higher heating value (HHV)* is the final form of the water. You will calculate the LHV first and get the HHV by adding the water heat of vaporization to the LHV. Use the `get_specindex` method to find the species index of water `MyGasMech`.

**Note**

The water heat of vaporization is computed by the `getwaterheatofvaporization` method you created earlier in this project. Reactivate `MyGasMech` (`MyGasMech.activate`) after calling `getwaterheatofvaporization` because another `Chemistry Set` `WaterMech` is used there.

```
# find the index for water vapor
watervapor_index = MyGasMech.get_specindex("h2o")
```

(continues on next page)

(continued from previous page)

```
# water heat of vaporization [erg/g-water] at 298.15 [K]
# either call this method before creating the current Chemistry Set
# or use the activate method to switch back to the current Chemistry Set
# after the call
heatvaporization = getwaterheatofvaporization(thistemperature)

# switch back to MyGasMech
MyGasMech.activate()
```

### Set up the unburned fuel-oxygen mixture

Set up the *unburned* "fuel-oxygen" mixture for the heating value calculations. Here the `x_by_equivalence_ratio` method with $\phi = 1$ is employed to generate a stoichiometric fuel-oxygen mixture. The advantage of using this method is that you do not need to calculate the "fuel-oxygen" composition for every fuel mixture. You can use a *lean* "fuel-oxygen" mixture, too, because the standard heat of formation of $O_2$ is zero by definition.

Here you will compute the heating values of four fuel mixtures: $CH_4$, $C_4H_{10}$,PRF 80 (20% $C_7H_{16}$ + 80% $C_8H_{18}$), and a mock-up biodiesel mixture (90% $C_{19}H_{38}O_2$ + 10% $CH_3OH$).

```
# prepare the fuel mixtures
fuel = ck.Mixture(MyGasMech)
fuel.pressure = thispressure
fuel.temperature = thistemperature
# list of fuel compositions (mole/volume fractions) of which the heating values
# will be computed.
# [Methane, n-Butane, PRF RON 80, biodiesel]
fuels = [
    [("ch4", 1.0)],
    [("c4h10", 1.0)],
    [("nc7h16", 0.2), ("ic8h18", 0.8)],
    [("mhd", 0.9), ("ch3oh", 0.1)],
]

# specify oxidizers = pure oxygen
oxid = ck.Mixture(MyGasMech)
oxid.x = [("o2", 1.0)]
oxid.pressure = thispressure
oxid.temperature = thistemperature
# get o2 index
oxy_index = MyGasMech.get_specindex("o2")

# specify the complete combustion product species
products = ["co2", "h2o"]
# no added species
add_frac = np.zeros(MyGasMech.kk, dtype=np.double)

# instantiate the unburned fuel-oxygen mixture
unburned = ck.Mixture(MyGasMech)
unburned.pressure = thispressure
unburned.temperature = thistemperature
```

### Compute the fuel heating value of the fuel mixtures

Now compute the heating values: LHV and HHV, from the enthalpy different between the unburned stoichiometric fuel-oxygen mixture `unburned` and the complete combustion product mixture `burned`. The complete combustion product mixture is found by using the `find_equilibrium` method with the default *fixed pressure* and *fixed temperature* option, that is, the product mixture is also at the standard state condition.

```
lhv = np.zeros(len(fuels), dtype=np.double)
hhv = np.zeros_like(lhv, dtype=np.double)
fuelcount = 0
for f in fuels:
    # re-set the fuel composition (mole/volume fractions)
    fuel.x = f
    # create a soichiometric fuel-oxygen mixture
    _ = unburned.x_by_equivalence_ratio(
        MyGasMech, fuel.x, oxid.x, add_frac, products, equivalenceratio=1.0
    )
    # get the mixture enthalpy of the initial mixture [erg/g]
    h_unburned = unburned.hml() / unburned.wtm

    # compute the complete combustion state (fixed temperature and pressure)
    # this step mimics the complete burning of the initial fuel-oxygen mixture
    # at constant pressure and the subsequent cooling of the combustion products
    # back to the original temperature
    burned = unburned.find_equilibrium()
    # get the mixture enthalpy of the final mixture [erg/g]
    h_burned = burned.hml() / burned.wtm

    # get total fuel mass fraction
    fmass = 0.0e0
    bmassfrac = unburned.y
    for i in range(MyGasMech.kk):
        if i != oxy_index:
            fmass += bmassfrac[i]
    # water vapor mass fraction in the urned mixture
    wmass = burned.y[watervapor_index]
    #
    if np.isclose(fmass, 0.0, atol=1.0e-10):
        # no fuel species exists in the unburned mixture
        print(f">>> error no fuel species in the unburned mixture {f}")
        exit()

    # compute the heating values [erg/g-fuel]
    lhv[fuelcount] = -(h_burned - h_unburned) / fmass
    hhv[fuelcount] = -(h_burned - (h_unburned + heatvaporization * wmass)) / fmass
    fuelcount += 1
```

### Display the heating values

List the fuel mixtures and their LHV and HHV. The heating values are converted from the cgs units [erg/g] to [kJ/g].

```
print(
    f"Fuel Heating Values at {thistemperature} [K] and {thispressure * 1.0e-6} [bar]\n"
)
```

(continues on next page)

(continued from previous page)

```
for i in range(len(fuels)):
    print(f"fuel composition:  {fuels[i]}")
    print(f" LHV [kJ/g-fuel]:  {lhv[i] / ck.ERGS_PER_JOULE / 1.0e3}")
    print(f" HHV [kJ/g-fuel]:  {hhv[i] / ck.ERGS_PER_JOULE / 1.0e3}\n")

##########
# Clean up
# ========
# Delete arrays, mixture objects, and temporary files no longer needed.
del hhv, lhv
del fuel, oxid, unburned, burned
# delete the local mechanism file just created
Path(mymechfile).unlink()
ck.done()
```

### 22.2.6 Combine gas mixtures

PyChemkin provides a set of basic mixture utilities enabling the creation of new mixtures from existing ones. The mixing methods let you combine two mixtures at constant pressure with specific constraints.

- The `adiabatic_mixing()` method combines two mixtures while keeping the overall enthalpy constant. The temperature of the combined mixture is determined by the conservation of the overall enthalpy of the two parent mixtures.
- The `isothermal_mixing()` method simply combines two mixtures and determines the composition of the final mixture according to the mole or mass ratios specified. Because this method does not consider the enthalpy conservation, you must assign the temperature value of the combined mixture.

This example shows how to use these two mixing methods and understand the differences in the temperatures of their combined mixtures. It first creates a fuel ($CH_4$) mixture and an air ($O_2$+$N_2$) mixture and then makes a fuel-air mixture by mixing them *isothermally* (that is, without considering the enthalpy conservation) with a given air-to-fuel mass ratio. The example then dilutes the fuel-air mixture with argon (AR) *adiabatically* with the molar/volumetric ratio specified.

```

```

#### Import PyChemkin packages and start the logger

Import the PyChemkin packages, check the working directory, and start the logger in verbose mode.

```
from pathlib import Path

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core.logger import logger

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
```

### Create a chemistry set

Load the GRI 3.0 mechanism and its associated data files, which come with the standard Ansys Chemkin installation in the `/reaction/data` directory.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir
# create a chemistry set based on GRI 3.0
MyGasMech = ck.Chemistry(label="GRI 3.0")
# set mechanism input files
# including the full file path is recommended
MyGasMech.chemfile = str(mechanism_dir / "grimech30_chem.inp")
MyGasMech.thermfile = str(mechanism_dir / "grimech30_thermo.dat")
# transport data is not needed
```

### Preprocess the chemistry set

Preprocess the mechanism files to prepare the chemistry set.

```
# preprocess the mechanism files
_ = MyGasMech.preprocess()
```

### Set up gas mixtures

Set up gas mixtures based on the species in this chemistry set. Use the equivalence ratio method to set up the combustible mixture so that you can easily change the mixture composition by assigning a different equivalence ratio value.

### Create a fuel mixture

Create a fuel mixture of 100% methane.

**Note**

Mixture pressures are not specified here because they are not required by the calculations. The mixing process assumes a fixed pressure, meaning that the mixtures are at the same pressure.

```
fuel = ck.Mixture(MyGasMech)
# set mole fraction
fuel.x = [("CH4", 1.0)]
fuel.temperature = 300.0
```

### Create an air mixture

Create an air mixture of oxygen and nitrogen.

```
air = ck.Mixture(MyGasMech)
# set mole fraction
air.x = [("O2", 0.21), ("N2", 0.79)]
air.temperature = 300.0
```

### Create a fuel-air mixture by mixing

Use the `isothermal_mixing()` method to mix the `fuel` and `air` mixtures created earlier. Define the mixing formula using `mixture_recipe` with this mass ratio: `fuel:air=1.00:17.19`. Use the `finaltemperature` parameter to set the temperature of the new `premixed` mixture to 300 [K]. Set `mode="mass"` because the ratios given in `mixture_recipe` are mass ratios.

```
# mix the fuel and the air with an air-fuel ratio of 17.19 (almost stoichiometric)
mixture_recipe = [(fuel, 1.0), (air, 17.19)]
# create the new mixture (the air-fuel ratio is by mass)
premixed = ck.isothermal_mixing(
    recipe=mixture_recipe, mode="mass", finaltemperature=300.0
)
```

### Display the molar composition of the premixed mixture

Use the `list_composition()` method with `mode="mole"` to list the mole fractions. The molar composition should resemble the stoichiometric methane-air mixture.

```
# list the molar composition
premixed.list_composition(mode="mole")
print()
```

### Create a diluent mixture with pure argon

Create an argon mixture to dilute the premixed mixture later. Set the mixture temperature to a temperature of 600 [K].

```
ar = ck.Mixture(MyGasMech)
# species composition
ar.x = [("AR", 1.0)]
# mixture temperature
ar.temperature = 600.0
```

### Dilute the fuel-air mixture with argon by adiabatic mixing

Dilute the premixed mixture by mixing 30% argon by volume adiabatically. The `adiabatic_mixing()` method determines the final mixture temperature based on the enthalpy conservation. Set `mode="mole"` to indicate that the ratios in `dilute_recipe` are molar ratios.

```
# create the mixing recipe
dilute_recipe = [(premixed, 0.7), (ar, 0.3)]
# create the diluted mixture
diluted = ck.adiabatic_mixing(recipe=dilute_recipe, mode="mole")
```

### Display information for the diluted mixture

Use the `list_composition()` method with `mode="mole"` to display the molar composition. Also display the temperatures of the three mixtures involved in the adiabatic mixing process for verification. The temperature of the diluted mixture should sit in between those of the `premixed` and `ar` mixtures.

```
# list molar composition
diluted.list_composition(mode="mole")
# show the mixture temperatures
print(f"The diluted mixture temperature is  {diluted.temperature:f} [K].")
```

(continues on next page)

(continued from previous page)

```
print(f"The ar mixture temperature is       {ar.temperature:f} [K].")
print(f"The premixed mixture temperature is {premixed.temperature:f} [K].")

# clean up
ck.done()
```

### 22.2.7 Rank reaction rates

When you have a mixture in PyChemkin, you can not only obtain its properties but also extract the rate information. The mixture can be any one of the following:

- Set up from scratch.
- Obtained from certain mixture operations.
- Based on a point (time or grid) solution of a reactor simulation.

The mixture rate utilities let you access the net production rate of species (ROP), forward and reverse reaction rates per reaction (RR), and net chemical heat release rate (HRR).

This example shows how to use PyChemkin mixture rate tools to derive useful information from the raw data, such as isolating dominant reactions at different mixture conditions.

#### Import PyChemkin packages and start the logger

```
from pathlib import Path

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core.logger import logger

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

#### Create a chemistry set

The first mechanism to load is the GRI 3.0 mechanism for methane combustion. This mechanism and its associated data files come with the standard Ansys Chemkin installation in the `/reaction/data` directory.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir
```

(continues on next page)

(continued from previous page)

```
# create a chemistry set based on GRI 3.0
MyGasMech = ck.Chemistry(label="GRI 3.0")
# set mechanism input files
# including the full file path is recommended
MyGasMech.chemfile = str(mechanism_dir / "grimech30_chem.inp")
MyGasMech.thermfile = str(mechanism_dir / "grimech30_thermo.dat")
MyGasMech.tranfile = str(mechanism_dir / "grimech30_transport.dat")
```

### Preprocess the chemistry set

```
# preprocess the mechanism files
_ = MyGasMech.preprocess()
```

### Set up gas mixtures based on the species in this chemistry set

Before you can create a premixed fuel-oxidizer mixture by the equivalence ratio, you must first create fuel and the oxidizer mixtures.

### Create the fuel mixture

The fuel mixture consists of 100% methane.

```
fuelmixture = ck.Mixture(MyGasMech)
# set fuel composition
fuelmixture.x = [("CH4", 1.0)]
# setting pressure and temperature is not required in this case
fuelmixture.pressure = 5.0 * ck.P_ATM
fuelmixture.temperature = 1500.0
```

### Create the air mixture

The air mixture consists of oxygen and nitrogen. The temperature and the pressure of this mixture are set to the values of the fuel mixture.

```
air = ck.Mixture(MyGasMech)
air.x = [("O2", 0.21), ("N2", 0.79)]
# setting pressure and temperature is not required in this case
air.pressure = 5.0 * ck.P_ATM
air.temperature = 1500.0
```

### Define the combustion products and additives

To use the equivalence ratio method, you must define the complete combustion products and the composition of the additives.

```
# products from the complete combustion of the fuel and air mixtures
products = ["CO2", "H2O", "N2"]
# Specify mole fractions of the added/inert mixture.
# You can also create an additives mixture here.
add_frac = np.zeros(MyGasMech.kk, dtype=np.double)  # no additives: all zeros
```

### Create a placeholder for the fuel-air mixture

You can create an empty mixture object that is fully defined later. The composition of the `premixed` mixture is determined when calling one of the mixture composition methods:

- `X()`
- `Y()`
- `X_byEquivalence_Ratio()`
- `Y_byEquivalence_Ratio()`

When calling the `X_byEquivalence_Ratio()` or `Y_byEquivalence_Ratio()` method, the equivalence ratio is specified by this parameter: `equivalenceratio=1.0`.

```
# create the premixed mixture to define
premixed = ck.Mixture(MyGasMech)
# define the actual composition by the equivalence ratio
ierror = premixed.x_by_equivalence_ratio(
    MyGasMech, fuelmixture.x, air.x, add_frac, products, equivalenceratio=1.0
)
if ierror != 0:
    # check fuel-air mixture creation status
    print("Error: Failed to create the fuel-air mixture.")
    exit()
```

### Display the molar composition of the premixed mixture

List the composition of the premixed mixture for verification.

```
premixed.list_composition(mode="mole")
```

### Evaluate reaction rates at a given mixture temperature and pressure

Compute and rank the *net* reaction rates in the `MyGasMech` chemistry set at 1600 [K] and 5 [atm]. The *net* rate of a reaction is computed by summing the forward rate `kf` and the reverse rate `kr`. The `rxn_rates` rate utility returns both the forward and the reverse rates of each reaction in the mechanism.

Get the net production rates of *all* species by using the `ROP` method. The ROP values obtained are in [mole/cm$^3$-sec]. Alternatively, you can use the `rop_mass` method to get mass-unit ROP values in [g/cm$^3$-sec].

Use the optional `threshold` parameter to get only the ROP values with absolute value above the given threshold value. For example, `premixed.ROP(threshold=1.0e-8)` returns a `rop` array in which any raw ROP value with an absolute value less than 1.0e-8 is set to zero. By default, `threshold=0.0`, in which case all raw ROP values are returned.

**Note**

Temperature and pressure are required to compute the reaction rates.

```
# set the temperature and the pressure of the premixed mixture
premixed.pressure = 5.0 * ck.P_ATM
premixed.temperature = 1600.0
```

### Obtain and sort the rates of production of all species

Use the `ROP()` method to get the rates of production (ROP) of all species in the `MyGasMech` chemistry set. You can use the `list_rop()` method to list and sort the ROP values. `spec_rate_order` contains the sorted species indices in descending order. `species_rates` contains the sorted ROP values.

```
rop = premixed.rop()
# list the non-zero rates in descending order
print()
spec_rate_order, species_rates = premixed.list_rop()
```

### Evaluate and sort the rates of production of all species

Use the `rxn_rates()` method to get the forward rate (`kf`) and reverse rate (`kr`) of each reaction. The `list_reaction_rates()` method computes the net rate of each reaction and sorts them in ascending or descending order. The net reaction rate is computed by summing the forward and reverse rates of each reaction. `rxn_order` is a list of the ranked reaction indices. `net_rxn_rates` contains the net reaction rates in the same order.

**Note**

The species and the reactions indexes returned by the `list_rop()` and `list_reaction_rates()` methods are 0-based indexes. Increment the returned index by 1 to get the actual 1-based index.

```
kf, kr = premixed.rxn_rates()
print()
print(f"Reverse reaction rates: (raw values of all {MyGasMech.ii_gas:d} reactions)")
print(str(kr))
print("=" * 40)
# list the non-zero net reaction rates
rxn_order, net_rxn_rates = premixed.list_reaction_rates()
```

### Plot the sorted reaction rates at 1600 [K]

Display the top net reaction rates and their reaction strings in the commonly used horizontal bar plot. Use the `get_gas_reaction_string` utility of the chemistry set to get the actual reaction for the Y-axis label.

```
# create a rate plot
plt.rcParams.update({"figure.autolayout": True})
plt.subplots(2, 1, sharex="col", figsize=(10, 5))
# convert reaction # from integers to strings
rxnstring = []
for i in range(len(rxn_order)):
    # the array index starting from 0 so the actual reaction # = index + 1
    rxnstring.append(MyGasMech.get_gas_reaction_string(rxn_order[i] + 1))
# use horizontal bar chart
plt.subplot(211)
plt.barh(rxnstring, net_rxn_rates, color="blue", height=0.4)
# use log scale on x axis
plt.xscale("symlog")
# plt.ylabel('reaction')
plt.text(-3.0e-4, 0.5, "T = 1600K", fontsize=10)
```

#### Evaluate reaction rates at a given mixture temperature and pressure

Compute and rank the net reaction rates in the `MyGasMech` chemistry set at 1800 [K] and 5 [atm].

```
# change the mixture temperature
premixed.temperature = 1800.0
```

#### Evaluate and sort the rates of production of all species at 1800 [K]

Get the list of non-zero net reaction rates at the new temperature.

```
rxn_order, net_rxn_rates = premixed.list_reaction_rates()
```

#### Plot the sorted reaction rates at 1800 [K]

Display the top net reaction rates and their reaction strings in the commonly used horizontal bar plot.

```
plt.subplot(212)
# convert reaction # from integers to strings
rxnstring.clear()
for i in range(len(rxn_order)):
    # the array index starting from 0 so the actual reaction # = index + 1
    rxnstring.append(MyGasMech.get_gas_reaction_string(rxn_order[i] + 1))
plt.barh(rxnstring, net_rxn_rates, color="orange", height=0.4)
plt.xlabel("reaction rate [mole/cm3-sec]")
# plt.ylabel('reaction')
plt.text(-3.0e-4, 0.5, "T = 1800K", fontsize=10)
# use log scale on x axis
plt.xscale("symlog")

# clean up
ck.done()

# plot both ranking results
if interactive:
    plt.show()
else:
    plt.savefig("plot_reaction_rates.png", bbox_inches="tight")
```

## 22.3 Batch reactor

The *batch reactor* is an idealized 0-D transient closed reactor model in which the gas inside the reactor is assumed to be homogeneous. When the pressure of the *batch reactor* is constrained, the reactor volume would vary to conserve the total gas mass. Likewise, the reactor pressure might change when the reactor volume is "given". The gas temperature can be "given" by piecewise linear profile data or solved by the energy equation.

The examples consist of projects that utilize the *batch reactor* model to perform simulations of various complexities, from tracking the evolution of the mixture properties to the A-factor sensitivity analysis.

### 22.3.1 Simulate hydrogen combustion in a constant-pressure reactor

Ansys Chemkin offers some idealized reactor models commonly used for studying chemical processes and for developing reaction mechanisms. The batch reactor is a transient 0-D numerical portrayal of the closed homogeneous/perfectly mixed gas-phase reactor. There are two basic types of batch reactor models:

- **constrained-pressure**
- **constrained-volume**

You can choose either to specify the reactor temperature (as a fixed value or by a piecewise-linear profile) or to solve the energy conservation equation for each reactor type. In total, you get four variations out of the base batch reactor model.

This example models the ignition of a stoichiometric hydrogen-air mixture ($\phi = 1$) in a balloon (constant-pressure) reactor. The reactor is created as an instance of the `GivenPressureBatchReactorEnergyConservation` object. The initial gas mixture in the batch reactor is set by the properties of the hydrogen-air mixture. You get the ignition delay time (if auto-ignition of the hydrogen-air mixture occurs during the simulation time) and plot the predicted profiles of gas temperature, gas density, $H_2O$ mole fraction, and the net production rate of $H_2O$.

#### Import PyChemkin packages and start the logger

```
from pathlib import Path

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color

# chemkin batch reactor models (transient)
from ansys.chemkin.core.batchreactors.batchreactor import (
    GivenPressureBatchReactorEnergyConservation,
)
from ansys.chemkin.core.logger import logger

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

#### Create a chemistry set

The mechanism to load is the GRI 3.0 mechanism for methane combustion. This mechanism and its associated data files come with the standard Ansys Chemkin installation in the `/reaction/data` directory.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir
# create a chemistry set based on the diesel 14-components mechanism
MyGasMech = ck.Chemistry(label="GRI 3.0")
# set mechanism input files
# including the full file path is recommended
MyGasMech.chemfile = str(mechanism_dir / "grimech30_chem.inp")
MyGasMech.thermfile = str(mechanism_dir / "grimech30_thermo.dat")
MyGasMech.tranfile = str(mechanism_dir / "grimech30_transport.dat")
```

### Preprocess the chemistry set

```
# preprocess the mechanism files
_ = MyGasMech.preprocess()
```

### Set up gas mixtures based on the species in this chemistry set

Compose the species molar ratios of the fuel-air mixture in a recipe list and use the `fuelmixture.x()` method to set the mixture composition (mole fractions) directly.

Since you are going to use the `fuelmixture` mixture to instantiate the reactor object later, setting the mixture pressure and temperature is equivalent to setting the initial temperature and pressure of the batch reactor.

```
# create the fuel mixture
fuelmixture = ck.Mixture(MyGasMech)
# set fuel composition (mole ratios)
fuelmixture.x = [("H2", 2.0), ("N2", 3.76), ("O2", 1.0)]
# setting the mixture pressure and temperature is equivalent to setting
# the initial temperature and pressure of the reactor in this case
fuelmixture.pressure = ck.P_ATM
fuelmixture.temperature = 1000

# list the composition of the premixed mixture
# this serves as the baseline for verification later
fuelmixture.list_composition(mode="mole")
```

### Set up a constant-pressure batch reactor (with energy equation)

Create the constant-pressure batch reactor as an instance of the `GivenPressureBatchReactorEnergyConservation` object because the reactor pressure is kept constant (or assigned as a function of time). The batch reactors must be associated with a mixture, which implicitly links the chemistry set (gas-phase mechanism and properties) to the batch reactor. Additionally, it defines the initial conditions (pressure, temperature, volume, and gas composition) of the batch reactor.

```
MyCONP = GivenPressureBatchReactorEnergyConservation(fuelmixture, label="tran")
```

### List the mixture composition

List the initial gas composition inside the reactor for verification. You can use the composition printout from the `fuelmixture` mixture to verify that the initial gas composition inside `MyCONP` is set correctly, that is, `MyCONP` has been initialized by the `fuelmixture` mixture.

```
MyCONP.list_composition(mode="mole")
```

### Set up additional reactor model parameters

Before you can run the simulation, you must provide reactor parameters, solver controls, and output instructions. For a batch reactor, the initial volume and the simulation end time are required inputs.

```
# set other reactor properties
# reactor volume [cm3]
MyCONP.volume = 1
MyCONP.temperature = 1000
# simulation end time [sec]
MyCONP.time = 0.0005
```

### Set output options

You can turn on adaptive solution saving to resolve the steep variations in the solution profile. Here, additional solution data points are saved for every 20 internal solver steps. You must include the `set_ignition_delay()` method for the reactor model to report the ignition delay times after the simulation is done. If `method="T_rise"` is set, the reactor model considers the gas is auto-ignited when the predicted gas temperature goes above the initial temperature by the amount indicated by the parameter `val=400`. You can choose a different auto-ignition definition.

**Note**

- Type `ansys.chemkin.core.show_ignition_definitions()` to get the list of all available ignition delay time definitions in Chemkin.
- By default, time intervals for both print and save solution are 1/100 of the simulation end time, which in this example is $dt = time/100 = 0.001$. You can change them to different values.

```
# turn on adaptive solution saving
MyCONP.adaptive_solution_saving(mode=True, steps=20)
# specify the ignition definitions
ck.show_ignition_definitions()
MyCONP.set_ignition_delay(method="T_rise", val=400)
```

### Set solver controls

You can overwrite the default solver controls by using solver-related methods, such as those for tolerances.

```
# set tolerances in tuple: (absolute tolerance, relative tolerance)
MyCONP.tolerances = (1.0e-20, 1.0e-8)

# get solver parameters
ATOL, RTOL = MyCONP.tolerances
print(f"Default absolute tolerance = {ATOL}.")
print(f"Default relative tolerance = {RTOL}.")
# turn on the force non-negative solutions option in the solver
MyCONP.force_nonnegative = True
# show solver option
print(f"Timestep between solution printing: {MyCONP.timestep_for_printing_solution}")
```

(continues on next page)

(continued from previous page)

```
# show timestep between printing solution
print(f"Forced non-negative solution values: {MyCONP.force_nonnegative}")
```

### Display the added parameters (keywords)

Use the `showkeywordinputlines()` method to verify that the preceding parameters are correctly assigned to the reactor model.

```
MyCONP.showkeywordinputlines()
```

### Run the simulation

Use the `run()` method to start the batch reactor simulation.

```
runstatus = MyCONP.run()

# check run status
if runstatus != 0:
    # Run failed.
    print(Color.RED + ">>> Run failed. <<<", end="\n" + Color.END)
    exit()
# Run succeeded.
print(Color.GREEN + ">>> Run completed. <<<", end="\n" + Color.END)
```

### Get the ignition delay time from the solution

Use the `get_ignition_delay()` method to extract the ignition delay time after the run is completed.

```
delaytime = MyCONP.get_ignition_delay()
print(f"Ignition delay time = {delaytime} [msec].")
```

### Postprocess the solution

The postprocessing step parses the solution and packages the solution values at each time point into a mixture object. There are two ways to access the solution profiles:

- The raw solution profiles (value as a function of time) are available for time, temperature, pressure, volume, and species mass fractions.
- The mixture objects that permit the use of all property and rate utilities to extract information such as viscosity, density, species production rates, and mole fractions.

To obtain the raw solution profiles, you can use the `get_solution_variable_profile()` method. To obtain the solution mixture objects, you can use either the `get_solution_mixture_at_index()` method for the solution mixture at a given time point or the `get_solution_mixture()` method for the solution mixture at a given time. (In this case, the mixture is constructed by # interpolation.)

**Note**

Use the `getnumbersolutionpoints()` method to get the size of the solution profiles before creating the arrays.

```
MyCONP.process_solution()
# get the number of solution time points
solutionpoints = MyCONP.getnumbersolutionpoints()
print(f"Number of solution points = {solutionpoints}.")

# get the time profile
timeprofile = MyCONP.get_solution_variable_profile("time")
# get the temperature profile
tempprofile = MyCONP.get_solution_variable_profile("temperature")
# more involving postprocessing by using mixtures
# create arrays for H2O mole fraction, H2O ROP, and mixture density
h2o_profile = np.zeros_like(timeprofile, dtype=np.double)
h2o_rop_profile = np.zeros_like(timeprofile, dtype=np.double)
denprofile = np.zeros_like(timeprofile, dtype=np.double)
current_rop = np.zeros(MyGasMech.kk, dtype=np.double)
# find H2O species index
h2o_index = MyGasMech.get_specindex("H2O")
# loop over all solution time points
for i in range(solutionpoints):
    # get the mixture at the time point
    solutionmixture = MyCONP.get_solution_mixture_at_index(solution_index=i)
    # get gas density [g/cm3]
    denprofile[i] = solutionmixture.rho
    # reactor mass [g]
    # get H2O mole fraction profile
    h2o_profile[i] = solutionmixture.x[h2o_index]
    # get H2O ROP profile
    current_rop = solutionmixture.rop()
    h2o_rop_profile[i] = current_rop[h2o_index]
```

### Plot the solution profiles

```
# plot the profiles
plt.subplots(2, 2, sharex="col", figsize=(12, 6))
plt.subplot(221)
plt.plot(timeprofile, tempprofile, "r-")
plt.ylabel("Temperature [K]")
plt.subplot(222)
plt.plot(timeprofile, h2o_profile, "b-")
plt.ylabel("H2O Mole Fraction")
plt.subplot(223)
plt.plot(timeprofile, denprofile, "m-")
plt.xlabel("time [sec]")
plt.ylabel("Mixture Density [g/cm3]")
plt.subplot(224)
plt.plot(timeprofile, h2o_rop_profile, "g-")
plt.xlabel("time [sec]")
plt.ylabel("H2O Production Rate [mol/cm3-sec]")

# clean up
ck.done()
```

(continues on next page)

(continued from previous page)

```
# display the plots
if interactive:
    plt.show()
else:
    plt.savefig("plot_close_homogeneous.png", bbox_inches="tight")
```

### 22.3.2 Predict the ignition delay time of a combustible mixture

One of the important properties of hydrocarbon fuels is the *ignition delay time*. For automobiles, the gasoline, a mixtures of many fuel species, is graded by the *octane number* which is closely related to the fuel mixture's ignition characteristics. Higher octane number fuel tends to ignite more easily. However, this does not mean you should blindly put the highest octane number fuel into your car. Car engines are designed for specific operating conditions, and you should always use the gasoline grades recommended by the car/engine manufacturer to prevent damages to the engine.

This tutorial employs the **constant-pressure batch reactor model** (with the energy equation **ON**) to predict the *auto-ignition delay time* of a **Primary Reference Fuel (PRF)**. PRF is created for automobile fuel researches and consists of two major components: n-heptane $C_7H_{16}$ and iso-octane $C_8H_{18}$. By definition, n-heptane is assigned with an octane number of 0, and an octane number of 100 for iso-octane. Thus, a PRF of 20% n-heptane and 80% iso-octane has an octane number of 80. In this tutorial, a PRF 60 fuel is used to show the **negative temperature coefficient (NTC)** behavior in the ignition delay curve. The ignition delay curve is constructed by collecting the predicted auto-ignition delay times with various initial gas conditions. Here, the initial gas temperature is increased from 700 to 1080 [K].

```

```

#### Import PyChemkin package and start the logger

```
from pathlib import Path
import time

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

import ansys.chemkin.core as ck  # chemkin
from ansys.chemkin.core import Color

# chemkin batch reactor models (transient)
from ansys.chemkin.core.batchreactors.batchreactor import (
    GivenPressureBatchReactorEnergyConservation,
)
from ansys.chemkin.core.logger import logger

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(False)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create a chemistry set

For PRF, the encrypted 14-component gasoline mechanism, `gasoline_14comp_WBencrypted.inp`, is used. `gasoline` is the name given to this `Chemistry Set`.

**Note**

This gasoline mechanism does not come with any transport data so you do not need to provide the transport data file.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir
# create a chemistry set based on GRI 3.0
gasoline = ck.Chemistry(label="gasoline 14comp")
# set mechanism input files
# including the full file path is recommended
gasoline.chemfile = str(mechanism_dir / "gasoline_14comp_WBencrypt.inp")
```

### Pre-process the gasoline `Chemistry Set`

```
# preprocess the mechanism files
_ = gasoline.preprocess()
```

### Set up the stoichiometric gasoline-air mixture

You need to set up the stoichiometric gasoline-air mixture for the subsequent ignition delay time calculations. Here the `x_by_equivalence_ratio``method is used. You create the ``fuel` and the `air` mixtures first. Then define the *complete combustion product species* and provide the *additives* composition if applicable. And finally you can simply set `equivalenceratio=1` to create the stoichiometric gasoline-air mixture.

For PRF 60 gasoline, the `recipe` is `[("ic8h18", 0.6), ("nc7h16", 0.4)]`.

```
# create the fuel mixture
fuelmixture = ck.Mixture(gasoline)
# set fuel = composition of PRF 60
fuelmixture.x = [("ic8h18", 0.6), ("nc7h16", 0.4)]
# setting pressure and temperature
fuelmixture.pressure = 5.0 * ck.P_ATM
fuelmixture.temperature = 1500.0

# create the oxidizer mixture: air
air = ck.Mixture(gasoline)
air.x = [("o2", 0.21), ("n2", 0.79)]
# setting pressure and temperature
air.pressure = 5.0 * ck.P_ATM
air.temperature = 1500.0

# products from the complete combustion of the fuel mixture and air
products = ["co2", "h2o", "n2"]
# species mole fractions of added/inert mixture.
# can also create an additives mixture here
add_frac = np.zeros(gasoline.kk, dtype=np.double)  # no additives: all zeros
```

(continues on next page)

(continued from previous page)

```
# create the premixed mixture to be defined
premixed = ck.Mixture(gasoline)
# define the composition by the equivalence ratio
ierror = premixed.x_by_equivalence_ratio(
    gasoline, fuelmixture.x, air.x, add_frac, products, equivalenceratio=1.0
)
# check fuel-oxidizer mixture creation status
if ierror != 0:
    print("Error: Failed to create the fuel-oxidizer mixture.")
    exit()
```

### List the mixture composition

list the composition of the premixed mixture for verification

```
premixed.list_composition(mode="mole")
# set mixture temperature and pressure
# (equivalent to setting the initial temperature and pressure of the reactor)
premixed.pressure = 40.0 * ck.P_ATM
premixed.temperature = 700.0
```

### Create the reactor object for ignition delay time calculations

Use the `GivenPressureBatchReactorEnergyConservation` method to instantiate a *constant pressure batch reactor that also includes the energy equation*. This `ReactorModel` method requires a `Mixture` object as an input parameter. This `Mixture` input serves two purposes: passing the `Chemistry Set` to be used by the reactor model and setting the *initial condition* (pressure, temperature, and gas composition) of the simulation. You should use the `premixed` mixture you just created.

```
MyCONP = GivenPressureBatchReactorEnergyConservation(premixed, label="CONP")
```

### Set up additional reactor model parameters

*Reactor parameters*, *solver controls*, and *output instructions* need to be provided before running the simulations. For a batch reactor, the *initial volume* and the *simulation end time* are required inputs.

```
# verify initial gas composition inside the reactor
MyCONP.list_composition(mode="mole")

# set other reactor parameters
# initial reactor volume [cm3]
MyCONP.volume = 10.0
# simulation end time [sec]
MyCONP.time = 1.0
```

### Set output options

You can turn on the *adaptive solution saving* to resolve the steep variations in the solution profile. Here additional solution data point will be saved for every **100 [K]** change in gas **temperature**. The `set_ignition_delay` method must be included for the reactor model to report the *ignition delay times* after the simulation is done. If `method="T_inflection"` is set, the reactor model will treat the *inflection points* in the predicted gas temperature profile as the indication of an auto-ignition. You can choose a different auto-ignition definition.

**Note**

Type `ansys.chemkin.core.show_ignition_definitions()` to get the list of all available ignition delay time definitions in Chemkin.

**Note**

By default, time intervals for both print and save solution are **1/100** of the *simulation end time*. In this case $dt = time/100 = 0.001$. You can change them to different values.

```
# set timestep between saving solution
MyCONP.timestep_for_saving_solution = 0.001
# change timestep between saving solution
MyCONP.timestep_for_saving_solution = 0.01
# turn ON adaptive solution saving
MyCONP.adaptive_solution_saving(mode=True, value_change=100, target="TEMPERATURE")
# set ignition delay
MyCONP.set_ignition_delay(method="T_inflection")
```

### Set solver controls

You can overwrite the default solver controls by using solver related methods, for example, `tolerances`.

```
# tolerances are given in tuple: (absolute tolerance, relative tolerance)
MyCONP.tolerances = (1.0e-10, 1.0e-8)
# stop after ignition is detected
# (not recommended for ignition delay time calculations)
# MyCONP.stop_after_ignition()
# show solver option
print(f"timestep between solution printing: {MyCONP.timestep_for_printing_solution}")
# show timestep between printing solution
print(f"forced non-negative solution values: {MyCONP.force_nonnegative}")
```

### Run the parameter study

Construct the ignition delay curve by varying the initial reactor temperature in 20 [K] increments from 700 to 1080 [K]. Use the `get_ignition_delay` method to extract the ignition delay times after finishing each run.

```
# loop over initial reactor temperature to create an ignition delay time plot
npoints = 20
delta_temp = 20.0
init_temp = premixed.temperature
delaytime = np.zeros(npoints, dtype=np.double)
temp_inv = np.zeros_like(delaytime, dtype=np.double)
# set the start wall time
start_time = time.time()
# loop over all cases with different initial gas/reactor temperatures
for i in range(npoints):
    # update the initial reactor temperature
    MyCONP.temperature = init_temp  # K
    # show the additional keywords given by user
```

(continues on next page)

(continued from previous page)

```
    # (verify inputs before running the simulation)
    # MyCONP.showkeywordinputlines()
    # run the reactor model
    runstatus = MyCONP.run()
    #
    if runstatus == 0:
        # plot 1/T instead of T
        temp_inv[i] = 1.0e0 / init_temp
        # get ignition delay time
        delaytime[i] = MyCONP.get_ignition_delay()
        print(f"ignition delay time = {delaytime[i]} [msec]")
    else:
        # if get this, most likely the END time is too short
        print(Color.RED + ">>> Run failed. <<<", end=Color.END)
    init_temp += delta_temp

# compute the total runtime
runtime = time.time() - start_time
print(f"total simulation duration: {runtime} [sec] for {npoints} cases")
```

### Plot the ignition delay curve

The ignition delay curve is customarily plotted against the inverse temperature. The corresponding temperature values are shown on the top axis.

Intuitively, you expect that the reactivity increases as the temperature increases. However, as you can see, in this case, the ignition delay curve does not vary monotonically with the temperature. Between 850 and 950 [K], the ignition delay time actually increases slightly (because the reactivity decreases). This is regarded as the **Negative Temperature Coefficient (NTC)** behavior which is often observed during low-temperature oxidation of large hydrocarbon fuels.

```
# create an ignition delay versus 1/T plot for the PRF fuel
# (should exhibit the NTC region)
plt.rcParams.update({"figure.autolayout": True})
fig, ax1 = plt.subplots()
ax1.semilogy(temp_inv, delaytime, "bs--")
ax1.set_xlabel("1/T [1/K]")
ax1.set_ylabel("Ignition delay time [msec]")


# Create a secondary x-axis for T (=1/(1/T))
def one_over(x):
    """Vectorized 1/x, treating x==0 manually."""
    x = np.array(x, float)
    near_zero = np.isclose(x, 0)
    x[near_zero] = np.inf
    x[~near_zero] = 1 / x[~near_zero]
    return x


# the function "1/x" is its own inverse
inverse = one_over
ax2 = ax1.secondary_xaxis("top", functions=(one_over, inverse))
```

(continues on next page)

(continued from previous page)

```
ax2.set_xlabel("T [K]")

# clean up
ck.done()

# plot results
if interactive:
    plt.show()
else:
    plt.savefig("plot_ignition_delay.png", bbox_inches="tight")
```

### 22.3.3 Simulate a rapid compression machine

Ansys Chemkin offers some idealized reactor models commonly used for studying chemical processes and for developing reaction mechanisms. The *batch reactor* is a transient 0-D numerical portrayal of the *closed homogeneous/perfectly mixed* gas-phase reactor. There are two basic types of batch reactor models:

- **constrained-pressure**
- **constrained-volume**

You can choose either to specify the reactor temperature (as a fixed value or by a piecewise-linear profile) or to solve the energy conservation equation for each reactor type. In total, you get four variations out of the base batch reactor model.

**Rapid Compression Machine (RCM)** is often employed to study fuel auto-ignition at high temperature and high-pressure conditions that are compatible to the engine-operating environments. The fuel-air mixture inside the RCM chamber is at relatively low pressure and temperature initially. The gas mixture is then suddenly compressed causing both the pressure and the temperature of the mixture to rise rapidly. The reactor/chamber pressure is monitored to identify the onset of auto-ignition after the compression stopped. This example models the RCM as a `GivenVolumeBatchReactorEnergyConservation`, and the compression process is simulated by a predetermined time-volume profile.

#### Import PyChemkin package and start the logger

```
from pathlib import Path

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color

# chemkin batch reactor models (transient)
from ansys.chemkin.core.batchreactors.batchreactor import (
    GivenVolumeBatchReactorEnergyConservation,
)
from ansys.chemkin.core.logger import logger

# check working directory
current_dir = str(Path.cwd())
```

(continues on next page)

(continued from previous page)

```
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create a chemistry set

The mechanism to load is the GRI 3.0 mechanism for methane combustion. This mechanism and its associated data files come with the standard Ansys Chemkin installation in the `/reaction/data` directory.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir
# create a chemistry set based on the GRI 3.0 mechanism
MyGasMech = ck.Chemistry(label="GRI 3.0")
# set mechanism input files
# including the full file path is recommended
MyGasMech.chemfile = str(mechanism_dir / "grimech30_chem.inp")
MyGasMech.thermfile = str(mechanism_dir / "grimech30_thermo.dat")
MyGasMech.tranfile = str(mechanism_dir / "grimech30_transport.dat")
```

### Preprocess the chemistry set

```
# preprocess the mechanism files
_ = MyGasMech.preprocess()
```

### Set up gas mixtures based on the species in this chemistry set

Use the *equivalence ratio method* so that you can easily set up the premixed fuel-oxidizer mixture composition by assigning an equivalence ratio value.

```
# create the fuel mixture
fuelmixture = ck.Mixture(MyGasMech)
# set fuel composition
fuelmixture.x = [("CH4", 1.0)]
# setting pressure and temperature is not required in this case
fuelmixture.pressure = 5.0 * ck.P_ATM
fuelmixture.temperature = 1500.0

# create the oxidizer mixture: air
air = ck.Mixture(MyGasMech)
air.x = [("O2", 0.21), ("N2", 0.79)]
# setting pressure and temperature is not required in this case
air.pressure = 5.0 * ck.P_ATM
air.temperature = 1500.0

# products from the complete combustion of the fuel mixture and air
products = ["CO2", "H2O", "N2"]
```

(continues on next page)

(continued from previous page)

```
# species mole fractions of added/inert mixture.
# can also create an additives mixture here
add_frac = np.zeros(MyGasMech.kk, dtype=np.double)  # no additives: all zeros

# create the premixed mixture to be defined
premixed = ck.Mixture(MyGasMech)

ierror = premixed.x_by_equivalence_ratio(
    MyGasMech, fuelmixture.x, air.x, add_frac, products, equivalenceratio=0.7
)
# check fuel-oxidizer mixture creation status
if ierror != 0:
    print("Error: Failed to create the fuel-oxidizer mixture.")
    exit()

# list the composition of the premixed mixture for verification
premixed.list_composition(mode="mole")

# set mixture temperature and pressure
# (equivalent to setting the initial temperature and pressure of the reactor)
premixed.temperature = 800.0
premixed.pressure = 3.0 * ck.P_ATM
```

### Set up the rapid-compression machine

Create the rapid-compression machine as an instance of the `GivenVolumeBatchReactorEnergyConservation` object because the reactor volume is assigned as a function of time. The batch reactors must be associated with a mixture that implicitly links the chemistry set (gas-phase mechanism and properties) to the batch reactor. Additionally, it also defines the initial reactor conditions (pressure, temperature, volume, and gas composition).

```
# create a constant volume batch reactor (with energy equation)
MyCONV = GivenVolumeBatchReactorEnergyConservation(premixed, label="RCM")
# show initial gas composition inside the reactor
MyCONV.list_composition(mode="mole")
```

### Set up additional reactor model parameters

You must provide reactor parameters, solver controls, and output instructions before running the simulations. For a batch reactor, the initial volume and the simulation end time are required inputs.

**Note**

You can reset the initial reactor temperature by using the `MyCONV.temperature = 800.0` method. In the run output, you see a warning message about the change.

```
# set other reactor properties
# set initial reactor volume [cm3]
MyCONV.volume = 10.0
# simulation end time [sec]
MyCONV.time = 0.1
```

### Set the volume profile

Create a time-volume profile by using two arrays. Use the `set_volume_profile()` method to add the profile to the reactor model. The profile data overrides the initial volume value set earlier with the `volume()` method.

```
# number of profile data points
npoints = 3
# position array of the profile data
x = np.zeros(npoints, dtype=np.double)
# value array of the profile data
vol_profile = np.zeros_like(x, dtype=np.double)
# set reactor volume data points
x = [0.0, 0.01, 2.0]  # [sec]
vol_profile = [10.0, 4.0, 4.0]  # [cm3]
```

### Set output options

You can turn on the adaptive solution saving to resolve the steep variations in the solution profile. Here additional solution data points are saved for every **100 [K]** change in gas temperature. The `set_ignition_delay()` method must be included for the reactor model to report the ignition delay times after the simulation is done. If `method="T_inflection"` is set, the reactor model treats the inflection points in the predicted gas temperature profile as the indication of an auto-ignition. You can choose a different auto-ignition definition.

**Note**

Type `ansys.chemkin.core.show_ignition_definitions()` to getthe list of all available ignition delay time definitions in Chemkin.

**Note**

By default, time intervals for both print and save solution are **1/100** of the simulation end time. In this case $dt = time/100 = 0.001$. You can change them to different values.

```
# set the volume profile
MyCONV.set_volume_profile(x, vol_profile)
# output controls
# set timestep between saving solution
MyCONV.timestep_for_saving_solution = 0.01
# turn ON adaptive solution saving
MyCONV.adaptive_solution_saving(mode=True, value_change=100, target="TEMPERATURE")
# specify the ignition definitions
MyCONV.set_ignition_delay(method="T_inflection")
```

### Set solver controls

You can overwrite the default solver controls by using solver-related methods, such as those for tolerances.

```
# set tolerances in tuple: (absolute tolerance, relative tolerance)
MyCONV.tolerances = (1.0e-10, 1.0e-8)
# get solver parameters
atol, rtol = MyCONV.tolerances
```

(continues on next page)

(continued from previous page)

```
print(f"Default absolute tolerance = {atol}.")
print(f"Default relative tolerance = {rtol}.")
# turn on the force non-negative solutions option in the solver
MyCONV.force_nonnegative = True
# show solver option
print(f"Timestep between solution printing: {MyCONV.timestep_for_printing_solution}.")
# show timestep between printing solution
print(f"Forced non-negative solution values: {MyCONV.force_nonnegative}.")
```

### Display the added parameters (keywords)

You can use the `showkeywordinputlines()` method to verify the preceding parameters are correctly assigned to the reactor model.

```
# show the additional keywords given by user
MyCONV.showkeywordinputlines()
```

### Run the simulation

Use the `run()` method to start the RCM simulation.

```
# run the CONV reactor model
runstatus = MyCONV.run()
# check run status
if runstatus != 0:
    # Run failed.
    print(Color.RED + ">>> Run failed. <<<", end=Color.END)
    exit()

# Run succeeded.
print(Color.GREEN + ">>> Run completed. <<<", end=Color.END)
```

### Get the ignition delay time from the solution

Use the `get_ignition_delay()` method to extract the ignition delay time after the run is completed.

**Note**

You need to deduct the initial compression time = 0.01 [sec] to get the *actual* ignition delay time.

```
# get ignition delay time (need to deduct the initial compression time = 0.01 [sec])
delaytime = MyCONV.get_ignition_delay() - 0.01 * 1.0e3
print(f"Ignition delay time = {delaytime} [msec].")
```

### Postprocess the solution

The postprocessing step parses the solution and package the solution values at each time point into a mixture. There are two ways to access the solution profiles:

- The raw solution profiles (value as a function of distance) are available for distance, temperature, pressure, volume, and species mass fractions.

**-The mixtures permit the use of all property and rate utilities to extract**
information such as viscosity, density, and mole fractions.

You can use the `get_solution_variable_profile()` method to get the raw solution profiles. You can get solution mixtures using either the `get_solution_mixture_at_index()` method for the solution mixture at the given saved location or the `get_solution_mixture()` method for the solution mixture at the given distance. (In this case, the mixture is constructed by interpolation.)

**Note**

Use the `getnumbersolutionpoints()` method to get the size of the solution profiles before creating the arrays.

```
# postprocess the solutions
MyCONV.process_solution()
# get the number of solution time points
solutionpoints = MyCONV.getnumbersolutionpoints()
print(f"Number of solution points = {solutionpoints}.")

# easily access raw solution profiles
# get the time profile
timeprofile = MyCONV.get_solution_variable_profile("time")
# get the temperature profile
tempprofile = MyCONV.get_solution_variable_profile("temperature")
# get the volume profile
volprofile = MyCONV.get_solution_variable_profile("volume")

# more involved postprocessing using mixtures
# reactor mass
massprofile = np.zeros_like(timeprofile, dtype=np.double)
# create arrays for CH4 mole fraction, CH4 ROP, and mixture viscosity
ch4_profile = np.zeros_like(timeprofile, dtype=np.double)
ch4_rop_profile = np.zeros_like(timeprofile, dtype=np.double)
viscprofile = np.zeros_like(timeprofile, dtype=np.double)
current_rop = np.zeros(MyGasMech.kk, dtype=np.double)
# find CH4 species index
ch4_index = MyGasMech.get_specindex("CH4")

# loop over all solution time points
for i in range(solutionpoints):
    # get the mixture at the time point
    solutionmixture = MyCONV.get_solution_mixture_at_index(solution_index=i)
    # get gas density [g/cm3]
    den = solutionmixture.rho
    # reactor mass [g]
    massprofile[i] = den * volprofile[i]
    # get CH4 mole fraction profile
    ch4_profile[i] = solutionmixture.x[ch4_index]
    # get CH4 ROP profile
    current_rop = solutionmixture.rop()
    ch4_rop_profile[i] = current_rop[ch4_index]
    # get mixture vicosity profile
    viscprofile[i] = solutionmixture.mixture_viscosity()
```

#### Validate the simulation result

Since the RCM is a closed reactor, the total gas mass inside RCM must be kept constant. You can verify it by computing the maximum mass deviation in the solution profile. If you find the mass variations are too large to be acceptable, you can set smaller tolerance values and/or adjust some solver parameters such as the maximum solver time step size and re-run the simulation.

```
del_mass = np.zeros_like(timeprofile, dtype=np.double)
mass0 = massprofile[0]
for i in range(solutionpoints):
    del_mass[i] = abs(massprofile[i] - mass0)
#
print(f">>> Maximum magnitude of reactor mass deviation = {np.max(del_mass)} [g].")
```

#### Plot the RCM solution profiles

```
# plot the profiles
plt.subplots(2, 2, sharex="col", figsize=(12, 6))
plt.subplot(221)
plt.plot(timeprofile, tempprofile, "r-")
plt.ylabel("Temperature [K]")
plt.subplot(222)
plt.plot(timeprofile, ch4_profile, "b-")
plt.ylabel("CH4 Mole Fraction")
plt.subplot(223)
plt.plot(timeprofile, ch4_rop_profile, "g-")
plt.xlabel("time [sec]")
plt.ylabel("CH4 Production Rate [mol/cm3-sec]")
plt.subplot(224)
plt.plot(timeprofile, viscprofile, "m-")
plt.xlabel("time [sec]")
plt.ylabel("Mixture Viscosity [g/cm-sec]")

# clean up
ck.done()

# plot results
if interactive:
    plt.show()
else:
    plt.savefig("plot_RCM_solution.png", bbox_inches="tight")
```

### 22.3.4 Perform brute-force sensitivity analysis

One of the advantages of PyChemkin is customizability. You can easily build a specialized workflow to facilitate your simulation goals.

This tutorial demonstrates how to create a purpose-built workflow with PyChemkin by conducting a brute-force A-factor sensitivity analysis for ignition delay time of a premixed natural gas-air mixture at given initial temperature and pressure. Most Chemkin reactor models have the A-factor sensitivity analysis capability. The catch is that the subject variable of the analysis must be a member of the solution variables such as temperature, species mass fractions, and mass flow rate. However, for derived variables such as the ignition delay time, the built-in sensitivity analysis work as convenient. Thus, in this case, you may want to resort to the brute-force method to obtain those A-factor sensitivity coefficients with respect to the ignition delay time.

To conduct the brute-force A-factor sensitivity analysis, you will have to repeat the three steps for every reaction in the mechanism one by one

1. perturb the A-factor (the Arrhenius pre-exponent parameters) of a reaction
2. obtain the ignition delay time with this perturbed A-factor by running a constant pressure batch reactor simulation
3. restore the A-factor to its original value

The normalized ignition delay time sensitivity coefficient of reaction $j$ is the difference between the original and the perturbed ignition delay time values divided by the size of the A-factor disturbance

$$S_j = frac(I_{j,ptb} - I_{j,org})(A_{j,ptb} - A_{j,org})/A_{j,org}$$

### Import PyChemkin package and start the logger

```
from pathlib import Path
import time

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color

# chemkin batch reactor models (transient)
from ansys.chemkin.core.batchreactors.batchreactor import (
    GivenPressureBatchReactorEnergyConservation,
)
from ansys.chemkin.core.logger import logger

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(False)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create a chemistry set

The mechanism to load is the GRI 3.0 mechanism for methane combustion. This mechanism and its associated data files come with the standard Ansys Chemkin installation in the `/reaction/data` directory.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir
# create a chemistry set based on the diesel 14 components mechanism
```

(continues on next page)

(continued from previous page)

```
MyGasMech = ck.Chemistry(label="GRI 3.0")
# set mechanism input files
# including the full file path is recommended
MyGasMech.chemfile = str(mechanism_dir / "grimech30_chem.inp")
MyGasMech.thermfile = str(mechanism_dir / "grimech30_thermo.dat")
```

### Pre-process the `Chemistry Set`

```
ierror = MyGasMech.preprocess()
# check preprocess status
if ierror == 0:
    print("mechanism information:")
    print(f"number of gas species = {MyGasMech.kk:d}")
    print(f"number of gas reactions = {MyGasMech.ii_gas:d}")
else:
    # When a non-zero value is returned from the process, check the text output files
    # chem.out, tran.out, or summary.out for potential error messages about
    # the mechanism data.
    print(f"Preprocessing error encountered. Code = {ierror:d}.")
    print(f"see the summary file {MyGasMech.summaryfile} for details")
    exit()
```

### Set up gas mixtures based on the species in this `Chemistry Set`

Use the *equivalence ratio method* so that you can easily set up the premixed fuel-oxidizer mixture composition by assigning an *equivalence ratio* value. In this case, the fuel mixture consists of methane, ethane, and propane as the simulated "natural gas". The premixed air-fuel mixture has an equivalence ratio of 1.1.

```
oxid = ck.Mixture(MyGasMech)
# set mole fraction
oxid.x = [("O2", 1.0), ("N2", 3.76)]
oxid.temperature = 900
oxid.pressure = ck.P_ATM  # 1 atm

fuel = ck.Mixture(MyGasMech)
# set mole fraction
fuel.x = [("C3H8", 0.1), ("CH4", 0.8), ("H2", 0.1)]
fuel.temperature = oxid.temperature
fuel.pressure = oxid.pressure

mixture = ck.Mixture(MyGasMech)
mixture.pressure = oxid.pressure
mixture.temperature = oxid.temperature
products = ["CO2", "H2O", "N2"]
add_frac = np.zeros(MyGasMech.kk, dtype=np.double)
# create the air-fuel mixture by using the equivalence ratio method
ierror = mixture.x_by_equivalence_ratio(
    MyGasMech, fuel.x, oxid.x, add_frac, products, equivalenceratio=1.1
)
# check fuel-oxidizer mixture creation status
if ierror != 0:
```

(continues on next page)

(continued from previous page)

```
    print("Error: Failed to create the fuel-oxidizer mixture.")
    exit()
```

### List the mixture composition

list the composition of the premixed mixture for verification.

```
if ck.verbose():
    mixture.list_composition(mode="mole")
```

### Preparing for the sensitivity analysis

You need to perform some preparation work before running the brute-force sensitivity analysis. These tasks include: making backup copies of the original Arrhenius rate parameters, set up a *constant pressure* batch reactor object to compute the ignition delay times, and establish the baseline ignition delay time value with the original mechanism.

### Get the original rate parameters

The first step is to save a copy of the Arrhenius rate parameters of all reactions. You can use the `get_reaction_parameters` method associated with the `MyGasMech` object. You can also verify the rate parameters by "screening" their values.

```
a_factor, beta, active_energy = MyGasMech.get_reaction_parameters()
if ck.verbose():
    for i in range(MyGasMech.ii_gas):
        print(f"reaction: {i + 1}")
        print(f"A  = {a_factor[i]}")
        print(f"B  = {beta[i]}")
        print(f"Ea = {active_energy[i]}\n")
        if np.isclose(0.0, a_factor[i], atol=1.0e-15):
            print("reaction pre-exponential factor = 0")
            exit()
```

### Create the reactor object for ignition delay time calculations

Use the `GivenPressureBatchReactorEnergyConservation` method to instantiate a *constant pressure batch reactor that also includes the energy equation*. You should use the `mixture` you just created.

```
MyCONP = GivenPressureBatchReactorEnergyConservation(mixture, label="CONP")
# show initial gas composition inside the reactor for verification
MyCONP.list_composition(mode="mole")
```

### Set up additional reactor model parameters

*Reactor parameters*, *solver controls*, and *output instructions* need to be provided before running the simulations. For a batch reactor, the *initial volume* and the *simulation end time* are required inputs. The `set_ignition_delay` method must be included for the reactor model to report the *ignition delay times* after the simulation is done. The *inflection points* definition is employed to detect the auto-ignition time because `method="T_inflection"` is specified. You can choose a different auto-ignition definition. Allow additional solution data point to be saved so that the predicted temperature profile can have enough resolution to provide more precise ignition delay time value. Here the adoptive solution saving is turned on by the `adaptive_solution_saving` method and the solution will be recorded for every **20** solver internal steps. Remember to set a simulation end time `time` that is long enough to catch the occurrence of auto-ignition.

**Note**

By default, time intervals for both print and save solution are **1/100** of the *simulation end time*. In this case $dt = time/100 = 0.001$. You can change them to different values.

```
# reactor volume [cm3]
MyCONP.volume = 10.0
# simulation end time [sec]
MyCONP.time = 2.0

# turn ON adaptive solution saving
MyCONP.adaptive_solution_saving(mode=True, steps=20)
# set ignition delay
MyCONP.set_ignition_delay(method="T_inflection")

# set tolerances in tuple: (absolute tolerance, relative tolerance)
MyCONP.tolerances = (1.0e-10, 1.0e-8)

# set the start wall time to get the total simulation run time
start_time = time.time()
```

### Establish the baseline result

Run the nominal case and get the baseline ignition delay time value by using the `get_ignition_delay` method.

```
runstatus = MyCONP.run()
#
if runstatus == 0:
    # get ignition delay time
    delaytime_org = MyCONP.get_ignition_delay()
    print(f"ignition delay time = {delaytime_org} [msec]")
else:
    # if get this, most likely the END time is too short
    print(Color.RED + ">>> Run failed. <<<", end=Color.END)
    print("failed to find the ignition delay time of the nominal case")
    exit()
```

### Run the sensitivity analysis cases

Now compute the "raw" A-factor sensitivity coefficients of ignition delay time. Firstly, you create an array `IGsen` to store the sensitivity coefficients, the size of `ig_sen` must be no less than the number of reactions in the mechanism `MyGasMech`. Secondly, you introduce a small perturbation to the A-factor one reaction at a time by using the `set_reaction_afactor` method. The advantage of this method is that you do not need to preprocess the `Chemistry Set` every time you make a change to the rate parameter. Then you run the same batch reactor `MyCONP` to get the ignition delay time.

Once the simulation is complete successfully, use the `get_ignition_delay` method to extract the ignition delay time. Compute the difference between this ignition delay time value (with altered A-factor) and the baseline value (from the original mechanism) and save the result to array `ig_sen`. Remember to restore the A-factor to its original value before moving on to the next reaction.

```
# create sensitivity coefficient array
ig_sen = np.zeros(MyGasMech.ii_gas, dtype=np.double)
```

(continues on next page)

(continued from previous page)

```
# set perturbation magnitude
perturb = 0.001  # increase by 0.1%
perturb_plus_1 = 1.0 + perturb
# loop over all reactions
for i in range(MyGasMech.ii_gas):
    a_new = a_factor[i] * perturb_plus_1
    # actual reaction index
    ireac = i + 1
    # update the A factor
    MyGasMech.set_reaction_afactor(ireac, a_new)
    # run the reactor model
    runstatus = MyCONP.run()
    #
    if runstatus == 0:
        # get ignition delay time
        delaytime = MyCONP.get_ignition_delay()
        print(f"ignition delay time = {delaytime} [msec]")
        # compute d(delaytime)
        ig_sen[i] = delaytime - delaytime_org
        # restore the A factor
        MyGasMech.set_reaction_afactor(ireac, a_factor[i])
    else:
        # if get this, most likely the END time is too short
        print(f"trouble finding ignition delay time for raection {ireac}")
        print(Color.RED + ">>> Run failed. <<<", end=Color.END)
        exit()

# compute and report the total runtime (wall time)
runtime = time.time() - start_time
print(f"\ntotal simulation time: {runtime} [sec] over {MyGasMech.ii_gas + 1} runs")
```

### Compute the normalized sensitivity coefficients

Compute the normalized sensitivity coefficient = d(delaytime) * A[i] / d(A[i]).

```
ig_sen /= perturb
```

### Screen and rank the coefficients

Print top 5 positive and negative ignition delay time sensitivity coefficients to reveal the reactions of which the A-factor values have the strongest impact on the auto-ignition timing (positively or negatively). The ranking will change when the mixture composition or condition is changed.

```
top = 5
# rank the positive coefficients
posindex = np.argpartition(ig_sen, -top)[-top:]
poscoeffs = ig_sen[posindex]

# rank the negative coefficients
neg_ig_sen = np.negative(ig_sen)
negindex = np.argpartition(neg_ig_sen, -top)[-top:]
negcoeffs = ig_sen[negindex]
```

(continues on next page)

(continued from previous page)

```
# print the top sensitivity coefficients
if ck.verbose():
    print("positive sensitivity coefficients")
    for i in range(top):
        print(f"reaction {posindex[i] + 1}: coefficient = {poscoeffs[i]}")
    print()
    print("negative sensitivity coefficients")
    for i in range(top):
        print(f"reaction {negindex[i] + 1}: coefficient = {negcoeffs[i]}")
```

### Plot the ranked sensitivity coefficients

Create plots to show the reactions whose A-factors have most positive and negative influence on the ignition delay time.

```
plt.rcParams.update({"figure.autolayout": True, "ytick.color": "blue"})
plt.subplots(2, 1, sharex="col", figsize=(10, 5))
# convert reaction # from integers to strings
rxnstring = []
for i in range(len(posindex)):
    # the array index starting from 0 so the actual reaction # = index + 1
    rxnstring.append(MyGasMech.get_gas_reaction_string(posindex[i] + 1))
# use horizontal bar chart
plt.subplot(211)
plt.barh(rxnstring, poscoeffs, color="orange", height=0.4)
plt.axvline(x=0, c="gray", lw=1)
# convert reaction # from integers to strings
rxnstring.clear()
fnegindex = np.flip(negindex)
fnegcoeffs = np.flip(negcoeffs)
for i in range(len(negindex)):
    # the array index starting from 0 so the actual reaction # = index + 1
    rxnstring.append(MyGasMech.get_gas_reaction_string(fnegindex[i] + 1))
plt.subplot(212)
plt.barh(rxnstring, fnegcoeffs, color="orange", height=0.4)
plt.axvline(x=0, c="gray", lw=1)
plt.xlabel("Sensitivity Coefficients")
plt.suptitle("Ignition Delay Time Sensitivity", fontsize=16)

# clean up
ck.done()

# plot results
if interactive:
    plt.show()
else:
    plt.savefig("plot_sensitivity_analysis.png", bbox_inches="tight")
```

### 22.3.5 Explore cooling water vapor

How does the volume of water vapor evolves when it is cooled from a temperature above the boiling point to a temperature that is just above the freezing point at constant pressure? How fast does the vapor volume drop? This example uses the `GivenPressureBatchReactorFixedTemperature` model to explore the different behaviors between an *ideal gas* water vapor and its *real gas* counterpart.

#### Import PyChemkin packages and start the logger

```
from pathlib import Path

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color

# chemkin batch reactor model (transient)
from ansys.chemkin.core.batchreactors.batchreactor import (
    GivenPressureBatchReactorFixedTemperature,
)
from ansys.chemkin.core.logger import logger

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

#### Create a chemistry set

To compare the different behaviors of the water vapor under the ideal gas and real gas assumptions, you must use a *real gas EOS-enabled gas mechanism. The 'C2 NOx' mechanism that includes information about the *Soave* cubic* Equation of State (EOS) is suitable for this endeavor. Therefore, the `MyMech` chemistry set is created from this gas phase mechanism.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir
# create a chemistry set based on C2_NOx using an alternative method
MyMech = ck.Chemistry(label="C2 NOx")
# set mechanism input files individually
# this mechanism file contains all the necessary thermodynamic and transport data
# thus, there is no need to specify thermodynamic and transport data files
MyMech.chemfile = str(Path(mechanism_dir) / "C2_NOx_SRK.inp")
```

### Preprocess the C2 NOx chemistry set

```
# preprocess the mechanism file
ierror = MyMech.preprocess()
if ierror == 0:
    print(Color.GREEN + ">>> Preprocessing succeeded.", end=Color.END)
else:
    print(Color.RED + ">>> Preprocessing failed.", end=Color.END)
    exit()
```

### Set up the air/water vapor mixture

Create the mixture of air and water vapor inside the imaginary strong but elastic container. The initial gas temperature is set to 500 [K] and at a constant pressure of 100 [atm]. At this temperature and pressure, the mixture should be entirely in gas form.

```
mist = ck.Mixture(MyMech)
# set mole fraction
mist.x = [("H2O", 2.0), ("O2", 1.0), ("N2", 3.76)]
mist.temperature = 500.0  # [K]
mist.pressure = 100.0 * ck.P_ATM
```

### Create the reactor `tank` to perform the vapor cooling simulation

Use the `GivenPressureBatchReactorFixedTemperature()` method to create a constant-pressure batch reactor (with a given temperature). Use the `mist` mixture that you just created to set the initial gas condition inside the `tank` reactor.

```
tank = GivenPressureBatchReactorFixedTemperature(mist, label="tank")
```

### Set up additional reactor model parameters

You must provide reactor parameters, solver controls, and output instructions before running the simulations. For a batch reactor, the initial volume and simulation end time are required inputs.

```
# verify initial gas composition inside the reactor
tank.list_composition(mode="mole")

# set other reactor properties
tank.volume = 10.0  # cm3
tank.time = 0.5  # sec
```

### Set the gas temperature profile of the `tank` reactor

Create a time-temperature profile by using two arrays. Use the `set_temperature_profile()` method to add the profile to the reactor model.

```
# number of profile data points
npoints = 3
# position array of the profile data
x = np.zeros(npoints, dtype=np.double)
# value array of the profile data
tpro_profile = np.zeros_like(x, dtype=np.double)
```

(continues on next page)

(continued from previous page)

```
# set tank temperature data points
x = [0.0, 0.2, 2.0]  # [sec]
tpro_profile = [500.0, 275.0, 275.0]  # [K]
# set the temperature profile
tank.set_temperature_profile(x, tpro_profile)
```

### Switch on the real-gas EOS model

Use the `use_realgas_cubicEOS()` method to turn on the real-gas EOS model. For more information, type either `ansys.chemkin.core.help("real gas")` for information on real-gas model usage or `ansys.chemkin.core.help("manuals")` to access the online **Chemkin Theory** manual for descriptions of the real-gas EOS models.

**Note**

By default the *Van der Waals* mixing rule is applied to evaluate thermodynamic properties of a real-gas mixture. You can use `set_realgas_mixing_rule` to switch to a different mixing rule.

```
tank.userealgas_eos(mode=True)
```

### Set output options

```
# set timestep between saving solution
tank.timestep_for_saving_solution = 0.01
```

### Set solver controls

You can overwrite the default solver controls by using solver related methods, for example, `tolerances`.

```
# set tolerances in tuple: (absolute tolerance, relative tolerance)
tank.tolerances = (1.0e-10, 1.0e-8)
# get solver parameters
atol, rtol = tank.tolerances
print(f"default absolute tolerance = {atol}")
print(f"default relative tolerance = {rtol}")
# turn on the force non-negative solutions option in the solver
tank.force_nonnegative = True
```

### Run the vapor cooling simulation with the *real gas EOS*

Run the CONP reactor model with given temperature profile with the *real gas EOS* model switched *ON*.

```
runstatus = tank.run()

# check run status
if runstatus != 0:
    # run failed!
    print(Color.RED + ">>> Run failed. <<<", end=Color.END)
    exit()
# run success!
print(Color.GREEN + ">>> Run completed. <<<", end=Color.END)
```

### Postprocess the solution

The postprocessing step parses the solution and packages the solution values at each time point into a mixture. There are two ways to access the solution profiles:

- The raw solution profiles (value as a function of distance) are available for distance, temperature, pressure, volume, and species mass fractions.

  **-The mixtures permit the use of all property and rate utilities to extract** information such as viscosity, density, and mole fractions.

You can use the `get_solution_variable_profile()` method to get the raw solution profiles. You can get solution mixtures using either the `get_solution_mixture_at_index()` method for the solution mixture at the given saved location or the `get_solution_mixture()` method for the solution mixture at the given distance. (In this case, the mixture is constructed by interpolation.)

**Note**

Use the `getnumbersolutionpoints()` method to get the size of the solution profiles before creating the arrays.

```
tank.process_solution()
# get the number of solution time points
solutionpoints = tank.getnumbersolutionpoints()
print(f"number of solution points = {solutionpoints}")
# get the time profile
timeprofile = tank.get_solution_variable_profile("time")
# get the temperature profile
temp_profile = tank.get_solution_variable_profile("temperature")
# get the volume profile
vol_profile = tank.get_solution_variable_profile("volume")
# create array for mixture density
den_profile = np.zeros_like(timeprofile, dtype=np.double)
# create array for mixture enthalpy
h_profile = np.zeros_like(timeprofile, dtype=np.double)
# loop over all solution time points
for i in range(solutionpoints):
    # get the mixture at the time point
    solutionmixture = tank.get_solution_mixture_at_index(solution_index=i)
    # get mixture density profile
    den_profile[i] = solutionmixture.rho
    # get mixture enthalpy profile
    h_profile[i] = solutionmixture.hml() / ck.ERGS_PER_JOULE * 1.0e-3
```

### Turn off the real gas EOS model

Use the `use_realgas_cubicEOS()` method to turn off the real gas EOS model. Alternatively, use the `use_idealgas_law()` method to turn on the ideal gas law.

```
tank.userealgas_eos(mode=False)
```

### Run the vapor cooling simulation with the ideal gas law

Run the CONP reactor model with given temperature profile with the *ideal gas law* turned back on.

```
runstatus = tank.run()

# check run status
if runstatus != 0:
    # run failed!
    print(Color.RED + ">>> Run failed. <<<", end=Color.END)
    exit()
# run success!
print(Color.GREEN + ">>> Run completed. <<<", end=Color.END)
```

### Postprocess the ideal gas solution

postprocess the solutions

```
tank.process_solution()
# get the number of solution time points
solutionpoints = tank.getnumbersolutionpoints()
print(f"number of solution points = {solutionpoints}")
# get the time profile
timeprofile_ig = tank.get_solution_variable_profile("time")
# get the volume profile
vol_profile_ig = tank.get_solution_variable_profile("volume")
# create array for mixture density
den_profile_ig = np.zeros_like(timeprofile, dtype=np.double)
# create array for mixture enthalpy
h_profile_ig = np.zeros_like(timeprofile, dtype=np.double)
# loop over all solution time points
for i in range(solutionpoints):
    # get the mixture at the time point
    solutionmixture = tank.get_solution_mixture_at_index(solution_index=i)
    # get mixture density profile
    den_profile_ig[i] = solutionmixture.rho
    # get mixture enthalpy profile
    h_profile_ig[i] = solutionmixture.hml() / ck.ERGS_PER_JOULE * 1.0e-3

ck.done()
```

### Plot the RCM solution profiles

You should observe that the mixture volume obtained by the real gas model is noticeably lower than the ideal gas volume. When water vapor is cooled below the boiling point, formation of liquid water is expected due to condensation. The ideal gas law assumes that the mixture is always in gas phase and is unable to address the phase change phenomenon. The real gas EOS, on the other hand, can capture the formation of the liquid water as indicated by the sharper rise of the mixture density during the cooling process. The real gas mixture also has a lower enthalpy level than that of the ideal gas mixture. The enthalpy differences should largely represent the heat of vaporization of water at the temperature.

```
# plot the profiles
plt.subplots(2, 2, sharex="col", figsize=(12, 6))
thispres = str(mist.pressure / ck.P_ATM)
thistitle = "Cooling Vapor + Air at " + thispres + " atm"
```

(continues on next page)

(continued from previous page)

```
plt.suptitle(thistitle, fontsize=16)
plt.subplot(221)
plt.plot(timeprofile, temp_profile, "r-")
plt.ylabel("Temperature [K]")
plt.subplot(222)
plt.plot(timeprofile, vol_profile, "b-", label="real gas")
plt.plot(timeprofile_ig, vol_profile_ig, "b--", label="ideal gas")
plt.legend(loc="upper right")
plt.ylabel("Volume [cm3]")
plt.subplot(223)
plt.plot(timeprofile, h_profile, "g-", label="real gas")
plt.plot(timeprofile_ig, h_profile_ig, "g--", label="ideal gas")
plt.legend(loc="upper right")
plt.xlabel("time [sec]")
plt.ylabel("Mixture Enthalpy [kJ/mole]")
plt.subplot(224)
plt.plot(timeprofile, den_profile, "m-", label="real gas")
plt.plot(timeprofile_ig, den_profile_ig, "m--", label="ideal gas")
plt.legend(loc="upper left")
plt.xlabel("time [sec]")
plt.ylabel("Mixture Density [g/cm3]")

# clean up
ck.done()

# plot results
if interactive:
    plt.show()
else:
    plt.savefig("plot_vapor_condensation.png", bbox_inches="tight")
```

## 22.4 0-D engine models

**PyChemkin** offers two types of 0-D engine models: the *Homogeneous Charged Compression Ignition (HCCI)* engine model and the *Spark Ignition (SI)* engine model. These 0-D engine models simulate the combustion process (or the lack of, in case of a misfire) inside an engine cylinder between the *Intake Valve Close (IVC)* and the *Exhaust Valve Open (EVO)* when the cylinder is considered as a closed system. The Chemkin *Theory* manual has detailed descriptions of these 0-D engine models.

The examples show the steps of setting up simulations with different *Chemkin* engine models.

### 22.4.1 Simulate a single-zone HCCI engine

Ansys Chemkin offers some idealized internal combustion (IC) engine models commonly used for fuel combustion and engine performance research. The Chemkin IC engine model is a specialized transient 0-D *closed* gas-phase reactor that mainly performs combustion simulation between the intake valve closing (IVC) and the exhaust valve opening (EVO), that is, when the engine cylinder resembles a closed chamber. The cylinder volume is derived from the piston motion as a function of the engine crank angle (CA) and engine parameters such as engine speed (RPM) and stroke. The energy equation is always solved, and there are several wall heat transfer models specifically designed for engine simulations.

**Note**

For additional information on Chemkin IC engine models, use the `ansys.chemkin.core.manuals()` method to view the online **Theory** manual.

This example shows how to set up and run the simplest Chemkin IC engine model: the single-zone homogeneous charged compression ignition (HCCI) engine model. In addition to the basic engine parameters, many engine model-specific features such as the *exhaust gas recirculation* and *wall heat transfer* can be included in the engine simulation.

### Import PyChemkin packages and start the logger

```
from pathlib import Path

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color

# chemkin homonegeous charge compression ignition (HCCI) engine model (transient)
from ansys.chemkin.core.engines.HCCI import HCCIengine
from ansys.chemkin.core.logger import logger

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create a chemistry set

The mechanism to load is the GRI 3.0 mechanism for methane combustion. This mechanism and its associated data files come with the standard Ansys Chemkin installation in the `/reaction/data` directory.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir
# create a chemistry set based on the GRI mechanism
MyGasMech = ck.Chemistry(label="GRI 3.0")
# set mechanism input files
# including the full file path is recommended
MyGasMech.chemfile = str(mechanism_dir / "grimech30_chem.inp")
MyGasMech.thermfile = str(mechanism_dir / "grimech30_thermo.dat")
MyGasMech.tranfile = str(mechanism_dir / "grimech30_transport.dat")
```

### Preprocess the chemistry set

```
# preprocess the mechanism files
ierror = MyGasMech.preprocess()
if ierror != 0:
    msg = "preprocess failed"
    logger.critical("msg")
    Color.ckprint("critical", [msg, "!!!"])
    exit()
```

### Set up the fuel-air mixture

You must set up the fuel-air mixture inside the engine cylinder right after the intake valve is closed. Here the `x_by_equivalence_ratio()` method is used. You create the `fuelmixture` and `air` mixtures first. You then define the *complete combustion product species* and provide the *additives* composition if there is any. And finally, you can simply set the value of `equivalenceratio` to create the fuel-air mixture. In this case, the fuel mixture consists of methane, ethane, and propane as the simulated natural gas. Because HCCI engines typically run on lean fuel-air mixtures, the equivalence ratio is set to 0.8.

```
# create a premixed fuel-oxidizer mixture by assigning the equivalence ratio
# create the fuel mixture
fuelmixture = ck.Mixture(MyGasMech)
# set fuel composition
fuelmixture.x = [("CH4", 0.9), ("C3H8", 0.05), ("C2H6", 0.05)]
# setting pressure and temperature is not required in this case
fuelmixture.pressure = 1.5 * ck.P_ATM
fuelmixture.temperature = 400.0

# create the oxidizer mixture: air
air = ck.Mixture(MyGasMech)
air.x = [("O2", 0.21), ("N2", 0.79)]
# setting pressure and temperature is not required in this case
air.pressure = 1.5 * ck.P_ATM
air.temperature = 400.0

# products from the complete combustion of the fuel mixture and air
products = ["CO2", "H2O", "N2"]
# species mole fractions of added/inert mixture.
# You can also create an additives mixture here.
add_frac = np.zeros(MyGasMech.kk, dtype=np.double)  # no additives: all zeros

# create the unburned fuel-air mixture
fresh = ck.Mixture(MyGasMech)
# mean equivalence ratio
equiv = 0.8
ierror = fresh.x_by_equivalence_ratio(
    MyGasMech, fuelmixture.x, air.x, add_frac, products, equivalenceratio=equiv
)
# check fuel-oxidizer mixture creation status
if ierror != 0:
    print("Error: Failed to create the fuel-oxidizer mixture.")
    exit()
```

(continues on next page)

(continued from previous page)

```
# list the composition of the unburned fuel-air mixture
fresh.list_composition(mode="mole")
```

### Specify pressure and temperature of the fuel-air mixture

Since you are going to use `fresh` to instantiate the engine object later, setting the mixture pressure and temperature is equivalent to setting the initial temperature and pressure of the engine cylinder.

```
fresh.temperature = 447.0
fresh.pressure = 1.065 * ck.P_ATM
```

### Add EGR to the fresh fuel-air mixture

Many engines have the configuration for exhaust gas recirculation (EGR). Chemkin engine models allow you to add the EGR mixture to the fresh fuel-air mixture entered the cylinder. If the engine that you are modeling has EGR, you should have the EGR ratio, which is generally the volume ratio between the EGR mixture and the fresh fuel-air ratio. However, you know nothing about the composition of the exhaust gas so you cannot simply combine these two mixtures. In this case, you can use the `get_egr_mole_fraction()` method to estimate the major components of the exhaust gas from the combustion of the fresh fuel-air mixture. The parameter `threshold=1.0e-8` tells the method to ignore any species with mole fractions below the threshold value. Once you have the EGR mixture composition, use the `x_by_equivalence_ratio()` method a second time to re-create the fuel-air mixture `fresh` with the original `fuelmixture` and `air` mixtures along with the EGR composition that you just got as the *additives*.

```
egr_ratio = 0.3
# compute the EGR stream composition in mole fractions
add_frac = fresh.get_egr_mole_fraction(egr_ratio, threshold=1.0e-8)
# re-create the initial mixture with EGR
_ = fresh.x_by_equivalence_ratio(
    MyGasMech,
    fuelmixture.x,
    air.x,
    add_frac,
    products,
    equivalenceratio=equiv,
    threshold=1.0e-8,
)

# list the composition of the fuel+air+EGR mixture for verification
fresh.list_composition(mode="mole", bound=1.0e-8)
```

### Set up the HCCI engine reactor

Use the `HCCIengine()` method to create a single-zone HCCI engine named `MyEngine` and make the *new* `fresh` mixture the initial in-cylinder gas mixture at IVC. Set the `nzones` parameter to 1 for the *single-zone* HCCI simulation.

```
MyEngine = HCCIengine(reactor_condition=fresh, nzones=1)
# show initial gas composition inside the reactor
MyEngine.list_composition(mode="mole", bound=1.0e-8)
```

### Set up basic engine parameters

Set the required engine parameters as shown in the following code. These engine parameters are used to describe the cylinder volume during the simulation. The `starting_ca` should be the crank angle corresponding to the cylinder IVC. The `ending_ca` is typically the EVC crank angle.

```
# cylinder bore diameter [cm]
MyEngine.bore = 12.065
# engine stroke [cm]
MyEngine.stroke = 14.005
# connecting rod length [cm]
MyEngine.connecting_rod_length = 26.0093
# compression ratio [-]
MyEngine.compression_ratio = 16.5
# engine speed [RPM]
MyEngine.rpm = 1000

# set piston pin offset distance [cm] (optional)
MyEngine.set_piston_pin_offset(offset=-0.5)

# set other required parameters
# simulation start CA [degree]
MyEngine.starting_ca = -142.0
# simulation end CA [degree]
MyEngine.ending_ca = 116.0

# list the engine parameters for verification
MyEngine.list_engine_parameters()
print(f"Engine displacement volume = {MyEngine.get_displacement_volume()} [cm3].")
print(f"Engine clearance volume = {MyEngine.get_clearance_volume()} [cm3].")
print(f"Number of zones = {MyEngine.get_number_of_zones()}.")
```

### Set up engine wall heat transfer model

By default, the engine cylinder is adiabatic. You must set up a wall heat transfer model to include the heat loss effects in your engine simulation. Chemkin supports three widely used engine wall heat transfer models. These models and their parameters follow:

- `dimensionless`: [<a> <b> <c> <Twall>]
- `dimensional`: [<a> <b> <c> <Twall>]
- `hohenburg`: [<a> <b> <c> <d> <e> <Twall>]

There is also the incylinder gas velocity correlation (the Woschni correlation) that is associated with the engine wall heat transfer models. Here are the parameters of the Woschni correlation:

`[<C11> <C12> <C2> <swirl ratio>]`

You can also specify the surface areas of the piston head and the cylinder head for more precision heat transfer wall area. By default, both the piston head and the cylinder head surfaces are flat.

```
heattransferparameters = [0.035, 0.71, 0.0]
# set cylinder wall temperature [K]
t_wall = 400.0
MyEngine.set_wall_heat_transfer("dimensionless", heattransferparameters, t_wall)
# in-cylinder gas velocity correlation parameter (Woschni)
```

(continues on next page)

(continued from previous page)

```
# [<C11> <C12> <C2> <swirl ratio>]
gv_parameters = [2.28, 0.308, 3.24, 0.0]
MyEngine.set_gas_velocity_correlation(gv_parameters)
# set piston head top surface area [cm2]
MyEngine.set_piston_head_area(area=124.75)
# set cylinder clearance surface area [cm2]
MyEngine.set_cylinder_head_area(area=123.5)
```

### Set output options

You can turn on adaptive solution saving to resolve the steep variations in the solution profile. Here additional solution data points are saved for every 20 solver internal steps. You must include the `set_ignition_delay()` method for the engine model to report the ignition delay crank angle after the simulation is done. If `method="T_inflection"` is set, the reactor model treats the inflection points in the predicted gas temperature profile as the indication of an auto-ignition. You can choose a different auto-ignition definition.

**Note**

Type `ansys.chemkin.core.show_ignition_definitions()` to get the list of all available ignition delay time definitions in Chemkin.

**Note**

By default, time/crank angle intervals for both print and save solution are 1/100 of the simulation duration, which is in this case $dCA = (EVO - IVC)/100 = 2.58$. You can make the model report more frequently by using the `ca_step_for_saving_solution()` or `ca_step_for_printing_solution()` method to set different interval values in the crank angle.

```
# set the number of crank angles between saving solution
MyEngine.ca_step_for_saving_solution = 0.5
# set the number of crank angles between printing solution
MyEngine.ca_step_for_printing_solution = 10.0
# turn on adaptive solution saving
MyEngine.adaptive_solution_saving(mode=True, steps=20)
# specify the ignition definitions
MyEngine.set_ignition_delay(method="T_inflection")
```

### Set solver controls

You can overwrite the default solver controls by using solver-related methods, such as those for tolerances.

```
# set tolerances in tuple: (absolute tolerance, relative tolerance)
MyEngine.tolerances = (1.0e-12, 1.0e-10)
# get solver parameters
atol, rtol = MyEngine.tolerances
print(f"Default absolute tolerance = {atol}.")
print(f"Default relative tolerance = {rtol}.")
# turn on the force non-negative solutions option in the solver
MyEngine.force_nonnegative = True
```

(continues on next page)

(continued from previous page)

```
# show solver and output options
# show the number of crank angles between printing solution
print(
    f"Crank angles between solution printing: {MyEngine.ca_step_for_printing_solution}"
)
# show other transient solver setup
print(f"Forced non-negative solution values: {MyEngine.force_nonnegative}")
```

### Display the added parameters (keywords)

Use the `showkeywordinputlines()` method to verify that the preceding parameters are correctly assigned to the engine model.

```
MyEngine.showkeywordinputlines()
```

### Run the simulation

Use the `run()` method to start the single-zone HCCI engine simulation.

```
runstatus = MyEngine.run()
# check run status
if runstatus != 0:
    # Run failed.
    print(Color.RED + ">>> Run failed. <<<", end=Color.END)
    logger.error("Run failed.")
    exit()
# Run succeeded.
print(Color.GREEN + ">>> Run completed. <<<", end=Color.END)
logger.info("Run Completed.")
```

### Get the ignition delay crank angle from the solution

Use the `get_ignition_delay()` method to extract the ignition delay crank angle (CA) after the run is completed.

```
# get ignition delay "time"
delay_ca = MyEngine.get_ignition_delay()
print(f"Ignition delay CA = {delay_ca} [degree].")
```

### Get the heat release crank angles

The engine models also report the crank angles when the accumulated heat release reaches 10%, 50%, and 90% of the total heat release. Use the `get_engine_heat_release_cas` method to extract these heat release crank angles (CA).

```
# get heat release information
hr10, hr50, hr90 = MyEngine.get_engine_heat_release_cas()
print("Engine Heat Release Information")
print(f"10% heat release CA = {hr10} [degree].")
print(f"50% heat release CA = {hr50} [degree].")
print(f"90% heat release CA = {hr90} [degree].\n")
```

### Postprocess the solution

The postprocessing step parses the solution and packages the solution values at each time point into a `Mixture` object. There are two ways to access the solution profiles:

- The raw solution profiles (value as a function of time) are available for time, temperature, pressure, volume, and species mass fractions.
- The mixtures permit the use of all property and rate utilities to extract information such as viscosity, density, and mole fractions.

You can use the `get_solution_variable_profile()` method to get the raw solution profiles. You can get solution mixtures using either the `get_solution_mixture_at_index()` method for the solution mixture at a given time point or the `get_solution_mixture()` method for the solution mixture at a given time. (In this case, the mixture is constructed by interpolation.)

**Note**

- For engine models, use the `process_engine_solution()` method to postprocess the solutions.
- Use the `getnumbersolutionpoints()` method to get the size of the solution profiles before creating the arrays.
- Use the `get_ca()` method to convert the time values reported in the solution to crank angles.

```
# postprocess the solutions
MyEngine.process_engine_solution()
# get the number of solution time points
solutionpoints = MyEngine.getnumbersolutionpoints()
print(f"Number of solution points = {solutionpoints}.")
# get the time profile
timeprofile = MyEngine.get_solution_variable_profile("time")
# convert time to crank angle
ca_profile = np.zeros_like(timeprofile, dtype=np.double)
count = 0
for t in timeprofile:
    ca_profile[count] = MyEngine.get_ca(timeprofile[count])
    count += 1
# get the cylinder pressure profile
presprofile = MyEngine.get_solution_variable_profile("pressure")
presprofile *= 1.0e-6
# get the volume profile
volprofile = MyEngine.get_solution_variable_profile("volume")
# create arrays for mixture density, NO mole fraction,
# and mixture-specific heat capacity
denprofile = np.zeros_like(timeprofile, dtype=np.double)
cp_profile = np.zeros_like(timeprofile, dtype=np.double)
# loop over all solution time points
for i in range(solutionpoints):
    # get the mixture at the time point
    solutionmixture = MyEngine.get_solution_mixture_at_index(solution_index=i)
    # get gas density [g/cm3]
    denprofile[i] = solutionmixture.rho
    # get mixture-specific heat capacity profile [erg/mole-K]
    cp_profile[i] = solutionmixture.cpbl() / ck.ERGS_PER_JOULE * 1.0e-3
```

### Plot the engine solution profiles

Plot the profiles from the HCCI engine simulation.

**Note**

You can get profiles of the thermodynamic and the transport properties by applying `Mixture` utility methods to the solution mixtures.

```
plt.subplots(2, 2, sharex="col", figsize=(12, 6))
plt.subplot(221)
plt.plot(ca_profile, presprofile, "r-")
plt.ylabel("Pressure [bar]")
plt.subplot(222)
plt.plot(ca_profile, volprofile, "b-")
plt.ylabel("Volume [cm3]")
plt.subplot(223)
plt.plot(ca_profile, denprofile, "g-")
plt.xlabel("Crank Angle [degree]")
plt.ylabel("Mixture Density [g/cm3]")
plt.subplot(224)
plt.plot(ca_profile, cp_profile, "m-")
plt.xlabel("Crank Angle [degree]")
plt.ylabel("Mixture Cp [kJ/mole]")

# clean up
ck.done()

# plot results
if interactive:
    plt.show()
else:
    plt.savefig("plot_HCCI_engine.png", bbox_inches="tight")
```

## 22.4.2 Simulate a multi-zone HCCI engine

Ansys Chemkin offers some idealized internal combustion (IC) engine models commonly used for fuel combustion and engine performance research. The Chemkin IC engine model is a specialized transient 0-D *closed* gas-phase reactor that mainly performs combustion simulation between the intake valve closing (IVC) and the exhaust valve opening (EVO), that is, when the engine cylinder resembles a closed chamber. The cylinder volume is derived from the piston motion as a function of the engine crank angle (CA) and engine parameters such as engine speed (RPM) and stroke. The energy equation is always solved and there are several wall heat transfer models specifically designed for engine simulations.

**Note**

For additional information on the Chemkin IC engine models, use the `ansys.chemkin.core.manuals()` method to view the online **Theory** manual.

The multi-zone homogeneous charged compression ignition (HCCI) model is mainly intended to address the temperature variation inside the cylinder caused by the wall heat transfer and the imperfect mixing of the in-cylinder gas mixture. In addition, the multi-zone HCCI engine model allows the introduction of non-uniform temperature and/or

the equivalence ratio distribution of the gas mixture at the IVC.

This example shows how to set up and run the Chemkin multi-zone HCCI engine model. Many engine model-specific features, such as basic engine parameters, exhaust gas recirculation, and wall heat transfer, are applied to the engine simulation. This example also shows how to create initial distributions of zone size, temperature, and composition.

### Import PyChemkin packages and start the logger

```
from pathlib import Path

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color

# Chemkin homogeneous charge compression ignition (HCCI) engine model (transient)
from ansys.chemkin.core.engines.HCCI import HCCIengine
from ansys.chemkin.core.logger import logger

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create a chemistry set

The mechanism to load is the GRI 3.0 mechanism for methane combustion. This mechanism and its associated data files come with the standard Ansys Chemkin installation in the `/reaction/data` directory.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir
# create a chemistry set based on the GRI mechanism
MyGasMech = ck.Chemistry(label="GRI 3.0")
# set mechanism input files
# including the full file path is recommended
MyGasMech.chemfile = str(mechanism_dir / "grimech30_chem.inp")
MyGasMech.thermfile = str(mechanism_dir / "grimech30_thermo.dat")
MyGasMech.tranfile = str(mechanism_dir / "grimech30_transport.dat")
```

### Preprocess the chemistry set

```
# preprocess the mechanism files
_ = MyGasMech.preprocess()
```

### Set up the fuel-air mixture

You must set up the fuel-air mixture inside the engine cylinder right after the intake valve is closed. Here the `x_by_equivalence_ratio()` method is used. You create the `fuelmixture` and the `air` mixtures first. You then define the *complete combustion product species* and provide the *additives* composition if there is any. Finally, you set the `equivalenceratio` value to create the fuel-air mixture. In this case, the fuel mixture consists of methane, ethane, and propane as the simulated natural gas. Because HCCI engines generally run on lean fuel-air mixtures, the equivalence ratio is set to 0.8.

```
# create the fuel mixture
fuelmixture = ck.Mixture(MyGasMech)
# set fuel composition
fuelmixture.x = [("CH4", 0.9), ("C3H8", 0.05), ("C2H6", 0.05)]
# setting pressure and temperature is not required in this case
fuelmixture.pressure = 1.5 * ck.P_ATM
fuelmixture.temperature = 400.0
# create the oxidizer mixture: air
air = ck.Mixture(MyGasMech)
air.x = [("O2", 0.21), ("N2", 0.79)]
# setting pressure and temperature is not required in this case
air.pressure = 1.5 * ck.P_ATM
air.temperature = 400.0
# create the unburned fuel-air mixture
fresh = ck.Mixture(MyGasMech)
# products from the complete combustion of the fuel mixture and air
products = ["CO2", "H2O", "N2"]
# species mole fractions of added/inert mixture.
# can also create an additives mixture here
add_frac = np.zeros(MyGasMech.kk, dtype=np.double)  # no additives: all zeros
# mean equivalence ratio
equiv = 0.8
ierror = fresh.x_by_equivalence_ratio(
    MyGasMech, fuelmixture.x, air.x, add_frac, products, equivalenceratio=equiv
)
# check fuel-oxidizer mixture creation status
if ierror != 0:
    print("Error: Failed to create the fuel-oxidizer mixture.")
    exit()

# list the composition of the unburned fuel-air mixture
fresh.list_composition(mode="mole")
```

### Specify pressure and temperature of the fuel-air mixture

Since you are going to use the `fresh` fuel-air mixture to instantiate the engine object later, setting the mixture pressure and temperature is equivalent to setting the initial temperature and pressure of the engine cylinder.

```
fresh.temperature = 447.0
fresh.pressure = 1.065 * ck.P_ATM
```

### Add EGR to the fresh fuel-air mixture

Many engines have the configuration for exhaust gas recirculation (EGR). Chemkin engine models allow you to add the EGR mixture to the fresh fuel-air mixture entered the cylinder. If the engine you are modeling has EGR, you should have the EGR ratio, which is generally the volume ratio between the EGR mixture and the fresh fuel-air ratio. However, because you know nothing about the composition of the exhaust gas, you cannot simply combine these two mixtures. In this case, you use the `get_egr_mole_fraction()` method to estimate the major components of the exhaust gas from the combustion of the fresh fuel-air mixture. The `threshold=1.0e-8` parameter tells the method to ignore any species with a mole fraction below the threshold value. Once you have the EGR mixture composition, use the `x_by_equivalence_ratio()` method a second time to re-create the `fresh` fuel-air mixture with the original `fuelmixture` and `air` mixtures along with the EGR composition you just got as the *"additives"*.

```
egr_ratio = 0.3
# compute the EGR stream composition in mole fractions
add_frac = fresh.get_egr_mole_fraction(egr_ratio, threshold=1.0e-8)
# recreate the initial mixture with EGR
_ = fresh.x_by_equivalence_ratio(
    MyGasMech,
    fuelmixture.x,
    air.x,
    add_frac,
    products,
    equivalenceratio=equiv,
    threshold=1.0e-8,
)

# list the composition of the fuel+air+EGR mixture for verification
fresh.list_composition(mode="mole", bound=1.0e-8)
```

### Set up the HCCI engine reactor

Use the `HCCIengine()` method to create a multi-zone HCCI engine named `MyMZEngine` and make the new `fresh` mixture as the initial incylinder gas mixture at IVC. Set the `nzones` parameter to the number of zones in your multi-zone HCCI engine model.

```
# create a five-zone HCCI engine
numbzones = 5
MyMZEngine = HCCIengine(reactor_condition=fresh, nzones=numbzones)
# show initial gas composition inside the reactor
MyMZEngine.list_composition(mode="mole", bound=1.0e-8)
```

### Set basic engine parameters

Set the required engine parameters as shown in the following code. These engine parameters are used to describe the cylinder volume during the simulation. The `starting_ca` argument should be the crank angle corresponding to the cylinder IVC. The `ending_ca` is typically the EVC crank angle.

```
# cylinder bore diameter [cm]
MyMZEngine.bore = 12.065
# engine stroke [cm]
MyMZEngine.stroke = 14.005
# connecting rod length [cm]
MyMZEngine.connecting_rod_length = 26.0093
# compression ratio [-]
```

(continues on next page)

(continued from previous page)

```
MyMZEngine.compression_ratio = 16.5
# engine speed [RPM]
MyMZEngine.rpm = 1000

# set other parameters
# simulation start CA [degree]
MyMZEngine.starting_ca = -142.0
# simulation end CA [degree]
MyMZEngine.ending_ca = 116.0

# list the engine parameters
MyMZEngine.list_engine_parameters()
print(f"engine displacement volume {MyMZEngine.get_displacement_volume()} [cm3]")
print(f"engine clearance volume {MyMZEngine.get_clearance_volume()} [cm3]")
print(f"number of zone(s) = {MyMZEngine.get_number_of_zones()}")
```

### Set up the engine wall heat transfer model

By default, the engine cylinder is adiabatic. You must set up a wall heat transfer model to include the heat loss effects in your engine simulation. Chemkin support three widely used engine wall heat transfer models. The models and their parameters follow.

- `dimensionless`: [<a> <b> <c> <Twall>]
- `dimensional`: [<a> <b> <c> <Twall>]
- `hohenburg`: [<a> <b> <c> <d> <e> <Twall>]

There is also the in-cylinder gas velocity correlation (the Woschni correlation) that is associated with the engine wall heat transfer models. Here are the parameters of the Woschni correlation:

`[<C11> <C12> <C2> <swirl ratio>]`

You can also specify the surface areas of the piston head and the cylinder head for more precision heat transfer wall area. By default, both the piston head and the cylinder head surfaces are flat.

```
heattransferparameters = [0.035, 0.71, 0.0]
# set cylinder wall temperature [K]
t_wall = 400.0
MyMZEngine.set_wall_heat_transfer("dimensionless", heattransferparameters, t_wall)
# in-cylinder gas velocity correlation parameter (Woschni)
# [<C11> <C12> <C2> <swirl ratio>]
gv_parameters = [2.28, 0.308, 3.24, 0.0]
MyMZEngine.set_gas_velocity_correlation(gv_parameters)
# set piston head top surface area [cm2]
MyMZEngine.set_piston_head_area(area=124.75)
# set cylinder clearance surface area [cm2]
MyMZEngine.set_cylinder_head_area(area=123.5)
```

### Set zonal properties

By default, all zones in the multi-zone HCCI engine model have the same properties. You can artificially stratify the temperature and/or the equivalence ratio distribution in the cylinder at the IVC by utilizing the `set_zonal` methods of the `HCCI` object.

```
# zonal temperatures [K]
ztemperature = [447.5, 447.5, 447, 447, 447]
MyMZEngine.set_zonal_temperature(zonetemp=ztemperature)
# zonal volume fractions
zvolumefrac = [0.3, 0.25, 0.2, 0.2, 0.05]
MyMZEngine.set_zonal_volume_fraction(zonevol=zvolumefrac)
# wall heat transfer area fractions
zht_area = [0.0, 0.15, 0.2, 0.25, 0.4]
MyMZEngine.set_zonal_heat_transfer_area_fraction(zonearea=zht_area)
# zonal equivalence ratios
zphi = [equiv, equiv, equiv, equiv, equiv]
MyMZEngine.set_zonal_equivalence_ratio(zonephi=zphi)
# zonal EGR ratios
zegrr = [0.3, 0.3, 0.3, 0.35, 0.35]
MyMZEngine.set_zonal_egr_ratio(zoneegr=zegrr)
# set fuel "molar" composition
MyMZEngine.define_fuel_composition([("CH4", 0.9), ("C3H8", 0.05), ("C2H6", 0.05)])
# set oxidizer "molar' composition
MyMZEngine.define_oxid_composition([("O2", 0.21), ("N2", 0.79)])
# set products
MyMZEngine.define_product_composition(["CO2", "H2O", "N2"])
# set EGR composition in mole fractions
zadd = [add_frac, add_frac, add_frac, add_frac, add_frac]
MyMZEngine.define_additive_fractions(addfrac=zadd)
```

### Set output options

You can turn on the adaptive solution saving to resolve the steep variations in the solution profile. Here additional solution data points are saved for every 20*solver internal steps. You must include the `set_ignition_delay()` method for the engine model to report the ignition delay crank angle after the simulation is done. If `method="T_inflection"` is set, the reactor model treats the inflection points in the predicted gas temperature profile as the indication of an auto-ignition. You can choose a different auto-ignition definition.

**Note**

- Type `ansys.chemkin.core.show_ignition_definitions()` to get the list of all available ignition delay time definitions in Chemkin.
- The `set_ignition_delay()` method is required for the engine model to report the ignition delay time for each zone as well as the cylinder averaged ignition delay time derived from the cylinder averaged temperature profile.
- By default, time/crank angle intervals for both print and save solution are 1/100 of the simulation duration, which in this case is $dCA = (EVO - IVC)/100 = 2.58$. You can make the model report more frequently by using the `ca_step_for_saving_solution()` or the `ca_step_for_printing_solution()` method to set different interval values in the crank angle.

```
# set the number of crank angles between saving solution
MyMZEngine.ca_step_for_saving_solution = 0.5
# set the number of crank angles between printing solution
MyMZEngine.ca_step_for_printing_solution = 10.0
# turn on adaptive solution saving
```

(continues on next page)

(continued from previous page)

```
MyMZEngine.adaptive_solution_saving(mode=True, steps=20)
# specify the ignition definitions
MyMZEngine.set_ignition_delay(method="T_inflection")
```

### Set solver controls

You can overwrite the default solver controls by using solver-related methods, such as those for tolerances.

```
# set tolerances in tuple: (absolute tolerance, relative tolerance)
MyMZEngine.tolerances = (1.0e-12, 1.0e-10)
# get solver parameters
atol, rtol = MyMZEngine.tolerances
print(f"Default absolute tolerance = {atol}.")
print(f"Default relative tolerance = {rtol}")
# turn on the force non-negative solutions option in the solver
MyMZEngine.force_nonnegative = True
# show solver and output options
# show the number of crank angles between printing solution
print(
    f"Crank angles between solution printing: "
    f"{MyMZEngine.ca_step_for_printing_solution}"
)
# show other transient solver setup
print(f"Forced non-negative solution values: {MyMZEngine.force_nonnegative}")
```

### Display the added parameters (keywords)

Use the `showkeywordinputlines()` method to verify that the preceding parameters are correctly assigned to the engine model.

```
MyMZEngine.showkeywordinputlines()
```

### Run the simulation

Use the `run()` method to start the multi-zone HCCI engine simulation.

```
runstatus = MyMZEngine.run()
# check run status
if runstatus != 0:
    # Run failed.
    print(Color.RED + ">>> Run failed. <<<", end=Color.END)
    exit()
# Run succeeded.
print(Color.GREEN + ">>> Run completed. <<<", end=Color.END)
```

### Get the ignition delay crank angle from the solution

Use the `get_ignition_delay()` method to extract the cylinder averaged ignition delay crank angle (CA) after the run is completed.

```
# get ignition delay "time"
delay_ca = MyMZEngine.get_ignition_delay()
print(f"Ignition delay CA = {delay_ca} [degree].")
```

### Get the heat release crank angles

The engine models also report the crank angles when the accumulated heat release reaches 10%, 50%, and 90% of the total heat release. Use the `get_engine_heat_release_cas()` method to extract these heat release crank angles (CA).

```
hr10, hr50, hr90 = MyMZEngine.get_engine_heat_release_cas()
print("Engine Heat Release Information:")
print(f"10% heat release CA = {hr10} [degree].")
print(f"50% heat release CA = {hr50} [degree].")
print(f"90% heat release CA = {hr90} [degree].\n")
```

Postprocess the solution ==========_== The postprocessing step parses the solution and packages the solution values at each time point into a mixture. There are two ways to access the solution profiles:

- The raw solution profiles (value as a function of time) are available for time, temperature, pressure, volume, and species mass fractions.
- The mixtures permit the use of all property and rate utilities to extract information such as viscosity, density, and mole fractions.

You can use the `get_solution_variable_profile()` method to get the raw solution profiles. You can get solution mixtures using either the `get_solution_mixture_at_index()` method for the solution mixture at a given time point or the `get_solution_mixture()` method for the solution mixture at a given time. (In this case, the mixture is constructed by interpolation.)

**Note**

- For engine models, use the `process_engine_solution()` method to postprocess the solutions.
- Use the `getnumbersolutionpoints()` method to get the size of the solution profiles before creating the arrays.
- Use the `get_ca()` method to convert the time values reported in the solution to crank angles.

### Postprocess the solution profiles in selected zone

The solution of the multi-zone HCCI engine model contains the results of the individual zones plus the cylinder averaged results. This means that if there are n zones in the multi-zone engine model, there are (n+1) solution records: n zonal results and the cylinder averaged results.

To process the result of the zone number $j$, $(1 \leq j \leq n)$, set the parameter value of `zone_id` to $j$ when you call the engine postprocessor with the `process_engine_solution()` method. Otherwise, the cylinder averaged results are postprocessed by default, that is, when the `zone_id` parameter is omitted.

**Note**

Because The `process_engine_solution()` method can process only one set of results at a time (one zonal result or the cylinder averaged result), you must postprocess the zones one by one to obtain all solution data of the multi-zone simulation.

```
thiszone = 1
MyMZEngine.process_engine_solution(zone_id=thiszone)
plottitle = "Zone " + str(thiszone) + " Solution"
# get the number of solution time points
```

(continues on next page)

(continued from previous page)

```
solutionpoints = MyMZEngine.getnumbersolutionpoints()
print(f"Number of solution points = {solutionpoints}.")
# get the time profile
timeprofile = MyMZEngine.get_solution_variable_profile("time")
# convert time to crank angle
ca_profile = np.zeros_like(timeprofile, dtype=np.double)
count = 0
for t in timeprofile:
    ca_profile[count] = MyMZEngine.get_ca(timeprofile[count])
    count += 1
# get the cylinder pressure profile
presprofile = MyMZEngine.get_solution_variable_profile("pressure")
presprofile *= 1.0e-6
# get the zonal volume profile
volprofile = MyMZEngine.get_solution_variable_profile("volume")
# create arrays for zonal mixture density and mixture specific heat capacity
denprofile = np.zeros_like(timeprofile, dtype=np.double)
viscprofile = np.zeros_like(timeprofile, dtype=np.double)
# loop over all solution time points
for i in range(solutionpoints):
    # get the zonal mixture at the time point
    solutionmixture = MyMZEngine.get_solution_mixture_at_index(solution_index=i)
    # get zonal gas density [g/cm3]
    denprofile[i] = solutionmixture.rho
    # get zonal mixture viscosity profile [g/cm-sec] or [Poise]
    viscprofile[i] = solutionmixture.mixture_viscosity() * 1.0e2

# post-process cylinder-averged solution
# do NOT set the zone_id parameter
MyMZEngine.process_average_engine_solution()
# get the cylinder volume profile
cylindervolprofile = MyMZEngine.get_solution_variable_profile("volume")
# create arrays for cylinder-averaged mixture density
cylinderdenprofile = np.zeros_like(timeprofile, dtype=np.double)
# loop over all solution time points
for i in range(solutionpoints):
    # get the zonal mixture at the time point
    solutionmixture = MyMZEngine.get_solution_mixture_at_index(solution_index=i)
    # get zonal gas density [g/cm3]
    cylinderdenprofile[i] = solutionmixture.rho
```

### Plot the engine solution profiles

Plot the zonal and the cylinder averaged profiles from the multi-zone HCCI engine simulation.

**Note**

You can get profiles of the thermodynamic and the transport properties by applying `Mixture` utility methods to the solution mixtures.

```
plt.subplots(2, 2, sharex="col", figsize=(12, 6))
plt.suptitle(plottitle, fontsize=16)
plt.subplot(221)
plt.plot(ca_profile, presprofile, "r-")
plt.ylabel("Pressure [bar]")
plt.subplot(222)
plt.plot(ca_profile, volprofile, "b-")
plt.plot(ca_profile, cylindervolprofile, "b--")
plt.ylabel("Volume [cm3]")
plt.legend(["Zone", "Cylinder"], loc="upper right")
plt.subplot(223)
plt.plot(ca_profile, denprofile, "g-")
plt.plot(ca_profile, cylinderdenprofile, "g--")
plt.xlabel("Crank Angle [degree]")
plt.ylabel("Mixture Density [g/cm3]")
plt.legend(["Zone", "Averaged"], loc="upper left")
plt.subplot(224)
plt.plot(ca_profile, viscprofile, "m-")
plt.xlabel("Crank Angle [degree]")
plt.ylabel("Mixture Viscosity [cP]")

# clean up
ck.done()

# plot results
if interactive:
    plt.show()
else:
    plt.savefig("plot_multizone_HCCI_engine.png", bbox_inches="tight")
```

### 22.4.3 Simulate a spark ignition engine

Ansys Chemkin provides a set of idealized internal combustion (IC) engine models that are widely used for fuel combustion analysis and engine performance research. The Chemkin IC engine model is a specialized, transient, zero-dimensional (0D) closed gas-phase reactor designed to simulate combustion processes occurring between intake valve closing (IVC) and exhaust valve opening (EVO). During this period, the engine cylinder is treated as a closed chamber.

The cylinder volume is calculated based on piston motion as a function of crank angle (CA) and key engine parameters, including engine speed (RPM) and stroke. The energy equation is always solved, and several wall heat transfer models specifically developed for engine simulations are available to account for thermal losses.

**Note**

For additional information on Chemkin IC engine models, use the `ansys.chemkin.core.manuals()` method to view the online **Theory** manual.

The Chemkin spark-ignition (SI) engine model provides a straightforward framework for simulating chemical kinetics in spark-ignition engines. Rather than predicting the fuel mass burning rate, the model requires the burning rate profile as an input. This profile can be specified using Wiebe function parameters, burn profile anchor points, or normalized burn rate data.

The primary applications of the Chemkin SI engine model include parametric studies of combustion burn rate effects on engine performance, emission formation, and the onset of engine knock caused by end-gas autoignition. This example

demonstrates how to set up and run the simplest Chemkin IC engine configuration—the SI engine model. In addition to basic engine parameters, the simulation can incorporate advanced features such as exhaust gas recirculation (EGR) and wall heat transfer models to enhance realism.

```
```

### Import PyChemkin packages and start the logger

```
from pathlib import Path
import time

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color

# chemkin spark ignition (SI) engine model (transient)
from ansys.chemkin.core.engines.SI import SIengine
from ansys.chemkin.core.logger import logger

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory:" + current_dir)
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create a chemistry set

For PRF, the encrypted 14-component gasoline mechanism, `gasoline_14comp_WBencrypted.inp`, is used. The chemistry set is named `gasoline`.

**Note**

Because this gasoline mechanism does not come with any transport data, you do not need to provide a transport data file.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir
# create a chemistry set based on the gasoline 14 components mechanism
MyGasMech = ck.Chemistry(label="Gasoline")
# set mechanism input files
# including the full file path is recommended
MyGasMech.chemfile = str(mechanism_dir / "gasoline_14comp_WBencrypt.inp")
```

### Preprocess the gasoline chemistry set

```
# preprocess the mechanism files
_ = MyGasMech.preprocess()
print("Mechanism information:")
print(f"Number of gas species = {MyGasMech.kk:d}.")
print(f"Number of gas reactions = {MyGasMech.ii_gas:d}.")
```

### Set up the stoichiometric gasoline-air mixture

You must set up the stoichiometric gasoline-air mixture for the subsequent SI engine calculations. Here the `x_by_equivalence_ratio()``method is used. You create the ``fuel` and the `air` mixtures first. You then define the *complete combustion product species* and provide the *additives* composition if applicable. Finally, you simply set `equivalenceratio=1` to create the stoichiometric gasoline-air mixture.

For PRF 90 gasoline, the recipe is `[("ic8h18", 0.9), ("nc7h16", 0.1)]`.

```
# create the fuel mixture
fuelmixture = ck.Mixture(MyGasMech)
# set fuel composition
fuelmixture.x = [("ic8h18", 0.9), ("nc7h16", 0.1)]
# setting pressure and temperature is not required in this case
fuelmixture.pressure = ck.P_ATM
fuelmixture.temperature = 353.0

# create the oxidizer mixture: air
air = ck.Mixture(MyGasMech)
air.x = [("o2", 0.21), ("n2", 0.79)]
# setting pressure and temperature is not required in this case
air.pressure = ck.P_ATM
air.temperature = 353.0

# products from the complete combustion of the fuel mixture and air
products = ["co2", "h2o", "n2"]
# species mole fractions of added/inert mixture.
# You can also create an additives mixture here.
add_frac = np.zeros(MyGasMech.kk, dtype=np.double)  # no additives: all zeros

# create the unburned fuel-air mixture
fresh = ck.Mixture(MyGasMech)

# mean equivalence ratio
equiv = 1.0
ierror = fresh.x_by_equivalence_ratio(
    MyGasMech, fuelmixture.x, air.x, add_frac, products, equivalenceratio=equiv
)
# check fuel-oxidizer mixture creation status
if ierror != 0:
    print("Error: Failed to create the fuel-oxidizer mixture.")
    exit()
```

### List the mixture composition

List the composition of the premixed mixture for verification.

```
fresh.list_composition(mode="mole")
```

### Specify pressure and temperature of the fuel-air mixture

Since you are going to use `fresh` fuel-air mixture to instantiate the engine object later, setting the mixture pressure and temperature is equivalent to setting the initial temperature and pressure of the engine cylinder.

```
fresh.temperature = fuelmixture.temperature
fresh.pressure = fuelmixture.pressure
```

### Add EGR to the fresh fuel-air mixture

Many engines have the configuration for exhaust gas recirculation (EGR). Chemkin engine models let you add the EGR mixture to the fresh fuel-air mixture entering the cylinder. If the engine you are modeling has EGR, you should have the EGR ratio, which is generally the volume ratio between the EGR mixture and the fresh fuel-air ratio. However, because you know nothing about the composition of the exhaust gas, you cannot simply combine these two mixtures. In this case, you use the `get_egr_mole_fraction()` method to estimate the major components of the exhaust gas from the combustion of the fresh fuel-air mixture. The `threshold=1.0e-8` parameter tells the method to ignore any species with a mole fraction below the threshold value. Once you have the EGR mixture composition, use the `x_by_equivalence_ratio()` method a second time to re-create the fuel-air mixture `fresh` with the original `fuelmixture` and `air` mixtures, along with the EGR composition that you just got as the *additives*.

```
egr_ratio = 0.3
# compute the EGR stream composition in mole fractions
add_frac = fresh.get_egr_mole_fraction(egr_ratio, threshold=1.0e-8)
# recreate the initial mixture with EGR
_ = fresh.x_by_equivalence_ratio(
    MyGasMech,
    fuelmixture.x,
    air.x,
    add_frac,
    products,
    equivalenceratio=equiv,
    threshold=1.0e-8,
)
# list the composition of the fuel+air+EGR mixture for verification
fresh.list_composition(mode="mole", bound=1.0e-8)
```

### Set up the SI engine reactor

Use the `SIengine()` method to create an SI engine named `MyEngine` and make the new `fresh` mixture as the initial in-cylinder gas mixture at the IVC. The Chemkin SI engine model consists of two*zones: the unburned zone and the burned zone. At the IVC, the unburned zone is filled with the `fresh` mixture and occupies the entire engine cylinder. On the other hand, the burned zone is empty and has zero volume/mass.

```
MyEngine = SIengine(reactor_condition=fresh)
# show initial gas composition inside the reactor
MyEngine.list_composition(mode="mole", bound=1.0e-8)
```

### Set up basic engine parameters

Set the required engine parameters as shown in the following code. These engine parameters are used to describe the cylinder volume during the simulation. The `starting_ca` argument should be the crank angle corresponding to the cylinder IVC. The `ending_ca` argument is typically the EVC crank angle.

```
# cylinder bore diameter [cm]
MyEngine.bore = 8.5
# engine stroke [cm]
MyEngine.stroke = 10.82
# connecting rod length [cm]
MyEngine.connecting_rod_length = 17.853
# compression ratio [-]
MyEngine.compression_ratio = 12
# engine speed [RPM]
MyEngine.rpm = 600

# set other parameters
# simulation start CA [degree]
MyEngine.starting_ca = -120.2
# simulation end CA [degree]
MyEngine.ending_ca = 139.8
# list the engine parameters
MyEngine.list_engine_parameters()
print(f"Engine displacement volume = {MyEngine.get_displacement_volume()} [cm3].")
print(f"Engine clearance volume = = {MyEngine.get_clearance_volume()} [cm3].")
```

### Set up the SI engine specific parameters

### Set up the mass burned fraction profile

The burning rate profile is required because the SI engine model uses it to determine the mass transfer rate from the unburned zone to the burned zone during the simulation. You can use one of three methods to specify the mass burning fraction profile for the SI engine simulation:

- Wiebe function

  You must provide two sets of parameters:

  - `set_burn_timing`: Start of combustion crank angle (CA) and burn duration.
  - `wiebe_parameters`: Wiebe function parameters.

- Burn profile anchor points

  You must provide one set of parameters: `set_burn_anchor_points`.

- Burned mass fraction profile data

  You must provide two sets of parameters:

  - `set_burn_timing`: Start of combustion CA and burn duration.
  - `set_mass_burned_profile`: Normalized burn rate profile.

```
# start of combustion CA
MyEngine.set_burn_timing(soc=-14.5, duration=45.6)
MyEngine.wiebe_parameters(n=4.0, b=7.0)
# set minimum zonal mass [g]
MyEngine.set_minimum_zone_mass(minmass=1.0e-5)
```

### Set up engine wall heat transfer model

By default, the engine cylinder is adiabatic. You must set up a wall heat transfer model to include the heat loss effects in your engine simulation. Chemkin supports three widely used engine wall heat transfer models. The models and their parameters follow:

- `dimensionless`: [<a> <b> <c> <Twall>]
- `dimensional`: [<a> <b> <c> <Twall>]
- `hohenburg`: [<a> <b> <c> <d> <e> <Twall>]

There is also the in-cylinder gas velocity correlation (the Woschni correlation) that is associated with the engine wall heat transfer models. Here are the parameters of the Woschni correlation:

`[<C11> <C12> <C2> <swirl ratio>]`

You can also specify the surface areas of the piston head and the cylinder head for more precision heat transfer wall area. By default, both the piston head and the cylinder head surfaces are flat.

```
heattransferparameters = [0.1, 0.8, 0.0]
# set cylinder wall temperature [K]
Twall = 434.0
MyEngine.set_wall_heat_transfer("dimensionless", heattransferparameters, Twall)
# in-cylinder gas velocity correlation parameter (Woschni)
# [<C11> <C12> <C2> <swirl ratio>]
GVparameters = [2.28, 0.318, 0.324, 0.0]
MyEngine.set_gas_velocity_correlation(GVparameters)
# set piston head top surface area [cm2]
MyEngine.set_piston_head_area(area=56.75)
# set cylinder clearance surface area [cm2]
MyEngine.set_cylinder_head_area(area=56.75)
```

### Set output options

You can turn on the adaptive solution saving to resolve the steep variations in the solution profile. Here additional solution data point are saved for every 20 solver internal steps. Since ignition in SI engine is controlled by the spark timing, that is, the start of combustion CA of the chemkin SI engine model, the ignition delay time is not reported.

**Note**

By default, time/crank angle intervals for both print and save solution are 1/100 of the simulation duration, which in this case is $dCA = (EVO - IVC)/100 = 2.58$. You can make the model report more frequently by using the `ca_step_for_saving_solution()` or `ca_step_for_printing_solution()` method to set different interval values in the CA.

```
# set the number of crank angles between saving solution
MyEngine.ca_step_for_saving_solution = 0.5
# set the number of crank angles between printing solution
MyEngine.ca_step_for_printing_solution = 10.0
# turn ON adaptive solution saving
MyEngine.adaptive_solution_saving(mode=True, steps=20)
```

### Set solver controls

You can overwrite the default solver controls by using solver-related methods, such as those for tolerances.

```
# set tolerances in tuple: (absolute tolerance, relative tolerance)
MyEngine.tolerances = (1.0e-15, 1.0e-6)
# get solver parameters
atol, rtol = MyEngine.tolerances
print(f"Default absolute tolerance = {atol}.")
print(f"Default relative tolerance = {rtol}.")
# turn on the force non-negative solutions option in the solver
MyEngine.force_nonnegative = True
# show solver option
# show the number of crank angles between printing solution
print(
    f"Crank angles between solution printing: {MyEngine.ca_step_for_printing_solution}"
)
# show other transient solver setup
print(f"Forced non-negative solution values: {MyEngine.force_nonnegative}")
```

### Display the added parameters (keywords)

Use the `showkeywordinputlines()` method to verify the preceding parameters are correctly assigned to the engine model.

```
MyEngine.showkeywordinputlines()
# set the start wall time
start_time = time.time()
```

### Run the simulation

Use the `run()` method to start the SI engine simulation. The simulation might take more than 30 seconds because the gasoline mechanism contains a lot of species and reactions.

```
runstatus = MyEngine.run()
# compute the total runtime
runtime = time.time() - start_time
# check run status
if runstatus != 0:
    # Run failed.
    print(Color.RED + ">>> Run failed. <<<", end=Color.END)
    exit()
# run success!
print(Color.GREEN + ">>> Run completed. <<<", end=Color.END)
print(f"Total simulation duration: {runtime} [sec]")
```

### Postprocess the solution

The postprocessing step parses the solution and packages the solution values at each time point into a mixture. There are two ways to access the solution profiles:

- The raw solution profiles (value as a function of time) are available for time, temperature, pressure, volume, and species mass fractions.
- The mixtures permit the use of all property and rate utilities to extract information such as viscosity, density, and mole fractions.

You can use the `get_solution_variable_profile()` method to get the raw solution profiles. You can get solution mixtures using either the `get_solution_mixture_at_index()` method for the solution mixture at a given time point or the `get_solution_mixture()` method for the solution mixture at a given time. (In this case, the mixture is constructed by interpolation.)

**Note**

- For engine models, use the `process_engine_solution()` method to postprocess the solutions.
- Use the `getnumbersolutionpoints()` method to get the size of the solution profiles before creating the arrays.
- Use the `get_ca()` method to convert the time values reported in the solution to crank angles.

### Postprocess the solution profiles in selected zone

The solution of the SI engine model contains the results of the *unburned zones* and *burned zone*, along with the cylinder averaged results. That is, there are three solution records. The unburned zone is designated as `zone 1`, and the burned zone is designated as `zone 2`. To process the result of the burned zone, you use `zone_id=2` when you call the engine postprocessor with the `process_engine_solution()` method. If you omit the `zone_id` parameter, the cylinder averaged results are postprocessed by default.

**Note**

Because the `process_engine_solution` method can process only one set of result at a time (one zonal result or the cylinder averaged result), you must postprocess the zones one by one to obtain all solution data of the multi-zone simulation.

```
unburnedzone = 1
burnedzone = 2
zonestrings = ["Unburned Zone", "Burned Zone"]
thiszone = burnedzone
MyEngine.process_engine_solution(zone_id=thiszone)
plottitle = zonestrings[thiszone - 1] + " Solution"
# get the number of solution time points
solutionpoints = MyEngine.getnumbersolutionpoints()
print(f"Number of solution points = {solutionpoints}.")
# get the time profile
timeprofile = MyEngine.get_solution_variable_profile("time")
# convert time to crank angle
ca_profile = np.zeros_like(timeprofile, dtype=np.double)
count = 0
for t in timeprofile:
    ca_profile[count] = MyEngine.get_ca(timeprofile[count])
    count += 1
# get the cylinder pressure profile
presprofile = MyEngine.get_solution_variable_profile("pressure")
presprofile *= 1.0e-6
# get zonal temperature profile [K]
tempprofile = MyEngine.get_solution_variable_profile("temperature")
# get the zonal volume profile
volprofile = MyEngine.get_solution_variable_profile("volume")
```

(continues on next page)

(continued from previous page)

```
# create arrays for zonal CO mole fraction
# find CO species index
co_index = MyGasMech.get_specindex("co")
co_profile = np.zeros_like(timeprofile, dtype=np.double)
# loop over all solution time points
for i in range(solutionpoints):
    # get the zonal mixture at the time point
    solutionmixture = MyEngine.get_solution_mixture_at_index(solution_index=i)
    # get zonal CO mole fraction
    co_profile[i] = solutionmixture.x[co_index]

# postprocess cylinder-averged solution
MyEngine.process_average_engine_solution()
# get the cylinder volume profile
cylindervolprofile = MyEngine.get_solution_variable_profile("volume")
# create arrays for cylinder-averaged mixture temperature
cylindertempprofile = MyEngine.get_solution_variable_profile("temperature")
```

### Plot the engine solution profiles

Plot the zonal and the cylinder averaged profiles from the SI engine simulation.

**Note**

You can get profiles of the thermodynamic and the transport properties by applying `Mixture` utility methods to the solution mixtures.

```
plt.subplots(2, 2, sharex="col", figsize=(13, 6.5))
plt.suptitle(plottitle, fontsize=16)
plt.subplot(221)
plt.plot(ca_profile, presprofile, "r-")
plt.ylabel("Pressure [bar]")
plt.subplot(222)
plt.plot(ca_profile, volprofile, "b-")
plt.plot(ca_profile, cylindervolprofile, "b--")
plt.ylabel("Volume [cm3]")
plt.legend(["Zone", "Cylinder"], loc="lower right")
plt.subplot(223)
plt.plot(ca_profile, tempprofile, "g-")
plt.plot(ca_profile, cylindertempprofile, "g--")
plt.xlabel("Crank Angle [degree]")
plt.ylabel("Mixture Temperature [K]")
plt.legend(["Zone", "Averaged"], loc="upper left")
plt.subplot(224)
plt.plot(ca_profile, co_profile, "m-")
plt.xlabel("Crank Angle [degree]")
plt.ylabel("CO Mole Fraction")

# clean up
ck.done()
```

(continues on next page)

(continued from previous page)

```
# plot results
if interactive:
    plt.show()
else:
    plt.savefig("plot_spark_ignition_engine.png", bbox_inches="tight")
```

## 22.5 Perfectly stirred reactor

The *perfectly Stirred reactor (PSR)* model is a 0-D steady-state reactor. This ideal reactor model assumes the gas mixture inside is uniform and the properties of the outlet stream are exactly the same as the gas properties inside the reactor. A PSR can have either its pressure or its volume "specified", and the reactor temperature can either be solved from the energy equation or be "given" as an input parameter.

The examples will show the procedures of setting up single PSR simulations with different constraints and with multiple inlet streams.

### 22.5.1 Set up a PSR parameter study for the inlet stream equivalence ratio

Ansys Chemkin offers some idealized reactor models commonly used for studying chemical processes and for developing reaction mechanisms. The PSR (perfectly stirred reactor) model is a steady-state 0-D model of the open perfectly mixed gas-phase reactor. There is no limit on the number of inlets to the PSR. As soon as the inlet gases enter the reactor, they are thoroughly mixed with the gas mixture inside. The PSR has only one outlet, and the outlet gas is assumed to be exactly the same as the gas mixture in the PSR.

There are two basic types of PSR models:

- **constrained-pressure** (or set residence time)
- **constrained-volume**

By default, the PSR model is running under constant pressure. PyChemkin PSR models always require the connected inlets to be defined, that is, the total inlet flow rate to the PSR is always known. Therefore, either the residence time or the reactor volume is needed to satisfy the basic setup of the PSR model. In this case, you specify the residence time of the PSR, and the PSR model automatically calculates the reactor volume from the given residence time and the total inlet volumetric flow rate.

For each type of the PSR, you can choose either to specify the reactor temperature (as a fixed value or by a piecewise-linear profile) or to solve the energy conservation equation. In total, you get four variations of the PSR model.

The PSR model is mostly employed in chemical kinetics studies. By controlling the reactor temperature, pressure, and/or residence time, you can gain knowledge about the major intermediates of a complex chemical process and postulate possible reaction pathways. The parameter study in this example shows how the inlet equivalence ratio impacts the hydrogen combustion process in a fixed residence time PSR.

```

```

#### Import PyChemkin packages and start the logger

```
from pathlib import Path
import time

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

```

(continues on next page)

(continued from previous page)

```
import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color
from ansys.chemkin.core.inlet import Stream  # external gaseous inlet
from ansys.chemkin.core.logger import logger

# Chemkin PSR model (steady-state)
from ansys.chemkin.core.stirreactors.PSR import PSRSetResTimeEnergyConservation as Psr
from ansys.chemkin.core.utilities import find_file

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create a chemistry set

This example uses the encrypted hydrogen-ammonia mechanism, `Hydrogen-Ammonia-NOx_chem_MFL2021.inp`. This mechanism is developed under Chemkin's *Model Fuel Library* (MFL) project. Like the rest of the MFL mechanisms, it is in the *ModelFuelLibrary* in the `/reaction/data` directory of the standard Ansys Chemkin installation.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data" / "ModelFuelLibrary" / "Skeletal"
mechanism_dir = str(data_dir.resolve())
# create a chemistry set based on the gasoline 14-components mechanism
MyGasMech = ck.Chemistry(label="hydrogen")
# set mechanism input files
# including the full file path is recommended
MyGasMech.chemfile = find_file(
    mechanism_dir,
    "Hydrogen-Ammonia-NOx_chem_MFL",
    "inp",
)
```

### Preprocess the gasoline chemistry set

```
# preprocess the mechanism files
_ = MyGasMech.preprocess()
```

### Set up the $H_2$-air stream

Instantiate a stream feed for the inlet gas mixture. The stream is a mixture with the addition of the inlet flow rate. You specify inlet gas properties in the same way as you set up a mixture. Here, the `x_by_equivalence_ratio()` method is used.

First create the fuel and air mixtures. Then, define the complete combustion product species and provide the additives composition if applicable. Finally, set the equivalence ratio to 1 to create the stoichiometric hydrogen-air mixture. Use

the `mass_flowrate()` method to assign the inlet mass flow rate. Use a fixed inlet mass flow rate of 432 [g/sec] since you are to change the PSR residence time in the parameter study.

```
# create the fuel mixture
fuel = ck.Mixture(MyGasMech)
# set fuel composition: hydrogen diluted by nitrogen
fuel.x = [("h2", 0.8), ("n2", 0.2)]
# setting the pressure and temperature is not required in this case
fuel.pressure = ck.P_ATM
fuel.temperature = 298.0  # inlet temperature

# create the oxidizer mixture: air
air = ck.Mixture(MyGasMech)
air.x = [("o2", 0.21), ("n2", 0.79)]
# setting the pressure and temperature is not required in this case
air.pressure = fuel.pressure
air.temperature = fuel.temperature

# create the fuel-oxidizer inlet to the PSR
feed = Stream(MyGasMech, label="feed_1")
# products from the complete combustion of the fuel mixture and air
products = ["h2o", "n2"]
# species mole fractions of added/inert mixture.
# You can also create an additives mixture here.
add_frac = np.zeros(MyGasMech.kk, dtype=np.double)  # no additives: all zeros
# mean equivalence ratio
equiv = 1.0
ierror = feed.x_by_equivalence_ratio(
    MyGasMech, fuel.x, air.x, add_frac, products, equivalenceratio=equiv
)
# check fuel-oxidizer mixture creation status
if ierror != 0:
    print("Error: Failed to create the fuel-oxidizer mixture.")
    exit()

# set reactor pressure [dynes/cm2]
feed.pressure = fuel.pressure
# set inlet gas temperature [K]
feed.temperature = fuel.temperature
# set inlet mass flow rate [g/sec]
feed.mass_flowrate = 432.0
```

### Create the PSR to predict the gas composition of the outlet stream

Use the `PSRSetResTimeEnergyConservation()` method to instantiate the PSR `sphere` object because the goal is to see how the residence time affects the hydrogen combustion process. The gas property of the inlet feed is applied as the estimated reactor condition of the `sphere` object by default. You can overwrite any estimated reactor conditions by using appropriate methods. For example, `sphere.temperature = 1700.0` changes the estimated reactor temperature from 298 to 1700 [K]. The residence time of the nominal case is set by the `residence_time()` method.

```
sphere = Psr(feed, label="PSR_1")
```

### Set up additional reactor model parameters

Before you can run the simulation, you must provide reactor parameters, solver controls, and output instructions. For a steady-state PSR, you must provide either the residence time or reactor volume. You can also make changes to any estimated reactor conditions if desired.

```
# reset the estimated reactor temperature [K]
sphere.temperature = 1700.0
# set PSR residence time (sec): required for PSRSetResTimeEnergyConservation model
sphere.residence_time = 3.0 * 1.0e-5
```

### Connect the inlet to the reactor

You must connect at least one inlet to the open reactor. Use the `set_inlet()` method to add a stream to the PSR. Inversely, use the `remove_inlet()` to disconnect an inlet from the PSR.

**Note**

There is no limit on the number of inlets that can be connected to a PSR.

```
# connect the inlet to the reactor
sphere.set_inlet(feed)
```

### Set solver controls

You can overwrite the default solver controls by using solver-related methods, such as those for tolerances. Here the tolerances that the steady-state solver is to use for the *steady-state search* and the *pseudo time stepping* stages are changed. Sometimes, during the iterations, some species mass fractions might become negative, causing the solver to report an error and stop. To overcome this issue, you can use the `set_species_floor()` method to provide a small cushion, allowing species mass fractions to go slightly negative by resetting the mass fraction floor value.

```
# reset the tolerances in the steady-state solver
sphere.steady_state_tolerances = (1.0e-9, 1.0e-6)
sphere.timestepping_tolerances = (1.0e-9, 1.0e-6)
# reset the gas species floor value in the steady-state solver
sphere.set_species_floor(-1.0e-10)
```

### Run the inlet equivalence ratio parameter study

In the parameter study, the equivalence ratio $\phi$ of the inlet gas mixture is increased from 1.0 to 1.4. This is done by applying the `x_by_equivalence_ratio()` method on the feed inlet. Changing the equivalence ratio of the inlet gas mixture inevitably has impact on the inlet stream density and hence the inlet volumetric flow rate. By using the `PSRSetResTimeEnergyConservation` model, the PSR residence time remains constant for all runs. The effects from any variations of the inlet are reflected on the PSR volume.

Use the `process_solution()` method to convert the result from each PSR to a mixture. You can either overwrite the solution mixture or use a new one for each simulation result.

```
# inlet gas equivalence ratio increment
deltaequiv = 0.05
```

(continues on next page)

(continued from previous page)

```
numbruns = 9
# solution arrays
inletequiv = np.zeros(numbruns, dtype=np.double)
temp_ss_solution = np.zeros_like(inletequiv, dtype=np.double)
# set the start wall time
start_time = time.time()
# loop over all inlet temperature values
for i in range(numbruns):
    # run the PSR model
    runstatus = sphere.run()
    # check run status
    if runstatus != 0:
        # Run failed.
        print(Color.RED + ">>> Run failed. <<<", end=Color.END)
        exit()
    # Run succeeded.
    print(Color.GREEN + ">>> Run completed. <<<", end=Color.END)
    # postprocess the solution profiles
    solnmixture = sphere.process_solution()
    # print the steady-state solution values
    # print(f"Steady-state temperature = {solnmixture.temperature} [K].")
    # solnmixture.list_composition(mode="mole")
    # store solution values
    inletequiv[i] = equiv
    temp_ss_solution[i] = solnmixture.temperature
    # update inlet gas equivalence ratio (composition)
    equiv += deltaequiv
    ierror = feed.x_by_equivalence_ratio(
        MyGasMech, fuel.x, air.x, add_frac, products, equivalenceratio=equiv
    )
    # check fuel-oxidizer mixture creation status
    if ierror != 0:
        print(f"Error encountered with inlet equivalence ratio = {equiv}.")
        exit()

# compute the total runtime
runtime = time.time() - start_time
print(f"Total simulation duration: {runtime} [sec] over {numbruns} runs.")
```

### Plot the parameter study results

Plot the steady-state PSR temperature against the equivalence ratio of the inlet $H_2$-air mixture. You should see that the maximum combustion temperature does not correspond to the $\phi = 1$ mixture. Instead, the temperature peak occurs when the mixture is slightly fuel rich. You can run the same parameter study on a different fuel species such as $CH_4$ to see if you observe the same behavior.

```
plt.plot(inletequiv, temp_ss_solution, "b-")
plt.xlabel("Inlet Gas Equivalence Ratio")
plt.ylabel("Reactor Temperature [K]")
plt.title("PSR Solution")

# clean up
```

(continues on next page)

(continued from previous page)

```
ck.done()

# plot results
if interactive:
    plt.show()
else:
    plt.savefig("plot_PSR_gas.png", bbox_inches="tight")
```

### 22.5.2 Run PSR calculations for mechanism validation

Ansys Chemkin offers some idealized reactor models commonly used for studying chemical processes and for developing reaction mechanisms. The PSR (perfectly stirred reactor) model is a steady-state 0-D model of the open perfectly mixed gas-phase reactor. There is no limit on the number of inlets to the PSR. As soon as the inlet gases enter the reactor, they are thoroughly mixed with the gas mixture inside. The PSR has only one outlet, and the outlet gas is assumed to be exactly the same as the gas mixture in the PSR.

There are two basic types of PSR models:

- **constrained-pressure** (or set residence time)
- **constrained-volume**

By default, the PSR is running under constant pressure. In this case, you specify the residence time of the PSR. For the constrained-volume type of application, you must provide the PSR volume. You can calculate the residence time from the reactor volume and the total inlet volumetric flow rate.

For each type of PSR, you either specify the reactor temperature (as a fixed value or by a piecewise-linear profile) or solve the energy conservation equation. In total, you get four variations of the PSR model.

The JSR (jet-stirred reactor is mostly employed in chemical kinetics studies. By controlling the reactor temperature, pressure, and/or residence time, you can gain knowledge about the major intermediates of a complex chemical process and postulate possible reaction pathways.

This example shows how to use the PSR model to validate the reaction mechanism against the measured data from hydrogen oxidation experiments.

```

```

#### Import PyChemkin packages and start the logger

```
from pathlib import Path
import time

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color
from ansys.chemkin.core.inlet import Stream  # external gaseous inlet
from ansys.chemkin.core.logger import logger

# chemkin perfectly stirred reactor (PSR) model (steady-state)
from ansys.chemkin.core.stirreactors.PSR import PSRSetResTimeFixedTemperature as Psr
from ansys.chemkin.core.utilities import find_file

```

(continues on next page)

(continued from previous page)

```
# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create a chemistry set

This code uses the encrypted hydrogen-ammonia mechanism, `Hydrogen-Ammonia-NOx_chem_MFL2021.inp`. This mechanism is developed under Chemkin's **Model Fuel Library (MFL)** project. Like the rest of the MFL mechanisms, it is in the `ModelFuelLibrary` in the `/reaction/data` directory of the standard Ansys Chemkin installation.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data" / "ModelFuelLibrary" / "Skeletal"
mechanism_dir = str(data_dir)
# create a chemistry set based on the hydrogen-ammonia mechanism
MyGasMech = ck.Chemistry(label="hydrogen")
# set mechanism input files
# including the full file path is recommended
MyGasMech.chemfile = find_file(
    mechanism_dir,
    "Hydrogen-Ammonia-NOx_chem_MFL",
    "inp",
)
```

### Preprocess the gasoline chemistry set

```
# preprocess the mechanism files
_ = MyGasMech.preprocess()
```

### Set up the $H_2$-$O_2$-$N_2$ stream

Instantiate a stream feed for the inlet gas mixture. A stream is a mixture with the addition of the inlet flow rate. You set up the inlet gas properties in the same way you set up a mixture. Set up the gas properties of the feed as described by the experiment.

Use the `mass_flowrate()` method to assign the inlet mass flow rate. The experiments use a fixed inlet mass flow rate of 0.11 [g/sec].

```
# create the fuel-oxidizer inlet to the JSR
feed = Stream(MyGasMech)
# set H2-O2-N2 composition
feed.x = [("h2", 1.1e-2), ("n2", 9.62e-1), ("o2", 2.75e-2)]
# set reactor pressure [dynes/cm2]
feed.pressure = ck.P_ATM
# set inlet gas temperature [K]
temp = 800.0
```

(continues on next page)

(continued from previous page)

```
feed.temperature = temp
# set inlet mass flow rate [g/sec]
feed.mass_flowrate = 0.11
```

### Create the PSR to predict the gas composition of the outlet stream

Use the `PSRSetResTimeFixedTemperature()` method to instantiate the JSR because both the reactor temperature and residence time are fixed during the experiments. The gas property of the inlet feed is applied as the estimated reactor condition of the JSR.

```
Jsr = Psr(feed, label="JSR")
```

### Connect the inlets to the reactor

You must connect at least one inlet to the open reactor. Use the `set_inlet()` method to add a stream to the PSR. Inversely, use the `remove_inlet()` method to disconnect an inlet from the PSR.

```
# connect the inlet to the reactor
Jsr.set_inlet(feed)
```

### Set up additional reactor model parameters

You must provide reactor parameters, solver controls, and output instructions before running the simulations. For the steady-state PSR, you must provide either the residence time or the reactor volume.

```
# set PSR residence time (sec): required for PSRSetResTimeFixedTemperature model
Jsr.residence_time = 120.0 * 1.0e-3
```

### Set solver controls

You can overwrite the default solver controls by using solver-related methods, such as those for tolerances. Here, particular to this mechanism, a number of *pseudo timesteps* is required before attempting to actually search for the steady-state solution. This is done by using the steady-state solver control method, `set_initial_timesteps()`.

```
# set the number of initial pseudo timesteps in the steady-state solver
Jsr.set_initial_timesteps(1000)
```

### Run the parameter study to replicate the experiments

Different inlet and reactor temperatures are used in the experiments. The temperature value varies from 800 to 1050 [K].

Use the `process_solution()` method to convert the result from each PSR run to a mixture. You can either overwrite the solution mixture or use a new one for each simulation result.

```
# inlet gas temperature increment
deltatemp = 25.0
numbruns = 19
# find "h2o" species index
h2o_index = MyGasMech.get_specindex("h2o")
# solution arrays
inlet_temp = np.zeros(numbruns, dtype=np.double)
```

(continues on next page)

(continued from previous page)

```
h2o_ss_solution = np.zeros_like(inlet_temp, dtype=np.double)
# set the start wall time
start_time = time.time()
# loop over all inlet temperature values
for i in range(numbruns):
    # run the PSR model
    runstatus = Jsr.run()
    # check run status
    if runstatus != 0:
        # Run failed.
        print(Color.RED + ">>> Run failed. <<<", end=Color.END)
        exit()
    # Run succeeded.
    print(Color.GREEN + ">>> Run completed. <<<", end=Color.END)
    # postprocess the solution profiles
    solnmixture = Jsr.process_solution()
    # print the steady-state solution values
    # print(f"Steady-state temperature = {solnmixture.temperature} [K].")
    # solnmixture.list_composition(mode="mole")
    # store solution values
    inlet_temp[i] = solnmixture.temperature
    h2o_ss_solution[i] = solnmixture.x[h2o_index]
    # update reactor temperature
    temp += deltatemp
    Jsr.temperature = temp

# compute the total runtime
runtime = time.time() - start_time
print(f"Total simulation duration: {runtime} [sec] over {numbruns} runs")
#
# experimental data
# JSR temperature [K]
temp_data = [
    803.0,
    823.0,
    850.0,
    875.0,
    902.0,
    925.0,
    951.0,
    973.0,
    1002.0,
    1023.0,
    1048.0,
]
# measured H2O mole fractions in the JSR exit flow
h2o_data = [
    0.000312,
    0.000313,
    0.000318,
    0.000303,
    0.003292,
```

(continues on next page)

(continued from previous page)

```
    0.006226,
    0.008414,
    0.009745,
    0.010103,
    0.011036,
    0.010503,
]
```

#### Plot the ignition delay curve

Compare the predicted $H_2O$ mole fraction to the measurement.

```
# plot results
plt.plot(inlet_temp, h2o_ss_solution, "b-", label="prediction")
plt.plot(temp_data, h2o_data, "ro", label="data", markersize=4, fillstyle="none")
plt.xlabel("Reactor Temperature [K]")
plt.ylabel("H2O Mole Fraction")
plt.legend(loc="lower right")
plt.title("JSR Solution")

# clean up
ck.done()

# plot results
if interactive:
    plt.show()
else:
    plt.savefig("plot_jet_stirred_reactor.png", bbox_inches="tight")
```

### 22.5.3 Determine the impact of residence time on combustion in a PSR

Ansys Chemkin offers some idealized reactor models commonly used for studying chemical processes and for developing reaction mechanisms. The PSR (perfectly stirred reactor) model is a steady-state 0-D model of the open perfectly mixed gas-phase reactor. There is no limit on the number of inlets to the PSR. As soon as the inlet gases enter the reactor, they are thoroughly mixed with the gas mixture inside. The PSR has only one outlet, and the outlet gas is assumed to be exactly the same as the gas mixture in the PSR. There are two basic types of PSR models:

- **constrained-pressure** (or set residence time)
- **constrained-volume**

By default, the PSR model is running under constant pressure. The PyChemkin PSR models always require the connected inlets to be defined, that is, the total inlet flow rate to the PSR is always known. Therefore, either the residence time or the reactor volume is needed to satisfy the basic setup of the PSR model.

This example specifies the reactor volume of the PSR. The residence time is calculated from the reactor volume and the total inlet volumetric flow rate.

For each type of PSR model, you can either specify the reactor temperature (as a fixed value or by a piecewise-linear profile) or solve the energy conservation equation. In total, you get four variations of the PSR model.

PSR models are mostly employed in chemical kinetics studies. By controlling the reactor temperature, pressure, and/or residence time, you can gain knowledge about the major intermediates of a complex chemical process and postulate possible reaction pathways.

This example describes a parameter study of the influence of the PSR residence time on the hydrogen combustion process. It uses two inlet streams, one for the fuel mixture and the other for the air mixture. The fuel-to-air ratio inside the PSR is determined by the mass or the volumetric flow rate ratio of the two inlet streams.

### Import PyChemkin packages and start the logger

```
from pathlib import Path
import time

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color
from ansys.chemkin.core.inlet import Stream  # external gaseous inlet
from ansys.chemkin.core.logger import logger

# Chemkin PSR model (steady-state)
from ansys.chemkin.core.stirreactors.PSR import PSRSetVolumeEnergyConservation as Psr
from ansys.chemkin.core.utilities import find_file

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create a chemistry set

This example uses the encrypted hydrogen-ammonia mechanism, `Hydrogen-Ammonia-NOx_chem_MFL2021.inp`. This mechanism is developed under Chemkin's **Model Fuel Library (MFL)** project. Like the rest of the MFL mechanisms, it is located in `ModelFuelLibrary` in the `/reaction/data` directory of the standard Ansys Chemkin installation.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data" / "ModelFuelLibrary" / "Skeletal"
mechanism_dir = str(data_dir)
# create a chemistry set based on the gasoline 14 components mechanism
MyGasMech = ck.Chemistry(label="hydrogen")
# set mechanism input files
# including the full file path is recommended
MyGasMech.chemfile = find_file(
    mechanism_dir,
    "Hydrogen-Ammonia-NOx_chem_MFL",
    "inp",
)
```

### Preprocess the gasoline chemistry set

```
# preprocess the mechanism files
_ = MyGasMech.preprocess()
```

### Set up the fuel and air streams

Instantiate a stream named `fuel` for the inlet stream containing the fuel mixture and a stream named `air` for the inlet stream containing the air mixture. The `Inlet` object is a mixture with the addition of the inlet flow rate.

You specify inlet gas properties in the same way as you set up a mixture. Here, the `fuel` and `air` inlets are created separately. You can adjust their inlet volumetric flow rates to create the desired hydrogen-air mixture. You use the `vol_flowrate()` method to assign the inlet volumetric flow rate. In this project, the `fuel` and `air` inlets have fixed volumetric flow rates of 25 and 50 [cm3/sec], respectively.

**Note**

This equation is used to determine the hydrogen-to-oxygen molar ratio:

$$H_2 : O_2 = 0.21[cm^3/cm^3] * 25[cm^3/s] : 0.21[cm^3/cm^3] * 50[cm^3/s] = 1 : 2$$

**Note**

The PSR residence time is specified indirectly through the reactor volume in the parameter study.

```
# create the fuel inlet
fuel = Stream(MyGasMech, label="Fuel")
# set fuel composition
fuel.x = [("h2", 0.21), ("n2", 0.79)]
# setting pressure and temperature is not required in this case
fuel.pressure = ck.P_ATM
fuel.temperature = 450.0  # inlet temperature
# set inlet volumetric flow rate [cm3/sec]
fuel.vol_flowrate = 25.0

# create the oxidizer inlet: air
air = Stream(MyGasMech, label="Oxid")
air.x = [("o2", 0.21), ("n2", 0.79)]
# setting pressure and temperature is not required in this case
air.pressure = fuel.pressure
air.temperature = fuel.temperature
# set inlet volumetric flow rate [cm3/sec]
air.vol_flowrate = 50.0
```

### Create the PSR to predict the gas composition of the outlet stream

Use the `PSRSetVolumeEnergyConservation()` method to instantiate a PSR named `combustor`. You must include the energy equation because the goal is to see how the residence time would affect the hydrogen combustion process. The `combustor` PSR is initiated with the parameter set to the `fuel` inlet, Hence, the gas property of the `fuel` inlet is applied as the estimated reactor condition of the `combustor` PSR. You can overwrite any estimated reactor conditions by using appropriate methods. For example, `combustor.temperature = 2000.0` changes the estimated reactor

temperature from 450 (the temperature of the `fuel` inlet) to 2000 [K]. The `volume()` method sets the reactor volume of the nominal case.

```
# create a PSR with fixed reactor volume and
# with the fuel inlet composition as the estimated reactor condition
combustor = Psr(fuel, label="tincan")
```

### Set up additional reactor model parameters

You must provide reactor parameters, solver controls, and output instructions before running the simulations. For the steady-state PSR, you must provide either the residence time or the reactor volume. You can also make changes to any estimated reactor conditions if desired.

```
# reset the estimated reactor temperature [K]
combustor.temperature = 2000.0
# set the reactor volume (cm3): required for PSRSetVolumeEnergyConservation model
combustor.volume = 200.0
```

### Connect the inlets to the reactor

You must connect at least one inlet to the open reactor. Use the `set_inlet()` method to add a stream object to the PSR. Inversely, use the `remove_inlet()` to disconnect an inlet from the PSR. Here two inlets, `fuel` and `air`, are connected to the `combustor` PSR. The fuel-to-air ratio is controlled by the mass or the volumetric flow rate ratio of the `fuel` and the `air` inlets.

```
# add external inlets to the PSR
combustor.set_inlet(fuel)
combustor.set_inlet(air)
```

### Set solver controls

You can overwrite the default solver controls by using solver-related methods, such as those for tolerances. The following code changes the tolerances that the steady-state solver is to use for the steady-state search and changes the pseudo time stepping stages. Sometimes, during the iterations, some species mass fractions might become negative, causing the solver to report an error and stop. To overcome this issue, you can provide a small cushion to allow species mass fractions to go slightly negative by using the `set_species_floor()` method to reset the mass fraction floor value.

```
# reset the tolerances in the steady-state solver
combustor.steady_state_tolerances = (1.0e-9, 1.0e-6)
combustor.timestepping_tolerances = (1.0e-9, 1.0e-6)
# reset the gas species floor value in the steady-state solver
combustor.set_species_floor(-1.0e-10)
```

### Run the PSR residence time parameter study

The PSR residence time $\tau$ is calculated as follows:

$$\tau = \frac{reactor\ volume}{total\ volumetric\ flow\ rate}$$

In this parameter study, the reactor volume is decreased from 200 to 160 [cm3]. Accordingly, $\tau$ is decreased from 2.67 to 2.13 [sec] because the total inlet volumetric flow rate is kept constant at 75 [cm3/sec]. Usually you want to run the burning cases (large residence times) first in the PSR parameter study.

The `process_solution` method converts the result from each PSR run to a mixture. You can either overwrite the solution mixture or use a new one for each simulation result.

```
# reactor volume increment
delta_vol = -5
numbruns = 9
# solution arrays
residencetime = np.zeros(numbruns, dtype=np.double)
temp_ss_solution = np.zeros_like(residencetime, dtype=np.double)
# set the start wall time
start_time = time.time()
# loop over all inlet temperature values
for i in range(numbruns):
    # run the PSR model
    runstatus = combustor.run()
    # check run status
    if runstatus != 0:
        # Run failed.
        print(Color.RED + ">>> Run failed. <<<", end=Color.END)
        exit()
    # Run succeeded.
    print(Color.GREEN + ">>> Run completed. <<<", end=Color.END)
    # postprocess the solution profiles
    solnmixture = combustor.process_solution()
    # print the steady-state solution values
    print(f"steady-state temperature = {solnmixture.temperature} [K]")
    # solnmixture.list_composition(mode="mole")
    # store solution values
    # final reactor gas density [g/cm3]
    # density = solnmixture.RHO
    # final reactor mass [g]
    # mass = density * combustor.volume
    # PSR apparent residence time [sec]
    residencetime[i] = combustor.volume / combustor.net_vol_flowrate
    temp_ss_solution[i] = solnmixture.temperature
    # update reactor volume
    combustor.volume += delta_vol

# compute the total runtime
runtime = time.time() - start_time
print(f"Total simulation duration: {runtime} [sec] over {numbruns} runs")
```

### Plot the parameter study results

Plot the predicted PSR temperature against the residence time. You can observe that the hydrogen-air mixture ceases to burn when the residence time becomes too small. Gas turbine terminology refers to this as *blown out* because the fuel-air mixture gets blown out of the combustor before any significant chemical reaction can take place.

```
plt.plot(residencetime, temp_ss_solution, "bo-")
plt.xlabel("Apparent Residence Time [sec]")
plt.ylabel("Exit Gas Temperature [K]")
plt.title("PSR Solution")

# clean up
ck.done()
```

(continues on next page)

(continued from previous page)

```
# plot results
if interactive:
    plt.show()
else:
    plt.savefig("plot_multi_inlet_PSR.png", bbox_inches="tight")
```

### 22.5.4 Transient simulation of methane-air ignition in a PSR

This project demonstrates the process of running a transient perfectly stirred reactor model (transPSR) with multiple inlets. Normally the PSR model is a 0-D steady-state model with constant pressure. However, the dynamic responses of a chemistry domanted process are in interest, performing transient simulations of the system becomes necessary. In this example, a transient PSR model with the fuel streamand the air stream introduced separately. The "mean" reactor residence time is given to cap the time available for ignition to take place inside the reactor. The reactor is initially filled with hot air to promote the chemical reactions. The fuel-air mixture must ignite within the time duration of 1 "residence time" or the PSR would go cold. The solution profiles also indicate that it takes several "residence times" for the PSR to reach steady state.

```

```

#### Import PyChemkin packages and start the logger

```
from pathlib import Path
import time

import matplotlib.pyplot as plt  # plotting

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color
from ansys.chemkin.core.inlet import Stream  # external gaseous inlet
from ansys.chemkin.core.logger import logger

# Chemkin 0-D transient PSR model with given reactor residence time
from ansys.chemkin.core.stirreactors.transient_PSR import (
    TransientPSRSetResTimeEnergyConservation as TransPsr,
)

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create an instance of the Chemistry Set

The mechanism loaded is the GRI 3.0 mechanism for methane combustion. The mechanism and its associated data files come with the standard Ansys Chemkin installation under the subdirectory *"/reaction/data"*.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir
# including the full file path is recommended
chemfile = str(mechanism_dir / "grimech30_chem.inp")
thermfile = str(mechanism_dir / "grimech30_thermo.dat")
# create a chemistry set based on GRI 3.0
MyGasMech = ck.Chemistry(chem=chemfile, therm=thermfile, label="GRI 3.0")
```

### Preprocess the Chemistry Set

```
# preprocess the mechanism files
ierror = MyGasMech.preprocess()
if ierror != 0:
    print("Error: failed to preprocess the mechanism!")
    print(f"       error code = {ierror}")
    exit()
```

### Set up the fuel and the air inlet streams

Instantiate a stream feed for the inlet gas mixture. The stream is a mixture with the addition of the inlet flow rate. You specify inlet gas properties in the same way as you set up a mixture.

Use the `x` method of the `Stream` object to specify the inlet gas composition with a `recipe`. In this project, the `mass_flowrate` method, [g/sec] is used set the volumetric flow rates of the two inlet streams. The `air` composition is copied from the pre-defined `Air` object in Pychemkin.

```
# set the fuel stream composition and conditions
fuel = Stream(MyGasMech, label="methane")
fuel.pressure = 2.0 * ck.P_ATM
fuel.temperature = 300.0
fuel.x = [("CH4", 1.0)]
# methane mass flow rate [g/sec]
fuel.mass_flowrate = 0.5

# set the air composition and conditions
air = Stream(MyGasMech, label="air")
air.pressure = fuel.pressure
air.temperature = 350.0
air.x = ck.Air.x()
# air stream mass flow rate to create a stoichiiometric combustion
# condition with methane [g/sec]
air.mass_flowrate = 8.1
```

### Set up the transient PSR bomb reactor

Instantiate the transient PSR `fire_bomb` as a `TransientPSRSetResTimeEnergyConservation` object. Since this is a transient simulation, a initial reactor condition is required. In this example, the `fire_bomb` is initially filled with `air`.

```
fire_bomb = TransPsr(air, label="Fire_Bomb")
```

### Set up additional reactor model parameters

Before you can run the simulation, you must provide reactor parameters, solver controls, and output instructions. For a PSR, you must provide either the residence time or the reactor volume. You can also make changes to any reactor initial conditions such as the reactor temperature and the gas composition. For example, the `reset_initial_temperature()` method and the `reset_initial_composition()` method. You can use the solver parameters to improve the convergence performance.

```
# set PSR residence time (sec): required for the
# ``TransientPSRSetResTimeEnergyConservation`` model
fire_bomb.residence_time = 0.02
# transient simulation end time [sec]
# for PSR, the simulation time should be greater than the reactor residence time
fire_bomb.time = 0.2
# raise the initial reactor temperature to trigger ignition
fire_bomb.reset_initial_temperature(1500.0)
```

### Connect the inlet to the reactor

You must connect at least one inlet to the open reactor. Use the `set_inlet()` method to add a stream to the PSR. Inversely, use the `remove_inlet()` to disconnect an inlet from the PSR.

**Note**

There is no limit on the number of inlets that can be connected to a PSR.

```
# connect the fuel stream to the reactor
fire_bomb.set_inlet(fuel)

# connect the air stream to the reactor
fire_bomb.set_inlet(air)
```

### Set solver controls

You can overwrite the default solver controls by using solver-related methods, such as those for tolerances.

```
# set solver the tolerances
fire_bomb.tolerances = (1.0e-10, 1.0e-8)
# solver non-negative mode
fire_bomb.force_nonnegative = True
# transient solver max time step size [sec]
fire_bomb.set_solver_max_timestep_size(2.0e-3)
# set adaptive solution saving distance
fire_bomb.adaptive_solution_saving(mode=True, steps=60)
# use the legacy solver to get better coonvergence performance for
# small mechanism
fire_bomb.set_legacy_option(option=True)
# set ignition delay definition
fire_bomb.set_ignition_delay(method="T_inflection")
```

### Run the transient PSR model

Once all the necessary input parameters are provided, use the `run()` method to start the transient PSR simulation. After the simulation is completed successfully, use the `get_ignition_delay` method to extract the ignition delay time.

```
# set the start wall time
start_time = time.time()
# run the reactor model
runstatus = fire_bomb.run()
#
if runstatus == 0:
    # get ignition delay time
    delaytime = fire_bomb.get_ignition_delay()
    print(f"ignition delay time = {delaytime} [msec]")
else:
    # if get this, most likely the END time is too short
    print(Color.RED + ">>> Run failed. <<<", end=Color.END)

runtime = time.time() - start_time
print(f"Total simulation duration: {runtime} [sec].")

# postprocess the solution profiles
fire_bomb.process_solution()

# get the number of data points in the solution profiles
n_points = fire_bomb.getnumbersolutionpoints()
print(f"number of solution time points = {n_points}")
# store solution profiles
# simulation time [sec]
sim_time = fire_bomb.get_solution_variable_profile("time")
# convert [sec] to [msec]
sim_time *= 1.0e3
# temperature solution profile [K]
temp_profile = fire_bomb.get_solution_variable_profile("temperature")
# total heat release rate from the gas-phase reactions [erg/sec]
gashrr_profile = fire_bomb.get_solution_variable_profile("gashrr")
# convert [erg] to [kJ]
gashrr_profile /= 1.0e3 * ck.ERGS_PER_JOULE
# gas phase CO solution profile [mass fraction]
co_profile = fire_bomb.get_solution_variable_profile("CO")
# gas phase CH4 solution profile [mass fraction]
ch4_profile = fire_bomb.get_solution_variable_profile("CH4")
# gas phase CO solution profile [mass fraction]
h2o_profile = fire_bomb.get_solution_variable_profile("H2O")
# gas phase NO solution profile [mass fraction]
no_profile = fire_bomb.get_solution_variable_profile("NO")

# clean up
ck.done()
```

#### Plot the transient PSR solution profiles

Plot the solution profiles over the entire simulation time span. You could observe the ignition occurs around time = 0.002 [msec] by the spike in the heat release rate profile.

```
plt.subplots(2, 2, sharex="col", figsize=(12, 6))
plt.suptitle("Transient PSR Solution", fontsize=16)
plt.subplot(221)
plt.plot(sim_time, temp_profile, "r-")
plt.ylabel("Temperature [K]")
plt.subplot(222)
plt.plot(sim_time, gashrr_profile, "b-")
plt.ylabel("Total Heat Release Rate [kJ/sec]")
plt.subplot(223)
plt.plot(sim_time, co_profile, "g-")
plt.plot(sim_time, ch4_profile, "r--")
plt.plot(sim_time, h2o_profile, "b:")
plt.legend(["CO", "CH4", "H2O"], loc="upper right")
plt.yscale("log")
plt.xlabel("Time [msec]")
plt.ylabel("Mass Fraction")
plt.subplot(224)
plt.plot(sim_time, no_profile, "b-")
plt.xlabel("Time [msec]")
plt.ylabel("NO Mass Fraction")
# plot results
if interactive:
    plt.show()
else:
    plt.savefig("plot_trans_PSR_ignition.png", bbox_inches="tight")
```

## 22.6 Plug-flow reactor

*Plug-Flow Reactor (PFR)* model is an idealized 1-D steady-state flow reactor model with single inlet stream. The reactor pressure is assumed to be given, and the mass (flow rate) conservation is maintained by the velocity change. The gas temperature along the reactor can be either specified or solved by the energy equation.

### 22.6.1 Simulate NO reduction in combustion exhaust

The PFR (plug flow reactor) model is widely adopted in the chemical kinetic studies of emission abatement processes because of its simplicity in flow treatment as well as its resemblance of an exhaust duct or a tailpipe.

The PFR model is a steady-state open reactor because materials are allowed to move in and out of the reactor. The solutions are obtained along the length of the reactor as the inlet materials march toward the reactor exit. Typically, the pressure of the PFR is fixed. However, Chemkin does permit the use of a pressure profile to enforce a certain pressure gradient in the PFR.

Use the `PFRFixedTemperature()` or `PFREnergyConservation()` method to create a PFR. Unlike the closed batch reactor model, which is instantiated by a mixture, the open reactor model, such as the PFR model, is initiated by a stream, which is simply a mixture with the addition of the inlet mass/volumetric flow rate or velocity. You already know how to create a stream if you know how to create a mixture. You can specify the inlet flow rate using one of these methods: `velocity()`, `mass_flowrate()`, `vol_flowrate()`, or `sccm()` (standard cubic centimeters per minute).

This example shows how to use the Chemkin PFR model to study the reduction of nitric oxide (NO) in the combustion exhaust by using the $CH_4$ reburning process. This is achieved by injecting $CH_4$ into the hot exhaust gas mixture. As

the exhaust gas travels along the tubular reactor, the injected $CH_4$ is oxidized by the NO to form $N_2$, CO, and $H_2$. This is why this NO reduction process is called *methane reburning*.

```
```

### Import PyChemkin packages and start the logger

```
from pathlib import Path
import time

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color

# chemkin plug flow reactor model
from ansys.chemkin.core.flowreactors.PFR import PFRFixedTemperature
from ansys.chemkin.core.inlet import Stream
from ansys.chemkin.core.logger import logger

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create a chemistry set

The mechanism to load is the C2 NOx mechanism for the combustion of C1-C2 hydrocarbons. The mechanism comes with the standard Ansys Chemkin installation in the `/reaction/data` directory.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir
# create a chemistry set based on the GRI mechanism
MyGasMech = ck.Chemistry(label="C2 NOx")
# set mechanism input files
# including the full file path is recommended
MyGasMech.chemfile = str(mechanism_dir / "C2_NOx_SRK.inp")
```

### Preprocess the GRI 3.0 chemistry set

```
# preprocess the mechanism files
_ = MyGasMech.preprocess()
```

### Instantiate and set up the stream

Create a hot exhaust gas mixture by assigning the mole fractions of the species. A stream is simply a mixture with the addition of mass/volumetric flow rate or velocity. Here, the gas composition `exhaust.x` is given by a recipe consisting of the common species in the combustion exhaust, such as $CO_2$ and $H_{2`O and the CH:sub:`4}$ injected. The inlet velocity is given by `exhaust.velocity = 26.815`.

```
# create the inlet (mixture + flow rate)
exhaust = Stream(MyGasMech)
# set inlet temperature [K]
exhaust.temperature = 1750.0
# set inlet/PFR pressure [atm]
exhaust.pressure = 0.83 * ck.P_ATM
# set inlet molar composition directly
exhaust.x = [
    ("CO", 0.0145),
    ("CO2", 0.0735),
    ("H2O", 0.1107),
    ("NO", 0.0012),
    ("N2", 0.7981),
    ("CH4", 0.002),
]
# set inlet velocity [cm/sec]
exhaust.velocity = 26.815
#
```

### Create the plug flow reactor object

Use the `PFRFixedTemperature()` method to create a plug flow reactor. The required input parameter of the open reactor models in PyChemkin is the stream, while PyChemkin batch reactor models take a mixture as the input parameter. In this case, the exhaust stream is used to create a PFR named `tubereactor`.

```
tubereactor = PFRFixedTemperature(exhaust)
```

### Set up additional reactor model parameters

For the PFR, the required reactor parameters are the reactor diameter [cm] or the cross-sectional flow area [cm2] and the reactor length [cm]. The mixture condition and inlet mass flow rate of the inlet are already defined by the stream when the `tubereactor` PFR is instantiated.

```
# set PFR diameter [cm]
tubereactor.diameter = 5.8431
# set PFR length [cm]
tubereactor.length = 5.0
```

### Verify the inlet condition of the PFR

```
print(f"PFR inlet mass flow rate {tubereactor.mass_flowrate} [g/sec]")
print(f"PFR inlet velocity {tubereactor.velocity} [cm/sec]")
# show inlet gas composition of the PFR
print("PFR inlet gas compsition")
tubereactor.list_composition(mode="mass", bound=1.0e-8)
```

### Set output options

You can turn on adaptive solution saving to resolve the steep variations in the solution profile. Here, additional solution data points are saved for every **100** internal solver steps.

**Note**

By default, distance intervals for both print and save solution are 1/100 of the reactor length. In this case, $dt = time/100 = 0.001$. You can change them to different values.

```
# set distance between saving solution
tubereactor.timestep_for_saving_solution = 0.0005
# turn on adaptive solution saving
tubereactor.adaptive_solution_saving(mode=True, steps=100)
```

### Display the added parameters (keywords)

Use the `showkeywordinputlines()` method to verify that the preceding parameters are correctly assigned to the reactor model.

```
tubereactor.showkeywordinputlines()
```

### Run the parameter study to replicate the experiments

Use the `run()` method to start the plug flow simulation.

**Note**

You can use two `time` calls (one before the run and one after the run) to get the simulation run time (wall time).

```
# set the start wall time
start_time = time.time()
# run the PFR model
runstatus = tubereactor.run()
# compute the total runtime
runtime = time.time() - start_time
# check run status
if runstatus != 0:
    # Run failed.
    print(Color.RED + ">>> Run failed. <<<", end=Color.END)
    exit()
# run succeeded.
print(Color.GREEN + ">>> Run completed. <<<", end=Color.END)
print(f"Total simulation duration: {runtime * 1.0e3} [msec]")
```

### Postprocess the solution

The postprocessing step parses the solution and packages the solution values at each time point into a mixture. There are two ways to access the solution profiles:

- The raw solution profiles (value as a function of distance) are available for distance, temperature, pressure, volume, and species mass fractions.

- The mixtures permit the use of all property and rate utilities to extract information such as viscosity, density, and mole fractions.

You can use the `get_solution_variable_profile()` method to get the raw solution profiles. You can get solution mixtures using either the `get_solution_mixture_at_index()` method for the solution mixture at the given saved location or the `get_solution_mixture()` method for the solution mixture at the given distance. (In this case, the mixture is constructed by interpolation.)

You can get the outlet solution by simply grabbing the solution values at the very last point by using $(outletsolutionindex) = (numberofsolutions) - 1$.

```
# postprocess the solution profiles
tubereactor.process_solution()

# get the number of solution time points
solutionpoints = tubereactor.getnumbersolutionpoints()
print(f"number of solution points = {solutionpoints}")

# get the grid profile [cm]
xprofile = tubereactor.get_solution_variable_profile("distance")
# get the temperature profile [K]
tempprofile = tubereactor.get_solution_variable_profile("temperature")
# get the CH4 mass fraction profile
ch4_y_profile = tubereactor.get_solution_variable_profile("CH4")
# get the CO mass fraction profile
co_y_profile = tubereactor.get_solution_variable_profile("CO")
# get the H2O mass fraction profile
h2o_y_profile = tubereactor.get_solution_variable_profile("H2O")
# get the H2 mass fraction profile
h2_y_profile = tubereactor.get_solution_variable_profile("H2")
# get the NO mass fraction profile
no_y_profile = tubereactor.get_solution_variable_profile("NO")
# get the N2 mass fraction profile
n2_y_profile = tubereactor.get_solution_variable_profile("N2")

# inlet NO mass fraction
no_y_inlet = exhaust.y[MyGasMech.get_specindex("NO")]
# outlet grid index
xout_index = solutionpoints - 1
print("At the reactor inlet: x = 0 [cm]")
print(f"The NO mass fraction = {no_y_inlet}.")
print(f"At the reactor outlet: x = {xprofile[xout_index]} [cm]")
print(f"The NO mass fraction = {no_y_profile[xout_index]}.")
print(
    "The NO conversion rate = "
    + f"{(no_y_inlet - no_y_profile[xout_index]) / no_y_inlet * 100.0} %.\n"
)

# more involved postprocessing using mixtures
#
# create arrays for the gas velocity profile
velocityprofile = np.zeros_like(xprofile, dtype=np.double)
# reactor mass flow rate (constant) [g/sec]
massflowrate = tubereactor.mass_flowrate
```

(continues on next page)

(continued from previous page)

```
# reactor cross-section area [cm2]
areaflow = tubereactor.flowarea
print(f"Mass flow rate: {massflowrate} [g/sec]\nflow area: {areaflow} [cm2]")
# ratio
ratio = massflowrate / areaflow
# loop over all solution time points
for i in range(solutionpoints):
    # get the mixture at the time point
    solutionmixture = tubereactor.get_solution_mixture_at_index(solution_index=i)
    # get gas density [g/cm3]
    den = solutionmixture.rho
    # gas velocity [g]
    velocityprofile[i] = ratio / den
```

### Plot the solution profiles

Plot the species and the gas velocity profiles along the tubular reactor.

You can see from the plots that $CH_4$ is mainly oxidized by NO to form $N_2$ and $H_2$in the hot exhaust. The gas velocity accelerates because of the net increase in total number of moles of gas species due to the chemical reactions. You can conduct more in depth analyses of the reaction pathways of the $CH_4$ reburning mechanism by applying other PyChemkin utilities.

```
plt.subplots(2, 2, sharex="col", figsize=(12, 6))
plt.suptitle("Constant Temperature Plug-Flow Reactor", fontsize=16)
plt.subplot(221)
plt.plot(xprofile, n2_y_profile, "r-")
plt.ylabel("N2 Mass Fraction")
plt.subplot(222)
plt.plot(xprofile, h2o_y_profile, "b-", label="H2O")
plt.ylabel("H2O Mass Fraction")
plt.subplot(223)
plt.plot(xprofile, no_y_profile, "g-", label="NO")
plt.plot(xprofile, ch4_y_profile, "g:", label="CH4")
plt.plot(xprofile, h2_y_profile, "g--", label="H2")
plt.legend()
plt.xlabel("distance [cm]")
plt.ylabel("Mass Fraction")
plt.subplot(224)
plt.plot(xprofile, velocityprofile, "m-")
plt.xlabel("distance [cm]")
plt.ylabel("Gas Velocity [cm/sec]")

# clean up
ck.done()

# plot results
if interactive:
    plt.show()
else:
    plt.savefig("plot_plug_flow_reactor.png", bbox_inches="tight")
```

## 22.7 Shock tube reactor

The *shock tube reactor* model is a 0-D transient reactor. This ideal reactor model is normally used to reproduce the shock tube experiments for chemical kinetics researches. The *incident shock* model predicts species profiles behind the incident shock front which can be used to compare against corresponding experimental data. Similarly, the *reflected shock* model complements the experiments for kinetics researches at the high temperature conditions behind a reflected shock front.

The examples will show the procedures of setting up an incident shock (with boundary layer correction) simulation and a Zeldovich-von Neumann-Deoring (ZND) analysis of a combustible gas mixture.

### 22.7.1 Perform ZND analysis of a combustible hydrogen mixture

The ZND (Zeldovich-von Neumann-Deoring) model is commonly applied to study the evolution of the gas mixture behind a detonation wave.

The ZND model is a transient reactor, and the solutions describe the state of the gas mixture (particle) as it moving away from the detonation front in the shock tube reactor.

Use the `ZNDCalculator()` method to create a ZND shock tube reactor. The shock tube reactor models, such as the IncidentShcok, the ReflectedShock and the ZNDCalculator models, are initiated by a stream, which is simply a mixture with the addition of the shock wave velocity. You already know how to create a stream if you know how to create a mixture. You can specify the shock wave velocity using the combination of the `velocity` method of the initial gas stream and the `location` parameter when you instantiate the `incidentShock` or the `ReflectedShock` object.

This example shows how to use the Chemkin ZND shock tube model to study the gas mixture evolution behind an incident detonation wave front. The speed of the incident shock wave is estimated by the ZND model automatically by finding the stable detonation wave speed corresponding to the given gas mixture before the incident shock front. Once the solution is obtained, you can extract the induction zone length behind the wave front and utilize this information to gain insights about the characteristic size of the cellular structure in multi-dimensional flow as well as the stability of the cellular structure.

#### Import PyChemkin packages and start the logger

```
from pathlib import Path
import time

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color
from ansys.chemkin.core.inlet import Stream
from ansys.chemkin.core.logger import logger

# chemkin plug flow reactor model
from ansys.chemkin.core.shock.shocktubereactors import ZNDCalculator as Znd
from ansys.chemkin.core.utilities import find_file

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set interactive mode for plotting the results
```

(continues on next page)

(continued from previous page)

```
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create a chemistry set

The mechanism used here is the MFL 2021 hydrogen mechanism. The mechanism comes with the standard Ansys Chemkin installation in the `/reaction/data/ModelFuelLibrary/Skeletal` directory.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data" / "ModelFuelLibrary" / "Skeletal"
mechanism_dir = data_dir
# create a chemistry set based on the MFL 2021 hydrogen mechanism
MyGasMech = ck.Chemistry(label="hydrogen")
# set mechanism input files
# including the full file path is recommended
MyGasMech.chemfile = find_file(mechanism_dir, "Hydrogen_chem_MFL", "inp")
```

### Preprocess the hydrogen chemistry set

```
# preprocess the mechanism files
_ = MyGasMech.preprocess()
```

### Instantiate and set up the stream

Create a combustible gas stream by assigning the mole fractions of the species. Here, several gas streams are created with different recipes consisting of $H_2$, $O_2$ and $N_2$. You can pick any one of the streams to instantiate the `ZNDCalculator`. You can compare the "induction lengths" by different streams to see how the combustion intensity (reversely proportion to the amount of $N_2$ dilution) impacts the size of the induction zone behind the detonation wave front.

```
stream_a = Stream(MyGasMech)
# initial gas state before the incident shock front
# 1 [atm]
stream_a.pressure = ck.P_ATM
# 300 [K]
stream_a.temperature = 300.0
# 50% hydrogen + 50% oxygen by volume
stream_a.x = [("h2", 1.0), ("o2", 1.0)]
#
stream_b = Stream(MyGasMech)
# a diluted initial gas state before the incident shock front
# 1 [atm]
stream_b.pressure = ck.P_ATM
# 300 [K]
stream_b.temperature = 300.0
# the 50% hydrogen + 50% oxygen mixture diluted by 40% nitrogen by volume
stream_b.x = [("h2", 0.3), ("o2", 0.3), ("n2", 0.4)]
#
stream_c = Stream(MyGasMech)
# a diluted initial gas state before the incident shock front
```

(continues on next page)

(continued from previous page)

```
# 1 [atm]
stream_c.pressure = ck.P_ATM
# 300 [K]
stream_c.temperature = 300.0
# the 50% hydrogen + 50% oxygen mixture diluted by 40% nitrogen by volume
stream_c.x = [("h2", 0.2), ("o2", 0.2), ("n2", 0.6)]
```

### Create the shock tube reactor object

Use the `ZNDCalculator()` method to create an incident shock reactor. The required input parameter is the stream representing the state of the initial gas mixture before the incident shock front. In this case, `stream_a` is used to initialize the ZND model `ZNDIncident`.

```
ZNDIncident = Znd(stream_c, label="ZND")
```

### Set up additional reactor model parameters

For the ZND calculator model, the required reactor parameters is the total simulation time [sec]. The initial gas mixture conditions are defined by the stream when the `ZNDIncident` is instantiated.

```
# set total simulation time (particle time) [sec]
ZNDIncident.time = 3.0e-6
```

### Set solver controls

You can overwrite the default solver controls by using solver related methods, for example, `tolerances`.

```
# tolerances are given in tuple: (absolute tolerance, relative tolerance)
ZNDIncident.tolerances = (1.0e-10, 1.0e-6)
```

### Run the ZND analysis

Use the `run()` method to start the ZND analysis.

**Note**

You can use two `time` calls (one before the run and one after the run) to get the simulation run time (wall time).

```
# set the start wall time
start_time = time.time()
# run the ZND model
runstatus = ZNDIncident.run()
# compute the total runtime
runtime = time.time() - start_time
# check run status
if runstatus != 0:
    # Run failed.
    print(Color.RED + ">>> Run failed. <<<", end=Color.END)
    exit()
# run succeeded.
```

(continues on next page)

(continued from previous page)

```
print(Color.GREEN + ">>> Run completed. <<<", end=Color.END)
print(f"Total simulation duration: {runtime * 1.0e3} [msec]")
```

### Postprocess the solution

The postprocessing step parses the solution and packages the solution values at each time point into a mixture. There are two ways to access the solution profiles:

- The raw solution profiles (value as a function of time) are available for distance, temperature, pressure, velocity, density, and species mass fractions.
- The mixtures permit the use of all property and rate utilities to extract information such as viscosity, density, total thermicity, speed of sound, Mach number, and mole fractions.

You can use the `get_solution_variable_profile()` method to get the raw solution profiles. You can get solution mixtures using either the `get_solution_stream_at_index()` method for the solution mixture at the given saved location or the `get_solution_stream()` method for the solution mixture at the given distance. (In this case, the mixture is constructed by interpolation.)

You can get ZND analysis information about the induction zone length by using the `get_inductionlength_size()` and the `get_induction_lengths()` methods. The induction zone length can be used to derive the characteristic cell size and the stability of the multi-dimensional cellular wave structure.

```
# postprocess the solution profiles
ZNDIncident.process_solution()

# get the number of solution time points
solutionpoints = ZNDIncident.get_solution_size()
print(f"number of solution points = {solutionpoints}")
# get information about the induction length
numb_induct_length = ZNDIncident.get_inductionlength_size()
print(f"number of induction length = {numb_induct_length}")
if numb_induct_length > 0:
    induct_legth, sigmamax = ZNDIncident.get_induction_lengths(numb_induct_length)
    for i in range(numb_induct_length):
        print(f"Induction length # {i + 1}:")
        print(f"        Induction length = {induct_legth[i] * 1.0e1} [mm].")
        print(f"    Local max thermicity = {sigmamax[i]} [1/sec].\n")

# estimated detonation wave cell structure parameters
cell_width, chi = ZNDIncident.calculate_cell_width_ng()
print(f"Instability parameter Chi = {chi}")  # stable = Chi > 1
print(f"Estimated cell size = {cell_width} [cm].")

# get the grid profile [cm]
xprofile = ZNDIncident.get_solution_variable_profile("distance")
# get the temperature profile [K]
tempprofile = ZNDIncident.get_solution_variable_profile("temperature")
# get the pressure profile [dynes/cm2]
pressprofile = ZNDIncident.get_solution_variable_profile("pressure")
# get the velocity profile [cm/sec]
velprofile = ZNDIncident.get_solution_variable_profile("velocity")
# get the total thermiocity profile [1/sec]
sigmaprofile = ZNDIncident.get_solution_variable_profile("thermicity")
```

(continues on next page)

(continued from previous page)

```
# create arrays for the gas Mach number profile
machprofile = np.zeros_like(xprofile, dtype=np.double)
# loop over all solution time points
for i in range(solutionpoints):
    # get the mixture at the time point
    solutionmixture = ZNDIncident.get_solution_stream_at_index(solution_index=i)
    # gas speed of sound [cm/sec]
    soundspeed = solutionmixture.sound_speed()
    # gas Mach number
    machprofile[i] = velprofile[i] / soundspeed
    # convert pressure from [dynes/cm2] to [bar]
    pressprofile[i] /= 1.0e6
```

#### Plot the solution profiles

Plot the temperature, pressure, total thermicity, and the gas Mach number profiles as a function of distance behind the wave front.

```
plt.subplots(2, 2, sharex="col", figsize=(12, 6))
plt.suptitle("ZND Analysis", fontsize=16)
plt.subplot(221)
plt.plot(xprofile, tempprofile, "r-")
plt.ylabel("Temperature [K]")
plt.subplot(222)
plt.plot(xprofile, pressprofile, "b-")
plt.ylabel("Pressure [bar]")
plt.subplot(223)
plt.plot(xprofile, machprofile, "g-")
plt.xlabel("distance behind shock [cm]")
plt.ylabel("Mach Number [-]")
plt.subplot(224)
plt.plot(xprofile, sigmaprofile, "m-")
plt.xlabel("distance behind shock [cm]")
plt.ylabel("Total Thermicity [1/sec]")

# clean up
ck.done()

# plot results
if interactive:
    plt.show()
else:
    plt.savefig("plot_shock_ZND_reactor.png", bbox_inches="tight")
```

### 22.7.2 Nitric oxide formation in air heated by an incident shock wave

Shock tube experiments are commonly used to study reaction paths and to measure reaction rates at elevated temperatures. You can apply the incident shock reactor model `IncidentShock()` to validate the reaction mechanism or kinetic parameters derived from such experiments.

The shock tube reactor models, such as the IncidentShcok, the ReflectedShock and the ZNDCalculator models, are initiated by a stream, which is simply a mixture with the addition of the shock wave velocity. You already know how to

create a stream if you know how to create a mixture. You can specify the shock wave velocity using the combination of the `velocity` method of the initial gas stream and the `location` parameter when you instantiate the `incidentShock` or the `ReflectedShock` object.

This example aims to reproduce one of the shock tube experiments done by Camac and Feinberg. Camac and Feinberg measured the production rates of nitric oxide (NO) in shock-heated air over the temperature range of 2300 [K] to 6000 [K]. The NO mole fraction behind the incident shock will be plotted as a function of time (after the passing of the incident shock front). The NO mole fraction profile rapidly rises to a peak value then gradually falls back to its equilibrium level. The predicted peak NO mole fraction is 0.04609 and is in good agreement with the measured and the computed data by Camac and Feinberg.

Reference: M. Camac and R.M. Feinberg, Proceedings of Combustion Institute, vol. 11, p. 137-145 (1967)

### Import PyChemkin packages and start the logger

```
from pathlib import Path
import time

import matplotlib.pyplot as plt  # plotting

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color
from ansys.chemkin.core.inlet import Stream
from ansys.chemkin.core.logger import logger

# chemkin plug flow reactor model
from ansys.chemkin.core.shock.shocktubereactors import IncidentShock

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create the hot air dissociation mechanism file

Create a new mechanism input file 'no_hot_air_chem.inp' that contains the reactions to describe NO formation in heated air. This file is saved to the working directory `current_dir`.

```
mymechfile = Path(current_dir) / "no_hot_air_chem.inp"
m = mymechfile.open(mode="w")
# the mechanism contains only the necessary species
# (oxygen, nitrogen, nitric oxide, and major byproducts)
# declare elements
m.write("ELEMENT O N AR END\n")
# declare species
m.write("SPECIES\n")
m.write("O2 N2 NO N O AR\n")
m.write("END\n")
```

(continues on next page)

(continued from previous page)

```
# write reactions for N2O dissociation
# Reference:
# M. Camac and R.M. Feinberg, Proceedings of Combustion Institute,
# vol. 11, pp. 137-145 (1967)
m.write("REACTIONS\n")
m.write("N2+O2=NO+NO             9.1E24   -2.5   128500.\n")
m.write("N2+O=NO+N               7.0E13    0.     75000.\n")
m.write("O2+N=NO+O               1.34E10   1.0     7080.\n")
m.write("O2+M=O+O+M              3.62E18  -1.0   118000.\n")
m.write("N2/2/  O2/9/   O/25/\n")
m.write("N2+M=N+N+M              1.92E17  -0.5   224900.\n")
m.write("N2/2.5/  N/0/\n")
m.write("N2+N=N+N+N              4.1E22   -1.5   224900.\n")
m.write("NO+M=N+O+M              4.0E20   -1.5   150000.\n")
m.write("NO/20/  O/20/  N/20/\n")
m.write("END\n")
# close the mechnaism file
m.close()
```

### Create a chemistry set

The mechanism used here is the air dissociation mechanism. The mechanism will be created in situ in the working directory. The thermodynamic data file is the standard one that comes with the Ansys Chemkin installation in the `/reaction/data` directory.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir
# create a chemistry set based on the N2O dissociation mechanism
MyGasMech = ck.Chemistry(label="NO_from_hot_air")
# set mechanism input files
# including the full file path is recommended
MyGasMech.chemfile = str(mymechfile)
MyGasMech.thermfile = str(data_dir / "therm.dat")
```

### Preprocess the hydrogen chemistry set

```
# preprocess the mechanism files
_ = MyGasMech.preprocess()
```

### Instantiate and set up the stream

Create a diluted air stream by assigning the mole fractions of the species.

```
diluted_air = Stream(MyGasMech)
# initial gas state before the incident shock front
# 5 [torrs]
diluted_air.pressure = 5.0 * ck.P_TORRS
# 296 [K]
diluted_air.temperature = 296.0
# AR diluted air based on the experiment setup
```

(continues on next page)

(continued from previous page)

```
diluted_air.x = [("AR", 0.0093), ("O2", 0.2095), ("N2", 0.7812)]
#
```

### Create the shock tube reactor object

Use the `IncidentShock()` method to create an incident shock reactor. The `IncidentShock()` method has two required input parameters. The first parameter is the "stream" representing the state of the initial gas mixture. In this case, it is the `diluted_air`. The second required parameter is the location of the initial gas stream relative to the incident shock front. A value of '1' indicate the initial gas stream is before the incident shock front (state 1), and a value of '2' indicates the gas stream is behind the incident shock (state 2). The gas velocity (same as the incident shock velocity) must be given by the `velocity` method for `IncidentShock()` model.

```
# set the incident shock velocity [cm/sec]
diluted_air.velocity = 2.8e5

# instantiate the shock tube reactor
# the location '1' means the gas stream is before the incident shock front
Incident = IncidentShock(diluted_air, location=1, label="incident_shock")
```

### Set up additional reactor model parameters

For the incident shock model, the required reactor parameters is the total simulation time [sec]. The initial gas mixture conditions are defined by the stream when the `IncidentShock` is instantiated. To include the boundary layer correction in the shock tube model, you must specify both the 'shock tube diameter' and the 'gas viscosity' at 300 [k] using the `diameter` and the `set_inlet_viscosity()` methods, respectively.

```
# to use the boundary layer correction, both diameter and viscocity must be given
# shock tube diameter [cm]
Incident.diameter = 3.81
# mixture viscosity [g/cm-sec] at 300 [K]
Incident.set_inlet_viscosity(2.0e-4)

# set total simulation time (particle time) [sec]
Incident.time = 2.0e-3
```

### Set solver controls

You can overwrite the default solver controls by using solver related methods, for example, `tolerances`.

```
# tolerances are given in tuple: (absolute tolerance, relative tolerance)
Incident.tolerances = (1.0e-8, 1.0e-4)
```

### Run the ZND analysis

Use the `run()` method to start the ZND analysis.

**Note**

You can use two `time` calls (one before the run and one after the run) to get the simulation run time (wall time).

```
# set the start wall time
start_time = time.time()
# run the ZND model
runstatus = Incident.run()
# compute the total runtime
runtime = time.time() - start_time
# check run status
if runstatus != 0:
    # Run failed.
    print(Color.RED + ">>> Run failed. <<<", end=Color.END)
    exit()
# run succeeded.
print(Color.GREEN + ">>> Run completed. <<<", end=Color.END)
print(f"Total simulation duration: {runtime * 1.0e3} [msec]")
```

### Postprocess the solution

The postprocessing step parses the solution and packages the solution values at each time point into a mixture. There are two ways to access the solution profiles:

- The raw solution profiles (value as a function of time) are available for distance, temperature, pressure, velocity, density, and species mass fractions.
- The mixtures permit the use of all property and rate utilities to extract information such as viscosity, density, total thermicity, speed of sound, Mach number, and mole fractions.

You can use the `get_solution_variable_profile()` method to get the raw solution profiles. You can get solution mixtures using either the `get_solution_stream_at_index()` method for the solution mixture at the given saved location or the `get_solution_stream()` method for the solution mixture at the given distance. (In this case, the mixture is constructed by interpolation.)

```
# postprocess the solution profiles
Incident.process_solution()

# get the number of solution time points
solutionpoints = Incident.get_solution_size()
print(f"number of solution points = {solutionpoints}")

# get the time profile [sec]
timeprofile = Incident.get_solution_variable_profile("time")
# convert to [msec]
timeprofile *= 1.0e3
# get the temperature profile [K]
tempprofile = Incident.get_solution_variable_profile("temperature")
# get the velocity profile [cm/sec]
velprofile = Incident.get_solution_variable_profile("velocity")
# convert to [m/sec]
velprofile *= 1.0e-2
# get the NO mass fraction profile [-]
no_profile = Incident.get_solution_variable_profile("NO")
# get the O mass fraction profile [-]
o_profile = Incident.get_solution_variable_profile("O")

# clean up
```

(continues on next page)

(continued from previous page)

```
if mymechfile.exists and mymechfile.is_file():
    mymechfile.unlink()
```

#### Plot the solution profiles

Plot the temperature, the velocity, and the NO and the O species mass fraction profiles as a function of time.

```
plt.subplots(2, 2, sharex="col", figsize=(12, 6))
plt.suptitle("Incident Shock", fontsize=16)
plt.subplot(221)
plt.plot(timeprofile, tempprofile, "r-")
plt.ylabel("Temperature [K]")
plt.subplot(222)
plt.plot(timeprofile, velprofile, "b-")
plt.ylabel("Velocity [m/sec]")
plt.subplot(223)
plt.plot(timeprofile, no_profile, "g-")
plt.xlabel("time [msec]")
plt.ylabel("NO Mass Fraction [-]")
plt.subplot(224)
plt.plot(timeprofile, o_profile, "m-")
plt.xlabel("time [msec]")
plt.ylabel("O Mass Fraction [-]")

# clean up
ck.done()

# plot results
if interactive:
    plt.show()
else:
    plt.savefig("plot_incident_shock.png", bbox_inches="tight")
```

### 22.7.3 Dissociation of nitrous oxide behind a reflected shock wave

Shock tube experiments are commonly used to study reaction paths and to measure reaction rates at elevated temperatures. You can apply the incident shock reactor model `IncidentShock()` to validate the reaction mechanism or kinetic parameters derived from such experiments.

The shock tube reactor models, such as the IncidentShcok, the ReflectedShock and the ZNDCalculator models, are initiated by a stream, which is simply a mixture with the addition of the shock wave velocity. You already know how to create a stream if you know how to create a mixture. You can specify the shock wave velocity using the combination of the `velocity` method of the initial gas stream and the `location` parameter when you instantiate the `incidentShock` or the `ReflectedShock` object.

The gas condition behind a reflected shock is hot and uniform, and is favorable for studying chemical kinetics at high temperatures. This example utilizes the $N_2O$ dissociation mechanism proposed by Baber and Dean to simulate the evolution of $N_2O$ behind a reflected shock. The temperature profile and the mass fraction profiles of major species $N_2O$, NO, and $O_2$ in the hot zone behind the reflected shock will be plotted as a function of time.

Reference: S.C. Baber and A.M. Dean, International Journal of Chemical Kinetics, vol. 7, pp. 381-398 (1975)

```

```

### Import PyChemkin packages and start the logger

```
from pathlib import Path
import time

import matplotlib.pyplot as plt  # plotting

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color
from ansys.chemkin.core.inlet import Stream
from ansys.chemkin.core.logger import logger

# chemkin plug flow reactor model
from ansys.chemkin.core.shock.shocktubereactors import ReflectedShock

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create the nitrous oxide dissociation mechanism file

Create a new mechanism input file 'n2o_dissociation_chem.inp' that contains the reactions to describe N2O dissociation. This file is saved to the working directory `current_dir`.

```
mymechfile = Path(current_dir) / "n2o_dissociation_chem.inp"
m = mymechfile.open(mode="w")
# the mechanism contains only the necessary species
# (oxygen, nitrogen, nitrous oxide, nitric oxide, and major byproducts)
# declare elements
m.write("ELEMENT O N AR END\n")
# declare species
m.write("SPECIES\n")
m.write("O N O2 N2 NO N2O NO2 AR\n")
m.write("END\n")
# write reactions for N2O dissociation
# Reference:
# S.C. Baber and A.M. Dean, International Journal of Chemical Kinetics,
# vol. 7, pp. 381-398 (1975)
# S.C. Baber and A.M. Dean, Journal of Chemical Physics, vol. 60, Mo. 1,
# pp. 307-313 (1974)
m.write("REACTIONS\n")
m.write("N2O+M=N2+O+M            7.83E+14  0.0    56845.32\n")
m.write("N2O+O=NO+NO             1.15E+13  0.0    25078.82\n")
m.write("N2O+O=N2+O2             1.15E+13  0.0    25078.82\n")
m.write("O+NO=N+O2               1.55E+09  1.0    38640.\n")
m.write("N+NO=N2+O               3.10E+13  0.0      334.\n")
m.write("NO+O2=O+NO2             1.99E+12  0.0    47740.\n")
m.write("END\n")
```

(continues on next page)

(continued from previous page)

```
# close the mechnaism file
m.close()
```

### Create a chemistry set

The mechanism used here is the air dissociation mechanism. The mechanism will be created in situ in the working directory. The thermodynamic data file is the standard one that comes with the Ansys Chemkin installation in the /reaction/data directory.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir
# create a chemistry set based on the N2O dissociation mechanism
MyGasMech = ck.Chemistry(label="N2O_dissociation")
# set mechanism input files
# including the full file path is recommended
MyGasMech.chemfile = str(mymechfile)
MyGasMech.thermfile = str(data_dir / "therm.dat")
```

### Preprocess the hydrogen chemistry set

```
# preprocess the mechanism files
_ = MyGasMech.preprocess()
```

### Instantiate and set up the stream

Create a diluted air stream by assigning the mole fractions of the species.

```
n2o_mixture = Stream(MyGasMech)
# initial gas state after the reflected shock wave
# 2290 [K]
n2o_mixture.temperature = 2290.0
# AR + N2O ixture composition based on the experiment setup
n2o_mixture.x = [("AR", 0.9899), ("N2O", 0.0101)]
# calculate the gas pressure after the reflected shock from the temperature and density
# density [g/cm3]
state3_den = 0.00015937
n2o_mixture.set_pressure_by_density(rho=state3_den)
# check the pressure value
print(
    f"Gas pressure after the reflected shock = {n2o_mixture.pressure / ck.P_ATM} [atm]."
)
#
```

### Create the shock tube reactor object

Use the `ReflectedShock()` method to create an incident shock reactor. The `ReflectedShock()` method has two required input parameters. The first parameter is the "stream" representing the state of the initial gas mixture. In this case, it is the `n2o_mixture`. The second required parameter is the location of the initial gas stream relative to the incident shock front. A value of '1' indicate the initial gas stream is before the incident shock front, and a value of '2' indicates the gas stream is behind the reflected shock. When the gas condition before the incident shock are

provided (location = 1), the gas velocity (same as the incident shock velocity) must be given by the `velocity` method for `ReflectedShock()` model.

```
# instantiate the shock tube reactor
# the location '2' means the gas stream is after the reflected shock front
hottube = ReflectedShock(n2o_mixture, location=2, label="reflected_shock")
```

### Set up additional reactor model parameters

For the reflected shock model, the required reactor parameters is the total simulation time [sec]. The initial gas mixture conditions are defined by the stream when the `ReflectedShock` is instantiated.

```
# set total simulation time (particle time) [sec]
hottube.time = 3.0e-4
```

### Set solver controls

You can overwrite the default solver controls by using solver related methods, for example, `tolerances`.

```
# tolerances are given in tuple: (absolute tolerance, relative tolerance)
hottube.tolerances = (1.0e-10, 1.0e-6)
```

### Run the N2O dissociation simulation

Use the `run()` method to start the reflected shock simulation.

**Note**

You can use two `time` calls (one before the run and one after the run) to get the simulation run time (wall time).

```
# set the start wall time
start_time = time.time()
# run the reflected shock tube reactor model
runstatus = hottube.run()
# compute the total runtime
runtime = time.time() - start_time
# check run status
if runstatus != 0:
    # Run failed.
    print(Color.RED + ">>> Run failed. <<<", end=Color.END)
    exit()
# run succeeded.
print(Color.GREEN + ">>> Run completed. <<<", end=Color.END)
print(f"Total simulation duration: {runtime * 1.0e3} [msec]")
```

### Postprocess the solution

The postprocessing step parses the solution and packages the solution values at each time point into a mixture. There are two ways to access the solution profiles:

- The raw solution profiles (value as a function of time) are available for distance, temperature, pressure, velocity, density, and species mass fractions.

- The mixtures permit the use of all property and rate utilities to extract information such as viscosity, density, total thermicity, speed of sound Mach number, and mole fractions.

You can use the `get_solution_variable_profile()` method to get the raw solution profiles. You can get solution mixtures using either the `get_solution_stream_at_index()` method for the solution mixture at the given saved location or the `get_solution_stream()` method for the solution mixture at the given distance. (In this case, the mixture is constructed by interpolation.)

```
# postprocess the solution profiles
hottube.process_solution()

# get the number of solution time points
solutionpoints = hottube.get_solution_size()
print(f"number of solution points = {solutionpoints}")

# get the time profile [sec]
timeprofile = hottube.get_solution_variable_profile("time")
# convert to [msec]
timeprofile *= 1.0e3
# get the temperature profile [K]
tempprofile = hottube.get_solution_variable_profile("temperature")
# get the NO mass fraction profile [-]
no_profile = hottube.get_solution_variable_profile("NO")
# get the N2O mass fraction profile [-]
n2o_profile = hottube.get_solution_variable_profile("N2O")
# get the O2 mass fraction profile [-]
o2_profile = hottube.get_solution_variable_profile("O2")

# clean up
if mymechfile.exists and mymechfile.is_file():
    mymechfile.unlink()
```

### Plot the solution profiles

Plot the temperature and the NO, N2O, and O2 species mass fractions profiles as a function of time.

```
plt.subplots(2, 2, sharex="col", figsize=(12, 6))
plt.suptitle("Behind Reflected Shock", fontsize=16)
plt.subplot(221)
plt.plot(timeprofile, tempprofile, "r-")
plt.ylabel("Temperature [K]")
plt.subplot(222)
plt.plot(timeprofile, no_profile, "b-")
plt.ylabel("NO Mass Fraction [-]")
plt.subplot(223)
plt.plot(timeprofile, n2o_profile, "g-")
plt.xlabel("time [msec]")
plt.ylabel("N2O Mass Fraction [-]")
plt.subplot(224)
plt.plot(timeprofile, o2_profile, "m-")
plt.xlabel("time [msec]")
plt.ylabel("O2 Mass Fraction [-]")

# clean up
```

(continues on next page)

(continued from previous page)

```
ck.done()

# plot results
if interactive:
    plt.show()
else:
    plt.savefig("plot_reflected_shock.png", bbox_inches="tight")
```

## 22.8 PSR network

**PyChemkin** can be used to create an *Equivalence Reactor Network (ERN)* as a reduced-order (in geometry) model for complex combustion applications such as gas turbine combustor. Each reactor in the network represents a sub-region (zone) in the actual equipment. Normally these zones are created according to specific criteria such as temperature, equivalence ratio, or location; and the gas flow (mean, turbulent, and diffusive) between the zones becomes the internal streams between the reactors.

There are several ways to build an *ERN* in **PyChemkin**. The first method is to create and solve the reactors one-by-one starting from the most upstream (first) reactor. Adjust/manipulate the external and outlet streams from connected reactors (that is, solutions from the reactors) using the *Mixture*/*Stream* utilities and apply the desired stream as the inlet for the downstream reactors. The second method takes advantage of the *hybridreactornetwork* "model" of **PyChemkin**. Simply defined the reactors with the associated external inlets and add the reactors to the *hybridreactornetwork*. If there are *recycling* streams from the downstream reactor to the upstream ones, the *hybridreactornetwork* will solve the entire *ERN* iteratively. In this case, a *tear point* must be explicitly specified. The last method, the coupled method, is to create the network and its connectivity first, and the entire *ERN* will be solved in a coupled manner. This method is the default method in Chemkin GUI and is available as the *PSRCluster* "model" in **PyChemkin**.

**Chemkin *ERN* has a few limitations:**

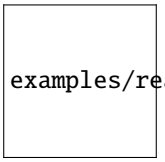
- The first reactor of the reactor network must have at least one external inlet.
- when using the *coupled* model, the entire reactor network can have only one outlet to the surroundings, and the outlet must be attached to the last reactor in the network.
- PSR is the only reactor model allowed in the *PSRCluster* and the *hybridreactornetwork* reactor networks.

The *PSRChain_xxx* examples show the two methods to build and run a series of linked PSRs (no stream recycling) can be modeled in **PyChemkin**. The *PSRnetwork* example goes over the steps of creating and running an *ERN* with recycling streams by using the *hybridreactornetwork* method. The *PSRnetwork_coupled* example solves the same *ERN* as the *PSRnetwork* example but utilizes the *coupled* method. Because the sole outlet from the *coupled ERN* must be attached to the last reactor, the order of the downstream zones is different from the *PSRnetwork* example.

### 22.8.1 Use a chain of individual reactors to model a gas combustor

This example shows how to set up and solve a series of linked PSRs (perfectly-stirred reactors). This is the simplest reactor network as it does not contain any recycling streams or outflow splittings.

Here is the PSR chain model of a fictional gas combustor.



The primary inlet stream to the first reactor, the *combustor*, is the fuel-lean methane-air mixture that is formed by mixing the fuel (methane) and the heated air. The exhaust from the combustor enters the second reactor, the *dilution zone*, where the hot combustion products are cooled by the introduction of additional cool air. The cooled and diluted gas mixture in the *dilution zone* then travels to the third reactor, the *reburning zone*. A mixture of fuel (methane) and carbon dioxide is injected to the gas in the reburning zone, attempting to convert any remaining carbon monoxide or nitric oxide in the exhaust gas to carbon dioxide or nitrogen, respectively.

This example solves the reactors one by one, from upstream to downstream. Once the solution of the upstream reactor is obtained, it is used to set up the external inlet of the immediate downstream reactor. This process continues until all reactors in the chain network are solved. Since there is no recycling stream in this configuration, the entire reactor network can be solved in one sweep.

### Import PyChemkin packages and start the logger

```
from pathlib import Path
import time

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color
from ansys.chemkin.core.inlet import (
    Stream,  # external gaseous inlet
    adiabatic_mixing_streams,
)
from ansys.chemkin.core.logger import logger

# Chemkin PSR model (steady-state)
from ansys.chemkin.core.stirreactors.PSR import PSRSetResTimeEnergyConservation as Psr

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create a chemistry set

The mechanism to load is the GRI 3.0 mechanism for methane combustion. This mechanism and its associated data files come with the standard Ansys Chemkin installation in the `/reaction/data` directory.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir
# create a chemistry set based on the GRI mechanism
MyGasMech = ck.Chemistry(label="GRI 3.0")
# set mechanism input files
# including the full file path is recommended
```

(continues on next page)

(continued from previous page)

```
MyGasMech.chemfile = str((mechanism_dir / "grimech30_chem.inp").resolve())
MyGasMech.thermfile = str((mechanism_dir / "grimech30_thermo.dat").resolve())
```

### Preprocess the gasoline chemistry set

```
# preprocess the mechanism files
_ = MyGasMech.preprocess()
```

### Set up gas mixtures based on the species in this chemistry set

Create the `fuel` and `air` mixtures to initialize the external inlet streams. The fuel for this case is pure methane.

```
# fuel is pure methane
fuel = Stream(MyGasMech)
fuel.temperature = 300.0  # [K]
fuel.pressure = 2.1 * ck.P_ATM  # [atm] => [dyne/cm2]
fuel.x = [("CH4", 1.0)]
fuel.mass_flowrate = 3.275  # [g/sec]

# air is modeled as a mixture of oxygen and nitrogen
air = Stream(MyGasMech)
air.temperature = 550.0  # [K]
air.pressure = 2.1 * ck.P_ATM
air.x = ck.Air.x()  # use predefined "air" recipe in mole fractions
air.mass_flowrate = 45.0  # [g/sec]
```

### Create external inlet streams from the mixtures

Use the stream `adiabatic_mixing_streams()` method to combine the `fuel` and `air` streams. The final gas temperature should land between the temperatures of the two source streams. The mass flow rate of the `premixed` stream should be the sum of the sources. A simple PyChemkin composition recipe is used to create the `reburn_fuel` stream.

```
# premixed stream for the combustor
premixed = adiabatic_mixing_streams(fuel, air)
print(str(premixed.temperature))
print(str(premixed.mass_flowrate))

# additional fuel injection for the reburning zone
reburn_fuel = Stream(MyGasMech)
reburn_fuel.temperature = 300.0  # [K]
reburn_fuel.pressure = 2.1 * ck.P_ATM  # [atm] => [dyne/cm2]
reburn_fuel.x = [("CH4", 0.6), ("CO2", 0.4)]
reburn_fuel.mass_flowrate = 0.12  # [g/sec]

# find the species index
ch4_index = MyGasMech.get_specindex("CH4")
o2_index = MyGasMech.get_specindex("O2")
no_index = MyGasMech.get_specindex("NO")
co_index = MyGasMech.get_specindex("CO")
```

### Create PSRs for each zone

Set up the PSR for each zone one by one with external inlets only. PyChemkin requires that the first reactor/zone must have at least one external inlet. There are three reactors in the network. From upstream to downstream, they are `combustor`, `dilution zone`, and `reburning zone`. All of them have one external inlet. For the two downstream reactors, you must create the through-flow stream from their respective upstream reactor by using the solution of the upstream reactor. Use the `process_solution()` method to get the solution stream from the upstream reactor. Then, use the `set_inlet()` method to connect the resulting solution stream to the downstream reactor.

**Note**

- PyChemkin requires that the first reactor/zone must have at least one external inlet. The rest of the reactors have at least the through-flow from the immediate upstream reactor so they do not require an external inlet.
- The `Stream` parameter used to instantiate a PSR object is used to establish the *guessed reactor solution* and is modified when the network is solved by the `ERN`.
- The reactors in the network must be postprocessed individually.

```
# PSR #1: combustor
combustor = Psr(premixed, label="combustor")
# use the equilibrium state of the inlet gas mixture as the guessed solution
combustor.set_estimate_conditions(option="HP")
# set PSR residence time (sec): required for PSRSetResTimeEnergyConservation model
combustor.residence_time = 2.0 * 1.0e-3
# add external inlet
combustor.set_inlet(premixed)
# set the start wall time
start_time = time.time()
# run PSR #1
status = combustor.run()
if status != 0:
    print(Color.RED + combustor.label + " Run failed.")
    exit()
# postprocess the solution profiles
solnstream1 = combustor.process_solution()
solnstream1.label = "PSR1"
print("=" * 40)
print("Combustor exited.")
print("=" * 40)
print(f"Temperature = {solnstream1.temperature} [K].")
print(f"Mass flow rate = {solnstream1.mass_flowrate} [g/sec].")
print(f"CH4 = {solnstream1.x[ch4_index]}.")
print(f"O2 = {solnstream1.x[o2_index]}.")
print(f"CO = {solnstream1.x[co_index]}.")
print(f"NO = {solnstream1.x[no_index]}.")

# PSR #2: cooling
cooling = Psr(solnstream1, label="cooling zone")
# set PSR residence time (sec): required for PSRSetResTimeEnergyConservation model
cooling.residence_time = 1.5 * 1.0e-3
# add external inlet
air.mass_flowrate = 62.0  # [g/sec]
```

(continues on next page)

(continued from previous page)

```
cooling.set_inlet(air)
# add the through-flow from PSR #1
cooling.set_inlet(solnstream1)
# run PSR #2
status = cooling.run()
if status != 0:
    print(Color.RED + cooling.label + " Run failed.")
    exit()
# post-process the solution profiles
solnstream2 = cooling.process_solution()
solnstream2.label = "PSR2"
print()
print("=" * 40)
print("Dilution zone exited.")
print("=" * 40)
print(f"Temperature = {solnstream2.temperature} [K].")
print(f"Mass flow rate = {solnstream2.mass_flowrate} [g/sec].")
print(f"CH4 = {solnstream2.x[ch4_index]}.")
print(f"O2 = {solnstream2.x[o2_index]}.")
print(f"CO = {solnstream2.x[co_index]}.")
print(f"NO = {solnstream2.x[no_index]}.")

# PSR #3: reburn
reburn = Psr(solnstream2, label="reburn zone")
# set PSR residence time (sec): required for PSRSetResTimeEnergyConservation model
reburn.residence_time = 3.5 * 1.0e-3
# add external inlet
reburn.set_inlet(reburn_fuel)
# add the through flow from PSR #2
reburn.set_inlet(solnstream2)
# run PSR #3
status = reburn.run()
if status != 0:
    print(Color.RED + reburn.label + " Run failed.")
    exit()
# post-process the solution profiles
outflow = reburn.process_solution()
outflow.label = "outflow"
print()
print("=" * 40)
print("Outflow exited.")
print("=" * 40)
print(f"Temperature = {outflow.temperature} [K].")
print(f"Mass flow rate = {outflow.mass_flowrate} [g/sec].")
print(f"CH4 = {outflow.x[ch4_index]}.")
print(f"O2 = {outflow.x[o2_index]}.")
print(f"CO = {outflow.x[co_index]}.")
print(f"NO = {outflow.x[no_index]}.")

# compute the total runtime
runtime = time.time() - start_time
print()
```

(continues on next page)

(continued from previous page)

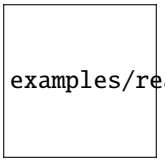
```
print(f"Total simulation duration: {runtime} [sec].")

# clean up
ck.done()
```

### 22.8.2 Use a chain reactor network to model a gas combustor

This example shows how to set up and solve a series of linked PSRs (perfectly-stirred reactors). This is the simplest reactor network as it does not contain any recycling streams or outflow splittings.

Here is a PSR chain model of a fictional gas combustor:




The primary inlet stream to the first reactor, the *combustor*, is the fuel-lean methane-air mixture that is formed by mixing the fuel (methane) and the heated air. The exhaust from the combustor enters the second reactor, the *dilution zone*, where the hot combustion products are cooled by the introduction of additional cool air. The cooled and diluted gas mixture in the dilution zone then travel to the third reactor, the *reburning zone*. A mixture of fuel (methane) and carbon dioxide is injected to the gas in the reburning zone, attempting to convert any remaining carbon monoxide or nitric oxide in the exhaust gas to carbon dioxide or nitrogen, respectively.

This example uses the `ReactorNetwork` module to configure and solve this chain reactor network. This module automatically handles the tasks of running the individual reactors and setting up the inlet to the downstream reactor.

```

```

#### Import PyChemkin packages and start the logger

```
from pathlib import Path
import time

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color
from ansys.chemkin.core.hybridreactornetwork import ReactorNetwork as Ern
from ansys.chemkin.core.inlet import (
    Stream,  # external gaseous inlet
    adiabatic_mixing_streams,
)
from ansys.chemkin.core.logger import logger

# Chemkin PSR model (steady-state)
from ansys.chemkin.core.stirreactors.PSR import PSRSetResTimeEnergyConservation as Psr

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
```

### Create a chemistry set

The mechanism to load is the GRI 3.0 mechanism for methane combustion. This mechanism and its associated data files come with the standard Ansys Chemkin installation in the `/reaction/data` directory.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir
# create a chemistry set based on the GRI mechanism
MyGasMech = ck.Chemistry(label="GRI 3.0")
# set mechanism input files
# including the full file path is recommended
MyGasMech.chemfile = str(mechanism_dir / "grimech30_chem.inp")
MyGasMech.thermfile = str(mechanism_dir / "grimech30_thermo.dat")
```

### Preprocess the gasoline chemistry set

```
# preprocess the mechanism files
_ = MyGasMech.preprocess()
```

### Set up gas mixtures based on the species in this chemistry set

Create the `fuel` and `air` streams before setting up the external inlet streams. The fuel in this case is pure methane. The main `premixed` inlet stream to the combustor is formed by mixing the `fuel` and `air` streams adiabatically. The fuel-to-air mass ratio is provided implicitly by the mass flow rates of the two streams. The external inlet to the second reactor, the `dilution zone`, is simply the `air` stream with a different mass flow rate (and different temperature if desirable). The `reburn_fuel` stream to inject to the downstream reburning zone is a mixture of methane and carbon dioxide.

**Note**

PyChemkin has `air` predefined as a convenient way to set up the air stream/mixture in simulations. Use the `ansys.chemkin.core.Air.x()` or `ansys.chemkin.core.Air.y()` method when the mechanism uses "O2" and "N2" for oxygen and nitrogen. Use the `ansys.chemkin.core.Air.x('L')` or `ansys.chemkin.core.Air.y('L')` method when the mechanism uses "o2" and "n2" for oxygen and nitrogen.

```
# fuel is pure methane
fuel = Stream(MyGasMech)
fuel.temperature = 300.0  # [K]
fuel.pressure = 2.1 * ck.P_ATM  # [atm] => [dyne/cm2]
fuel.x = [("CH4", 1.0)]
fuel.mass_flowrate = 3.275  # [g/sec]

# air is modeled as a mixture of oxygen and nitrogen
air = Stream(MyGasMech)
air.temperature = 550.0  # [K]
air.pressure = 2.1 * ck.P_ATM
# use predefined "air" recipe in mole fractions (with upper cased symbols)
air.x = ck.Air.x()
air.mass_flowrate = 45.0  # [g/sec]
```

### Create external inlet streams from the mixtures

Use the `adiabatic_mixing_streams()` method to combine the `fuel` and the `air` streams. The final gas temperature should land between the temperatures of the two source streams. The mass flow rate of the `premixed` stream should be the sum of the sources. Use a simple PyChemkin composition recipe to create the `reburn_fuel` stream.

```
# premixed stream for the combustor
premixed = adiabatic_mixing_streams(fuel, air)

# verify the premixed stream properties
print(f"Premixed stream temperature = {premixed.temperature} [K].")
print(f"Premixed stream mass flow rate = {premixed.mass_flowrate} [g/sec].")

# additional fuel injection for the reburning zone
reburn_fuel = Stream(MyGasMech)
reburn_fuel.temperature = 300.0  # [K]
reburn_fuel.pressure = 2.1 * ck.P_ATM  # [atm] => [dyne/cm2]
reburn_fuel.x = [("CH4", 0.6), ("CO2", 0.4)]
reburn_fuel.mass_flowrate = 0.12  # [g/sec]

# find the species index
ch4_index = MyGasMech.get_specindex("CH4")
o2_index = MyGasMech.get_specindex("O2")
no_index = MyGasMech.get_specindex("NO")
co_index = MyGasMech.get_specindex("CO")
```

### Create PSRs for each zone

Set up the PSR for each zone one by one with *external inlets only*. For PSR creation, use the `set_inlet()` method to add the external inlets to the reactor. A PFR always requires one external inlet when it is instantiated.

There are three reactors in the network. From upstream to downstream, they are `combustor`, `dilution zone`, and `reburning zone`. All of them have one external inlet.

**Note**

PyChemkin requires that the first reactor/zone must have at least one external inlet. Because the rest of the reactors have at least the through flow from the immediate upstream reactor, they do not require an external inlet.

**Note**

The `Stream` parameter used to instantiate a PSR is used to establish the *guessed reactor solution* and is modified when the network is solved by the `ERN`.

```
# PSR #1: combustor
combustor = Psr(premixed, label="combustor")
# use the equilibrium state of the inlet gas mixture as the guessed solution
combustor.set_estimate_conditions(option="HP")
# set PSR residence time (sec): required for PSRSetResTimeEnergyConservation model
combustor.residence_time = 2.0 * 1.0e-3
# add external inlet
```

(continues on next page)

(continued from previous page)

```
combustor.set_inlet(premixed)

# PSR #2: dilution zone
dilution = Psr(premixed, label="dilution zone")
# set PSR residence time (sec): required for PSRSetResTimeEnergyConservation model
dilution.residence_time = 1.5 * 1.0e-3
# add external inlet
# assign the correct mass flow rate to the "air" stream
air.mass_flowrate = 62.0  # [g/sec]
dilution.set_inlet(air)

# PSR #3: reburning zone
reburn = Psr(premixed, label="reburning zone")
# set PSR residence time (sec): required for PSRSetResTimeEnergyConservation model
reburn.residence_time = 3.5 * 1.0e-3
# add external inlet
reburn.set_inlet(reburn_fuel)
```

### Create the reactor network

Create a hybrid reactor network named `PSRChain` and use the `add_reactor()` method to add the reactors one by one from upstream to downstream. For a simple chain network such as the one used in this example, you do not need to define the connectivity among the reactors. The reactor network model automatically figures out the through-flow connections.

**Note**

- Use the `show_reactors()` method to get the list of reactors in the network in the order they are added.
- Use the `remove_reactor()` method to remove an existing reactor from the network by the reactor `name/label`. Similarly, use the `clear_connections()` method to undo the network connectivity.
- The order of the reactor addition is important as it dictates the solution sequence and thus the convergence rate.

```
# instantiate the chain PSR network as a hybrid reactor network
PSRChain = Ern(MyGasMech)

# add the reactors from upstream to downstream
PSRChain.add_reactor(combustor)
PSRChain.add_reactor(dilution)
PSRChain.add_reactor(reburn)

# list the reactors in the network
PSRChain.show_reactors()
```

### Solve the reactor network

Use the `run()` method to solve the entire reactor network. The hybrid reactor network solves the reactors one by one in the order that they are added to the network.

```
# set the start wall time
start_time = time.time()

# solve the reactor network
status = PSRChain.run()
if status != 0:
    print(Color.RED + "Failed to solve the reactor network." + Color.END)
    exit()

# compute the total runtime
runtime = time.time() - start_time
print()
print(f"Total simulation duration: {runtime} [sec]")
```

### Postprocess reactor network results

There are two ways to process results from a reactor network. You can extract the solution of an individual reactor member as a stream by using the `get_reactor_stream()` method to get the reactor by its name. Or, you can get the stream properties of a specific network outlet by using the `get_external_stream()` method to get the stream by its outlet index. Once you get the solution as a stream, you can use any stream or mixture method to further manipulate the solutions.

**Note**

Use the `number_external_outlets()` method to find out the number of external outlets of the reactor network.

```
# get the outlet stream from the reactor network solutions
# find the number of external outlet streams from the reactor network
print(f"Number of outlet streams = {PSRChain.number_external_outlets}.")

# get the first (and the only) external outlet stream properties
network_outflow = PSRChain.get_external_stream(1)
# set the stream label
network_outflow.label = "outflow"

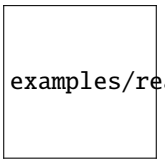
# print the desired outlet stream properties
print()
print("=" * 10)
print("outflow")
print("=" * 10)
print(f"temperature = {network_outflow.temperature} [K]")
print(f"mass flow rate = {network_outflow.mass_flowrate} [g/sec]")
print(f"CH4 = {network_outflow.x[ch4_index]}")
print(f"O2 = {network_outflow.x[o2_index]}")
print(f"CO = {network_outflow.x[co_index]}")
print(f"NO = {network_outflow.x[no_index]}")

# clean up
ck.done()
```

### 22.8.3 A simple reactor network with heat exchange

This example shows how to set up and solve a series of linked PSRs (perfectly-stirred reactors) with heat exchange.

Here is a PSR chain model of a fictional gas combustor with heat exchange:




The primary fuel inlet stream to the first reactor, the *pre-heater*, contains pure methane. The outlet flow from the *pre-heater* enters the second reactor, the *mixer*, where the hot fuel would mix with the cool air from the external inlet to form a combustible mathen-air mixture. The combustible mixture from the *mixer* then travel to the third reactor, the *combustor* in which the methane-air mixture would ignite and burn. The *combustor* is connected to the *pre-heater* by a heat exchanger so that a portion of the heat generated from the combustion in the *combustor* will be recycled to the *pre-heeater* to heat up the fuel stream.

This example uses the `ReactorNetwork` module to configure and solve this chain reactor network with heat exchange iteratively. This module automatically handles the tasks of running the individual reactors and setting up the inlet to the downstream reactor.

#### Import PyChemkin packages and start the logger

```
from pathlib import Path
import time

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color
from ansys.chemkin.core.hybridreactornetwork import ReactorNetwork as Ern
from ansys.chemkin.core.inlet import (
    Stream,  # external gaseous inlet
    adiabatic_mixing_streams,
)
from ansys.chemkin.core.logger import logger

# Chemkin PSR model (steady-state)
from ansys.chemkin.core.stirreactors.PSR import PSRSetResTimeEnergyConservation as Psr

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
```

#### Create a chemistry set

The mechanism to load is the GRI 3.0 mechanism for methane combustion. This mechanism and its associated data files come with the standard Ansys Chemkin installation in the `/reaction/data` directory.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir
# create a chemistry set based on the GRI mechanism
MyGasMech = ck.Chemistry(label="GRI 3.0")
# set mechanism input files
# including the full file path is recommended
MyGasMech.chemfile = str(mechanism_dir / "grimech30_chem.inp")
MyGasMech.thermfile = str(mechanism_dir / "grimech30_thermo.dat")
```

### Preprocess the gasoline chemistry set

```
# preprocess the mechanism files
_ = MyGasMech.preprocess()
```

### Set up gas mixtures based on the species in this chemistry set

Create the `fuel` and `air` streams before setting up the external inlet streams. The fuel in this case is pure methane. The main `premixed` inlet stream to the combustor is formed by mixing the `fuel` and `air` streams adiabatically. The fuel-to-air mass ratio is provided implicitly by the mass flow rates of the two streams. The external inlet to the second reactor, the `dilution zone`, is simply the `air` stream with a different mass flow rate (and different temperature if desirable). The `reburn_fuel` stream to inject to the downstream reburning zone is a mixture of methane and carbon dioxide.

**Note**

PyChemkin has `air` predefined as a convenient way to set up the air stream/mixture in simulations. Use the `ansys.chemkin.Air.x()` or `ansys.chemkin.Air.y()` method when the mechanism uses "O2" and "N2" for oxygen and nitrogen. Use the `ansys.chemkin.Air.x('L')` or `ansys.chemkin.Air.y('L')` method when the mechanism uses "o2" and "n2" for oxygen and nitrogen.

```
# fuel is pure methane
fuel = Stream(MyGasMech)
fuel.temperature = 300.0  # [K]
fuel.pressure = ck.P_ATM  # [atm] => [dyne/cm2]
fuel.x = [("CH4", 1.0)]
fuel.mass_flowrate = 3.275  # [g/sec]

# air is modeled as a mixture of oxygen and nitrogen
air = Stream(MyGasMech)
air.temperature = 300.0  # [K]
air.pressure = ck.P_ATM
# use predefined "air" recipe in mole fractions (with upper cased symbols)
air.x = ck.Air.x()
air.mass_flowrate = 66.75  # [g/sec]

# prepare a premixed stream as the guessed condition for the combustor
premixed = adiabatic_mixing_streams(fuel, air)

# find the species index
ch4_index = MyGasMech.get_specindex("CH4")
```

(continues on next page)

(continued from previous page)

```
o2_index = MyGasMech.get_specindex("O2")
no_index = MyGasMech.get_specindex("NO")
co_index = MyGasMech.get_specindex("CO")
```

### Create PSRs for each zone

Set up the PSR for each zone one by one with *external inlets only*. For PSR creation, use the `set_inlet()` method to add the external inlets to the reactor. A PFR always requires one external inlet when it is instantiated.

There are three reactors in the network. From upstream to downstream, they are `combustor`, `dilution zone`, and `reburning zone`. All of them have one external inlet.

**Note**

PyChemkin requires that the first reactor/zone must have at least one external inlet. Because the rest of the reactors have at least the through flow from the immediate upstream reactor, they do not require an external inlet.

**Note**

The `Stream` parameter used to instantiate a PSR is used to establish the *guessed reactor solution* and is modified when the network is solved by the `ERN`.

```
# PSR #1: pre-heater
preheater = Psr(air, label="pre-heater")
# set PSR residence time (sec): required for PSRSetResTimeEnergyConservation model
preheater.residence_time = 1.5 * 1.0e-3
# add external inlet
preheater.set_inlet(air)

# PSR #2: mixer
mixer = Psr(fuel, label="mixer")
# set PSR residence time (sec): required for PSRSetResTimeEnergyConservation model
mixer.residence_time = 0.5 * 1.0e-3
# add external inlet
mixer.set_inlet(fuel)

# PSR #3: combustor
# use the premixed methane-air mixture to set up the combustor
# premixed.temperature = 1800.0
combustor = Psr(premixed, label="combustor")
# use the equilibrium state of the premixed methane-air mixture as the guessed solution
combustor.set_estimate_conditions(option="HP")
# set PSR residence time (sec): required for PSRSetResTimeEnergyConservation model
combustor.residence_time = 4.0 * 1.0e-3
```

### Create the reactor network

Create a hybrid reactor network named `PSRChain` and use the `add_reactor()` method to add the reactors one by one from upstream to downstream. For a simple chain network such as the one used in this example, you do not need to define the connectivity among the reactors. The reactor network model automatically figures out the through-flow connections.

**Note**

- Use the `show_reactors()` method to get the list of reactors in the network in the order they are added.
- Use the `remove_reactor()` method to remove an existing reactor from the network by the reactor `name/label`. Similarly, use the `clear_connections()` method to undo the network connectivity.
- The order of the reactor addition is important as it dictates the solution sequence and thus the convergence rate.

```
# instantiate the chain PSR network as a hybrid reactor network
PSRChain = Ern(MyGasMech)

# add the reactors from upstream to downstream
PSRChain.add_reactor(preheater)
PSRChain.add_reactor(mixer)
PSRChain.add_reactor(combustor)

# list the reactors in the network
PSRChain.show_reactors()
```

### Add the heat transfer connection

Use `add_heat_exchange()` method to define a heat exchange connection between the *pre-heater* and the *combustor*.

```
# effecctive heat transfer coefficient of the heat exchanger [cal/cm2-K-sec]
ht_coeff = 0.0025
# effective surface area of the heat exchanger [cm2]
ht_area = 100.0
PSRChain.add_heat_exchange(preheater.label, combustor.label, ht_coeff, ht_area)

# reset the network relative tolerance
PSRChain.set_tear_tolerance(1.0e-7)
```

### Solve the reactor network

Use the `run()` method to solve the entire reactor network. The hybrid reactor network solves the reactors one by one in the order that they are added to the network.

```
# set the start wall time
start_time = time.time()

# solve the reactor network
status = PSRChain.run()
if status != 0:
    print(Color.RED + "Failed to solve the reactor network." + Color.END)
    exit()
```

(continues on next page)

(continued from previous page)

```
# compute the total runtime
runtime = time.time() - start_time
print()
print(f"Total simulation duration: {runtime} [sec]")
```

### Postprocess reactor network results

There are two ways to process results from a reactor network. You can extract the solution of an individual reactor member as a stream by using the `get_reactor_stream()` method to get the reactor by its name. Or, you can get the stream properties of a specific network outlet by using the `get_external_stream()` method to get the stream by its outlet index. Once you get the solution as a stream, you can use any stream or mixture method to further manipulate the solutions.

**Note**

Use the `number_external_outlets()` method to find out the number of external outlets of the reactor network.

```
# display the final temperatures in reactor #1 (pre-heater) and reactor #2 (mixer)
psr1_mixture = PSRChain.get_reactor_stream(preheater.label)
psr2_mixture = PSRChain.get_reactor_stream(mixer.label)
print(f"temperature in {preheater.label} = {psr1_mixture.temperature} [K].")
print(f"temperature in {mixer.label} = {psr2_mixture.temperature} [K].")

# get the outlet stream from the reactor network solutions
# find the number of external outlet streams from the reactor network
print(f"Number of outlet streams = {PSRChain.number_external_outlets}.")

# get the first (and the only) external outlet stream properties
network_outflow = PSRChain.get_external_stream(1)
# set the stream label
network_outflow.label = "outflow"

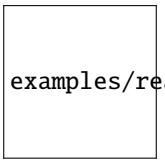
# print the desired outlet stream properties
print()
print("=" * 10)
print("outflow")
print("=" * 10)
print(f"temperature = {network_outflow.temperature} [K]")
print(f"mass flow rate = {network_outflow.mass_flowrate} [g/sec]")
print(f"CH4 = {network_outflow.x[ch4_index]}")
print(f"O2 = {network_outflow.x[o2_index]}")
print(f"CO = {network_outflow.x[co_index]}")
print(f"NO = {network_outflow.x[no_index]}")

# clean up
ck.done()
```

### 22.8.4 A PSR cluster with heat exchange

This example shows how to set up and solve a series of linked PSRs (perfectly-stirred reactors) with heat exchange.

Here is a PSR chain model of a fictional gas combustor with heat exchange:




The primary fuel inlet stream to the first reactor, the *pre-heater*, contains pure methane. The outlet flow from the *pre-heater* enters the second reactor, the *mixer*, where the hot fuel would mix with the cool air from the external inlet to form a combustible mathen-air mixture. The combustible mixture from the *mixer* then travel to the third reactor, the *combustor* in which the methane-air mixture would ignite and burn. The *combustor* is connected to the *pre-heater* by a heat exchanger so that a portion of the heat generated from the combustion in the *combustor* will be recycled to the *pre-heeater* to heat up the fuel stream.

This example uses the `ReactorNetwork` module to configure and solve this chain reactor network with heat exchange iteratively. This module automatically handles the tasks of running the individual reactors and setting up the inlet to the downstream reactor.

#### Import PyChemkin packages and start the logger

```
from pathlib import Path
import time

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color
from ansys.chemkin.core.inlet import (
    Stream,  # external gaseous inlet
    adiabatic_mixing_streams,
)
from ansys.chemkin.core.logger import logger

# Chemkin PSR model (steady-state)
from ansys.chemkin.core.stirreactors.PSR import (
    PSRSetResTimeEnergyConservation as Psr,
)
from ansys.chemkin.core.stirreactors.PSRcluster import PSRCluster as Ern

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
```

### Create a chemistry set

The mechanism to load is the GRI 3.0 mechanism for methane combustion. This mechanism and its associated data files come with the standard Ansys Chemkin installation in the `/reaction/data` directory.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir
# create a chemistry set based on the GRI mechanism
MyGasMech = ck.Chemistry(label="GRI 3.0")
# set mechanism input files
# including the full file path is recommended
MyGasMech.chemfile = str(mechanism_dir / "grimech30_chem.inp")
MyGasMech.thermfile = str(mechanism_dir / "grimech30_thermo.dat")
```

### Preprocess the gasoline chemistry set

```
# preprocess the mechanism files
_ = MyGasMech.preprocess()
```

### Set up gas mixtures based on the species in this chemistry set

Create the `fuel` and `air` streams before setting up the external inlet streams. The fuel in this case is pure methane. The main `premixed` inlet stream to the combustor is formed by mixing the `fuel` and `air` streams adiabatically. The fuel-to-air mass ratio is provided implicitly by the mass flow rates of the two streams. The external inlet to the second reactor, the `dilution zone`, is simply the `air` stream with a different mass flow rate (and different temperature if desirable). The `reburn_fuel` stream to inject to the downstream reburning zone is a mixture of methane and carbon dioxide.

**Note**

PyChemkin has `Air` predefined as a convenient way to set up the air stream/mixture in simulations. Use the `ansys.chemkin.Air.x()` or `ansys.chemkin.Air.y()` method when the mechanism uses "O2" and "N2" for oxygen and nitrogen. Use the `ansys.chemkin.Air.x('L')` or `ansys.chemkin.Air.y('L')` method when the mechanism uses "o2" and "n2" for oxygen and nitrogen.

```
# fuel is pure methane
fuel = Stream(MyGasMech)
fuel.temperature = 300.0  # [K]
fuel.pressure = ck.P_ATM  # [atm] => [dyne/cm2]
fuel.x = [("CH4", 1.0)]
fuel.mass_flowrate = 3.275  # [g/sec]

# air is modeled as a mixture of oxygen and nitrogen
air = Stream(MyGasMech)
air.temperature = 300.0  # [K]
air.pressure = ck.P_ATM
# use predefined "air" recipe in mole fractions (with upper cased symbols)
air.x = ck.Air.x()
air.mass_flowrate = 66.75  # [g/sec]

# prepare a premixed stream as the guessed condition for the combustor
```

(continues on next page)

(continued from previous page)

```
premixed = adiabatic_mixing_streams(fuel, air)

# find the species index
ch4_index = MyGasMech.get_specindex("CH4")
o2_index = MyGasMech.get_specindex("O2")
no_index = MyGasMech.get_specindex("NO")
co_index = MyGasMech.get_specindex("CO")
```

### Create PSRs for each zone

Set up the PSR for each zone one by one with *external inlets only*. For PSR creation, use the `set_inlet()` method to add the external inlets to the reactor. A PFR always requires one external inlet when it is instantiated.

There are three reactors in the network. From upstream to downstream, they are `combustor`, `dilution zone`, and `reburning zone`. All of them have one external inlet.

**Note**

PyChemkin requires that the first reactor/zone must have at least one external inlet. Because the rest of the reactors have at least the through flow from the immediate upstream reactor, they do not require an external inlet.

**Note**

The `Stream` parameter used to instantiate a PSR is used to establish the *guessed reactor solution* and is modified when the network is solved by the `ERN`.

```
# PSR #1: pre-heater
preheater = Psr(air, label="pre-heater")
# set PSR residence time (sec): required for PSR_SetResTime_EnergyConservation model
preheater.residence_time = 1.5 * 1.0e-3
# add external inlet
preheater.set_inlet(air)

# PSR #2: mixer
mixer = Psr(fuel, label="mixer")
# set PSR residence time (sec): required for PSR_SetResTime_EnergyConservation model
mixer.residence_time = 0.5 * 1.0e-3
# add external inlet
mixer.set_inlet(fuel)

# PSR #3: combustor
# use the premixed methane-air mixture to set up the combustor
combustor = Psr(premixed, label="combustor")
# use the equilibrium state of the premixed methane-air mixture as the guessed solution
combustor.set_estimate_conditions(option="HP")
# set PSR residence time (sec): required for PSR_SetResTime_EnergyConservation model
combustor.residence_time = 4.0 * 1.0e-3
```

### Create the reactor network

Create a coupled reactor network named `PSRcluster` with the list of the PSRs in the correct order. For a simple chain network, you do not need to define the connectivity among the reactors. The reactor network model automatically figures out the through-flow connections.

**Note**

- The order of the reactor in the list is important as it dictates the solution sequence and thus the convergence rate.

```
# instantiate the PSR network as a coupled reactor network
PSR_list = [preheater, mixer, combustor]
PSRcluster = Ern(PSR_list, label="combustor_cluster")
```

### Add the heat transfer connection

Use `add_heat_exchange()` method to define a heat exchange connection between the *pre-heater* and the *combustor*.

```
# effecctive heat transfer coefficient of the heat exchanger [cal/cm2-K-sec]
ht_coeff = 0.0025
# effective surface area of the heat exchanger [cm2]
ht_area = 100.0
PSRcluster.add_heat_exchange(preheater.label, combustor.label, ht_coeff, ht_area)
```

### Solve the reactor network

Use the `run()` method to solve the entire reactor network. The hybrid reactor network solves the reactors one by one in the order that they are added to the network.

```
# set the start wall time
start_time = time.time()

# solve the reactor network
status = PSRcluster.run()
if status != 0:
    print(Color.RED + "Failed to solve the reactor network." + Color.END)
    exit()

# compute the total runtime
runtime = time.time() - start_time
print()
print(f"Total simulation duration: {runtime} [sec]")
```

### Postprocess the reactor network solutions

Since the reactors in the same cluster are solved together, you must get use the `process_cluster_solution()` method first, and extract the solution stream of a specific PSR outlet by using the `get_reactor_stream()` method with the (1-based) reactor index. Once you get the solution as a stream, you can use any stream or mixture method to further manipulate the solutions.

```
# postprocess the solutions of all PSRs in the cluster
ierr = PSRcluster.process_cluster_solution()
```

(continues on next page)

(continued from previous page)

```
if ierr == 0:
    # verify the mass flow rate in and out of the PSR cluster
    print(
        f"net external inlet mass flow rate = "
        f"{PSRcluster.total_inlet_mass_flow_rate} [g/sec]."
    )
    print(
        f"net outlet mass flow rate = "
        f"{PSRcluster.get_cluster_outlet_flowrate()} [g/sec]."
    )
else:
    print("Failed to post-process the network solution.")
    print(f"error code = {ierr}")
    exit()

# display the reactor solutions
print("=" * 10)
print("reactor/zone")
print("=" * 10)
for index in range(PSRcluster.numb_psrs):
    id = index + 1
    name = PSRcluster.get_reactor_label(id)
    # get the solution stream of the reactor
    sstream = PSRcluster.get_reactor_stream(name)
    print(f"Reactor: {name}.")
    print(f"Temperature = {sstream.temperature} [K].")
    print(f"Mass flow rate = {sstream.mass_flowrate} [g/sec].")
    print(f"CH4 = {sstream.x[ch4_index]}.")
    print(f"O2 = {sstream.x[o2_index]}.")
    print(f"CO = {sstream.x[co_index]}.")
    print(f"NO = {sstream.x[no_index]}.")
    print("-" * 10)

psr1_mixture = PSRcluster.get_reactor_stream("pre-heater")
psr2_mixture = PSRcluster.get_reactor_stream("mixer")
print(f"pre-heater temperature = {psr1_mixture.temperature} [K]")
print(f"mixer temperature = {psr2_mixture.temperature} [K]")

# clean up
ck.done()
```

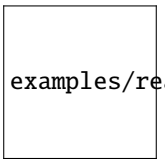
### 22.8.5 Simulate a combustor using an equivalent reactor network with stream recycling

This example shows how to set up and solve an ERN (equivalent reactor network) in PyChemkin.

An ERN is employed as a reduced-order model to simulate the steady-state combustion process inside a gas turbine combustor chamber. This reduced-order reactor network model retains the complexity of the combustion chemistry by sacrificing details of the combustor geometries, the spatial resolution, and the mass and energy transfer processes. An ERN usually comprises PSRs (perfectly-stirred reactors) and PFRs (plug-flow reactors). The network configuration and connectivity, the reactor parameters, and the mass flow rates can be determined from "hot" steady-state CFD simulation results and/or from observations and measured data of the actual/similar devices. Once a reactor network is calibrated

against the experimental data of a gas combustor, it becomes a handy tool for quickly estimating the emissions from the combustor when it is subjected to certain variations in the fuel compositions.

This figure shows a proposed ERN model of a fictional gas turbine combustor.




The *primary fuel* is mixed with the incoming air to form a *fuel-lean* mixture before entering the chamber through the primary inlet. Additional air, the *primary air*, is introduced to the combustion chamber separately through openings surrounding the primary inlet. The *secondary air* is entrained into the combustion chamber through well-placed holes on the liners at a location slightly downstream from the primary inlet.

The first PSR (reactor #1) represents the *mixing zone* around the main injector where the cool *premixed fuel-air* stream and the *primary air* stream are preheated by mixing with the hot combustion products from the *recirculation zone*.

Downstream from PSR #1, PSR #2, the *flame zone* is where the combustion of the heated fuel-air mixture takes place. The secondary air is injected here to cool down the combustion exhaust before it exits the combustion chamber. A portion of the exhaust gas coming out of PSR #2 does not leave the combustion chamber directly and is diverted to PSR #3, the *recirculation zone*.

The majority of the outlet flow from PSR #3 is recirculated back to the flame zone to sustain the fuel-lean premixed flame there. The rest of the hot gas from PSR #3 travels further back to PSR #1, the mixing zone, to preheat the fuel-lean mixture just entering the combustion chamber. Finally, the cooled flue gas leaves the chamber in a stream-like manner. Typically, a PFR is applied to simulation of the outflow.

The reactors in the network are solved individually one by one. When there is a *tear stream* in the network, the ERN should be solved iteratively. A tear stream is usually a *recycle" stream* that can serve as a pivot point for solving the recycle network iteratively. The convergence of the ERN is then determined by the absence of the per-iteration variation of the computed tear stream properties, such as species composition. In the current project, the recycling streams from PSR #3 to PSR #1 and PSR #2 are tear streams.

Import PyChemkin packages and start the logger ==========+======

```
from pathlib import Path
import time

import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color
from ansys.chemkin.core.hybridreactornetwork import ReactorNetwork as Ern
from ansys.chemkin.core.inlet import (
    Mixture,
    Stream,  # external gaseous inlet
)
from ansys.chemkin.core.logger import logger

# Chemkin PSR model (steady-state)
from ansys.chemkin.core.stirreactors.PSR import PSRSetResTimeEnergyConservation as Psr
```

(continues on next page)

(continued from previous page)

```
# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create a chemistry set

The mechanism to load is the GRI 3.0 mechanism for methane combustion. This mechanism and its associated data files come with the standard Ansys Chemkin installation in the `/reaction/data` directory.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir
# create a chemistry set based on the GRI mechanism
MyGasMech = ck.Chemistry(label="GRI 3.0")
# set mechanism input files
# including the full file path is recommended
MyGasMech.chemfile = str(mechanism_dir / "grimech30_chem.inp")
MyGasMech.thermfile = str(mechanism_dir / "grimech30_thermo.dat")
```

### Preprocess the gasoline chemistry set

```
# preprocess the mechanism files
_ = MyGasMech.preprocess()
```

### Set up gas mixtures based on the species in this chemistry set

Create the “fuel” and the “air” mixtures to initialize the external inlet streams. The fuel for this case is pure methane.

```
# fuel is pure methane
fuel = Mixture(MyGasMech)
fuel.temperature = 650.0  # [K]
fuel.pressure = 10.0 * ck.P_ATM  # [atm] => [dyne/cm2]
fuel.x = [("CH4", 1.0)]

# air is modeled as a mixture of oxygen and nitrogen
air = Mixture(MyGasMech)
air.temperature = 650.0  # [K]
air.pressure = 10.0 * ck.P_ATM
air.x = ck.Air.x()  # mole fractions
```

### Create external inlet streams from the mixtures

Create the `fuel` and the `air` streams/mixtures before setting up the external inlet streams. The fuel in this case is pure methane. The `x_by_equivalence_ratio()` method is used to form the main `premixed` inlet stream to the *mixing zone* reactor. The external inlets to the first reactor, the *mixing zone*, and the second reactor, the *flame zone*, are simply `air` mixtures with different mass flow rates and temperatures.

**Note**

PyChemkin has *air* redefined as a convenient way to set up the air stream/mixture in the simulations. Use the `ansys.chemkin.core.Air.x()` or `ansys.chemkin.core.Air.y()` method when the mechanism uses `O2` and `N2` for oxygen and nitrogen. Use the `ansys.chemkin.core.Air.x('L')` or `ansys.chemkin.core.Air.y('L')` method when oxygen and nitrogen are represented by `o2` and `n2`.

```
# primary fuel-air mixture
# products from the complete combustion of the fuel mixture and air
products = ["CO2", "H2O", "N2"]
# species mole fractions of added/inert mixture.
# (You can also create an additives mixture here.)
add_frac = np.zeros(MyGasMech.kk, dtype=np.double)  # no additives: all zeros

# create the unburned fuel-air mixture
premixed = Stream(MyGasMech)
# mean equivalence ratio
equiv = 0.6
ierror = premixed.x_by_equivalence_ratio(
    MyGasMech, fuel.x, air.x, add_frac, products, equivalenceratio=equiv
)
# check fuel-oxidizer mixture creation status
if ierror != 0:
    print("Error: Failed to create the fuel-oxidizer mixture.")
    exit()

# list the composition of the unburned fuel-air mixture
premixed.list_composition(mode="mole")

# set premixed fuel-air inlet temperature and mass flow rate
premixed.temperature = fuel.temperature
premixed.pressure = fuel.pressure
premixed.mass_flowrate = 500.0  # [g/sec]

# primary air stream to mix with the primary fuel-air stream
primary_air = Stream(MyGasMech, label="Primary_Air")
primary_air.x = air.x
primary_air.pressure = air.pressure
primary_air.temperature = air.temperature
primary_air.mass_flowrate = 50.0  # [g/sec]

# secondary bypass air stream
secondary_air = Stream(MyGasMech, label="Secondary_Air")
secondary_air.x = air.x
secondary_air.pressure = air.pressure
```

(continues on next page)

(continued from previous page)

```
secondary_air.temperature = 670.0  # [K]
secondary_air.mass_flowrate = 100.0  # [g/sec]

# find the species index
ch4_index = MyGasMech.get_specindex("CH4")
o2_index = MyGasMech.get_specindex("O2")
no_index = MyGasMech.get_specindex("NO")
co_index = MyGasMech.get_specindex("CO")
```

### Define reactors in the reactor network

Set up the PSR for each zone one by one with *external inlets only*. For PSRs, use the `set_inlet()` method to add the external inlets to the reactor. A PFR always requires one external inlet when it is instantiated. From upstream to downstream, they are `premix zone`, `flame zone`, and `recirculation zone`. The `recirculation zone` does not have an external inlet.

**Note**

PyChemkin requires that the first reactor/zone must have at least one external inlet. Because the rest of the reactors have at least the through-flow from the immediate upstream reactor, they do not require an external inlet.

**Note**

The stream parameter used to instantiate a PSR is used to establish the *guessed reactor solution* and is modified when the network is solved by the ERN.

```
# PSR #1: mixing zone
mix = Psr(premixed, label="mixing zone")
# use different guess temperature
mix.set_estimate_conditions(option="TP", guess_temp=800.0)
# set PSR residence time (sec): required for PSRSetResTimeEnergyConservation model
mix.residence_time = 0.5 * 1.0e-3
# add external inlets
mix.set_inlet(premixed)
mix.set_inlet(primary_air)


# PSR #2: flame zone
flame = Psr(premixed, label="flame zone")
# use the equilibrium state of the inlet gas mixture as the guessed solution
flame.set_estimate_conditions(option="TP", guess_temp=1600.0)
# set PSR residence time (sec): required for PSRSetResTimeEnergyConservation model
flame.residence_time = 1.5 * 1.0e-3
# add external inlet
flame.set_inlet(secondary_air)

# PSR #3: recirculation zone
recirculation = Psr(premixed, label="recirculation zone")
# use the equilibrium state of the inlet gas mixture as the guessed solution
```

(continues on next page)

(continued from previous page)

```
recirculation.set_estimate_conditions(option="TP", guess_temp=1600.0)
# set PSR residence time (sec): required for PSRSetResTimeEnergyConservation model
recirculation.residence_time = 1.5 * 1.0e-3
```

### Create the reactor network

Create a hybrid reactor network named `PSRnetwork` and add the reactors one by one from upstream to downstream using the `add_reactor()` method. For a simple chain network you do not need to define the connectivity among the reactors. The reactor network model automatically figures out the through-flow connections.

**Note**

- Use the `show_reactors()` method to get the list of reactors in the network in the order that they are added.
- Use the `remove_reactor()` method to remove an existing reactor from the network by the reactor `name/label`. Similarly, use the `clear_connections()` method to undo the network connectivity.
- The order of the reactor addition is important as it dictates the solution sequence and thus the convergence rate.

```
# instantiate the PSR network as a hybrid reactor network
PSRnetwork = Ern(MyGasMech)

# add the reactors from upstream to downstream
PSRnetwork.add_reactor(mix)
PSRnetwork.add_reactor(flame)
PSRnetwork.add_reactor(recirculation)

# list the reactors in the network
PSRnetwork.show_reactors()
```

### Define the PSR connectivity

Because the current reactor network contains recycling streams, for example, from PSR #3 to PSR #1 and PSR #2, you must explicitly specify the network connectivity. To do this, you use a *split* dictionary to define the outflow connections among the PSRs in the network. The *key* of the split dictionary is the label of the *originate PSR*. The *value* is a list of tuples consisting of the label of the *target PSR* and its mass flow rate fraction.

```
{ "originate PSR" : [("target PSR1", fraction), ("target PSR2", fraction)],
  "another originate PSR" : [("target PSR1", fraction), ... ], ... }
```

For example, the outflow from reactor `PSR1``is split into two streams. 90% of the mass flow rate goes to ``PSR2` and the rest is diverted to `PSR5`. The split dictionary entry for the outflow split of `PSR1` is as follows:

```
"PSR1" : [("PSR2", 0.9), ("PSR5", 0.1)]
```

**Note**

The outlet flow from a reactor that is leaving the reactor network must be labeled as `EXIT>>` when you define the outflow splitting.

```
# PSR #1 outlet flow splitting
split_table = [(flame.label, 1.0)]
PSRnetwork.add_outflow_connections(mix.label, split_table)
# PSR #2 outlet flow splitting
# part of the outlet flow from PSR #2 exits the reactor network
split_table = [(recirculation.label, 0.2), ("EXIT>>", 0.8)]
PSRnetwork.add_outflow_connections(flame.label, split_table)
# PSR #3 outlet flow splitting
# PSR #3 is the last reactor of the network. However,
# it does not have an external outlet.
split_table = [(mix.label, 0.15), (flame.label, 0.85)]
PSRnetwork.add_outflow_connections(recirculation.label, split_table)
```

### Define the tear point for the iteration

Because the reactor network contains recycling streams, it must be solved iteratively by applying the *tear stream* method. You use the `add_tearingpoint() `` method to explicitly define the *tear points* of the network. In this example, the stream recycling occurs at reactor #3, where the outlet flow is sent back to the upstream reactors, reactor #1 and reactor #2. Therefore, reactor #3, the *recirculation zone*, should be set as the only tear point of this reactor network. You use the ``set_tear_tolerance()` method to set the relative tolerance for the overall reactor network convergence. The default tolerance is 1.0e-6.

```
# define the tear point reactor as reactor #3
PSRnetwork.add_tearingpoint(recirculation.label)

# reset the network relative tolerance
PSRnetwork.set_tear_tolerance(1.0e-5)
```

### Solve the reactor network

Use the `run()` method to solve the entire reactor network. The `ReactorNetwork` hybrid solves the reactors one by one in the order that they are added to the network.

**Note**

Use the `set_tear_iteration_limit(count)` method to change the limit on the number of tear loop iterations that can be taken before declaring failure. The default limit is 200.

**Note**

The `set_relaxation_factor(factor)` method can be employed to make solution updating at the end of each iteration to be more aggressive (factor > 1.0) or more conservative (factor < 1.0). By default, the relaxation factor is set to 1.

```
# set the start wall time
start_time = time.time()
# solve the reactor network iteratively
status = PSRnetwork.run()
if status != 0:
```

(continues on next page)

(continued from previous page)

```
    print(Color.RED + "Failed to solve the reactor network." + Color.END)
    exit()

# compute the total runtime
runtime = time.time() - start_time
print()
print(f"Total simulation duration: {runtime} [sec].")
print()
```

### Postprocess the reactor network solutions

There are two ways to process the results from a reactor network. You can extract the solution of an individual reactor member as a stream by using the `get_reactor_stream()` method with the reactor name. Or, you can get the stream properties of a specific network outlet by using the `get_external_stream()` method with the outlet index. Once you get the solution as a stream, you can use any stream or mixture method to further manipulate the solutions.

**Note**

Use the `number_external_outlets()` method to find out the number of external exists of the reactor network.

```
# get the outlet stream from the reactor network solutions
# find the number of external outlet streams from the reactor network
print(f"number of outlet streams = {PSRnetwork.number_external_outlets}.")

# display the external outlet stream properties
for m in range(PSRnetwork.number_external_outlets):
    n = m + 1
    network_outflow = PSRnetwork.get_external_stream(n)
    # set the stream label
    network_outflow.label = "outflow"

    # print the desired outlet stream properties
    print("=" * 10)
    print(f"outflow # {n}")
    print("=" * 10)
    print(f"Temperature = {network_outflow.temperature} [K].")
    print(f"Mass flow rate = {network_outflow.mass_flowrate} [g/sec].")
    print(f"CH4 = {network_outflow.x[ch4_index]}.")
    print(f"O2 = {network_outflow.x[o2_index]}.")
    print(f"CO = {network_outflow.x[co_index]}.")
    print(f"NO = {network_outflow.x[no_index]}.")
    print("-" * 10)

# display the reactor solutions
print()
print("=" * 10)
print("reactor/zone")
print("=" * 10)
for index, stream in PSRnetwork.reactor_solutions.items():
    name = PSRnetwork.get_reactor_label(index)
    print(f"Reactor: {name}.")
```

(continues on next page)

(continued from previous page)

```
    print(f"Temperature = {stream.temperature} [K].")
    print(f"Mass flow rate = {stream.mass_flowrate} [g/sec].")
    print(f"CH4 = {stream.x[ch4_index]}.")
    print(f"O2 = {stream.x[o2_index]}.")
    print(f"CO = {stream.x[co_index]}.")
    print(f"NO = {stream.x[no_index]}.")
    print("-" * 10)

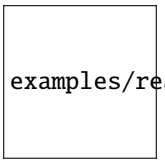
# clean up
ck.done()
```

### 22.8.6 Simulate a combustor using a coupled equivalent reactor network with stream recycling

This example shows how to set up and solve a "coupled" ERN (equivalent reactor network) in PyChemkin.

An ERN is employed as a reduced-order model to simulate the steady-state combustion process inside a gas turbine combustor chamber. This reduced-order reactor network model retains the complexity of the combustion chemistry by sacrificing details of the combustor geometries, the spatial resolution, and the mass and energy transfer processes. An ERN usually comprises PSRs (perfectly-stirred reactors) and PFRs (plug-flow reactors). The network configuration and connectivity, the reactor parameters, and the mass flow rates can be determined from "hot" steady-state CFD simulation results and/or from observations and measured data of the actual/similar devices. Once a reactor network is calibrated against the experimental data of a gas combustor, it becomes a handy tool for quickly estimating the emissions from the combustor when it is subjected to certain variations in the fuel compositions.

This figure shows a proposed ERN model of a fictional gas turbine combustor.




The *primary fuel* is mixed with the incoming air to form a *fuel-lean* mixture before entering the chamber through the primary inlet. Additional air, the *primary air*, is introduced to the combustion chamber separately through openings surrounding the primary inlet. The *secondary air* is entrained into the combustion chamber through well-placed holes on the liners at a location slightly downstream from the primary inlet.

The first PSR (reactor #1) represents the *mixing zone* around the main injector where the cool *premixed fuel-air* stream and the *primary air* stream are preheated by mixing with the hot combustion products from the *recirculation zone*.

The configurations of a "coupled" ERN (the same as the reactor network when you use the Chemkin GUI) has one major difference in requirements. The "hybrid" ERN in PyChemkin allows all PSRs to have an external outlet. However, for the "coupled" reactor network in PyChemkin and in the Chemkin GUI, ONLY the "last" PSR of the network can have an external outlet. Since the "recirculation zone* does not have any downstream outflow (through flow), it cannot be the "last" reactor of a "coupled" ERN. Consequently, even both the "PSRnetwork" and the current "PSRnetwork_coupled" represent exactly the same ERN, their configurations are different. In the current "coupled" ERN configuration, the *recirculation zone* becomes PSR #2 and the *flame zone* becomes the last reactor of the ENR, PSR #3.

Downstream from PSR #1, PSR #3, the *flame zone* is where the combustion of the heated fuel-air mixture takes place. The secondary air is injected here to cool down the combustion exhaust before it exits the combustion chamber. A portion of the exhaust gas coming out of PSR #3 does not leave the combustion chamber directly and is diverted to PSR #2, the *recirculation zone*. The external outlet of the ERN must be the direct downstream from this reactor.

The majority of the outlet flow from PSR #2 is recirculated back to the flame zone to sustain the fuel-lean premixed flame there. The rest of the hot gas from PSR #2 travels further back to PSR #1, the mixing zone, to preheat the fuel-lean mixture just entering the combustion chamber. Finally, the cooled flue gas leaves the chamber in a stream-like manner. Typically, a PFR is applied to simulation of the outflow.

The reactors in the "coupled" network are solved simultaneously. Therefore, there is no need to define any *tear stream*.

```
```

Import PyChemkin packages and start the logger ==========+======

```
from pathlib import Path
import time

import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color
from ansys.chemkin.core.inlet import (
    Mixture,
    Stream,  # external gaseous inlet
)
from ansys.chemkin.core.logger import logger

# Chemkin PSR model (steady-state)
from ansys.chemkin.core.stirreactors.PSR import PSRSetResTimeEnergyConservation as Psr
from ansys.chemkin.core.stirreactors.PSRcluster import PSRCluster as Ern

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create a chemistry set

The mechanism to load is the GRI 3.0 mechanism for methane combustion. This mechanism and its associated data files come with the standard Ansys Chemkin installation in the `/reaction/data` directory.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir
# create a chemistry set based on the GRI mechanism
MyGasMech = ck.Chemistry(label="GRI 3.0")
# set mechanism input files
# including the full file path is recommended
MyGasMech.chemfile = str(mechanism_dir / "grimech30_chem.inp")
MyGasMech.thermfile = str(mechanism_dir / "grimech30_thermo.dat")
```

### Preprocess the gasoline chemistry set

```
# preprocess the mechanism files
_ = MyGasMech.preprocess()
```

### Set up gas mixtures based on the species in this chemistry set

Create the "fuel" and the "air" mixtures to initialize the external inlet streams. The fuel for this case is pure methane.

```
# fuel is pure methane
fuel = Mixture(MyGasMech)
fuel.temperature = 650.0  # [K]
fuel.pressure = 10.0 * ck.P_ATM  # [atm] => [dyne/cm2]
fuel.x = [("CH4", 1.0)]

# air is modeled as a mixture of oxygen and nitrogen
air = Mixture(MyGasMech)
air.temperature = 650.0  # [K]
air.pressure = 10.0 * ck.P_ATM
air.x = ck.Air.x()  # mole fractions
```

### Create external inlet streams from the mixtures

Create the `fuel` and the `air` streams/mixtures before setting up the external inlet streams. The fuel in this case is pure methane. The `X_by_Equivalence_Ratio()` method is used to form the main `premixed` inlet stream to the *mixing zone* reactor. The external inlets to the first reactor, the *mixing zone*, and the second reactor, the *flame zone*, are simply `air` mixtures with different mass flow rates and temperatures.

**Note**

PyChemkin has *air* redefined as a convenient way to set up the air stream/mixture in the simulations. Use the `ansys.chemkin.Air.x()` or `ansys.chemkin.Air.y()` method when the mechanism uses `O2` and `N2` for oxygen and nitrogen. Use the `ansys.chemkin.Air.x('L')` or `ansys.chemkin.Air.y('L')` method when oxygen and nitrogen are represented by `o2` and `n2`.

```
# primary fuel-air mixture
# products from the complete combustion of the fuel mixture and air
products = ["CO2", "H2O", "N2"]
# species mole fractions of added/inert mixture.
# (You can also create an additives mixture here.)
add_frac = np.zeros(MyGasMech.kk, dtype=np.double)  # no additives: all zeros

# create the unburned fuel-air mixture
premixed = Stream(MyGasMech)
# mean equivalence ratio
equiv = 0.6
ierror = premixed.x_by_equivalence_ratio(
    MyGasMech, fuel.x, air.x, add_frac, products, equivalenceratio=equiv
)
# check fuel-oxidizer mixture creation status
if ierror != 0:
    print("Error: Failed to create the fuel-oxidizer mixture.")
```

(continues on next page)

(continued from previous page)

```
    exit()

# list the composition of the unburned fuel-air mixture
premixed.list_composition(mode="mole")

# set premixed fuel-air inlet temperature and mass flow rate
premixed.temperature = fuel.temperature
premixed.pressure = fuel.pressure
premixed.mass_flowrate = 500.0  # [g/sec]

# primary air stream to mix with the primary fuel-air stream
primary_air = Stream(MyGasMech, label="Primary_Air")
primary_air.x = air.x
primary_air.pressure = air.pressure
primary_air.temperature = air.temperature
primary_air.mass_flowrate = 50.0  # [g/sec]

# secondary bypass air stream
secondary_air = Stream(MyGasMech, label="Secondary_Air")
secondary_air.x = air.x
secondary_air.pressure = air.pressure
secondary_air.temperature = 670.0  # [K]
secondary_air.mass_flowrate = 100.0  # [g/sec]

# find the species index
ch4_index = MyGasMech.get_specindex("CH4")
o2_index = MyGasMech.get_specindex("O2")
no_index = MyGasMech.get_specindex("NO")
co_index = MyGasMech.get_specindex("CO")
```

### Define reactors in the reactor network

Set up the PSR for each zone one by one with *external inlets only*. For PSRs, use the `set_inlet()` method to add the external inlets to the reactor. A PFR always requires one external inlet when it is instantiated. From upstream to downstream, they are `premix zone`, `recirculation zone`, and `flame zone`. The `recirculation zone` does not have an external inlet.

**Note**

PyChemkin requires that the first reactor/zone must have at least one external inlet. Because the rest of the reactors have at least the through-flow from the immediate upstream reactor, they do not require an external inlet.

**Note**

The stream parameter used to instantiate a PSR is used to establish the *guessed reactor solution* and is modified when the network is solved by the ERN.

```
# PSR #1: mixing zone
mix = Psr(premixed, label="mixing zone")
```

(continues on next page)

(continued from previous page)

```
# use different guess temperature
mix.set_estimate_conditions(option="TP", guess_temp=800.0)
# set PSR residence time (sec): required for PSRSetResTimeEnergyConservation model
mix.residence_time = 0.5 * 1.0e-3
# add external inlets
mix.set_inlet(premixed)
mix.set_inlet(primary_air)

# PSR #2: recirculation zone
recirculation = Psr(premixed, label="recirculation zone")
# use the equilibrium state of the inlet gas mixture as the guessed solution
recirculation.set_estimate_conditions(option="TP", guess_temp=1600.0)
# set PSR residence time (sec): required for PSRSetResTimeEnergyConservation model
recirculation.residence_time = 1.5 * 1.0e-3

# PSR #3: flame zone
flame = Psr(premixed, label="flame zone")
# use the equilibrium state of the inlet gas mixture as the guessed solution
flame.set_estimate_conditions(option="TP", guess_temp=1600.0)
# set PSR residence time (sec): required for PSRSetResTimeEnergyConservation model
flame.residence_time = 1.5 * 1.0e-3
# add external inlet
flame.set_inlet(secondary_air)
```

### Create the reactor network

Create a coupled reactor network named `PSRcluster` with the list of the PSRs in the correct order. For a simple chain network, you do not need to define the connectivity among the reactors. The reactor network model automatically figures out the through-flow connections.

**Note**

- The order of the reactor in the list is important as it dictates the solution sequence and thus the convergence rate.

```
# instantiate the PSR network as a coupled reactor network
PSR_list = [mix, recirculation, flame]
PSRcluster = Ern(PSR_list, label="combustor_cluster")
```

### Define the PSR connectivity

Because the current reactor network contains recycling streams, for example, from PSR #3 to PSR #1 and PSR #2, you must explicitly specify the network connectivity. To do this, you use a *split* dictionary to define the outflow connections among the PSRs in the network. The *key* of the split dictionary is the label of the *originate PSR*. The *value* is a list of tuples consisting of the label of the *target PSR* and its mass flow rate fraction.

```
{ "originate PSR" : [("target PSR1", fraction), ("target PSR2", fraction)],
  "another originate PSR" : [("target PSR1", fraction), ... ], ... }
```

For example, the outflow from reactor `PSR1``is split into two streams. 90% of the mass flow rate goes to ``PSR2` and the rest is diverted to `PSR5`. The split dictionary entry for the outflow split of `PSR1` is as follows:

```
"PSR1" : [("PSR2", 0.9), ("PSR5", 0.1)]
```

**Note**

The outlet flow from a reactor that is leaving the reactor network must be labeled as `EXIT>>` when you define the outflow splitting.

```
# PSR #1 outlet flow splitting
split_table = [(flame.label, 1.0)]
PSRcluster.set_recycling_stream(mix.label, split_table)
# PSR #2 outlet flow splitting
# PSR #2 does not have an external outlet.
split_table = [(mix.label, 0.15), (flame.label, 0.85)]
PSRcluster.set_recycling_stream(recirculation.label, split_table)
# PSR #3 outlet flow splitting
# part of the outlet flow from PSR #3 exits the reactor network
split_table = [(recirculation.label, 0.2)]
PSRcluster.set_recycling_stream(flame.label, split_table)
```

### Solve the reactor network

Use the `run()` method to solve the entire reactor network. The `PSRCluster` solves the PSRs simultaneously.

```
# set the start wall time
start_time = time.time()
# solve the reactor network iteratively
status = PSRcluster.run()
if status != 0:
    print(Color.RED + "Failed to solve the reactor network." + Color.END)
    exit()

# compute the total runtime
runtime = time.time() - start_time
print()
print(f"Total simulation duration: {runtime} [sec].")
print()
```

### Postprocess the reactor network solutions

Since the reactors in the same cluster are solved together, you must get use the `process_cluster_solution()` method first, and extract the solution stream of a specific PSR outlet by using the `get_reactor_stream()` method with the (1-based) reactor index. Once you get the solution as a stream, you can use any stream or mixture method to further manipulate the solutions.

```
# postprocess the solutions of all PSRs in the cluster
_ = PSRcluster.process_cluster_solution()

# verify the mass flow rate in and out of the PSR cluster
print(
    f"net external inlet mass flow rate = "
    f"{PSRcluster.total_inlet_mass_flow_rate} [g/sec]."
```

(continues on next page)

(continued from previous page)

```
)
print(
    f"net outlet mass flow rate = {PSRcluster.get_cluster_outlet_flowrate()} [g/sec]."
)

# display the reactor solutions
print("=" * 10)
print("reactor/zone")
print("=" * 10)
for index in range(PSRcluster.numb_psrs):
    id = index + 1
    name = PSRcluster.get_reactor_label(id)
    # get the solution stream of the reactor
    sstream = PSRcluster.get_reactor_stream(name)
    print(f"Reactor: {name}.")
    print(f"Temperature = {sstream.temperature} [K].")
    print(f"Mass flow rate = {sstream.mass_flowrate} [g/sec].")
    print(f"CH4 = {sstream.x[ch4_index]}.")
    print(f"O2 = {sstream.x[o2_index]}.")
    print(f"CO = {sstream.x[co_index]}.")
    print(f"NO = {sstream.x[no_index]}.")
    print("-" * 10)

# clean up
ck.done()
```

## 22.9 Premixed flame models

The *Premixed Flame* model is a 1-D steady-state application, and it contains two sub-models: *"flame speed calculator"* (or the freely propagating flame model) and *"flat flame"* model. The *flame speed calculator* is commonly used to calculate the laminar flame speed of a premixed fuel-oxidizer mixture. The predicted/computed flame speeds can be used to validate a combustion mechanism or can serve as a prre-processor to construct a "flame speed table" or a "combustion progress table" for other applications. The *flat flame* burner-stablized model is primarily used to study the flame chemistry in conjunction with the flat flame experiments.

The examples show different ways to perform the "premixed flame" calculations.

### 22.9.1 Compute the laminar flame speed of diluted hydrogen at low pressure

The *freely propagating premixed flame* model can be utilized to obtain the laminar *flame speed* of a gas mixture at given initial temperature and pressure. The premixed flame model will calculate the temperature and species composition profiles across the flame, and the laminar flame speed will be derived from the mass flow rate of the gas mixture, a constant across the entire calculation domain because of the mass conservation.

This tutorial demonstrates the application of the "freely propagating" premixed flame model to calculate the laminar flame speed of an $N_2$ diluted hydrogen-air mixture at low pressure. Since the transport processes are critical for flame calculations, the *transport data* must be included in the mechanism data and pre-processed.

### Import PyChemkin packages and start the logger

```
from pathlib import Path
import time

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color
from ansys.chemkin.core.inlet import Stream  # external gaseous inlet
from ansys.chemkin.core.logger import logger

# Chemkin 1-D premixed freely propagating flame model (steady-state)
from ansys.chemkin.core.premixedflames.premixedflame import (
    FreelyPropagating as FlameSpeed,
)

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create a chemistry set

The 'C2 NOx' mechanism is from the default *"/reaction/data"* directory. This mechanism also includes information about the *Soave* cubic Equation of State (EOS) for the real-gas applications. PyChemkin preprocessor will indicate the availability of the real-gas model in the `Chemistry Set` processed.

**Note**

The transport data *must* be included and pre-processed because the transport processes, *convection and diffusion*, are important to sustain the flame structure.

**Note**

Use the `preprocess_transportdata` method if the transport data is embedded in the gas-phase mechanism file.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir
# create a chemistry set based on the C2 NOx mechanism
MyGasMech = ck.Chemistry(label="C2 NOx")
# set mechanism input files
```

(continues on next page)

(continued from previous page)

```
# including the full file path is recommended
MyGasMech.chemfile = str(mechanism_dir / "C2_NOx_SRK.inp")

# direct the preprocessor to include the transport properties
# only when the mechanism file contains all the transport data
MyGasMech.preprocess_transportdata()
```

### Pre-process the `Chemistry Set`

```
# preprocess the mechanism files
ierror = MyGasMech.preprocess()
if ierror != 0:
    print("Error: Failed to preprocess the mechanism!")
    print(f"       Error code = {ierror}")
    exit()
```

### Set up the ($H_2$ + $N_2$)-air mixture for the flame speed calculation

Instantiate an `Stream` object `premixed` for the inlet gas mixture. The `Stream` object is a `Mixture` object with the addition of the *inlet flow rate*. You can specify the inlet gas properties the same way you set up a `Mixture`. Here the `x_by_equivalence_ratio` method is used. You create the `fuel` and the `air` mixtures first. Then define the *complete combustion product species* and provide the *additives* composition if applicable. And finally you can simply set `equivalenceratio=1` to create the stoichiometric hydrogen-air mixture. The estimated inlet mass flow rate can be assigned by the `mass_flowrate()` method.

```
# create the fuel mixture
fuel = ck.Mixture(MyGasMech)
# set fuel composition: hydrogen diluted by nitrogen
fuel.x = [("H2", 0.7), ("N2", 0.3)]
# setting pressure and temperature is not required in this case
fuel.pressure = 0.0125 * ck.P_ATM
fuel.temperature = 300.0  # inlet temperature

# create the oxidizer mixture: air
air = ck.Mixture(MyGasMech)
air.x = ck.Air.x()
# setting pressure and temperature is not required in this case
air.pressure = fuel.pressure
air.temperature = fuel.temperature

# create the fuel-air Stream for the premixed flame speed calculation
premixed = Stream(MyGasMech, label="premixed")
# products from the complete combustion of the fuel mixture and air
products = ["H2O", "N2"]
# species mole fractions of added/inert mixture.
# can also create an additives mixture here
add_frac = np.zeros(MyGasMech.kk, dtype=np.double)  # no additives: all zeros
# mean equivalence ratio
equiv = 1.0
ierror = premixed.x_by_equivalence_ratio(
    MyGasMech, fuel.x, air.x, add_frac, products, equivalenceratio=equiv
)
```

(continues on next page)

(continued from previous page)

```
# check fuel-oxidizer mixture creation status
if ierror != 0:
    print("Error: Failed to create the hydrogen-air mixture!")
    exit()

# setting inlet pressure [dynes/cm2]
premixed.pressure = fuel.pressure
# set inlet/unburnt gas temperature [K]
premixed.temperature = fuel.temperature
# set estimated value of the flame speed [cm/sec]
premixed.velocity = 80.0
```

### Instantiate the laminar speed calculator

Set up the *freely propagating premixed flame* model by using the stream representing the premixed fuel-oxidizer mixture (with the estimated mass flow rate value). When the flame speed is expected to be very small (< 10 [cm/sec]) or very large (> 300 [cm/sec]), it might be beneficial to set the `mass_flowrate` of this inlet to the mass flux [g/cm$^2$-sec] based on the estimated flame speed value. There are many options and parameters related to the treatment of the species boundary condition, the transport properties. All the available options and parameters are described in the *Chemkin Input* manual.

**Note**

The stream parameter used to instantiate a `FlameSpeed` object is the properties of the unburned fuel-oxidizer mixture of which the *laminar flame speed* will be determined.

```
flamespeedcalculator = FlameSpeed(premixed, label="premixed_hydrogen")
```

### Set up initial mesh and grid adaption options

The premixed flame models provides several methods to set up the initial mesh. Here a uniform mesh of 35 grid points is used at the start of the simulation. The flame models would add more grid points to where they are needed as determined by the solution quality parameters specified by the `set_solution_quality` method.

The `end_poistion` is a required input as it defines the length of the calculation domain. Typically, the length of the calculation domain is between 1 to 10 [cm]. For low pressure conditions, the flame thickness becomes wider and a larger calculation domain is required.

**Note**

There are three methods to set up the initial mesh for the premixed flame calculations:

1. `use_tpro_grids` method (default) to use the grid points in the estimate temperature profile.
2. `set_numb_grid_points` method to create a uniform mesh of the given number of grid points.
3. `set_grid_profile` method to specify the initial grid point profile.

```
# set the initial mesh to 35 uniformly distributed grid points
flamespeedcalculator.set_numb_grid_points(35)
# set the maximum total number of grid points allowed
```

(continues on next page)

(continued from previous page)

```
# in the calculation (optional)
flamespeedcalculator.set_max_grid_points(150)
# define the calculation domain [cm]
flamespeedcalculator.end_position = 40.0
# maximum number of grid points can be added
# during each grid adaption event (optional)
flamespeedcalculator.set_max_adaptive_points(20)
# set the maximum values of the grdient and
# the curvature of the solution profiles (optional)
flamespeedcalculator.set_solution_quality(gradient=0.1, curvature=0.2)
```

### Set transport property options

Ansys Chemkin offers three methods for computing mixture properties:

- **Mixture averaged**
- **Multi-component**
- **Constant Lewis number**

When the system pressure is not too low, the mixture averaged method should be adequate. The multi-component method, although it is slightly more accurate, makes the simulation time longer and is harder to converge. Using the constant Lewis number method implies that all the species would have the same transport properties. Include the thermal diffusion effect, when there are large amount of light species (molecular weight < 5.0).

```
# use the mixture averaged formulism to evaluate the mixture transport properties
flamespeedcalculator.use_mixture_averaged_transport()
# include the thermal diffusion effect (because the unburned mixture has hydrogen
# (molecular weight < 5.0))
flamespeedcalculator.use_thermal_diffusion(mode=True)
```

### Set species composition boundary option

There two types of boundary condition treatments for the species composition available from the premixed flame models: `comp` and `flux`. You can find the descriptions of these two treatments in the *Chemkin Input* manual.

```
# specific the species composition boundary treatment ('comp' or 'flux')
# use 'comp' to keep the inlet species mass fraction values the same as
# the "given inlet stream".
flamespeedcalculator.set_species_boundary_types(mode="comp")
```

### Set solver parameters

The steady state solver parameters for the premixed flame model are optional because all the solver parameters have their own default values. Change the solver parameters when the premixed flame simulation does not converge with the default settings.

**Note**

The `FreelyPropagating` flame speed calculator has an option to automatically generate an estimate temperature profile that might improve the convergence performance. Use the `automatic_temperature_profile_estimate` method to turn this option on.

```
# reset the tolerances in the steady-state solver (optional)
flamespeedcalculator.steady_state_tolerances = (1.0e-9, 1.0e-6)
flamespeedcalculator.time_stepping_tolerances = (1.0e-6, 1.0e-4)
# reset the gas species floor value in the steady-state solver (optional)
flamespeedcalculator.set_species_floor(-1.0e-4)
# skip the fixed-temperature step (optional)
flamespeedcalculator.skip_fix_t_solution(mode=True)
# reduce the Jacobian age during the pseudo time stepping phase
flamespeedcalculator.set_pseudo_jacobian_age(10)

# turn ON/OFF auto ignition check (optional)
if ck.chemkin_version() >= 271:
    flamespeedcalculator.set_auto_ignition_check(check=False)
```

### Run the premixed flame calculation

Use the `run()` method to run the freely propagating premixed flame (flame speed) model. will solve the reactors one by one in the order they are added to the network. After the premixed flame calculation concludes successfully, use the `process_solution()` method to postprocess the solutions. The predicted laminar flame speed can be obtained by using the `get_flame_speed()` method. You can create other property profiles by looping through the solution streams with proper `Mixture` methods.

**Note**

When the inlet stream condition is close to the flammability limit, the flame speed calculation might fail. Remember that the reaction mechanism (reaction rates, thermodynamic properties, and transport properties) and the reactor model are *models* that contain assumptions and uncertainties.

```
# set the start wall time
start_time = time.time()

status = flamespeedcalculator.run()
if status != 0:
    print(Color.RED + "Failed to calculate the laminar flame speed!" + Color.END)
    exit()

# compute the total runtime
runtime = time.time() - start_time
print()
print(f"Total simulation duration: {runtime} [sec].")
print()
```

### Postprocess the premixed flame results

The postprocessing step will parse the solution and package the solution values at each time point into a stream. There are two ways to access the solution profiles:

1. the raw solution profiles (value as a function of time) are available for “distance”, “temperature”, and species “mass fractions”;

2. the streams that permit the use of all property and rate utilities to extract information such as viscosity, density, and mole fractions.

You can use the `get_solution_variable_profile()` method to get the raw solution profiles. The solution streams are accessed using either the `get_solution_stream_at_grid()` method for the solution stream at the given grid point or the `get_solution_stream()` method for the solution stream at the given location. (In this case, the stream is constructed by interpolation.)

**Note**

- Use the `get_solution_size()` to get the number of grid pints in the solution profiles before creating the arrays.
- The `mass_flowrate` from the solution streams is actually the *mass flux* [g/cm$^2$-sec]. It can used to derive the velocity at the corresponding location by dividing it by the local gas mixture density [g/cm$^3$]. The `get_flame_speed()` can also be used to retrieve the predicted laminar flame speed value.

```
# postprocess the solutions
flamespeedcalculator.process_solution()

# print the predicted laminar flame speed
print(
    f"The predicted laminar flame speed = "
    f"{flamespeedcalculator.get_flame_speed()} [cm/sec]."
)
# get the number of solution grid points
solutionpoints = flamespeedcalculator.get_solution_size()
print(f"Number of solution points = {solutionpoints}.")
# get the grid profile
mesh = flamespeedcalculator.get_solution_variable_profile("distance")
# get the temperature profile
tempprofile = flamespeedcalculator.get_solution_variable_profile("temperature")
# get HO2 mass fraction profile
h2o_profile = flamespeedcalculator.get_solution_variable_profile("HO2")

# create arrays for mixture density, mixture viscosity,
# and mixture specific heat capacity
denprofile = np.zeros_like(mesh, dtype=np.double)
viscprofile = np.zeros_like(mesh, dtype=np.double)
# loop over all solution grid points
for i in range(solutionpoints):
    # get the stream at the grid point
    solutionstream = flamespeedcalculator.get_solution_stream_at_grid(grid_index=i)
    # get gas density [g/cm3]
    denprofile[i] = solutionstream.rho
    # get mixture viscosity profile [g/cm-sec] or [Poise]
    viscprofile[i] = solutionstream.mixture_viscosity() * 1.0e2
```

### Plot the premixed flame solution profiles

Plot the solution profiles of the premixed flame.

**Note**

You can get profiles of the thermodynamic and the transport properties by applying `Mixture` utility methods to the solution `Stream`.

```
plt.subplots(2, 2, sharex="col", figsize=(12, 6))
plt.subplot(221)
plt.plot(mesh, tempprofile, "r-")
plt.ylabel("Temperature [K]")
plt.subplot(222)
plt.plot(mesh, denprofile, "b-")
plt.ylabel("Mixture Density [g/cm3]")
plt.subplot(223)
plt.plot(mesh, h2o_profile, "g-")
plt.xlabel("Distance [cm]")
plt.ylabel("HO2 Mass Fraction")
plt.subplot(224)
plt.plot(mesh, viscprofile, "m-")
plt.xlabel("Distance [cm]")
plt.ylabel("Mixture Viscosity [cP]")

# clean up
ck.done()

# plot results
if interactive:
    plt.show()
else:
    plt.savefig("plot_hydrogen_premixed_flame.png", bbox_inches="tight")
```

### 22.9.2 Construct atmospheric methane-air flame speed versus equivalence ratio table

One of the prevailing use case of the *freely propagating premixed flame* model is to build a *flame speed* table to be imported by another combustion simulation tools. PyChemkin provides the flexibility to customize the data structure of the flame speed table depending on the simulation goals and the tool. Furthermore, over the years, the chemkin flame speed calculator has derived a set of default solver settings that would greatly improve the convergence performance, especially for those widely adopted hydrocarbon fuel combustion mechanisms. The required input parameters the flame speed calculator are reduced to the composition of the fuel-oxidizer mixture, the initial/inlet pressure and temperature, and the calculation domain.

This tutorial shows the "minimal" effort to create a flame speed table of $CH_4$-air mixtures at the atmospheric pressure. The predicted flame speed values are compared against the experimental data as a function of the mixture equivalence ratio. Since the transport processes are critical for flame calculations, the transport data must be included in the mechanism data and preprocessed.

#### Import PyChemkin packages and start the logger

```
from pathlib import Path
import time

import matplotlib.pyplot as plt  # plotting
```

(continues on next page)

(continued from previous page)

```
import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color
from ansys.chemkin.core.inlet import Stream  # external gaseous inlet
from ansys.chemkin.core.logger import logger

# Chemkin 1-D premixed freely propagating flame model (steady-state)
from ansys.chemkin.core.premixedflames.premixedflame import (
    FreelyPropagating as FlameSpeed,
)

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create an instance of the Chemistry Set

The mechanism loaded is the GRI 3.0 mechanism for methane combustion. The mechanism and its associated data files come with the standard Ansys Chemkin installation under the subdirectory *"/reaction/data"*.

**Note**

The transport data *must* be included and preprocessed because the transport processes, *convection and diffusion*, are important to sustain the flame structure.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir
# including the full file path is recommended
chemfile = str(mechanism_dir / "grimech30_chem.inp")
thermfile = str(mechanism_dir / "grimech30_thermo.dat")
tranfile = str(mechanism_dir / "grimech30_transport.dat")
# create a chemistry set based on GRI 3.0
MyGasMech = ck.Chemistry(chem=chemfile, therm=thermfile, tran=tranfile, label="GRI 3.0")
```

### Preprocess the Chemistry Set

```
# preprocess the mechanism files
ierror = MyGasMech.preprocess()
if ierror != 0:
    print("Error: failed to preprocess the mechanism!")
    print(f"       error code = {ierror}")
```

(continues on next page)

(continued from previous page)

```
    exit()
```

### Set up the $CH_4$-air mixture for the flame speed calculation

Instantiate a stream named `premixed` for the inlet gas mixture. This stream is a mixture with the addition of the inlet flow rate. You can specify the inlet gas properties the same way you set up a `Mixture`. Here the `x_by_equivalence_ratio` method is used. You create the `fuel` and the `air` mixtures first. Then define the *complete combustion product species* and provide the *additives* composition if applicable. And finally, during the parameter iteration runs, you can simply set different values to `equivalenceratio` to create different methane-air mixtures.

```
# create the fuel mixture
fuel = ck.Mixture(MyGasMech)
# set fuel composition: methane
fuel.x = [("CH4", 1.0)]
# setting pressure and temperature condition for the flame speed calculations
fuel.pressure = 1.0 * ck.P_ATM
fuel.temperature = 300.0  # inlet temperature

# create the oxidizer mixture: air
air = ck.Mixture(MyGasMech)
air.x = ck.Air.x()
# setting pressure and temperature is not required in this case
air.pressure = fuel.pressure
air.temperature = fuel.temperature

# create the fuel-air Stream for the premixed flame speed calculation
premixed = Stream(MyGasMech, label="premixed")
# products from the complete combustion of the fuel mixture and air
products = ["CO2", "H2O", "N2"]
# species mole fractions of added/inert mixture.
# can also create an additives mixture here
add_frac = np.zeros(MyGasMech.kk, dtype=np.double)  # no additives: all zeros

# setting pressure and temperature is not required in this case
premixed.pressure = fuel.pressure
premixed.temperature = fuel.temperature

# set estimated value of the flame mass flux [g/cm2-sec]
premixed.mass_flowrate = 0.4

# equivalence ratio for the first case
phi = 0.6
# create mixture by using the equivalence ratio
ierror = premixed.x_by_equivalence_ratio(
    MyGasMech, fuel.x, air.x, add_frac, products, equivalenceratio=phi
)
# check fuel-oxidizer mixture creation status
if ierror != 0:
    print(
        "Error: failed to create the methane-air mixture "
        + "for equivalence ratio = "
        + str(phi)
```

(continues on next page)

(continued from previous page)

```
    )
    exit()
```

### Instantiate the laminar speed calculator

Set up the *freely propagating premixed flame* model by using the stream representing the premixed fuel-oxidizer mixture (with the estimated mass flow rate value). When the flame speed is expected to be very small (< 10 [cm/sec]) or very large (> 300 [cm/sec]), it might be beneficial to set the `mass_flowrate` of this inlet to the mass flux [g/cm$^2$-sec] based on the estimated flame speed value. There are many options and parameters related to the treatment of the species boundary condition, the transport properties. All the available options and parameters are described in the *Chemkin Input* manual.

**Note**

The stream parameter used to instantiate a `FlameSpeed` object is the properties of the unburned fuel-oxidizer mixture of which the *laminar flame speed* will be determined.

```
flamespeedcalculator = FlameSpeed(premixed, label="premixed_methane")
```

### Set up initial mesh and grid adaption options

The `end_poistion` is a required input as it defines the length of the calculation domain. Typically, the length of the calculation domain is between 1 to 10 [cm]. For low pressure conditions, the flame thickness becomes wider and a larger calculation domain is required.

```
# set the maximum total number of grid points allowed in the calculation (optional)
# flamespeedcalculator.set_max_grid_points(150)
# define the calculation domain [cm]
flamespeedcalculator.end_position = 1.0
```

### Run the flame speed parameter study

Use the `run()` method to run the freely propagating premixed flame (flame speed) model. After the premixed flame calculation concludes successfully, use the `process_solution()` method to postprocess the solutions. The predicted laminar flame speed can be obtained by using the `get_flame_speed()` method. You can create other property profiles by looping through the solution streams with proper `Mixture` methods. The parameter in this project is the equivalence ratio of the methane-air mixture. You can start the parameter run from the most fuel-lean or from the most fuel-rich case. Normally, the most "extreme" cases are difficult to converge. When running into these situations, start the parameter runs from the stoichiometric condition and go down the lean and/or the rich branch. Here the runs start from the most fuel-lean case ($\phi = 0.6$) and progress all the way to the most fuel-rich case ($\phi = 1.6$) in steps of 0.05.

**Note**

- When the inlet stream condition is close to the flammability limit, the flame speed calculation might fail. Remember that the reaction mechanism (reaction rates, thermodynamic properties, and transport properties) and the reactor model are *models* that contain assumptions and uncertainties.
- After complete the first run, you can use the `continuation()` method to start the new runs from the solution of the previous run. However, by doing this, the later runs will contain a lot of grid points accumulated from all previous runs.

- Use the `set_molefractions` method to update the inlet gas composition before each run. Similarly, use the `pressure` and the `temperature` methods to change the inlet condition.

```
# total number of parameter cases
points = 21
# equivalence ratio increment
delta_phi = 0.05
# create solution arrays
equival = np.zeros(points, dtype=np.double)
flamespeed = np.zeros_like(equival, dtype=np.double)
# set the start wall time
start_time = time.time()

# start the parameter study runs
for i in range(points):
    # run the flame speed calculation for this equivalence ratio
    status = flamespeedcalculator.run()
    if status != 0:
        print(
            Color.RED
            + "failed to calculate the laminar flame speed"
            + "for equivalence ratio = "
            + str(phi)
            + Color.END
        )
        exit()
    # get flame speed
    # postprocess the solutions
    flamespeedcalculator.process_solution()
    # save data
    equival[i] = phi
    # get flame speed
    flamespeed[i] = flamespeedcalculator.get_flame_speed()
    # print the predicted laminar flame speed
    print(
        f"methane-air equivalence ratio = {phi} :\n"
        + f"the predicted laminar flame speed = {flamespeed[i]} [cm/sec]"
    )
    #
    # update parameter
    phi += delta_phi
    # create mixture by using the equivalence ratio
    ierror = premixed.x_by_equivalence_ratio(
        MyGasMech, fuel.x, air.x, add_frac, products, equivalenceratio=phi
    )
    # check fuel-oxidizer mixture creation status
    if ierror != 0:
        print(
            "Error: failed to create the methane-air mixture ",
            "for equivalence ratio = ",
            str(phi),
        )
```

(continues on next page)

(continued from previous page)

```
        exit()
    # update initial gas composition
    flamespeedcalculator.set_molefractions(premixed.x)

# compute the total runtime
runtime = time.time() - start_time
print()
print(f"total simulation duration: {runtime} [sec]")
print()

# experimental data by Vagelopoulos
# equivalence ratios
data_equiv = [
    0.6126,
    0.6619,
    0.7109,
    0.7533,
    0.8268,
    0.9109,
    0.9826,
    1.0387,
    1.0901,
    1.1321,
    1.1858,
    1.2347,
    1.2695,
    1.325,
    1.3563,
    1.4279,
    1.4977,
]
# methane flame speeds at 1 atm
data_speed = [
    9.4434,
    12.7281,
    17.4088,
    21.0219,
    26.5237,
    33.2573,
    37.0347,
    38.677,
    38.8412,
    37.8558,
    34.9818,
    31.1223,
    25.9489,
    21.3504,
    17.2445,
    12.7281,
    9.7719,
]
```

#### Plot the premixed flame solution profiles

Plot the predicted flame speeds against the experimental data

```
plt.plot(data_equiv, data_speed, label="data", linestyle="", marker="^", color="blue")
plt.plot(equival, flamespeed, label=MyGasMech.label, linestyle="-", color="blue")
plt.legend()
plt.ylabel("Flame Speed [cm/sec]")
plt.xlabel("Equivalence Ratio")

# clean up
ck.done()

# plot results
if interactive:
    plt.show()
else:
    plt.savefig("plot_flame_speed_table.png", bbox_inches="tight")
```

### 22.9.3 Simulate a burner stabilized premixed flame

The *burner stabilized premixed flame* model is frequently used as a numerical tool to develop and to validate combustion mechanisms in the context of a burner stabilized flat flame. The premixed burner stabilized flat flame experiments are important because, in addition to the reaction pathways and rate parameters, they provide opportunities to learn about the impact of the transport processes on the combustion chemistry and flame structure. The premixed burner stabilized flame model calculates the temperature and the species concentrations along the burner centerline, and the results will be compared against the measurements. The burner stabilized flame model assumes that the flow is laminar and the temperature at the burner rim is constant (in order to anchor the flame).

This example shows how to use the burner stabilized premixed flame model to calculate the species profiles of a $C_2H_4$-$O_2$-AR mixture along the burner centerline with the measured temperature profile data. Since the transport processes are critical for flame calculations, transport data must be included in the mechanism data and preprocessed.

```

```

#### Import PyChemkin packages and start the logger

```
from pathlib import Path
import time

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color
from ansys.chemkin.core.inlet import Stream  # external gaseous inlet
from ansys.chemkin.core.logger import logger
from ansys.chemkin.core.premixedflames.premixedflame import (
    BurnedStabilizedGivenTemperature as Burner,
)

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
```

(continues on next page)

(continued from previous page)

```
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create the first chemistry set

The mechanism to load is the GRI 3.0 mechanism for methane combustion. This mechanism and its associated data files come with the standard Ansys Chemkin installation in the `/reaction/data` directory.

**Note**

The transport data *must* be included and preprocessed because the transport processes, *convection and diffusion*, are important to sustain the flame structure.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir
# including the full file path is recommended
chemfile = str(mechanism_dir / "grimech30_chem.inp")
thermfile = str(mechanism_dir / "grimech30_thermo.dat")
tranfile = str(mechanism_dir / "grimech30_transport.dat")
# create a chemistry set based on GRI 3.0
MyGasMech = ck.Chemistry(chem=chemfile, therm=thermfile, tran=tranfile, label="GRI 3.0")
```

### Preprocess the chemistry set

```
# preprocess the mechanism files
ierror = MyGasMech.preprocess()
if ierror != 0:
    print("Error: Failed to preprocess the mechanism.")
    print(f"       Error code = {ierror}.")
    exit()
```

### Set up the ($C_2H_4$ + $O_2$+ AR ) mixture to the flat burner

Instantiate a stream named `premixed` for the inlet gas mixture. This stream is a mixture with the addition of the inlet flow rate. You specify the inlet gas properties in the same way you set up a mixture. You can simply use a composition *"recipe"* to create the $C_2H_4$ + $O_2$+ AR mixture. You use the `mass_flowrate()` method to assign the burner inlet mass flow rate.

**Note**

By default, the burner flow area is set to unity (1.0 [$cm^2$]) because the flame models use the mass flux [g/$cm^2$-sec] in the calculations instead of the flow rate [g/sec]. If the mass flow rate is used, the actual burner cross-sectional flow area must be provided by using the `flowarea()` stream method.

```
# create the fuel-air stream for the premixed flame speed calculation
premixed = Stream(MyGasMech, label="premixed")
# set inlet premixed stream molar composition
premixed.x = [("C2H4", 0.163), ("O2", 0.237), ("AR", 0.6)]
# setting inlet pressure [dynes/cm2]
premixed.pressure = 1.0 * ck.P_ATM
# set inlet/unburnt gas temperature [K]
premixed.temperature = 300.0  # inlet temperature

# set the burner outlet cross sectional flow area to unity (1 [cm2])
# because the flame models use the mass flux in the calculation instead of
# the mass flow rate
premixed.flowarea = 1.0
# set value for the inlet mass flow rate [g/sec]
premixed.velocity = 0.0147
```

### Instantiate the laminar flat flame burner

Set up the premixed burner stabilized flame model by using the stream representing the premixed fuel-oxidizer mixture.There are many options and parameters related to the treatment of the species boundary condition and the transport properties. All the available options and parameters are described in the *Chemkin Input* manual.

**Note**

The stream parameter used to instantiate a `Burner` object consists of the properties of the unburned fuel-oxidizer mixture leaving the burner outlet.

```
flatflame = Burner(premixed, label="ethylene flame")
```

### Set up the temperature profile along the burner centerline

Use the `set_temperature_profile()` method to set up the temperature profile data. The temperature profile along the burner centerline is typically obtained from the corresponding experiments.

```
tpro_data_points = 21
gridpoints = np.zeros(tpro_data_points, dtype=np.double)
temp_data = np.zeros_like(gridpoints, dtype=np.double)
gridpoints = [
    0.0,
    0.01,
    0.02,
    0.0345643,
    0.0602659,
    0.100148,
    0.118759,
    0.135598,
    0.15421,
    0.179911,
    0.211817,
    0.233087,
    0.260561,
```

(continues on next page)

(continued from previous page)

```
    0.293353,
    0.348301,
    0.436928,
    0.517578,
    0.7161,
    0.935894,
    1.16632,
    1.2,
]
temp_data = [
    300.0,
    450.0,
    600.0,
    828.498,
    1104.4,
    1496.7,
    1634.65,
    1714.85,
    1768.33,
    1801.12,
    1807.2,
    1803.78,
    1799.51,
    1788.35,
    1778.09,
    1767.01,
    1760.23,
    1744.13,
    1737.55,
    1730.99,
    1729.31,
]
flatflame.set_temperature_profile(x=gridpoints, temp=temp_data)
```

### Set up initial mesh and grid adaption options

The premixed flame models provides several methods to set up the initial mesh. In this case, the temperature profile is given by the experimental data. The `use_TPRO_grid()` method lets the flame model recycle the grid points of the temperature profile as the initial mesh at the start of the calculation. The flame models would add more grid points to where they are needed as determined by the solution quality parameters specified by the `set_solution_quality()` method.

The `end_position` argument is a required input as it defines the length of the calculation domain. Typically, the length of the calculation domain is between 1 to 10 [cm].

```
# set the maximum total number of grid points allowed
# in the calculation (optional)
flatflame.set_max_grid_points(250)
# define the calculation domain [cm]
flatflame.end_position = 1.2
# maximum number of grid points can be added during
# each grid adaption event (optional)
```

(continues on next page)

(continued from previous page)

```
flatflame.set_max_adaptive_points(10)
# set the maximum values of the gradient and
# the curvature of the solution profiles (optional)
flatflame.set_solution_quality(gradient=0.1, curvature=0.1)
```

### Set transport property options

Ansys Chemkin offers three methods for computing mixture properties:

- **Mixture averaged**
- **Multi-component**
- **Constant Lewis number**

When the system pressure is not too low, the mixture averaged method should be adequate. The multi-component method, although it is slightly more accurate, makes the simulation time longer and is harder to converge. Using the constant Lewis number method implies that all the species would have the same transport properties.

```
# use the mixture averaged method to evaluate the mixture transport properties
flatflame.use_mixture_averaged_transport()
```

### Set species composition boundary option

There two types of boundary condition treatments for the species composition available from the premixed flame models: `comp` and `flux`. You can find the descriptions of these two treatments in the *Chemkin Input* manual.

```
# specify the species composition boundary treatment ('comp' or 'flux')
# use 'flux' to ensure that the "net" species mass fluxes are zero at
# the burner outlet.
flatflame.set_species_boundary_types(mode="flux")
```

### Set solver parameters

The steady-state solver parameters for the premixed flame model are optional because all the solver parameters have their own default values. Change the solver parameters when the premixed flame simulation does not converge with the default settings.

```
# reset the tolerances in the steady-state solver (optional)
flatflame.steady_state_tolerances = (1.0e-9, 1.0e-4)
# reset the gas species floor value in the steady-state solver (optional)
flatflame.set_species_floor(-1.0e-3)
```

### Run the premixed flame calculation

Use the `run()` method to run the freely propagating premixed flame (flame speed) model. This method solves the reactors one by one in the order that they are added to the network. After the premixed flame calculation concludes successfully, use the `process_solution()` method to postprocess the solutions. You can create other property profiles by looping through the solution streams with proper mixture methods.

```
# set the start wall time
start_time = time.time()

status = flatflame.run()
```

(continues on next page)

(continued from previous page)

```
if status != 0:
    print(Color.RED + "Failed to solve the reactor network." + Color.END)
    exit()

# get the number of solution grid points
solutionpoints = flatflame.get_solution_size()
print(f"Number of solution points = {solutionpoints}.")
```

### Refine the solution profiles

When the simulation is hard to converge in one go, you can use a number of continuations to gradually get to the solution of the intended condition. For example, you can get to the solution of a very fuel-lean mixture by starting with a slightly fuel-rich mixture flame and use the `continuation()` method to run the more difficult fuel-lean case from the converged solution.

```
# tightening the convergence criteria
flatflame.set_solution_quality(gradient=0.05, curvature=0.1)

# restart the flame simulation by continuation
flatflame.continuation()

# compute the total runtime
runtime = time.time() - start_time
print()
print(f"Total simulation duration: {runtime} [sec].")
print()
```

### Postprocess the premixed flame results

The postprocessing step parses the solution and packages the solution values at each time point into a mixture. There are two ways to access the solution profiles:

- The raw solution profiles (value as a function of distance) are available for distance, temperature, pressure, volume, and species mass fractions.

  **-The streams permit the use of all property and rate utilities to extract**
  information such as viscosity, density, and mole fractions.

You can use the `get_solution_variable_profile()` method to get the raw solution profiles. The solution streams are accessed using either the `get_solution_stream_at_grid()` method for the solution stream at the given grid point or the `get_solution_stream()` method for the solution stream at the given location. (In this case, the stream is constructed by interpolation.)

**Note**

- Use the `get_solution_size()` method to get the number of grid points in the solution profiles before creating the arrays.
- The `mass_flowrate` from the solution streams is actually the *mass flux* [g/cm$^2$-sec]. It can used to derive the velocity at the corresponding location by dividing it by the local gas mixture density [g/cm$^3$].

```
# postprocess the solutions
flatflame.process_solution()
```

(continues on next page)

(continued from previous page)

```
# get the number of solution grid points
solutionpoints = flatflame.get_solution_size()
print(f"Number of final solution points = {solutionpoints}.")
# get the grid profile
mesh = flatflame.get_solution_variable_profile("distance")
# get the temperature profile
tempprofile = flatflame.get_solution_variable_profile("temperature")
# get OH mass fraction profile
oh_profile = flatflame.get_solution_variable_profile("OH")

# create arrays for mixture conductivity and mixture-pecific heat capacity
cp_profile = np.zeros_like(mesh, dtype=np.double)
condprofile = np.zeros_like(mesh, dtype=np.double)
# loop over all solution grid points
for i in range(solutionpoints):
    # get the stream at the grid point
    solutionstream = flatflame.get_solution_stream_at_grid(grid_index=i)
    # get mixture-specific heat capacity profile [erg/mole-K]
    cp_profile[i] = solutionstream.cpbl() / ck.ERGS_PER_JOULE * 1.0e-3
    # get thermal conductivity profile [ergs/cm-K-sec]
    condprofile[i] = solutionstream.mixture_conductivity() * 1.0e-5
```

### Plot the premixed flame solution profiles

Plot the solution profiles of the premixed flame.

**Note**

You can get profiles of the thermodynamic and the transport properties by applying `Mixture` utility methods to the solution stream.

```
plt.subplots(2, 2, sharex="col", figsize=(12, 6))
plt.subplot(221)
plt.plot(mesh, tempprofile, "r-")
plt.ylabel("Temperature [K]")
plt.subplot(222)
plt.plot(mesh, cp_profile, "b-")
plt.ylabel("Mixture Cp [kJ/mole]")
plt.subplot(223)
plt.plot(mesh, oh_profile, "g-")
plt.xlabel("Distance [cm]")
plt.ylabel("OH Mass Fraction")
plt.subplot(224)
plt.plot(mesh, condprofile, "m-")
plt.xlabel("Distance [cm]")
plt.ylabel("Mixture conductivity [W/m-K]")

# clean up
ck.done()
```

(continues on next page)

(continued from previous page)

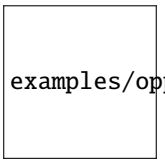
```
# plot results
if interactive:
    plt.show()
else:
    plt.savefig("plot_premixed_burner_stabilised_flame.png", bbox_inches="tight")
```

## 22.10 Opposed-flow flame model

The *opposed-flow Flame* model is a 1-D steady-state application to study the structure of stretched flames in the opposed-flow configuration. It can also be used to generate a "flamelet table" for CFD simulations. See the Chemkin *Theory* manual for additional information about this feature.

### 22.10.1 Study flame interactions in an opposed-flow flame configuration

The *opposed-flow flame* model is frequently used as a numerical tool to study the flame structures under fluid dynamic stress without the interferences from the wall. The opposed-flow flame experiments provide insights on the impact of the transport processes (mainly diffusion) on the combustion chemistry and flame structure. They also reveal how the flame structure would response to the strain rate imposed by the mean flow field (or to some extent by the turbulence).




Typically, the "fuel" and the "oxidizer" are introduced separately from the two opposing inlets, and a non-premixed (diffusion) flame would settle around the stagnating plan in the middle of the separation gap. Since the axial velocity near the stagnating plan is small, the flame is sustained mainly by the balance of combustion chemistry and the species/heat diffusion. The flame is stretched (strained) due to the radial velocity so changing the flow rates at the inlets will vary the stretching/strain rate applied to the flame. It is possible to establish more than one flame in the separation gap. For example, this project forms two flames by introducing a fuel-rich mixture from the "fuel" inlet. Alternatively, replacing the "oxidizer" with a fuel-lean mixture or replacing both inlet streams with premixed mixtures will also yields two flames. Having a fuel-rich mixture for the "fuel" and a fuel-lean mixture for the "oxidizer" might create three flames: one rich premixed flame, one lean premixed flame and one diffusion flame between them.

The flame model calculates the temperature, velocities, and the species concentrations along the centerline between the two opposing nozzles, and the results can be presented graphically. The mixture fraction profile is also available. The opposed-flow flame model evaluates the mixture fraction by using Bilger's elemental fraction formulation, therefore, the mixture fraction could be negative or have values greater than 1. Since there is no commonly recognized strain rate definition, you can derive your own strain rate from the velocity solution.

This tutorial demonstrates the application of the axisymmetric opposed-flow flame model to study the interactions (species and heat) between two strained flames, a fuel-rich premixed flame and a diffusion flame, situated in the space between the two inlets. The computed temperature profile does not appear to show two distinct flame zones. However, by plotting profile of radicals such as $NO_2$, the locations of the two flame fronts are clearly marked. The species and temperature gradients between the two flames indicate the two flames support each other by exchanging reactants and heat. If you plot the hydrogen cyanide (HCN) profile, you would see that the rich premixed flame can produce NOx through the Fenimore route (prompt NOx); while the oxygen atom (O) profile would further reveal that, while both flames form thermal NOx (Zeldovich route) , most of the thermal NOx is likely emitted from the diffusion flame. The nitrogen dioxide ($NO_2$) formed in the flames will convert to nitric oxide (NO) in the hot zone sandwiched by the two flames.

Since the transport processes are critical for flame calculations, the *transport data* must be included in the mechanism data and pre-processed.

```
```

### Import PyChemkin packages and start the logger

```
from pathlib import Path
import time

import matplotlib.pyplot as plt  # plotting

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color

# Chemkin 1-D opposed-flow flame model (steady-state)
from ansys.chemkin.core.diffusionflames.opposedflowflame import OpposedFlame as Flame
from ansys.chemkin.core.inlet import Stream  # external gaseous inlet
from ansys.chemkin.core.logger import logger

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create an instance of the Chemistry Set

The mechanism loaded is the GRI 3.0 mechanism for methane combustion. The mechanism and its associated data files come with the standard Ansys Chemkin installation under the subdirectory *"/reaction/data"*.

**Note**

The transport data *must* be included and preprocessed because the transport processes, *convection and diffusion*, are important to sustain the flame structure.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir
# including the full file path is recommended
chemfile = str(mechanism_dir / "grimech30_chem.inp")
thermfile = str(mechanism_dir / "grimech30_thermo.dat")
tranfile = str(mechanism_dir / "grimech30_transport.dat")
# create a chemistry set based on GRI 3.0
MyGasMech = ck.Chemistry(chem=chemfile, therm=thermfile, tran=tranfile, label="GRI 3.0")
```

### Preprocess the Chemistry Set

```
# preprocess the mechanism files
ierror = MyGasMech.preprocess()
if ierror != 0:
    print("Error: Failed to preprocess the mechanism!")
    print(f"       Error code = {ierror}")
    exit()
```

### Set up the opposed-flow inlet streams for the dual flame simulation

The opposed-flow flame has two opposing inlet streams separated by a small gap. Conventionally the "fuel" inlet is located at x = 0, and the "oxidizer" inlet is at the opposite end of the separation (in reality the nozzles are arranged vertically). A strained non-premixed flame (or diffusion flame) can be established in the gap between the inlets by tuning the velocities and the mixture properties of the two inlet streams.

This stream is a mixture with the addition of the *inlet flow rate*. You can specify the inlet gas properties the same way you set up a mixture. Here the recipe is used to set up the species mole fractions of the fuel mixture. For convenience, the pre-defined `Air()` in PyChemkin is used to set the composition of the `air` mixture. The inlet velocity is assigned by the `velocity` method.

```
# create the "fuel" stream
fuel = Stream(MyGasMech, label="FUEL")
# set fuel composition: fuel-rich methane-air mixture (equivalence ratio ~ 1.55)
fuel.x = [("CH4", 0.14001807), ("O2", 0.18066847), ("N2", 0.67931346)]
# system pressure [dynes/cm2]
fuel.pressure = ck.P_ATM
# fuel temperature [K]
fuel.temperature = 300.0
# fuel inlet velocity [cm/sec]
fuel.velocity = 16.0

# create the oxidizer mixture: air
air = Stream(MyGasMech, label="OXID")
air.x = ck.Air.x()
# oxidizer pressure (same as the fuel stream)
air.pressure = fuel.pressure
# oxidizer temperature (same as the fuel temperature)
air.temperature = fuel.temperature
# oxidizer inlet velocity [cm/sec]
air.velocity = 16.0
```

### Instantiate the opposed-flow flame model

Set up the *opposed-flow flame* model by using the stream representing the "fuel" mixture at the origin. The "oxidizer" inlet is added to the `OpposedFlame` object later by using the `set_oxidizer_inlet` method. There are many options and parameters related to the treatment of the species boundary condition, the transport properties. All the available options and parameters are described in the *Chemkin Input* manual.

**Note**

The parameter used to instantiate the `OpposedFlame` is the stream representing the "fuel" inlet at x = 0.

```
dual_flame = Flame(fuel, label="opposed_flame")
```

### Configure the opposed flame

Use the `set_oxidizer_inlet` method to specify the "oxidizer" stream at the opposite end of the separation gap. The distance between the two opposing inlets is defined by the `end_position` method. The `end_poistion` is a required input as it defines the length of the calculation domain. Typically, the length of the calculation domain is less than 10 [cm].

The opposed-flow flame model will set up a *flame zone* to improve the convergence performance. The center location of this "estimated" *flame zone* can be given by the `set_reaction_zone_center`, and the width of the *flame zone* is defined by the `set_reaction_zone_width`. The temperature and the gas species profiles are assumed to vary linearly from the inlets and plateaued inside the *flame zone*. The gas temperature value in the *flame zone* is specified by the `set_max_flame_temperature()` method (or 2200.0 [K] by default). The gas composition in the *flame zone* is set by the equilibrium composition of the mixture of the "fuel" and the "oxidizer" streams at the given "maximum flame temperature".

Alternatively, a different temperature profile can be set up by using the `setprofile()` method to replace the default "plateau" profile.

```
# add the "oxidizer" inlet
dual_flame.set_oxidizer_inlet(air)

# define the gap between the two opposing inlets (calculation domain) [cm]
dual_flame.end_position = 1.5
# set up of the "flame zone" to establish the guessed species profiles
# flame zone center location [cm]
dual_flame.set_reaction_zone_center(0.75)
# flame zone width [cm]
dual_flame.set_reaction_zone_width(0.5)
# set the estimated maximum gas temperature [K]
dual_flame.set_max_flame_temperature(2200.0)
```

### Set up initial mesh and grid adaption options

The opposed-flow flame models provides several methods to set up the initial mesh. Here a uniform mesh of 26 grid points is used at the start of the simulation. The flame models would add more grid points to where they are needed as determined by the solution quality parameters specified by the `set_solution_quality()` method.

**Note**

There are two ways to set up the initial mesh for the opposed-flow flame calculations:

1. **`set_numb_grid_points` method to create a uniform mesh of the given number** of grid points.
2. `set_grid_profile` method to specify the initial grid point profile.

```
# set the initial mesh to 26 uniformly distributed grid points
dual_flame.set_numb_grid_points(26)
# set the maximum total number of grid points allowed in the calculation (optional)
dual_flame.set_max_grid_points(250)
# maximum number of grid points can be added during each grid adaption event (optional)
```

(continues on next page)

(continued from previous page)

```
dual_flame.set_max_adaptive_points(5)
# set the maximum values of the grdient and the curvature
# of the solution profiles (optional)
dual_flame.set_solution_quality(gradient=0.1, curvature=0.3)
```

### Set transport property options

Ansys Chemkin offers three methods for computing mixture properties:

- **Mixture averaged**
- **Multi-component**
- **Constant Lewis number**

When the system pressure is not too low, the mixture averaged method should be adequate. The multi-component method, although it is slightly more accurate, makes the simulation time longer and is harder to converge. Using the constant Lewis number method implies that all the species would have the same transport properties. Include the thermal diffusion effect, when there are large amount of light species (molecular weight < 5.0).

```
# use the mixture averaged formulism to evaluate the mixture transport properties
dual_flame.use_mixture_averaged_transport()
# do NOT include the thermal diffusion effect
dual_flame.use_thermal_diffusion(mode=False)
```

### Set species composition boundary option

There two types of boundary condition treatments for the species composition available from the premixed flame models: `comp` and `flux`. You can find the descriptions of these two treatments in the *Chemkin Input* manual.

```
# specific the species composition boundary treatment ('comp' or 'flux')
# use 'flux' to keep the net species mass fluxes the same as given by
# the "inlet streams".
dual_flame.set_species_boundary_types(mode="flux")
```

### Set solver parameters

The steady state solver parameters for the opposed-flow flame model are optional because all the solver parameters have their own default values. Change the solver parameters when the flame simulation does not converge with the default settings.

```
# reset the tolerances in the steady-state solver (optional)
dual_flame.steady_state_tolerances = (1.0e-9, 1.0e-5)
dual_flame.time_stepping_tolerances = (1.0e-6, 1.0e-4)
```

### Run the opposed-flow flame simulation

Use the `run()` method to run the opposed-flow flame model. After the calculation concludes successfully, use the `process_solution()` method to postprocess the solutions. You can create other property profiles by looping through the solution streams by using proper `Mixture` methods.

```
# set the start wall time
start_time = time.time()

```

(continues on next page)

(continued from previous page)

```
status = dual_flame.run()
if status != 0:
    print(
        Color.RED
        + "Failed to get a converged solution of the opposed flow flame!"
        + Color.END
    )
    exit()

# compute the total runtime
runtime = time.time() - start_time
print()
print(f"Total simulation duration: {runtime} [sec].")
print()
```

### Postprocess the opposed-flow flame results

The post-processing step will parse the solution and package the solution values at each time point into a streams. There are two ways to access the solution profiles:

1. the raw solution profiles (value as a function of time) are available for "distance", "temperature", and species "mass fractions";

2. the streams that permit the use of all property and rate utilities to extract information such as viscosity, density, and mole fractions.

You can use the `get_solution_variable_profile()` method to get the raw solution profiles. Solution streams are accessed using either the `get_solution_stream_at_grid()` method for the solution stream at the given grid point or the `get_solution_stream()` method for the solution stream at the given location. (In this case, the stream is constructed by interpolation.)

**Note**

- Use the `get_solution_size()` to get the number of grid pints in the solution profiles before creating the arrays.
- The `mass_flowrate` from the solution streams is actually the *mass flux* [g/cm$^2$-sec]. It can used to derive the velocity at the corresponding location by dividing it by the local gas mixture density [g/cm$^3$].
- In addition to the usual raw solution profiles such as "distance", "temperature" and species mass fractions, the opposed-flow flame model provides solution profiles for the velocities ("axial_velocity" and "radial_velocity_gradient") and the mixture fraction ("mixture_fraction").
- The existence of two separated flames can be by shown by plotting the heat release rate or the concentrations of radical species found typically in hydrocarbon flames.

```
# postprocess the solutions
dual_flame.process_solution()

# get the number of solution grid points
solutionpoints = dual_flame.get_solution_size()
print(f"Number of solution points = {solutionpoints}.")
# get the grid profile
```

(continues on next page)

(continued from previous page)

```
mesh = dual_flame.get_solution_variable_profile("distance")
# get the temperature profile
tempprofile = dual_flame.get_solution_variable_profile("temperature")
# get the axial velocity profile
velprofile = dual_flame.get_solution_variable_profile("axial_velocity")
# get the mixture fraction profile
mfprofile = dual_flame.get_solution_variable_profile("mixture_fraction")
# get NO2 mass fraction profile
no2_profile = dual_flame.get_solution_variable_profile("NO2")
```

### Plot the opposed-flow flame solution profiles

Plot the solution profiles of the opposed-flow flame.

**Note**

You can get profiles of the thermodynamic and the transport properties by applying `Mixture` utility methods to the solution streams.

```
plt.subplots(2, 2, sharex="col", figsize=(12, 6))
plt.subplot(221)
plt.plot(mesh, tempprofile, "r-")
plt.ylabel("Temperature [K]")
plt.subplot(222)
plt.plot(mesh, velprofile, "b-")
plt.ylabel("Axial Velocity [cm/sec]")
plt.subplot(223)
plt.plot(mesh, no2_profile, "g-")
plt.xlabel("Distance [cm]")
plt.ylabel("NO2 Mass Fraction")
plt.subplot(224)
plt.plot(mesh, mfprofile, "m-")
plt.xlabel("Distance [cm]")
plt.ylabel("Mixture Fraction [-]")

# clean up
ck.done()

# plot results
if interactive:
    plt.show()
else:
    plt.savefig("plot_opposed_flow_flame.png", bbox_inches="tight")
```

## 22.11 Surface chemistry

The examples in this section demonstrate how *surface chemistry* can be included in a *Chemistry Set* and applied to simulate chemical vapor deposition (CVD) and catalytic combustion processes in **PyChemkin**.

### 22.11.1 Simulating the chemical vapor deposition (CVD) process

The chemical vapor deposition (CVD) process plays an important part in the semiconductor industries, especially for manufacturing and processing the wafers.

This example project presents a model for the CVD of silicon nitride (Si-N) in a steady-state PSR. The utilization of the simple PSR model is justified because the CVD process is operated under the "kinetic limited" condition and is running steadily. The CVD PSR is characterized by the volume, the substrate surface area, and the inlet precursor gas volumetric flow rate; consequently the reactor residence time remains constant even when the precursor stream composition or the reactor temperature is changed.

The current CVD process operates at a low pressure of 1.8 Torr, and a high temperature (for both gas phase and the substrate surface) of 1440 |deg|C. The precursor stream consists of $SiF_4$ and $NH_3$. The chemistry set contains the gas phase mechanism input file, "chem_SIN_cvd.inp", the surface mechanism input file "surf_SIN_cvd.inp", and the thermodynamic data file, "therm.dat". The two mechanism input files are in the "exampledataSIN_CVD" folder, and the generic thermodynamic data file is in the "data" folder of the Ansys Chemkin installation. The chemistry set is described in the Si-N CVD from Silicon Tetrafluoride and Ammonia section of the *Chemkin Tutorial* manual (p. 346).

When a reactor model is instantiated with a `Mixture` or a `Stream` that is made of a `Chemistry Set` with a surface mechanism, the surface chemistry module of the reactor model will be automatically initialized with the surface mechanism data from the `Chemistry Set`. There is no additional step to "turn on" the surface mechanism for the reactor model.

You will determine the effects of the inlet $SiF_4$ and $NH_3$ molar ratio on the linear growth rate (deposition rate) of the Si-N layers on the substrate while keeping the reactor temperature and pressure fixed. The inlet composition can be easily changed by resetting the inlet stream composition.

In addition to the bulk species (Si and N) growth rates, you can get the reactor gas composition and the substrate surface coverage from the steady-state PSR solution for each case. You can also obtain the total surface rate-of-production (surface ROPs) from the solutions to analyze the important reaction pathways of the CVD process.

The solution plots show the dependencies of the mole fraction of gas product HF, the substrate surface coverage, the Si linear growth rate, and the surface consumption rates of the precursors on the inlet $SiF_4$ molar ratios. The results shows that the deposition rate can be controlled by adjusting the precursor molar ratio. The growth rate could reach its max value When both precursors have the same molar concentration. Moreover, the negative surface ROPs indicate that surface reactions (rather than gas reactions) are the dominate decomposition pathways of the gas-phase precursors.

#### Import PyChemkin packages and start the logger

```
from pathlib import Path
import time

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color
from ansys.chemkin.core.inlet import Stream
from ansys.chemkin.core.logger import logger

# Chemkin PSR model (steady-state)
from ansys.chemkin.core.stirreactors.PSR import PSRSetVolumeFixedTemperature as Psr

# check working directory
```

(continues on next page)

(continued from previous page)

```
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create a chemistry set

This example uses the surface mechanism of Si-N chemical vapor deposition (Si-N CVD) from Silicon tetrafluoride ($SiF_4$). The reaction mechanism consists of two parts: the gas-phase mechanism, `chem_SIN_cvd.inp`, describing the breakdown of the precursors in the gas phase; and the surface mechanism, `surf_SIN_cvd.inp`, describing the reactions taking place on the substrate surface, such as the adsorption of the gas precursors, the reactions between the adsorbed surface species, and desorption of gas products for the overall process of the deposition of the Si and the N atoms.

This mechanism is provided in the *examplesdata* folder of the local *PyChemkin* installation. You must provide the mechanism file names, `chem_SIN_cvd.inp` and `surf_SIN_cvd.inp`, with the correct path `example_mech_data` to the Chemistry Set before pre-processing. The thermodynamic data file from the default Ansys Chemkin installation will be used.

```
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
# find the location of this example py file
try:
    script_dir_obj = Path(__file__).parent.resolve()
except NameError:
    script_dir_obj = Path(current_dir)
script_dir = str(script_dir_obj)
# use the relative path to locate the example mechanism folder
example_mech_data = script_dir_obj / ".." / "data"
print(f"example data = {str(example_mech_data)}")
# create a chemistry set based on the Si-N CVD mechanism
MyCVDMech = ck.Chemistry(label="Si-N_CVD")
# set mechanism input files
# including the full file path is recommended
local_data_dir = "SIN_CVD"
MyCVDMech.chemfile = str(example_mech_data / local_data_dir / "chem_SIN_cvd.inp")
MyCVDMech.surffile = str(example_mech_data / local_data_dir / "surf_SIN_cvd.inp")
MyCVDMech.thermfile = str(data_dir / "therm.dat")
```

### Preprocess the CVD chemistry set

preprocess the mechanism files

```
_ = MyCVDMech.preprocess()
```

### Set up the precursors stream

Instantiate a stream feed for the inlet gas mixture. The stream is a mixture with the addition of the inlet flow rate. You specify inlet gas properties in the same way as you set up a mixture.

Use the `mass_flowrate` method to assign the inlet mass flow rate. In this example, the `sccm` method, [cm3/min] @ the standard condition, is used specify the inlet volumetric flow rate. And the different inlet precursors compositions are given by a list of mole fractions recipes `inlet_recipes`.

```
# precursor stream compositions
inlet_recipes = [
    [("SIF4", 0.14286), ("NH3", 0.85714)],
    [("SIF4", 0.16), ("NH3", 0.84)],
    [("SIF4", 0.18), ("NH3", 0.82)],
    [("SIF4", 0.20), ("NH3", 0.80)],
    [("SIF4", 0.22), ("NH3", 0.78)],
]
# create the gas precursors mixture
precursors = Stream(MyCVDMech)
# set fuel composition: hydrogen diluted by nitrogen
# use the first inlet composition to instantiate the inlet stream object
precursors.x = inlet_recipes[0]
# setting the pressure and temperature is not required in this case
precursors.pressure = 1.8 * ck.P_TORRS
precursors.temperature = 1713.15  # inlet temperature
precursors.sccm = 1.13e4  # cm3/sec @ standard condition
```

### Create the PSR to predict the gas composition of the outlet stream

Instantiate the PSR `cvd_reactor` as a `PSRSetVolumeFixedTemperature` object. Since the goal is to show how precursor stream composition affects the Si-N deposition rate, both the gas and the surface temperatures are fixed. The inlet precursor mixture `precursors` composition will be applied as the estimated reactor condition of the `cvd_reactor` by default. You can overwrite any estimated reactor conditions by using appropriate methods. For example, `cvd_reactor.temperature = 2000.0` changes the estimated reactor temperature from 1713.15 to 2000 [K]. The residence time is calculated from the inlet steam velocity and the PSR volume.

```
cvd_reactor = Psr(precursors, label="CVD_PSR")
```

### Set up additional reactor model parameters

Before you can run the simulation, you must provide reactor parameters, solver controls, and output instructions. For a steady-state PSR, you must provide either the residence time or the reactor volume. You can also make changes to any estimated reactor conditions, such as the reactor temperature, the gas composition, and the surface coverage, to improve the solver convergence performance in addition to the solver parameters.

```
# set PSR volume (cm3): required for ``PSRSetVolumeFixedTemperature`` model
cvd_reactor.volume = 2000.0
```

### Connect the inlet to the reactor

You must connect at least one inlet to the open reactor. Use the `set_inlet()` method to add a stream to the PSR. Inversely, use the `remove_inlet()` to disconnect an inlet from the PSR.

**Note**

There is no limit on the number of inlets that can be connected to a PSR.

```
# connect the inlet to the reactor
cvd_reactor.set_inlet(precursors)
```

### Set up surface chemistry parameters

When the `Mixture` or the `Stream` used to initiate the reactor model includes *surface chemistry*, the surface chemistry data and properties will automatically set up by the reactor model. However, you need to provide some essential surface chemistry related model parameters. For example, you need to specify the activate surface area as well as the temperature for each material defined in the surface mechanism if it is different from the gas temperature. By default, all site species have the same fraction and all bulk have activity of unity. You can explicitly provide the site fractions and bulk activities when the default surface coverage fail to get a converged solution.

```
# set PSR total active internal surface area (cm2):
cvd_reactor.area = 950.0
# get the number of surface material defined in the surface mechanism
n_material = cvd_reactor.get_numb_material
# get all surface material names
cvd_material_names = cvd_reactor.get_material_names()
# set surface area fraction (by default, materials have the same area)
area_frac = 1.0 / n_material
# site species symbols: list[list[str]]
site_names = cvd_reactor.get_site_species_names()
# bulk species symbols: list[list[str]]
bulk_names = cvd_reactor.get_bulk_species_names()
# list the material names and the site species and the bulk species symbols
# per material
print(f"number of surface materials = {n_material}")
for m in range(n_material):
    print(f"material names: {cvd_material_names[m]}")
    print(f"   list of site species:\n     {site_names[m]}")
    print(f"   list of bulk species:\n     {bulk_names[m]}")

# set up surface coverage of all surface materials
site_recipe = [[("HN_SIF(S)", 0.5), ("HN_NH2(S)", 0.5)]]
bulk_recipe = [[("SI(D)", 1.0), ("N(D)", 1.0)]]
# set the surface condition of the surface material
for i, mname in enumerate(cvd_material_names):
    # set area fraction [-] (optional: by default, materials have the same area)
    cvd_reactor.set_material_area_fraction(mname, area_frac)
    # set material temperature [K] if it is different from the gas temperature
    # (optional: by default, it is the same as the mixture temperature)
    cvd_reactor.set_material_temperature(mname, precursors.temperature)
    # set reactor surface coverage of the material
    # site fractions (optional: by default, all sites have the same fraction)
    cvd_reactor.set_site_fraction(mname, site_recipe[i])
    # bulk activities (optional: by default, all bulk species activities = 1.0)
    cvd_reactor.set_bulk_activity(mname, bulk_recipe[i])
# scale surface reaction rates
```

(continues on next page)

(continued from previous page)

```
cvd_reactor.surface_ratemultiplier = 1.0
```

### Set steady-state solver controls

You can overwrite the default solver controls by using solver-related methods, such as those for tolerances. Here the tolerances that the steady-state solver is to use for the *steady-state search* and the *pseudo time stepping* stages are changed by using the `steady_state_tolerances` method. Sometimes, during the iterations, some species mass fractions might become negative, causing the solver to report an error and stop. To overcome this issue, you can use the `set_species_floor()` method to provide a small cushion, allowing species mass fractions to go slightly negative (during the iterations) by resetting the mass fraction floor value.

```
# reset the tolerances in the steady-state solver
cvd_reactor.steady_state_tolerances = (1.0e-12, 1.0e-8)
cvd_reactor.timestepping_tolerances = (1.0e-9, 1.0e-6)
# reset the gas species floor value in the steady-state solver
cvd_reactor.set_species_floor(-1.0e-10)
```

### Run the inlet precursor composition parameter study

In the parameter study, the composition of the inlet $SiF_4$molar ratio is done by resetting the molar composition of the inlet stream `precursors` with the recipes of the `inlet_recipes` list.

Use the `process_solution()` method to convert the result from each PSR run into a solution `Stream`. The surface site fractions and the bulk growth rates can be extract from the solution stream `solnmixture` with the `surface_chemistry` methods `get_all_site_frac()` and `get_all_bulk_growth_rates()`, respectively. You can use the `rop_surf()` method to get the surface ROP of the gas species, the site species, and the bulk species of a surface material.

**Note**

In Pychemkin, the surface material is referred by its name. Instead of the surface material index, you loop over the surface material names. Use `get_material_names()` method to get a list of surface material names defined in the surface mechanism.

```
# solution arrays
numb_runs = len(inlet_recipes)
# get gas-species index
i_sif4 = precursors.get_specindex("SIF4")
i_nh3 = precursors.get_specindex("NH3")
i_hf = precursors.get_specindex("HF")
# get surface species index
i_hn_sif_global, i_hn_sif_local = precursors.get_surf_specindex("HN_SIF(S)")
i_f2sinh_global, i_f2sinh_local = precursors.get_surf_specindex("F2SINH(S)")
i_hn_fsinh_2_global, i_hn_fsinh_2_local = precursors.get_surf_specindex("HN(FSINH)2(S)")
i_si_d_global, i_si_d_local = precursors.get_surf_specindex("SI(D)")
# solution arrays
# SiF4 inlet mole fraction
sif4_inlet = np.zeros(numb_runs, dtype=np.double)
# staedy-state gas mole fraction of HF
hf_ss_solution = np.zeros_like(sif4_inlet, dtype=np.double)
# steady-state site fraction of HN_SiF(S)
```

(continues on next page)

(continued from previous page)

```
hn_sif_s_ss_solution = np.zeros_like(sif4_inlet, dtype=np.double)
# steady-state site fraction of F2SiNH(S)
f2sinh_s_ss_solution = np.zeros_like(sif4_inlet, dtype=np.double)
# steady-state site fraction of HN(FSiNH)2(S)
hn_fsinh_2_s_ss_solution = np.zeros_like(sif4_inlet, dtype=np.double)
# net surface rate of production [mole/cm2-sec] of NH3
nh3_rop_solution = np.zeros_like(sif4_inlet, dtype=np.double)
sif4_rop_solution = np.zeros_like(sif4_inlet, dtype=np.double)
# linear deposition rate [micro/min] of Si(D)
si_d_growth_solution = np.zeros_like(sif4_inlet, dtype=np.double)
# set the start wall time
start_time = time.time()
# loop over all inlet precursor compositions
for i, r in enumerate(inlet_recipes):
    # update inlet stream composition and the estimated reactor gas composition
    precursors.x = r
    cvd_reactor.reset_inlet(precursors)
    cvd_reactor.reset_estimate_composition(r)
    # reset estimated surface coverage
    for n, mname in enumerate(cvd_material_names):
        cvd_reactor.set_site_fraction(mname, site_recipe[n])
        cvd_reactor.set_bulk_activity(mname, bulk_recipe[n])
    # run the PSR model
    runstatus = cvd_reactor.run()
    # check run status
    if runstatus != 0:
        # Run failed.
        print(Color.RED + ">>> Run failed. <<<", end=Color.END)
        exit()
    # Run succeeded.
    print(Color.GREEN + ">>> Run completed. <<<", end=Color.END)
    # postprocess the solution profiles
    solnmixture = cvd_reactor.process_solution()
    # print the steady-state solution values
    # print(f"Steady-state temperature = {solnmixture.temperature} [K].")
    # solnmixture.list_composition(mode="mole")
    # store solution values
    sif4_inlet[i] = precursors.x[i_sif4]
    # gas species solution
    hf_ss_solution[i] = solnmixture.x[i_hf]
    # get surface properties (use local index)
    s_frac = solnmixture.surface_chemistry.get_all_site_frac()
    b_rate = solnmixture.surface_chemistry.get_all_bulk_growth_rates()
    # surface site species fraction
    hn_sif_s_ss_solution[i] = s_frac[i_hn_sif_local]
    f2sinh_s_ss_solution[i] = s_frac[i_f2sinh_local]
    hn_fsinh_2_s_ss_solution[i] = s_frac[i_hn_fsinh_2_local]
    # linear growth rate of bulk species [cm/sec] -> [micron/min]
    si_d_growth_solution[i] = b_rate[i_si_d_local] * 6.0e5
    # get gas species surface chemistry ROP [mole/cm2-sec]
    nh3_rop_solution[i] = 0.0
    sif4_rop_solution[i] = 0.0
```

(continues on next page)

(continued from previous page)

```
    for m in cvd_material_names:
        surf_rop, _ = solnmixture.rop_surf(m)
        # gas species net rate of production due to surface chemistry
        nh3_rop_solution[i] += surf_rop[i_nh3]
        sif4_rop_solution[i] += surf_rop[i_sif4]

# compute the total runtime
runtime = time.time() - start_time
print(f"Total simulation duration: {runtime} [sec] over {numb_runs} runs.")

# clean up
ck.done()
```

### Plot the parameter study results

Plot the steady-state PSR solution against the inlet $SiF_4$ mole fraction. You should see that both the Si(D) growth rate and the $NH_3$ surface *consumption rate* increase with the increasing inlet $SiF_4$ mole fraction.

```
plt.subplots(2, 2, sharex="col", figsize=(12, 6))
plt.suptitle("PSR CVD Reactor Solution", fontsize=16)
plt.subplot(221)
plt.plot(sif4_inlet, hf_ss_solution, "b-")
plt.ylabel("HF Mole Fraction")
plt.subplot(222)
plt.plot(sif4_inlet, hn_sif_s_ss_solution, "b-")
plt.plot(sif4_inlet, f2sinh_s_ss_solution, "r-")
plt.plot(sif4_inlet, hn_fsinh_2_s_ss_solution, "g-")
plt.legend(["HN_SiF(S)", "F2SiNH(S)", "HN(FSiNH)2(S)"], loc="lower right")
plt.yscale("log")
plt.ylabel("Site Fraction")
plt.subplot(223)
plt.plot(sif4_inlet, nh3_rop_solution, "g-")
plt.plot(sif4_inlet, sif4_rop_solution, "r-")
plt.legend(["NH3", "SIF4"], loc="lower left")
plt.xlabel("Inlet SiF4 Mole Fraction")
plt.ylabel("Surface ROP [mole/cm2-sec]")
plt.subplot(224)
plt.plot(sif4_inlet, si_d_growth_solution, "b-")
plt.xlabel("Inlet SiF4 Mole Fraction")
plt.ylabel("Si(D) Linear Growth Rate [micron/min]")
# plot results
if interactive:
    plt.show()
else:
    plt.savefig("plot_PSR_cvd.png", bbox_inches="tight")
```

## 22.11.2 Simulating CH4 catalytic Combustion process

Catalytic combustion was once an promising technology of ultra low NOx emission for gas power generation. The catalytic combustion process converts the gas-phase reactants to the gas-phase products on the catalyst surface while releasing heat, it consists of three major steps: (1) small fuel species and oxygen get adsorbed onto the catalyst surface, (2) the absorbed species react on the catalyst surface, and (3) the products such as carbon dioxide and water get released

from the surface into the gas stream. Catalytic combustion can exhibit something similar to the gas-phase "ignition" phenomenon called "light-off" when the net heat release from the surface reactions suddenly spikes. This is where the catalytic combustion process transitions from being kinetic-limited to being transport-limited. Hence, this is why it is appropriate to use the PFR model in this project. By going through the "surface route", the combustion process can avoid triggering the NOx formation reaction pathways, and the "flame" is anchored at the "light-off" location on the catalyst surface.

Catalytic combustion applications might utilize a two PFRs in series configuration for large fuel species. The first PFR is for pure gas-phase reactions called the pre-oxidizer, its main purpose is to partially break down the large hydrocarbon fuel species for the downstream catalytic combustor. The second PFR allows both gas-phase and surface reactions where the broken down fuel species are oxidized on the catalyst surface to generate heat.

This project attempts to obtain the solution profiles in order to catch the surface light-off in the downstream catalytic combustor. Two `Chemistry Set` are created in this project. The first `mech_no_surface` chemistry set is for the first PFR, the `pre_burner`, and it is a modified GRI 3.0 combustion mechanism with the addition of the platinum element (PT). For the downstream `cat_combustor`, the chemistry set `mech_catalytic` includes the surface mechanism "surf_CH4_Cat_Combust_PT.inp" as well as the same gas mechanism for the `mech_no_surface` chemistry set. The gas-phase solution at the exit of the `pre_burner` can be applied to set up the inlet stream of the `cat_combustor` as long as they use the same gas-phase mechanism.

### Import PyChemkin packages and start the logger

```
from pathlib import Path
import time

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color

# Chemkin PFR model (steady-state)
from ansys.chemkin.core.flowreactors.PFR import PFREnergyConservation as Pfr
from ansys.chemkin.core.inlet import Mixture, Stream
from ansys.chemkin.core.logger import logger

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create the chemistry sets

This example uses the methane catalytic combustion mechanism on platinum that comes with the standard Chemkin example, `reactor_network__two_stage_catalytic_combustor`. The reaction mechanism consists of two parts: the gas-phase mechanism, `chem_GRImech3_PT.inp`, describing the gas-phase combustion of methane in air; and the

surface mechanism, `surf_CH4_Cat_Combust_PT.inp`, describing the reactions lead to the light-off (surface ignition) of the adsorbed methane and the adsorbed oxygen (or surface C and O atoms) on the platinum catalyst.

The mechanisms are provided in the *examplesdata* folder of the local *PyChemkin* installation. You must provide the mechanism file names, `chem_GRImech3_PT.inp` and `surf_CH4_Cat_Combust_PT.inp` with the correct path `example_mech_data` to the Chemistry Set before pre-processing. The thermodynamic data file from the default Ansys Chemkin installation will be used.

```
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
# find the location of this example py file
try:
    script_dir_obj = Path(__file__).parent.resolve()
except NameError:
    script_dir_obj = Path(current_dir)
script_dir = str(script_dir_obj)
# use the relative path to locate the example mechanism folder
example_mech_data = script_dir_obj / ".." / "data"
print(f"example data = {str(example_mech_data)}")
```

### Set up and run the gas phase only reactor

Create a chemistry set based on the GRI 3.0 methane combustion mechanism with no surface mechanism input.

```
# gas-phase chemistry only
mech_no_surface = ck.Chemistry(label="GRI3")
# set mechanism input files
# including the full file path is recommended
mech_no_surface.chemfile = str(data_dir / "grimech30_chem.inp")
mech_no_surface.thermfile = str(data_dir / "grimech30_thermo.dat")
```

### Preprocess the gas-phase only chemistry set

preprocess the mechanism files.

```
_ = mech_no_surface.preprocess()
```

### Set up the lean premixed fuel-air stream

Instantiate a stream feed for the inlet gas mixture. The stream is a mixture with the addition of the inlet flow rate. You specify inlet gas properties in the same way as you set up a mixture.

Use the `x` method of the `Stream` object to specify the natural gas composition with a `recipe`. In this project, the `sccm` method, [cm3/min] @ the standard condition, is used set the inlet volumetric flow rate. The `air` composition is copied from the pre-defined `Air` object in Pychemkin. To create the premixed fuel-air inlet stream with the given equivalence ratio, the `x_by_equivalence_ratio` method is applied.

Once the inlet stream `lean_premix` is created correctly, it is used to instantiate the gas-only PFR `pre-burner`. Provide the necessary input parameters and run the PFR. Use the `get_last_solution_mixture()` method to get the solution `Stream` `preburner_exhaust` at the exit of the `pre_burner`. `preburner_exhaust` will be used to define the inlet stream properties of the down stream catalytic combustor `cat_combustor`.

```
# set the fuel composition and conditions
natural_gas = Mixture(mech_no_surface)
natural_gas.pressure = 2.5 * ck.P_ATM
natural_gas.temperature = 890.0
```

(continues on next page)

(continued from previous page)

```
natural_gas.x = [
    ("CH4", 0.971),
    ("C2H6", 0.0092),
    ("C3H8", 0.0051),
    ("CO2", 0.0053),
    ("N2", 0.0094),
]

# set the air composition and conditions
air = Mixture(mech_no_surface)
air.pressure = natural_gas.pressure
air.temperature = natural_gas.temperature
air.x = ck.Air.x()

# create the lean premixed natural gas-air mixture for the pre-burner
lean_premix = Stream(mech_no_surface)
lean_premix.pressure = natural_gas.pressure
lean_premix.temperature = natural_gas.temperature
# set stream velocity [cm/sec]
lean_premix.velocity = 60.0
# set equivalence ratio
equiv = 0.3
# products from the complete combustion of the natural gas and the air
products = ["CO2", "H2O", "N2"]
# species mole fractions of added/inert mixture.
# You can also create an additives mixture here.
add_frac = np.zeros(mech_no_surface.kk, dtype=np.double)  # no additives: all zeros
# use the equivalence ratio to set the stream composition
ierror = lean_premix.x_by_equivalence_ratio(
    mech_no_surface, natural_gas.x, air.x, add_frac, products, equivalenceratio=equiv
)
# check fuel-oxidizer mixture creation status
if ierror != 0:
    print("Error: Failed to create the fuel-oxidizer mixture.")
    exit()

# instantiate the pre-burner object with the lean premixed natural gas-air stream
pre_burner = Pfr(lean_premix, label="Pre-Burner")
# pre-burner parameters
# reactor length [cm]
pre_burner.length = 100.0
# reactor diameter [cm]
pre_burner.diameter = 12.0
# solver parameters
pre_burner.force_nonnegative = True

# set the start wall time
start_time = time.time()
# run the pre-burner PFR model
runstatus = pre_burner.run()
# check run status
if runstatus != 0:
```

(continues on next page)

(continued from previous page)

```
    # Run failed.
    print(Color.RED + ">>> Run failed. <<<", end=Color.END)
    exit()
# Run succeeded.
print(Color.GREEN + ">>> Run completed. <<<", end=Color.END)

# post process the homogeneous stage solution
pre_burner.process_solution()
# get the number of solution time points
n_points = pre_burner.getnumbersolutionpoints()

# the exit/last solution stream from the pre-burner becomes the feed stream of the
# downstream catalytic combustor
preburner_exhaust = pre_burner.get_last_solution_mixture()
# list the composition of the unburned fuel-air mixture
print(f"preburner exhaust temperature = {preburner_exhaust.temperature} [K]")
print(f"preburner exhaust mass flow rate = {pre_burner.mass_flowrate} [g/sec]")
# preburner_exhaust.list_composition(mode="mole")

# save the mass flow rate [g/cm]
preburner_exhaust_mflrt = pre_burner.mass_flowrate
```

### Set up and run the catalytic reactor

Create a new chemistry set based on the GRI 3.0 methane combustion mechanism with the addition of the catalytic combustion surface mechanism.

```
mech_catalytic = ck.Chemistry(label="catalytic")
# set mechanism input files
# including the full file path is recommended
local_data_dir = "CH4_Cat_Combust"
mech_catalytic.chemfile = str(
    example_mech_data / local_data_dir / "chem_GRImech3_PT.inp"
)
mech_catalytic.surffile = str(
    example_mech_data / local_data_dir / "surf_CH4_Cat_Combust_PT.inp"
)
mech_catalytic.thermfile = str(data_dir / "grimech30_thermo.dat")
```

### Preprocess the catalytic combustion chemistry set

preprocess the mechanism files.

```
_ = mech_catalytic.preprocess()
```

### Set up the inlet stream for the catalytic combustor

Use the exhaust mixture from the `pre-burner` to define the inlet stream `cat_feed` for the catalytic combustor `cat_combustor`. Even though the `Chemistry Sets` for the two PFR have the same gas phase mechanism, you should avoid applying or copying the `preburner_exhaust` object to instantiate `cat_combustor` because `preburner_exhaust` is obtained from the `pre_burner` which does not have any surface chemistry data. You should assign individual properties of the `cat_feed` from the `preburner_exhaust`.

```
cat_feed = Stream(mech_catalytic)

# use the exhaust mixture (the solution) from the gas-phase only pre-burner
# to set up the feed stream properties of the catalytic combustor
cat_feed.x = preburner_exhaust.x
cat_feed.pressure = preburner_exhaust.pressure
cat_feed.temperature = preburner_exhaust.temperature
# the mass flow rate of the feed stream to the catalytic burner
# equals the mass flow rate out of the gas phase only pre-burner
cat_feed.mass_flowrate = preburner_exhaust_mflrt
# list the composition of the unburned fuel-air mixture
print(f"cat burner feed temperature = {cat_feed.temperature} [K]")
print(f"cat burner feed mass flow rate = {cat_feed.mass_flowrate} [g/sec]")
# cat_feed.list_composition(mode="mole")
```

### Create the PFR to predict the gas composition of the outlet stream

Instantiate the PFR `cat_combustor` as a `PFREnergyConservation` object.

```
cat_combustor = Pfr(cat_feed, label="cat_combustor")
```

### Set up additional reactor model parameters

Before you can run the simulation, you must provide reactor parameters, solver controls, and output instructions. For PFR, you must provide either the reactor length, the reactor diameter (or, separately, the flow area and the surface area per reactor length). You can also use profiles to assign certain reactor conditions, such as the reactor temperature, the reactor pressure, and the surface area.

```
# catalytic combustor parameters
# reactor length [cm]
cat_combustor.length = 10.0
# reactor cross-sectional flow area [cm2]
cat_combustor.flowarea = 5.0
# reactor chemically active surface area per reactor length [cm2/cm]
# cat_combustor.area = 3000.0
area_prof_x = [0.0, 0.5, 1.0, 25.0]
area_prof_a = [2.0e2, 1.0e3, 1.0e4, 1.0e4]
cat_combustor.set_surfacearea_profile(area_prof_x, area_prof_a)
# specify the preactor ressure profile to simulate the pressure drop across
# the reactor length
# x: [cm]
pres_prof_x = [0.0, 25.0]
# pressure: [dynes/cm2]
pres_prof_p = [2.5 * ck.P_ATM, 2.48 * ck.P_ATM]
cat_combustor.set_pressure_profile(pres_prof_x, pres_prof_p)
```

### Set up surface chemistry parameters

When the `Chemistry Set` of the `Stream` includes surface chemistry, you need to provide some essential surface chemistry related model parameters. For example, you need to specify the activate surface area as well as the temperature for each material defined in the surface mechanism. You can also provide the surface coverage, site fractions and bulk activities, of the materials at the PFR inlet to improve convergence performance. Use the `set_site_fraction()` and

the set_bulk_activity() method to specify estimated site fractions and bulk activities at the reactor entrance (x = 0). By default, all site species have the same fraction and all bulk species activity is set to 1.

You can use surface chemistry methods such as get_material_names()` and ``get_site_species_names() to obtain the material names and the surface site species symbols, respectively.

```
# get the number of surface material defined in the surface mechanism
n_material = cat_combustor.get_numb_material
# get all surface material names
cat_material_names = cat_combustor.get_material_names()
# set surface area fraction (by default, materials have the same area)
# site species symbols: list[list[str]]
site_names = cat_combustor.get_site_species_names()
# bulk species symbols: list[list[str]]
bulk_names = cat_combustor.get_bulk_species_names()
# list the material names and the site species and the bulk species symbols
# per material
print(f"number of surface materials = {n_material}")
for m in range(n_material):
    print(f"material names: {cat_material_names[m]}")
    print(f"   list of site species:\n     {site_names[m]}")
    print(f"   list of bulk species:\n     {bulk_names[m]}")

# set up surface coverage of all surface materials
site_recipe = [[("O(S)", 1.0), ("PT(S)", 0.0)]]
bulk_recipe = [[("PT(B)", 1.0)]]
# set the surface condition of the surface material
for i, mname in enumerate(cat_material_names):
    # set area fraction [-] (optional: by default, materials have the same area)
    # set material temperature [K] if it is different from the gas temperature
    # (optional: by default, it is the same as the mixture temperature)
    # cat_combustor.set_material_temperature(mname, 1000.0)
    # set reactor surface coverage of the material
    # site fractions (optional: by default, all sites have the same fraction)
    cat_combustor.set_site_fraction(mname, site_recipe[i])
    # bulk activities (optional: by default, all bulk species activities = 1.0)
    cat_combustor.set_bulk_activity(mname, bulk_recipe[i])
# scale surface reaction rates
cat_combustor.surface_ratemultiplier = 1.0
```

### Set solver controls

You can overwrite the default solver controls by using solver-related methods, such as those for tolerances.

```
# set solver the tolerances
cat_combustor.tolerances = (1.0e-8, 1.0e-6)
# solver non-negative mode
cat_combustor.force_nonnegative = False
# transient solver max time step size [cm]
# cat_combustor.set_solver_max_timestep_size(1.0e-2)
# set adaptive solution saving distance
cat_combustor.adaptive_solution_saving(mode=True, steps=20)
```

### Run the catalytic combustor reactor model

Once all the necessary input parameters are provided, run the `cat_combustor` with the `run()` method. Use the `process_solution()` method to retrieve the raw solution profiles. If you are interested in the exit solution, use the `get_last_solution_mixture()` method to get the solution at the exit as a `Stream` object `combustor_exhaust`.

**Note**

In Pychemkin, the surface material is referred by its name. Instead of the surface material index, you loop over the surface material names. Use `get_material_names()` method to get a list of surface material names defined in the surface mechanism.

**Note**

For a site or a bulk species, use any `Mixture` or `Stream` created with the `Chemistry Set` containing the surface mechanism to get its species index. There are two type of index for surface species, the global index includes all species (the gas plus the site and the bulk of all materials); the local index includes the same type of surface species (site or bulk) of all materials.

```
# get gas-species index
i_co = cat_feed.get_specindex("CO")
i_ch4 = cat_feed.get_specindex("CH4")
i_no = cat_feed.get_specindex("NO")
# get surface species index: <global index> <local index>
# global index includes all species: the gas, the site of all materials, and the bulk
# of all materials
# local index includes the same type of species (site or bulk) of all materials
i_h_s_global, i_h_s_local = cat_feed.get_surf_specindex("H(S)")
i_pt_s_global, i_pt_s_local = cat_feed.get_surf_specindex("PT(S)")
# run the catalytic combustor PFR model
runstatus = cat_combustor.run()
# check run status
if runstatus != 0:
    # Run failed.
    print(Color.RED + ">>> Run failed. <<<", end=Color.END)
    exit()
# Run succeeded.
print(Color.GREEN + ">>> Run completed. <<<", end=Color.END)
# compute the total runtime
runtime = time.time() - start_time
print(f"Total simulation duration: {runtime} [sec].")
# postprocess the solution profiles
cat_combustor.process_solution()
# get the exit/last solution stream from the catalytic combustor
combustor_exhaust = cat_combustor.get_last_solution_mixture()
# display the gas composition at the reactor exit
combustor_exhaust.list_composition(mode="mole")
# get the number of data points in the solution profiles
n_points = cat_combustor.getnumbersolutionpoints()
print(f"number of solution time points = {n_points}")
# store solution profiles
```

(continues on next page)

(continued from previous page)

```
# distance from the reactor entrance [cm]
distance = cat_combustor.get_solution_variable_profile("distance")  #
# temperature solution profile along the reactor length [K]
temp_profile = cat_combustor.get_solution_variable_profile("temperature")
# total heat release rate from the gas-phase reactions [erg/sec]
gashrr_profile = cat_combustor.get_solution_variable_profile("gashrr")
gashrr_profile /= 1.0e3 * ck.ERGS_PER_JOULE
# total heat release rate from all surface reactions [erg/sec]
surfhrr_profile = cat_combustor.get_solution_variable_profile("surfhrr")
surfhrr_profile /= 1.0e3 * ck.ERGS_PER_JOULE
# gas phase CO solution profile [mass fraction]
co_profile = cat_combustor.get_solution_variable_profile("CO")
# gas phase CH4 solution profile [mass fraction]
ch4_profile = cat_combustor.get_solution_variable_profile("CH4")
# gas phase NO solution profile [mass fraction]
no_profile = cat_combustor.get_solution_variable_profile("NO")
# surface PT(S) site fraction solution profile [site fraction]
pt_s_profile = cat_combustor.get_solution_variable_profile("PT(S)")
# surface OH(S) site fraction solution profile [site fraction]
oh_s_profile = cat_combustor.get_solution_variable_profile("OH(S)")

# clean up
ck.done()
```

### Plot the catalytic combustor solution profiles

Plot the solution profiles along the `cat_combustor` length. You could observe the surface "light-off" point around x = 1.0 by the spikes in the solution profiles. The much bigger peak value of the surface heat release rate in comparison to the peak gas-phase heat release rate indicates the combustion process takes place at the catalyst surface.

```
plt.subplots(2, 2, sharex="col", figsize=(12, 6))
plt.suptitle("PFR Catalytic Combustor Solution", fontsize=16)
plt.subplot(221)
plt.plot(distance, temp_profile, "r-")
plt.ylabel("Temperature [K]")
plt.subplot(222)
plt.plot(distance, gashrr_profile, "b-")
plt.plot(distance, surfhrr_profile, "r-")
plt.legend(["Gas Phase", "Surface"], loc="upper right")
plt.ylabel("Total Heat Release Rate [kJ/sec]")
plt.subplot(223)
plt.plot(distance, co_profile, "g-")
plt.plot(distance, ch4_profile, "r--")
plt.plot(distance, no_profile, "b:")
plt.legend(["CO", "CH4", "NO"], loc="upper right")
plt.yscale("log")
plt.xlabel("Distance [cm]")
plt.ylabel("Gas Mass Fraction")
plt.subplot(224)
plt.plot(distance, pt_s_profile, "b-")
plt.plot(distance, oh_s_profile, "g:")
plt.legend(["PT(S)", "OH(S)"], loc="lower right")
```

(continues on next page)

(continued from previous page)

```
plt.yscale("log")
plt.xlabel("Distance [cm]")
plt.ylabel("Surface Site Fraction")
# plot results
if interactive:
    plt.show()
else:
    plt.savefig("plot_PFR_catalytic_combustion.png", bbox_inches="tight")
```

### 22.11.3 Processing a surface mechanism with multiple materials

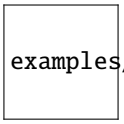
When the surface mechanism input file is added to the *Chemistry Set* via the `surffile` parameter, the `preprocess()` method will process the surface mechanism in the file. Typically, a surface mechanism consists of one surface *material* block. The *material* block defines the surface phases, the site and the bulk surface species of the surface phases, the thermodynamic data of all surface (site and bulk) species, and the surface reactions describing the chemical interactions between the gas-phase species and the surface species and between the surface species of the surface material. The gas phase is always the first phase of any surface material followed by all the site phases and then all the bulk phases of the material. The species are sorted in two different ways: the *global order* and the *local order*. The *global species index/order* combines species from all phases and arranges them in the order of gas phase species, species from all site phases of the material, followed by species from all bulk phases of the material. The *local species index/order* lists the species from the same type of surface phase. For example, local index of all site species of the surface material. Here is a simple graph showing different species arrangements of a surface material:




When there are multiple surface materials defined in the surface mechanism input file, all surface materials share the same gas phase (gas species and gas-phase reactions). Each surface material consists of its own surface phases, surface species, and surface reactions, and has independent *global* and *local* species orders/indices. Subsequently, you can access species and reaction properties one surface material at a time, and direct surface species interactions across different surface materials are *not* supported.

For detailed information about the *Surface Chemkin*, you can check out the *Chemkin theory* and the *Chemkin Input* manuals at the Ansys Product Help website.

When a `Chemistry Set` with surface chemistry is used to initiate a `Mixture` or a `Stream` object, Pychemkin will automatically set up the surface chemistry sub-module of the `Mixture` or the `Stream` object. Similarly, the surface chemistry and the surface governing equations will be ready when a `ReactorModel` is initiated with a surface chemistry enabled `Mixture` or `Stream`. There is no additional steps required to manually "activate" the surface chemistry in Pychemkin objects other than the `Chemistry Set`. The figure below shows the initialization process of the "surface chemistry modules" in different Pychemkin objects:




This example demonstrates the procedures of preprocessing a `Chemistry Set` that supports surface chemistry. It also shows how to access properties from two different surface materials. Typically, you would want to get information about the symbols, the index, the density, the molecular weights, and the elemental compositions of the species. For site species, you can get the "site occupancy". For bulk species, you can get the "enthalpy" and the "specific heat capacity".

```
```

### Import PyChemkin packages and start the logger

```
from pathlib import Path

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core.logger import logger

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = False
```

### Create a chemistry set with surface chemistry

This example uses mechanism files from the "examplesdata" folder, and assumes that this Python example file is located in the "examplessurface_chemistry" folder. Please verify the location and the availability of these files and modify the directories below to point to the correct files before running this example.

```
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
# find the floder where this py example file is located
try:
    script_dir_obj = Path(__file__).parent.resolve()
except NameError:
    script_dir_obj = Path(current_dir)
script_dir = str(script_dir_obj)
# use the relative path to find the location of the example mechanism files
example_mech_data = script_dir_obj / ".." / "data"
print(f"example data = {str(example_mech_data)}")
# create a chemistry set based on the gasoline 14-components mechanism
MySurfMech = ck.Chemistry(label="multi_materials")
# set mechanism input files
# including the full file path is recommended
local_data_dir = "CH4_MB89"
MySurfMech.chemfile = str(example_mech_data / local_data_dir / "chem_ch4_MB89.inp")
MySurfMech.surffile = str(example_mech_data / local_data_dir / "surf_ch4_MB89.inp")
MySurfMech.thermfile = str(data_dir / "therm.dat")
```

### Preprocess the surface chemistry set

preprocess the mechanism files

```
_ = MySurfMech.preprocess()
```

### Access surface chemistry information in the Chemistry Set

Access the *Material* and the *SurfacePhase* objects in the *Chemistry Set* containing surface mechanism by their names given in the surface mechanism input file. For example, `MySurfMech.materials['CLEANUP']` for the `Material` object associated with the surface material 'CLEANUP' defined in the surface mechanism input file, or `MySurfMech.materials['INITIATE'].phases['QUARTZ']` for the `SurfacePhase` object associated with the 'QUARTZ' phase of the surface material 'INITIATE'.

You can get the surface material names by using the `MySurfMech.material_names` method. The `Material` object provides information about the surface material such as the number of surface species and surface reactions, the surface phases, and the properties of the site species and the bulk species. Some of the available properties in the `Material` and the `SurfacePhase` are shown below.

```
# the number of surface materials in the Chemistry Set
print(f"Total number of surface materials = {MySurfMech.number_materials}")
# get all surface material names in the Chemistry Set with surface chemistry
m_symbols = MySurfMech.material_names
# use the material name to access the corresponding ``Material`` object
# in the Chemistry Set
# get the names of all phases (gas phase and surface (site and bulk) phases)
for s in m_symbols:
    print(
        f"Material '{s.rstrip()}' "
        f"contains phases: {MySurfMech.materials[s].phase_names}\n"
    )
```

### Information about the surface material, phase, and species

Display surface material phase and species information as well as the surface reactions of the material.

```
# list the commonly used properties of all surface materials of the Chemistry Set
for n, name in enumerate(m_symbols):
    # get the surface Material object by its name
    m = MySurfMech.materials[name]
    print("=" * 80)
    print(f"\nMaterial index = {n}")
    # display the general size information of the material
    m.information()
    print("=" * 40)
    # list the elemental compositions of all site species of the material
    print("elemental compositions of surface species on the material:")
    if m.num_site_species > 0:
        for i, p in enumerate(m.site_species_names):
            print(f"  site species {p.rstrip()}")
            k = m.site_species_map[p]
            for e, ele in enumerate(MySurfMech.element_symbols):
                print(f"    element {ele} = {m.material_species_composition(e, k)}")
        h_site = m.get_site_species_h(temp=400.0)
        for k, h in enumerate(h_site):
            # local index among all site species of this material
            print(
                f"  site species {k} {m.site_species_names[k]} "
                f"H = {h / ck.ERGS_PER_JOULE} [J/mole]"
            )
```

(continues on next page)

(continued from previous page)

```
    # list the elemental compositions of all bulk species of the material
    if m.num_bulk_species > 0:
        for i, p in enumerate(m.bulk_species_names):
            print(f"  bulk species {p.rstrip()}")
            k = m.bulk_species_map[p]
            for e, ele in enumerate(MySurfMech.element_symbols):
                print(f"    element {ele} = {m.material_species_composition(e, k)}")
        # also the specific heat capacity of the bulk species
        cp_bulk = m.get_bulk_species_cp(temp=450.0)
        for k, c in enumerate(cp_bulk):
            # local index among all bulk species of this material
            print(
                f"  bulk species {k} {m.bulk_species_names[k]} "
                f"Cp = {c / ck.ERGS_PER_JOULE} [J/mole-K]"
            )
    # get all phase names of the Chemistry Set, the "Gas" phase is always included
    # as the first phase
    p_symbols = m.phase_names
    # list the surface phase types and the index of the first species of the
    # surface phase.
    # There are three phase types: "Gas", "site", and "bulk". The surface species
    # (site and bulk) have two indices: global (of all species including the gas
    # species) and local (of the same type of surface species).
    for p in p_symbols:
        if p == "Gas":
            continue
        phase_type = m.phases[p].phase_type
        #
        if phase_type == "site":
            print(
                f" surface site phase: {p} "
                f"site density = {m.phases[p].site_density} [mole/cm2]\n"
            )
            # the global index is 1-based in Chemkin
            print(
                f" 1-base species index on phase {p} = "
                f"{m.phases[p].first_species_index}"
            )
            # the global index of the material, the phase, and the species
            # is 0-base in PyChemkin
            print(
                f" 0-base species index on site phase {p} "
                f"= {m.first_site_species_index}"
            )
        else:
            # the global index is 1-based in Chemkin
            print(
                f" 1-base species index on phase {p} = "
                f"{m.phases[p].first_species_index}"
            )
            # the global index of the material, the phase, and the species
            # is 0-base in PyChemkin
```

(continues on next page)

(continued from previous page)

```
            print(
                f" 0-base species index on bulk phase {p} "
                f"= {m.first_bulk_species_index}"
            )

    # list surface reaction information of the material, the reaction index
    # is 1-base in both Chemkin and PyChemkin.
    print("=" * 40)
    # the first surface reaction of the material
    rxn_id = 1
    # reaction string of the first surface reaction of the material
    print(f" surface reaction # 1 = {m.get_surface_reaction_string(rxn_id)}")
    # get the Arrhenius rate parameters of all surface reactions of the material
    a_factor, beta, act_energy = m.get_surface_reaction_parameters()
    # display the Arrhenius parameters of the first surface reaction of this material
    print("    original rate parameters:")
    print(f"    A = {a_factor[0]}   B = {beta[0]}  Ea = {act_energy[0]} [K]")
    # save a copy of the original value
    act_energy_org = act_energy[0]
    # change the activation temperature of the first surface reaction to 3000 [K]
    m.set_surface_reaction_act_energy(rxn_id, 3000.0)
    # retrieve and display the modified Arrhenius parameters
    # of the first surface reaction
    a_factor, beta, act_energy = m.get_surface_reaction_parameters()
    print("    modified rate parameters:")
    print(f"    A = {a_factor[0]}   B = {beta[0]}  Ea = {act_energy[0]} [K]")
    # restore the activation temperature of the first surface reaction
    m.set_surface_reaction_act_energy(rxn_id, act_energy_org)
    # verify the change
    a_factor, beta, act_energy = m.get_surface_reaction_parameters()
    print("    restored rate parameters:")
    print(f"    A = {a_factor[0]}   B = {beta[0]}  Ea = {act_energy[0]} [K]")
```

### 22.11.4 Modeling atomic layer deposition process in a transient PSR

Atomic layer deposition (ALD) is a technique used to deposit thin films of solid materials in a very controlled manner, and differs from CVD primarily in that it is a transient process with the deposition surface being exposed to pulses of alternating gases. Ideally, the deposition chemistry in ALD is self-limiting, with growth occurring in a layer-by-layer manner and the deposition thickness being controlled only by the number of cycles.

The major advantage of ALD over CVD is the improved control over the deposition process and more conformal deposition. The inherently lower deposition rates, however, lead to longer process times and higher costs. During a pulse, it is important that enough molecules react with all parts of the substrate to be coated. But many of the precursor materials are expensive. Thus one of the process optimization goals is to reduce the amount of precursor that flows through the reactor without reacting at the surface.

Normally the perfectly stirred reactor (PSR) model is a 0-D steady-state model with constant reactor pressure. When the dynamic responses of a chemistry dominated process are of interest, performing transient simulations of the system becomes necessary. The transient PSR (transPSR) model of the Ansys Chemkin is a useful tool for the ALD process development. It can be applied for optimizing the pulse sequences and the cycle times of the ALD process to obtain the desired pattern and thickness of the deposit layers. It can also be used to develop and to validate the chemical reaction mechanism for the ALD process.

This project demonstrates the process of setting up and running the transPSR model with multiple inlets of different

flow rate profiles. The alumina ALD chemistry used in this example is described in section 5.4.4 (Alumina ALD) of the Chemkin Tutorial manual.

### Import PyChemkin packages and start the logger

```
from pathlib import Path
import time

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color
from ansys.chemkin.core.inlet import Stream  # external gaseous inlet
from ansys.chemkin.core.logger import logger

# Chemkin 0-D transient PSR model with given reactor volume
from ansys.chemkin.core.stirreactors.transient_PSR import (
    TransientPSRSetVolumeFixedTemperature as TransPsr,
)

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create an instance of the Chemistry Set

The chemistry set describes the atomic layer deposition (ALD) of alumina from trimethylaluminum (TMA) and ozone. This mechanism is deliberately simplistic for illustration purposes only. It demonstrates one way of describing the ALD of alumina, but it should generally be considered as illustrative only and not used as a source of kinetic data for this process. This mechanism is designed to deposit stoichiometric alumina, with three oxygen atoms being deposited for each two aluminum atoms.

The ALD mechanism is provided in the *examplesdata* folder of the local *PyChemkin* installation. You must provide the mechanism file names, `chem_AL2O3_ald.inp` and `surf_AL2O3_ald.inp`, with the correct path `example_mech_data` to the Chemistry Set before pre-processing. The thermodynamic data file from the default Ansys Chemkin installation will be used.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
# find the location of this example py file
try:
    script_dir_obj = Path(__file__).parent.resolve()
except NameError:
```

(continues on next page)

(continued from previous page)

```
    script_dir_obj = Path(current_dir)
script_dir = str(script_dir_obj)
# use the relative path to locate the example mechanism folder
example_mech_data = script_dir_obj / ".." / "data"
print(f"example data = {str(example_mech_data)}")
# create a chemistry set based on the Si-N CVD mechanism
MyALDMech = ck.Chemistry(label="Al2O3_ALD")
# set mechanism input files
# including the full file path is recommended
local_data_dir = "AL2O3_ALD"
MyALDMech.chemfile = str(example_mech_data / local_data_dir / "chem_AL2O3_ald.inp")
MyALDMech.surffile = str(example_mech_data / local_data_dir / "surf_AL2O3_ald.inp")
MyALDMech.thermfile = str(data_dir / "therm.dat")
```

### Preprocess the Chemistry Set

```
# preprocess the mechanism files
ierror = MyALDMech.preprocess()
if ierror != 0:
    print("Error: failed to preprocess the mechanism!")
    print(f"       error code = {ierror}")
    exit()
```

### Set up the inlet streams

Instantiate a stream feed for the inlet gas mixture. The stream is a mixture with the addition of the inlet flow rate. You specify inlet gas properties in the same way as you set up a mixture.

Use the `x` method of the `Stream` object to specify the inlet gas composition with a `recipe`. In this project, the `mass_flowrate` method, [g/sec] is used set the volumetric flow rates of the two inlet streams. The `air` composition is copied from the pre-defined `Air` object in Pychemkin.

```
# set the TMA stream composition and conditions
met_organic = Stream(MyALDMech, label="TMA")
met_organic.pressure = 1.0 * ck.P_TORRS  # 1 [torr]
met_organic.temperature = 620.2  # [K]
met_organic.x = [("ALMe3", 0.01), ("AR", 0.99)]
# TMA stream sccm flow rate profile in pulses
# TMA stream sccm data points
tma_time = np.array(
    [
        0.0,
        0.19,
        0.2,
        5.19,
        5.2,
        5.39,
        5.4,
        10.39,
        10.4,
        10.59,
        10.6,
```

(continues on next page)

(continued from previous page)

```
        15.59,
        15.6,
        15.79,
        15.8,
        20.8,
    ]
)  # [sec]
tma_profile = np.array(
    [
        800.0,
        800.0,
        0.0,
        0.0,
        800.0,
        800.0,
        0.0,
        0.0,
        800.0,
        800.0,
        0.0,
        0.0,
        800.0,
        800.0,
        0.0,
        0.0,
    ]
)  # [sccm]
# set the TMA stream sccm profile
met_organic.set_sccm_flowrate_profile(tma_time, tma_profile)

# set the air composition and conditions
oxidizers = Stream(MyALDMech, label="O3")
oxidizers.pressure = met_organic.pressure
oxidizers.temperature = 620.2
oxidizers.x = [("O2", 0.4), ("O3", 0.05), ("AR", 0.55)]
# oxidizers stream sccm flow rate profile in pulses
# oxidizers stream sccm data points
oxid_time = np.array(
    [
        0.0,
        1.19,
        1.2,
        3.19,
        3.2,
        6.39,
        6.4,
        8.39,
        8.4,
        11.59,
        11.6,
        13.59,
        13.6,
```

(continues on next page)

(continued from previous page)

```
        16.79,
        16.8,
        18.79,
        18.8,
        20.8,
    ]
)  # [sec]
oxid_profile = np.array(
    [
        0.0,
        0.0,
        800.0,
        800.0,
        0.0,
        0.0,
        800.0,
        800.0,
        0.0,
        0.0,
        800.0,
        800.0,
        0.0,
        0.0,
        800.0,
        800.0,
        0.0,
        0.0,
    ]
)  # [sccm]
# set the oxidizers stream sccm profile
oxidizers.set_sccm_flowrate_profile(oxid_time, oxid_profile)

# set the air composition and conditions
purge = Stream(MyALDMech, label="AR")
purge.pressure = met_organic.pressure
purge.temperature = 620.2
purge.x = [("AR", 1.0)]
# purge stream sccm flow rate profile in pulses
# purge stream sccm data points
purge_time = np.array(
    [
        0.0,
        0.19,
        0.2,
        1.19,
        1.2,
        3.19,
        3.2,
        5.19,
        5.2,
        5.39,
        5.4,
```

(continues on next page)

(continued from previous page)

```
        6.39,
        6.4,
        8.39,
        8.4,
        10.39,
        10.4,
        10.59,
        10.6,
        11.59,
        11.6,
        13.59,
        13.6,
        15.59,
        15.6,
        15.79,
        15.8,
        16.79,
        16.8,
        18.79,
        18.8,
        20.8,
    ]
)  # [sec]
purge_profile = np.array(
    [
        0.0,
        0.0,
        800.0,
        800.0,
        0.0,
        0.0,
        800.0,
        800.0,
        0.0,
        0.0,
        800.0,
        800.0,
        0.0,
        0.0,
        800.0,
        800.0,
        0.0,
        0.0,
        800.0,
        800.0,
        0.0,
        0.0,
        800.0,
        800.0,
        0.0,
        0.0,
        800.0,
```

(continues on next page)

(continued from previous page)

```
        800.0,
        0.0,
        0.0,
        800.0,
        800.0,
    ]
)  # [sccm]
# set the purge stream sccm profile
purge.set_sccm_flowrate_profile(purge_time, purge_profile)
# check the initial flow rates of the streams
t = 0.0
print(
    "Initial 'TMA' stream flow rate = "
    f"{met_organic.get_sccm_flowrate_profile_data(time=t)} [SCCM]"
)
print(
    "Initial 'O3' stream flow rate = "
    f"{oxidizers.get_sccm_flowrate_profile_data(time=t)} [SCCM]"
)
print(
    "Initial 'AR purge' stream flow rate = "
    f"{purge.get_sccm_flowrate_profile_data(time=t)} [SCCM]"
)
```

### Set up the transient PSR bomb reactor

Instantiate the transient PSR `ald_reactor` as a `TransientPSRSetVolumeFixedTemperature` object. Since this is a transient simulation, a initial reactor condition is required. In this example, the `ald_reactor` is initially filled with pure argon.

```
ald_reactor = TransPsr(purge, label="ALD_reactor")
```

### Set up additional reactor model parameters

Before you can run the simulation, you must provide reactor parameters, solver controls, and output instructions. For a PSR, you must provide either the residence time or the reactor volume. You can also make changes to any reactor initial conditions such as the reactor temperature and the gas composition. For example, the `reset_initial_temperature()` method and the `reset_initial_composition()` method. You can use the solver parameters to improve the convergence performance.

```
# set PSR volume (cm3): required for the
# ``TransientPSRSetVolumeFixedTemperature`` model
ald_reactor.volume = 600.0
# reactor active surface area [cm2]
ald_reactor.area = 300.0
# transient simulation end time [sec]
ald_reactor.time = 20.8
# initial surface conditions
# get the number of surface material defined in the surface mechanism
n_material = ald_reactor.get_numb_material
# get all surface material names
ald_material_names = ald_reactor.get_material_names()
```

(continues on next page)

(continued from previous page)

```
# set up surface coverage of all surface materials
# initial surface coverage (site fractions)
site_recipe = [[("O(S)", 1.0)]]
# bulk activities
bulk_recipe = [[("AL2O3(B)", 1.0)]]
# set the surface condition of the surface material
for i, mname in enumerate(ald_material_names):
    # set area fraction [-] (optional: by default, materials have the same area)
    ald_reactor.set_material_area_fraction(mname, 1.0)
    # set material temperature [K] if it is different from the gas temperature
    # (optional: by default, it is the same as the mixture temperature)
    ald_reactor.set_material_temperature(mname, 723.0)
    # set reactor surface coverage of the material
    # site fractions (optional: by default, all sites have the same fraction)
    ald_reactor.set_site_fraction(mname, site_recipe[i])
    # bulk activities (optional: by default, all bulk species activities = 1.0)
    ald_reactor.set_bulk_activity(mname, bulk_recipe[i])
```

### Connect the inlet to the reactor

You must connect at least one inlet to the open reactor. Use the `set_inlet()` method to add a stream to the PSR. Inversely, use the `remove_inlet()` to disconnect an inlet from the PSR.

**Note**

There is no limit on the number of inlets that can be connected to a PSR.

```
# connect the metal organic stream to the reactor
ald_reactor.set_inlet(met_organic)

# connect the oxidizer stream to the reactor
ald_reactor.set_inlet(oxidizers)

# connect the purge stream to the reactor
ald_reactor.set_inlet(purge)

# check the net inlet flow rate to the ALD reactor at time = 0.0 [sec]
# the net mass flow rate to the PSR must not be zero at any given time
# during the simulation
print(
    f"initial net mass flow rate to the reactor = "
    f"{ald_reactor.net_mass_flowrate} [g/sec]"
)
```

### Set solver controls

You can overwrite the default solver controls by using solver-related methods, such as those for tolerances.

```
# set solver the tolerances
ald_reactor.tolerances = (1.0e-20, 1.0e-8)
# solver non-negative mode
```

(continues on next page)

(continued from previous page)

```
ald_reactor.force_nonnegative = True
# transient solver max time step size [sec]
# ald_reactor.set_solver_max_timestep_size(2.0e-3)
# set adaptive solution saving distance
ald_reactor.adaptive_solution_saving(mode=True, steps=20)
# use the legacy solver to get better coonvergence performance for
# small mechanism
# ald_reactor.set_legacy_option(option=True)
```

### Run the transient PSR model

Once all the necessary input parameters are provided, use the `run()` method to start the transient PSR simulation. After the simulation is completed successfully, Use the `process_solution()` method to retrieve the raw solution profiles. If you are interested in the exit solution, use the `get_last_solution_mixture()` method to get the solution at the exit as a `Stream` object `combustor_exhaust`.

**Note**

In Pychemkin, the surface material is referred by its name. Instead of the surface material index, you loop over the surface material names. Use `get_material_names()` method to get a list of surface material names defined in the surface mechanism.

**Note**

For a site or a bulk species, use any `Mixture` or `Stream` created with the `Chemistry Set` containing the surface mechanism to get its species index. There are two type of index for surface species, the global index includes all species (the gas plus the site and the bulk of all materials); the local index includes the same type of surface species (site or bulk) of all materials.

```
# set the start wall time
start_time = time.time()
# run the reactor model
runstatus = ald_reactor.run()
#
if runstatus != 0:
    # run failed
    print(Color.RED + ">>> Run failed. <<<", end=Color.END)

runtime = time.time() - start_time
print(f"Total simulation duration: {runtime} [sec].")

# postprocess the solution profiles
ald_reactor.process_solution()

# get the number of data points in the solution profiles
solutionpoints = ald_reactor.getnumbersolutionpoints()
print(f"number of solution time points = {solutionpoints}")
# store solution profiles
# simulation time [sec]
```

(continues on next page)

(continued from previous page)

```
sim_time = ald_reactor.get_solution_variable_profile("time")
# solution arrays
# get all surface material names
aldt_material_names = ald_reactor.get_material_names()
# AL2O3(B) linear growth rate [micron/min]
al2o3_b_growth_solution = np.zeros(solutionpoints, dtype=np.double)
al2o3_b_thickness = np.zeros_like(al2o3_b_growth_solution, dtype=np.double)
# get gas-species index
i_o = met_organic.get_specindex("O")
i_tma = met_organic.get_specindex("ALMe3")
o_solution = np.zeros_like(al2o3_b_growth_solution, dtype=np.double)
tma_solution = np.zeros_like(al2o3_b_growth_solution, dtype=np.double)
# get surface site fraction solution profiles
# get surface species index
# global index includes all species: the gas, the site of all materials, and the bulk
# of all materials
# local index includes the same type of species (site or bulk) of all materials
i_o_s_global, i_o_s_local = met_organic.get_surf_specindex("O(S)")
i_al2o3_b_global, i_al2o3_b_local = met_organic.get_surf_specindex("AL2O3(B)")
# surface O(S) site fraction
o_s_solution = ald_reactor.get_solution_variable_profile("O(S)")
# surface ALMe2(S) site fraction
alme2_s_solution = ald_reactor.get_solution_variable_profile("ALMe2(S)")
# surface ALMeOALMe(S) site fraction
almeoalme_s_solution = ald_reactor.get_solution_variable_profile("ALMeOALMe(S)")
# loop over all solution time points
for i in range(solutionpoints):
    # get the mixture at the time point
    solutionmixture = ald_reactor.get_solution_mixture_at_index(solution_index=i)
    # calculate species production rate due to surface reactions [mole/cm2-sec]
    brate = 0.0
    for m in ald_material_names:
        # calculate species ROPs due to surface reactions [mole/cm2-sec]
        surf_rop, _ = solutionmixture.rop_surf(m)
        # get AL2O3(B) linear growth rate [cm/sec] due to surface reactions
        brates = solutionmixture.surface_chemistry.get_bulk_linear_growth_rates(
            m, surf_rop
        )
        brate += brates[i_al2o3_b_local]
    # linear growth rate of AL2O3(B) bulk species [cm/sec] -> [micron/sec]
    al2o3_b_growth_solution[i] = brate * 1.0e4
    # compute deposit thickness [micron]
    if i > 0:
        im = i - 1
        delta_time = sim_time[i] - sim_time[im]
        delta_grate = al2o3_b_growth_solution[im] + al2o3_b_growth_solution[i]
        # integration
        al2o3_b_thickness[i] = al2o3_b_thickness[im] + delta_grate * delta_time / 2.0
    # gas phase O solution profile [mole fraction]
    o_solution[i] = solutionmixture.x[i_o]
    # gas phase ALMe3 solution profile [mole fraction]
    tma_solution[i] = solutionmixture.x[i_tma]
```

(continues on next page)

(continued from previous page)

```
# clean up
ck.done()
```

#### Plot the transient PSR solution profiles

Plot the solution profiles over the entire simulation time span. You could observe how the growth of the AL2O3(B) thickness is controlled by turning on and off the inlet streams.

```
plt.subplots(2, 2, sharex="col", figsize=(12, 6))
plt.suptitle("Transient PSR ALD Solution", fontsize=16)
plt.subplot(221)
plt.plot(tma_time, tma_profile, "b-")
plt.plot(oxid_time, oxid_profile, "r-")
plt.legend(["met_organic", "oxidizers"], loc="upper right")
plt.ylabel("Inlet volumetric flow rate [SCCM]")
plt.subplot(222)
plt.plot(sim_time, o_solution, "r-")
plt.ylabel("O Mole fraction")
plt.subplot(223)
plt.plot(sim_time, o_s_solution, "r-")
plt.plot(sim_time, almeoalme_s_solution, "b--")
plt.plot(sim_time, alme2_s_solution, "g:")
plt.legend(["O(S)", "ALMeOALMe(S)", "ALMe2(S)"], loc="upper right")
plt.xlabel("Time [sec]")
plt.ylabel("Site Fraction")
plt.subplot(224)
plt.plot(sim_time, al2o3_b_thickness, "b-")
plt.xlabel("Time [sec]")
plt.ylabel("Deposit thickness AL2O3(B) [micron]")
# plot results
if interactive:
    plt.show()
else:
    plt.savefig("plot_trans_PSR_ALD.png", bbox_inches="tight")
```

## 22.12 Multiprocessing parameter study

Chemkin application by itself does not support parallel computing, however, in the context of parameter study, each case can run on its own CPU core to make better use of any available computing power. The examples in this section describe the process of applying Python multi-threading and multi-processing packages to **PyChemkin** parameter study to improve the overall simulation performance.

### 22.12.1 SI engine knocking analysis

The 0-D SI engine model can be used to predict the occurrence of engine knock. There is no consensus on the determination of the onset of engine knock. In this example, the engine knock is defined as when the end gas (the gas mixture in the unburned zone) auto-ignites. And the knock intensity is associated to the peak value of the total thermicity of the end gas, that is, the higher the peak total thermicity the more intense the engine knock.

There are many parameters that have impact on the occurrence of engine knock, for example, the fuel composition, the start of combustion timing, the engine speed, the wall heat loss and etc. This tutorial focuses on the effect of start of

combustion timing (or spark timing) on the engine knock. It also shows the steps of setting up an SI engine parameter study for knock analysis. The predicted knocking occurrence (represented by the end gas autoignition timing) are presented as a function of the start of combustion (SOC) crank angle. Detailed description of the 0-D SI engine model can be found in the "0-D engine model" sections of the "Chemkin Theory" manual, respectively. You can use the `ansys.chemkin.manuals()` method to get to the latest version of the Chemkin online manuals.

The parameter study is performed in the multi-process (or multi-core) mode by using `ProcessPoolExecutor` from the `concurrent.futures` package. If the computer has enough number of CPU cores, the parameter study can be run in parallel with each case running on a designated CPU core.

### Import PyChemkin packages and start the logger

```
from concurrent.futures import ProcessPoolExecutor
import os
from pathlib import Path
import time

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin

# chemkin spark ignition (SI) engine model (transient)
from ansys.chemkin.core.engines.SI import SIengine
from ansys.chemkin.core.inlet import Stream  # external gaseous inlet
from ansys.chemkin.core.logger import logger
from ansys.chemkin.core.utilities import WorkingFolders

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create an SI engine calculator class

Create a local class that wraps around the actual `SIengine` class to make the setup of the SI engine knock analysis parameter study more convenient.

**Note**

The auto-ignition detection must be turned on to obtain the correct knock occurrence crank angle. Use the `set_ignition_delay()` method to specify the ignition definition criterion and to turn on the ignition delay time detection.

```
class SIengineCalculator:
    """SI engine calculator with fixed set up parameters."""

    def __init__(self, fresh_mixture: Stream, index: int, start_combustion: float):
        """Create an SI engine calculator."""
        """
        Create an SI engine calculator that instantiates an SI engine object with
        the given fresh (unburnt) mixture condition and predefined engine
        parameters.

        Parameters
        ----------
            fresh_mixture: Mixture object
                the initial/fresh/unburnt condition
            index: integer
                run index of this SI engine calculator
            start_combustion: double
                start of combustion crank angle [CA]
        """
        # instantiate an SI engine object
        # set up the run and working directory name
        self.name = "SI_engine_" + str(index)
        # instantiate the SIengine object for this run
        self.si_calculator = SIengine(fresh_mixture, label=self.name)
        # run status
        self.runstatus = -100
        # calculated engine knock occurrence crank angle [CA]
        self.knock_ca = -720.0
        # calculated maximum thermicity value in the unburned zone [1/sec]
        self.max_sigma = 0.0
        # peak end gas temperature [K]
        self.max_temp = 0.0
        # IMEP
        self.imep = 0.0
        # set the required premixed flame model parameters
        self.set_engine_parameters()
        # set burn profile
        # use fixed Wiebe function
        wiebe_b = 7.0
        wiebe_n = 4.0
        duration = 45.6  # degrees
        self.set_burn_profile(start_combustion, duration, wiebe_n, wiebe_b)

    def run(self):
        """Run the SI engine calculation in a separate working directory."""
        # run the flame speed calculation
        self.runstatus = self.si_calculator.run()
        # extract the laminar flame speed from the solution
        if self.runstatus == 0:
            # postprocess the unburned zone solution
            unburnedzone = 1
            # burnedzone = 2
            ierr = self.si_calculator.process_engine_solution(zone_id=unburnedzone)
```

(continues on next page)

(continued from previous page)

```
            if ierr == 0:
                # get the engine knock (end-gas auto-ignition)
                # occurrence crank angle [CA]
                # if no engine knock, the value = -720 CA
                # because the memory is shared, it must be done as soon as
                # the run is finished
                self.knock_ca = self.si_calculator.check_engine_knock()
                temperature = self.si_calculator.get_solution_variable_profile(
                    "temperature"
                )
                thermicity = self.si_calculator.get_solution_variable_profile(
                    "thermicity"
                )
                self.max_temp = np.max(temperature)
                self.max_sigma = np.max(thermicity)
                self.imep = self.si_calculator.get_engine_imep()
                del thermicity, temperature
            else:
                self.runstatus = ierr

    def set_engine_parameters(self):
        """Set up the basic engine parameters."""
        # Set up basic engine parameters
        # These engine parameters are used to describe the cylinder volume during the
        # simulation. The ``starting_ca`` argument should be the crank angle
        # corresponding to the cylinder IVC. The ``ending_ca`` argument is typically
        # the EVO crank angle.
        # cylinder bore diameter [cm]
        self.si_calculator.bore = 8.5
        # engine stroke [cm]
        self.si_calculator.stroke = 10.82
        # connecting rod length [cm]
        self.si_calculator.connecting_rod_length = 17.853
        # compression ratio [-]
        self.si_calculator.compression_ratio = 12
        # engine speed [RPM]
        self.si_calculator.rpm = 1200
        # simulation start CA [degree]
        self.si_calculator.starting_ca = -120.2
        # simulation end CA [degree]
        self.si_calculator.ending_ca = 139.8
        # wall heat transfer parameters
        # Set up engine wall heat transfer model
        heattransferparameters = [0.1, 0.8, 0.0]
        # set cylinder wall temperature [K]
        t_wall = 434.0
        self.si_calculator.set_wall_heat_transfer(
            "dimensionless", heattransferparameters, t_wall
        )
        # in-cylinder gas velocity correlation parameter (Woschni)
        # [<C11> <C12> <C2> <swirl ratio>]
        gv_parameters = [2.28, 0.318, 0.324, 0.0]
```

(continues on next page)

(continued from previous page)

```
        self.si_calculator.set_gas_velocity_correlation(gv_parameters)
        # set piston head top surface area [cm2]
        self.si_calculator.set_piston_head_area(area=56.75)
        # set cylinder clearance surface area [cm2]
        self.si_calculator.set_cylinder_head_area(area=56.75)
        # other engine model parameters
        # set tolerances in tuple: (absolute tolerance, relative tolerance)
        self.si_calculator.tolerances = (1.0e-10, 1.0e-6)
        # turn on the force non-negative solutions option in the solver
        self.si_calculator.force_nonnegative = False
        # set minimum zonal mass [g]
        self.si_calculator.set_minimum_zone_mass(minmass=1.0e-5)
        # set the maximum solver time step
        self.si_calculator.max_ca_step = 0.1
        # set the number of crank angles between saving solution
        self.si_calculator.ca_step_for_saving_solution = 0.5
        # set the number of crank angles between printing solution
        self.si_calculator.ca_step_for_printing_solution = 10.0
        # turn ON adaptive solution saving
        self.si_calculator.adaptive_solution_saving(mode=True, steps=20)
        # set to detect auto-ignition
        self.si_calculator.set_ignition_delay(method="Thermicity_peak")
        # suppress text output
        self.si_calculator.stop_output()

    def set_burn_profile(
        self,
        start_combustion: float,
        burn_duration: float,
        wiebe_n: float,
        wiebe_b: float,
    ):
        """Set up the fuel burn rate profile."""
        """
        Set up the fuel burn rate profile.

        Parameters
        ----------
            start_combustion: double
                start of combustion (SOC) crank angle [CA]
            burn_duration: double
                burn duration in crank angle [CA]
            wiebe_n: double
                Wiebe function n parameter value
            wiebe_b: double
                Wiebe function b parameter value
        """
        # start of combustion CA
        self.si_calculator.set_burn_timing(soc=start_combustion, duration=burn_duration)
        self.si_calculator.wiebe_parameters(n=wiebe_n, b=wiebe_b)
```

### Set up the SI engine knocking analysis parameter study for multi-processing

Create `si_engineCalculator` object with different start of combustion (SOC) crank angle. Each object represents one parameter study case and will be run on an designated cpu core when the parameter study is executed.

```
def si_engine_run(case: tuple[int, float]) -> tuple[float, float, float, float, float]:
    """Set up sSI engine parameter study."""
    """
    Set up the parameter study runs for multi-processing.

    Parameters
    ----------
        case: tuple (integer, double)
            SI engine calculation case condition: case index and start of
            combustion crank angle [CA]

    Returns
    -------
        start_ca: double
            the parameter: start of combustion crank angle [degree CA]
        knock_ca: double
            crank angle when engine knock occurs [degree CA]
        max_sigma: double
            the peak total thermicity of the end gas [1/sec]
        max_temp: double
            peak end gas temperature [K]
        imep: double
            indicated mean effective pressure [bar]
    """
    ########################################
    # Create an instance of the Chemistry Set
    # =======================================
    # For PRF, the encrypted 14-component gasoline mechanism,
    # ``gasoline_14comp_WBencrypted.inp``, is used. The chemistry set
    # is named ``gasoline``.
    #
    # .. note::
    #   Because this gasoline mechanism does not come with any transport data,
    #   you do not need to provide a transport data file.
    #
    # case index
    index = case[0]
    # start of combustion crank angle
    start_ca = case[1]
    # create and change to the working directory for this run
    name = "SI_engine_" + str(index)
    work_folder = WorkingFolders(name, current_dir)
    # set mechanism directory (the default Chemkin mechanism data directory)
    data_dir = Path(ck.ansys_dir) / "reaction" / "data"
    mechanism_dir = data_dir
    # create a chemistry set based on the gasoline 14 components mechanism
    MyGasMech = ck.Chemistry(label="Gasoline")
    # set mechanism input files
    # including the full file path is recommended
```

(continues on next page)

(continued from previous page)

```
    MyGasMech.chemfile = str(mechanism_dir / "gasoline_14comp_WBencrypt.inp")

    ######################################
    # Preprocess the gasoline chemistry set
    # ======================================

    # preprocess the mechanism files
    _ = MyGasMech.preprocess()

    ##############################################
    # Set up the stoichiometric gasoline-air mixture
    # ============================================
    # You must set up the stoichiometric gasoline-air mixture for the subsequent
    # SI engine calculations. Here the ``x_by_equivalence_ratio()`` method is used.
    # You create the ``fuel`` and the ``air`` mixtures first. You then define the
    # *complete combustion product species* and provide the *additives* composition
    # if applicable.
    # Finally, you simply set ``equivalenceratio=1`` to create the stoichiometric
    # gasoline-air mixture.
    #
    # For PRF 90 gasoline, the recipe is ``[("ic8h18", 0.9), ("nc7h16", 0.1)]``.

    # create the fuel mixture
    fuelmixture = ck.Mixture(MyGasMech)
    # set fuel composition
    fuelmixture.x = [("ic8h18", 0.6), ("nc7h16", 0.0), ("mch", 0.1), ("c6h5c3h7", 0.3)]
    # setting pressure and temperature is not required in this case
    fuelmixture.pressure = 2.5 * ck.P_ATM
    fuelmixture.temperature = 473.0

    # create the oxidizer mixture: air
    air = ck.Mixture(MyGasMech)
    air.x = [("o2", 0.21), ("n2", 0.79)]
    # setting pressure and temperature is not required in this case
    air.pressure = fuelmixture.pressure
    air.temperature = fuelmixture.temperature

    # products from the complete combustion of the fuel mixture and air
    products = ["co2", "h2o", "n2"]
    # species mole fractions of added/inert mixture.
    # You can also create an additives mixture here.
    add_frac = np.zeros(MyGasMech.kk, dtype=np.double)  # no additives: all zeros

    # create the unburned fuel-air mixture
    fresh = ck.Mixture(MyGasMech)

    # mean equivalence ratio
    equiv = 1.0
    _ = fresh.x_by_equivalence_ratio(
        MyGasMech, fuelmixture.x, air.x, add_frac, products, equivalenceratio=equiv
    )
```

(continues on next page)

(continued from previous page)

```
    ############################################################
    # Specify pressure and temperature of the fuel-air mixture
    # ============================================================
    # Since you are going to use ``fresh`` fuel-air mixture to instantiate
    # the engine object later, setting the mixture pressure and temperature
    # is equivalent to setting the initial temperature and pressure of the
    # engine cylinder.
    fresh.temperature = fuelmixture.temperature
    fresh.pressure = fuelmixture.pressure

    ##########################################
    # Add EGR to the fresh fuel-air mixture
    # ======================================
    # Many engines have the configuration for exhaust gas recirculation (EGR). Chemkin
    # engine models let you add the EGR mixture to the fresh fuel-air mixture entering
    # the cylinder. If the engine you are modeling has EGR, you should have
    # the EGR ratio, which is generally the volume ratio of the EGR mixture and
    # the fresh fuel-air mixture. However, because you know nothing about the
    # composition of the exhaust gas, you cannot simply combine these two mixtures.
    # In this case, you use the ``get_egr_mole_fraction()`` method to estimate
    # the major components of the exhaust gas from the combustion of the fresh
    # fuel-air mixture. The ``threshold=1.0e-8`` parameter tells the method to ignore
    # any species with a mole fraction below the threshold value. Once you have the
    # EGR mixture composition, use the ``x_by_equivalence_ratio()`` method a second
    # time to re-create the fuel-air mixture ``fresh`` with the original
    # ``fuelmixture`` and ``air`` mixtures, along with the EGR composition that
    # you just got as the *additives*.
    egr_ratio = 0.25
    # compute the EGR stream composition in mole fractions
    add_frac = fresh.get_egr_mole_fraction(egr_ratio, threshold=1.0e-8)
    # recreate the initial mixture with EGR
    ierror = fresh.x_by_equivalence_ratio(
        MyGasMech,
        fuelmixture.x,
        air.x,
        add_frac,
        products,
        equivalenceratio=equiv,
        threshold=1.0e-8,
    )
    if ierror != 0:
        print("error...creating the initial fuel-oxidizer mixture.")
        exit()
    ###################################
    # Set up the SI engine knock case
    # ===============================
    # create an SI engine calculation instance
    this_run = SIengineCalculator(fresh, index=index, start_combustion=start_ca)
    # run the case
    this_run.run()
    # change back to the original top folder
    work_folder.done()
```

(continues on next page)

(continued from previous page)

```
    return (
        start_ca,
        this_run.knock_ca,
        this_run.max_sigma,
        this_run.max_temp,
        this_run.imep,
    )
```

### Set up and start the multi-process runs

Use the `ProcessPoolExecutor()` method to assign the SI engine runs. In this project, each SI engine run/case runs on an exclusive cpu core. The first parameter of `map()` method should be `si_engine_run()`. Make the `si_engine_run()` method return the required parameter and results to make post-processing the results from the parameter study easier.

**Note**

`if __name__ == '__main__'` is required when `multiprocessing` package is used.

```
if __name__ == "__main__":
    # number of available cpu cores
    numb_cores = max(os.cpu_count(), 1)
    # set the number of worker cpu cores to be used by the parameter study
    numb_workers = numb_cores - 2
    numb_workers = max(numb_workers, 1)
    # start the multi-process
    # starting value for the start of combustion crank angle
    start_combustion = -10.5
    start_ca = start_combustion
    # increment
    dca = 2.0
    # number of cases to run in the parameter study
    numb_cases = 5
    # create the cases
    si_cases: list[tuple[int, float]] = []
    for i in range(numb_cases):
        si_cases.append((i, start_ca))
        start_ca += dca
    # set the start wall time
    start_time = time.time()
    #
    numb_workers = min(numb_workers, numb_cases)
    knock_ca: list[float] = []
    knock_intensity: list[float] = []
    soc_ca: list[float] = []
    knock_ca_soc: list[float] = []
    engine_imep: list[float] = []
    peak_temp: list[float] = []
    with ProcessPoolExecutor(max_workers=numb_workers) as e:
        for ret_value in e.map(si_engine_run, si_cases):
            # results returned by the SI engine calculator
```

(continues on next page)

(continued from previous page)

```
            # start of combustion CA
            soc_ca.append(ret_value[0])
            # knock CA (end gas auto-ignition CA)
            if ret_value[1] > -720.0:
                knock_ca.append(ret_value[1])
                knock_ca_soc.append(ret_value[0])
            # knock intensity as measured by the peak total thermicity
            # of the end gas in the unburned zone [1/sec]
            knock_intensity.append(ret_value[2])
            # peak end gas temperature [K]
            peak_temp.append(ret_value[3])
            # IMEP [bar]
            engine_imep.append(ret_value[4])
    # compute the total runtime
    runtime = time.time() - start_time
    print()
    print(f"total simulation duration: {runtime} [sec]")
    print()

    ########################################
    # Plot the premixed flame solution profiles
    # ========================================
    # Plot the predicted flame speeds against the experimental data.
    plt.subplots(2, 2, sharex="col", figsize=(12, 6))
    plt.subplot(221)
    plt.plot(knock_ca_soc, knock_ca, linestyle="-", marker="^", color="blue")
    plt.ylabel("Knock Crank Angle [degree CA]")
    plt.subplot(222)
    plt.plot(soc_ca, knock_intensity, linestyle="-", marker="o", color="green")
    plt.ylabel("Knock Intensity [1/sec]")
    plt.subplot(223)
    plt.plot(soc_ca, engine_imep, linestyle="-", marker="s", color="magenta")
    plt.ylabel("IMEP [bar]")
    plt.xlabel("Start of Combustion [degree CA]")
    plt.subplot(224)
    plt.plot(soc_ca, peak_temp, linestyle="-", marker="v", color="red")
    plt.ylabel("Peak End-Gas Temperature [K]")
    plt.xlabel("Start of Combustion [degree CA]")

    # clean up
    ck.done()

    # plot results
    if interactive:
        plt.show()
    else:
        plt.savefig("plot_SI_engine_knock.png", bbox_inches="tight")
```

### 22.12.2 Setting up a multi-process laminar flame speed parameter study

One of the prevailing use case of the *freely propagating premixed flame* model is to build a *flame speed* table to be imported by another combustion simulation tools. PyChemkin provides the flexibility to customize the data structure of the flame speed table depending on the simulation goals and the tool. Furthermore, over the years, the chemkin flame speed calculator has derived a set of default solver settings that would greatly improve the convergence performance, especially for those widely adopted hydrocarbon fuel combustion mechanisms. The required input parameters the flame speed calculator are reduced to the composition of the fuel-oxidizer mixture, the initial/inlet pressure and temperature, and the calculation domain.

This tutorial shows the steps of setting up a flame speed parameter study for $CH_4$-air mixtures at the 5 atmosphere pressure. The predicted flame speed values are compared against the experimental data as a function of the mixture equivalence ratio. The parameter study is performed in the multi-process (or multi-core) mode by using `ProcessPoolExecutor` from the `concurrent.futures` package. If the computer has enough number of CPU cores, the parameter study can be run in parallel with each case running on a designated CPU core.

Since the transport processes are critical for flame calculations, the transport data must be included in the mechanism data and preprocessed.

#### Import PyChemkin packages and start the logger

```
from concurrent.futures import ProcessPoolExecutor
import os
from pathlib import Path
import time

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core.inlet import Stream  # external gaseous inlet
from ansys.chemkin.core.logger import logger

# Chemkin 1-D premixed freely propagating flame model (steady-state)
from ansys.chemkin.core.premixedflames.premixedflame import (
    FreelyPropagating as FlameSpeed,
)
from ansys.chemkin.core.utilities import WorkingFolders

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create a flame speed calculator class

Create a local class that wraps around the actual `FlameSpeed` class to make the setup of the multi-process flame speed calculation parameter study more convenient.

```
class FlameSpeedCalculator:
    """Laminar flame speed calculator with fixed set up parameters."""

    def __init__(self, fresh_mixture: Stream, index: int):
        """Create a Laminar flame speed calculator."""
        """
        Create a Laminar flame speed calculator that instantiates a FlameSpeed object
        with the given fresh (unburnt) mixture condition.

        Parameters
        ----------
            fresh_mixture: Mixture object
                the initial/fresh/unburnt condition
            index: integer
                run index of this flame speed calculator
        """
        # instantiate a flame speed object
        # set up the run and working directory name
        self.name = "Flame_Speed_" + str(index)
        # instantiate the FlameSpeed object for this run
        self.fs_calculator = FlameSpeed(fresh_mixture, label=self.name)
        # set the required premixed flame model parameters
        #
        # set the maximum total number of grid points allowed
        # in the calculation (optional)
        # self.fs_calculator.set_max_grid_points(150)
        # define the calculation domain [cm]
        self.fs_calculator.end_position = 1.0
        # set the root directory
        self.root_dir = str(Path.cwd())
        # set the working directory
        self.work_dir = str(Path(self.root_dir) / self.name)
        # run status
        self.runstatus = -100
        # calculated laminar flame speed [cm/sec]
        self.flame_speed = 0.0

    def run(self):
        """Run the flame speed calculation in a separate working directory."""
        # run the flame speed calculation
        self.runstatus = self.fs_calculator.run()
        # extract the laminar flame speed from the solution
        if self.runstatus == 0:
            # postprocess the solutions
            self.fs_calculator.process_solution()
            # get the flame speed value [cm/sec]
            # because the memory is shared, it must be done as soon as
            # the run is finished
            self.flame_speed = self.fs_calculator.get_flame_speed()
```

(continues on next page)

(continued from previous page)

```
    def get_flame_speed(self) -> float:
        """Get the predicted laminar flame speed."""
        """
        Get the predicted laminar flame speed.

        Returns
        -------
            flame_speed: double
                predicted laminar flame speed [cm/sec]
        """
        return self.flame_speed
```

### Set up the flame speed parameter study for multi-processing

Create a list of `FlameSpeedCalculator` objects with different initial methane-air equivalence ratios. Each object represents one parameter study case and will be run on an designated cpu core when the parameter study is executed.

```
def flame_speed_run(case: tuple[int, float]) -> tuple[float, float]:
    """Set up the flame speed parameter study."""
    """
    Set up the parameter study runs for multi-processing.

    Parameters
    ----------
        case: tuple (integer, double)
            flame speed calculation case condition: case index and
            the equivalence ratio

    Returns
    -------
        phi: double
            the parameter: fresh mixture equivalence ratio
        flame_speed: double
            predicted laminar flame speed of the fresh mixture [cm/sec]
    """
    ########################################
    # Create an instance of the Chemistry Set
    # ======================================
    # The mechanism loaded is the GRI 3.0 mechanism for methane combustion.
    # The mechanism and its associated data files come with the standard Ansys Chemkin
    # installation under the subdirectory *"/reaction/data"*.
    #
    # case index
    index = case[0]
    # fresh mixture equivalence ratio
    phi = case[1]
    # create and change to the working directory for this run
    name = "Flame_Speed_" + str(index)
    work_folder = WorkingFolders(name, current_dir)
    # set mechanism directory (the default Chemkin mechanism data directory)
    data_dir = Path(ck.ansys_dir) / "reaction" / "data"
```

(continues on next page)

(continued from previous page)

```
    mechanism_dir = data_dir
    # including the full file path is recommended
    chemfile = str(mechanism_dir / "grimech30_chem.inp")
    thermfile = str(mechanism_dir / "grimech30_thermo.dat")
    tranfile = str(mechanism_dir / "grimech30_transport.dat")
    # create a chemistry set based on GRI 3.0
    MyGasMech = ck.Chemistry(
        chem=chemfile, therm=thermfile, tran=tranfile, label="GRI 3.0"
    )

    ##############################
    # Preprocess the Chemistry Set
    # ============================

    # preprocess the mechanism files
    ierror = MyGasMech.preprocess()
    if ierror != 0:
        print("Error: failed to preprocess the mechanism!")
        print(f"       error code = {ierror}")
        exit()

    ###########################################################################
    # Set up the CH\ :sub:`4`\ -air mixture for the flame speed calculation
    # ========================================================================
    # Instantiate a stream named ``premixed`` for the inlet gas mixture.
    # This stream  is a mixture with the addition of the
    # inlet flow rate. You can specify the inlet gas properties the same way you
    # set up a ``Mixture``. Here the ``x_by_equivalence_ratio`` method is used.
    # You create the ``fuel`` and the ``air`` mixtures first. Then define the
    # *complete combustion product species* and provide the *additives* composition
    # if applicable. And finally, during the parameter iteration runs, you can
    # simply set different values to ``equivalenceratio`` to create different
    # methane-air mixtures.
    #

    # create the fuel mixture
    fuel = ck.Mixture(MyGasMech)
    # set fuel composition: methane
    fuel.x = [("CH4", 1.0)]
    # setting pressure and temperature condition for the flame speed calculations
    fuel.pressure = 5.0 * ck.P_ATM
    fuel.temperature = 300.0  # inlet temperature

    # create the oxidizer mixture: air
    air = ck.Mixture(MyGasMech)
    air.x = ck.Air.x()
    # setting pressure and temperature is not required in this case
    air.pressure = fuel.pressure
    air.temperature = fuel.temperature

    # create the fuel-air Stream for the premixed flame speed calculation
    premixed = Stream(MyGasMech, label="premixed")
```

(continues on next page)

(continued from previous page)

```
    # products from the complete combustion of the fuel mixture and air
    products = ["CO2", "H2O", "N2"]
    # species mole fractions of added/inert mixture.
    # can also create an additives mixture here
    add_frac = np.zeros(MyGasMech.kk, dtype=np.double)  # no additives: all zeros

    # setting pressure and temperature is not required in this case
    premixed.pressure = fuel.pressure
    premixed.temperature = fuel.temperature

    # set estimated value of the flame mass flux [g/cm2-sec]
    premixed.mass_flowrate = 0.4
    # create mixture by using the equivalence ratio
    ierror = premixed.x_by_equivalence_ratio(
        MyGasMech, fuel.x, air.x, add_frac, products, equivalenceratio=phi
    )
    # check fuel-oxidizer mixture creation status
    if ierror != 0:
        print(
            "Error: failed to create the methane-air mixture "
            + "for equivalence ratio = "
            + str(phi)
        )
        exit()
    # create a flame speed calculation instance
    this_run = FlameSpeedCalculator(premixed, index=index)
    # run the case
    this_run.run()
    # change back to the original top folder
    work_folder.done()
    #
    return phi, this_run.flame_speed
```

### Set up and start the multi-process runs

Use the `ProcessPoolExecutor()` method to assign the flame speed runs. In this project, each flame speed run/case runs on an exclusive cpu core. The first parameter of `map()` method should be `flame_speed_run()`. Make the `flame_speed_run()` method return the required parameter and results to make post-processing the results from the parameter study easier.

**Note**

`if __name__ == '__main__'` is required when `multiprocessing` package is used.

```
if __name__ == "__main__":
    # number of available cpu cores
    numb_cores = max(os.cpu_count(), 1)
    # set the number of worker cpu cores to be used by the parameter study
    numb_workers = 14
    numb_workers = max(numb_workers, 1)
    numb_workers = min(numb_workers, numb_cores - 2)
```

(continues on next page)

(continued from previous page)

```
    # Set up the flame speed parameter study for multi-processing
    # equivalence ratio for the first case
    phi = 0.6
    # total number of parameter cases
    numb_cases = 21
    # equivalence ratio increment
    delta_phi = 0.05
    # set up flame speed calculation runs with different equivalence ratios
    flamespeed_cases: list[tuple[int, float]] = []
    for i in range(numb_cases):
        # create mixture by using the equivalence ratio
        flamespeed_cases.append((i, phi))
        # update parameter
        phi += delta_phi
    # start the multi-process
    case_id = 0
    # set the start wall time
    start_time = time.time()
    #
    numb_workers = min(numb_workers, numb_cases)
    equiv: list[float] = []
    flame_speed: list[float] = []
    with ProcessPoolExecutor(max_workers=numb_workers) as e:
        for ret_value in e.map(flame_speed_run, flamespeed_cases):
            # results returned by the flame speed calculator
            # equivalence ratio
            equiv.append(ret_value[0])
            # predicted laminar flame speed [cm/sec]
            flame_speed.append(ret_value[1])

    # compute the total runtime
    runtime = time.time() - start_time
    print()
    print(f"total simulation duration: {runtime} [sec]")
    print()

    # experimental data by Kochar
    # equivalence ratios
    data_equiv = [
        0.7005,
        0.8007,
        0.9009,
        1.001,
        1.1032,
        1.2014,
        1.3014,
    ]
    # methane flame speeds at 5 atm
    data_speed = [
        6.906,
        12.0094,
        15.9072,
```

(continues on next page)

(continued from previous page)

```
        19.2376,
        19.6601,
        15.8274,
        10.2925,
    ]

    ##########################################
    # Plot the premixed flame solution profiles
    # ==========================================
    # Plot the predicted flame speeds against the experimental data.

    plt.plot(
        data_equiv, data_speed, label="data", linestyle="", marker="^", color="blue"
    )
    plt.plot(equiv, flame_speed, label="GRI 3.0", linestyle="-", color="blue")
    plt.legend()
    plt.ylabel("Flame Speed [cm/sec]")
    plt.xlabel("Equivalence Ratio")

    # clean up
    ck.done()

    # plot results
    if interactive:
        plt.show()
    else:
        plt.savefig("plot_flame_speed_pooling.png", bbox_inches="tight")
```

### 22.12.3 Setting up a multi-thread laminar flame speed parameter study

One of the prevailing use case of the *freely propagating premixed flame* model is to build a *flame speed* table to be imported by another combustion simulation tools. PyChemkin provides the flexibility to customize the data structure of the flame speed table depending on the simulation goals and the tool. Furthermore, over the years, the chemkin flame speed calculator has derived a set of default solver settings that would greatly improve the convergence performance, especially for those widely adopted hydrocarbon fuel combustion mechanisms. The required input parameters the flame speed calculator are reduced to the composition of the fuel-oxidizer mixture, the initial/inlet pressure and temperature, and the calculation domain.

This tutorial shows the steps of setting up a flame speed parameter study for $CH_4$-air mixtures at the 5 atmosphere pressure. The predicted flame speed values are compared against the experimental data as a function of the mixture equivalence ratio. The parameter study is performed in the multi-thread mode by using the `threading` package.

Since the transport processes are critical for flame calculations, the transport data must be included in the mechanism data and preprocessed.

#### Import PyChemkin packages and start the logger

```
import os
from pathlib import Path
import threading
```

(continues on next page)

(continued from previous page)

```
import time

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core.inlet import Stream  # external gaseous inlet
from ansys.chemkin.core.logger import logger

# Chemkin 1-D premixed freely propagating flame model (steady-state)
from ansys.chemkin.core.premixedflames.premixedflame import (
    FreelyPropagating as FlameSpeed,
)

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create a flame speed calculator class

Create a local class that wraps around the actual `FlameSpeed` class to make the setup of the multi-thread flame speed calculation parameter study more convenient.

```
class FlameSpeedCalculator:
    """Laminar flame speed calculator with fixed set up parameters."""

    def __init__(self, fresh_mixture: Stream, index: int):
        """Create a Laminar flame speed calculator."""
        """
        Create a Laminar flame speed calculator that instantiates a FlameSpeed object
        with the given fresh (unburnt) mixture condition.

        Parameters
        ----------
            fresh_mixture: Mixture object
                the initial/fresh/unburnt condition
            index: integer
                run index of this flame speed calculator
        """
        # instantiate a flame speed object
        # set up the run and working directory name
        name = "Flame_Speed_" + str(index)
        # instantiate the FlameSpeed object for this run
        self.fs_calculator = FlameSpeed(fresh_mixture, label=name)
        # set the required premixed flame model parameters
```

(continues on next page)

(continued from previous page)

```
        #
        # set the maximum total number of grid points allowed
        # in the calculation (optional)
        # self.fs_calculator.set_max_grid_points(150)
        # define the calculation domain [cm]
        self.fs_calculator.end_position = 1.0
        # set the root directory
        self.root_dir = str(Path.cwd())
        # set the working directory
        self.work_dir = str(Path(self.root_dir) / name)
        # run status
        self.runstatus = -100
        # calculated laminar flame speed [cm/sec]
        self.flame_speed = 0.0

    def run(self):
        """Run the flame speed calculation in a separate working directory."""
        # create or clean up the working directory for this run
        this_work_folder = Path(self.work_dir)
        if this_work_folder.is_dir():
            # directory exists
            for f in this_work_folder.iterdir():
                if f.is_file():
                    # remove any existing file
                    try:
                        f.unlink()
                    except OSError as e:
                        print(f"Error removing {str(f)}: {e}")
        else:
            # create a new directory
            this_work_folder.mkdir()
        # change to the working directory for this run
        os.chdir(self.work_dir)
        # run the flame speed calculation
        self.runstatus = self.fs_calculator.run()
        # extract the laminar flame speed from the solution
        if self.runstatus == 0:
            # postprocess the solutions
            self.fs_calculator.process_solution()
            # get the flame speed value [cm/sec]
            # because the memory is shared, it must be done as soon as
            # the run is finished
            self.flame_speed = self.fs_calculator.get_flame_speed()
        # go back the root directory
        os.chdir(self.root_dir)

    def get_flame_speed(self) -> float:
        """Get the predicted laminar flame speed."""
        """
        Get the predicted laminar flame speed.

        Returns
```

(continues on next page)

(continued from previous page)

```
        -------
            flame_speed: double
                predicted laminar flame speed [cm/sec]
        """
        return self.flame_speed
```

### Set up the flame speed parameter study for multi-threading

Create a list of `FlameSpeedCalculator` objects with different initial methane-air equivalence ratios from 0.6 to 1.6. Each object represents one parameter study case and will be assigned to its own thread when the parameter study is executed.

```
def prepare_multi_thread_runs() -> dict[float, FlameSpeedCalculator]:
    """Set up the flame speed parameter study."""
    """
    Set up the parameter study runs for multi-threading.

    Returns
    -------
        flame_speed_runs: list of FlameSpeedCalculator objects
            flame speed calculation cases
    """
    ########################################
    # Create an instance of the Chemistry Set
    # ======================================
    # The mechanism loaded is the GRI 3.0 mechanism for methane combustion.
    # The mechanism and its associated data files come with the standard Ansys Chemkin
    # installation under the subdirectory *"/reaction/data"*.
    #

    # set mechanism directory (the default Chemkin mechanism data directory)
    data_dir = Path(ck.ansys_dir) / "reaction" / "data"
    mechanism_dir = data_dir
    # including the full file path is recommended
    chemfile = str(mechanism_dir / "grimech30_chem.inp")
    thermfile = str(mechanism_dir / "grimech30_thermo.dat")
    tranfile = str(mechanism_dir / "grimech30_transport.dat")
    # create a chemistry set based on GRI 3.0
    MyGasMech = ck.Chemistry(
        chem=chemfile, therm=thermfile, tran=tranfile, label="GRI 3.0"
    )

    ##############################
    # Preprocess the Chemistry Set
    # ============================

    # preprocess the mechanism files
    ierror = MyGasMech.preprocess()
    if ierror != 0:
        print("Error: failed to preprocess the mechanism!")
        print(f"       error code = {ierror}")
        exit()
```

(continues on next page)

(continued from previous page)

```
    ##########################################################################
    # Set up the CH\ :sub:`4`\ -air mixture for the flame speed calculation
    # ========================================================================
    # Instantiate a stream named ``premixed`` for the inlet gas mixture.
    # This stream  is a mixture with the addition of the
    # inlet flow rate. You can specify the inlet gas properties the same way you
    # set up a ``Mixture``. Here the ``x_by_equivalence_ratio`` method is used.
    # You create the ``fuel`` and the ``air`` mixtures first. Then define the
    # *complete combustion product species* and provide the *additives* composition
    # if applicable. And finally, during the parameter iteration runs, you can
    # simply set different values to ``equivalenceratio`` to create different
    # methane-air mixtures.
    #

    # create the fuel mixture
    fuel = ck.Mixture(MyGasMech)
    # set fuel composition: methane
    fuel.x = [("CH4", 1.0)]
    # setting pressure and temperature condition for the flame speed calculations
    fuel.pressure = 5.0 * ck.P_ATM
    fuel.temperature = 300.0  # inlet temperature

    # create the oxidizer mixture: air
    air = ck.Mixture(MyGasMech)
    air.x = ck.Air.x()
    # setting pressure and temperature is not required in this case
    air.pressure = fuel.pressure
    air.temperature = fuel.temperature

    # create the fuel-air Stream for the premixed flame speed calculation
    premixed = Stream(MyGasMech, label="premixed")
    # products from the complete combustion of the fuel mixture and air
    products = ["CO2", "H2O", "N2"]
    # species mole fractions of added/inert mixture.
    # can also create an additives mixture here
    add_frac = np.zeros(MyGasMech.kk, dtype=np.double)  # no additives: all zeros

    # setting pressure and temperature is not required in this case
    premixed.pressure = fuel.pressure
    premixed.temperature = fuel.temperature

    # set estimated value of the flame mass flux [g/cm2-sec]
    premixed.mass_flowrate = 0.4

    # Set up the flame speed parameter study for multi-threading
    # equivalence ratio for the first case
    phi = 0.6
    # total number of parameter cases
    points = 21
    # equivalence ratio increment
    delta_phi = 0.05
```

(continues on next page)

(continued from previous page)

```
    # set up flame speed calculation runs
    flame_speed_runs: dict[float, FlameSpeedCalculator] = {}
    for i in range(points):
        # create mixture by using the equivalence ratio
        ierror = premixed.x_by_equivalence_ratio(
            MyGasMech, fuel.x, air.x, add_frac, products, equivalenceratio=phi
        )
        # check fuel-oxidizer mixture creation status
        if ierror != 0:
            print(
                "Error: failed to create the methane-air mixture "
                + "for equivalence ratio = "
                + str(phi)
            )
            exit()
        # create a flame speed calculation instance
        flame_speed_runs[phi] = FlameSpeedCalculator(premixed, index=i)
        # update parameter
        phi += delta_phi

    # return the job setup parameters
    return flame_speed_runs
```

### Set up and start the multi-thread runs

Use the `Thread()` method to assign the flame speed runs. In this project, each flame speed run/case has its own thread. The *target* parameter of `Thread()` method should be the `run()` method of the `FlameSpeedCalculator`. Use the `start()` method to initiate the threads, and the `join()` method to sync the threads after they are done.

```
flame_speed_cases = prepare_multi_thread_runs()
# set the start wall time
start_time = time.time()
threads = []
for phi, fsc in flame_speed_cases.items():
    t = threading.Thread(target=fsc.run())
    threads.append(t)
    # start each thread
    t.start()
# wait for all threads to finish
for t in threads:
    t.join()

# compute the total runtime
runtime = time.time() - start_time
print()
print(f"total simulation duration: {runtime} [sec]")
print()
```

### Get the parameter study results

Use the `get_flame_speed()` method to get the calculated laminar flame speed values from each run. Then set up the experimental flame speed data for comparison.

```
points = len(flame_speed_cases.keys())
equival = np.zeros(points, dtype=np.double)
flamespeed = np.zeros_like(equival, dtype=np.double)
for i, case in enumerate(flame_speed_cases.items()):
    # equivalence ratio
    equival[i] = case[0]
    # flame speed calculator case
    fsc = case[1]
    flamespeed[i] = fsc.get_flame_speed()

# experimental data by Kochar
# equivalence ratios
data_equiv = [
    0.7005,
    0.8007,
    0.9009,
    1.001,
    1.1032,
    1.2014,
    1.3014,
]
# methane flame speeds at 5 atm
data_speed = [
    6.906,
    12.0094,
    15.9072,
    19.2376,
    19.6601,
    15.8274,
    10.2925,
]
```

### Plot the premixed flame solution profiles

Plot the predicted flame speeds against the experimental data.

```
plt.plot(data_equiv, data_speed, label="data", linestyle="", marker="^", color="blue")
plt.plot(equival, flamespeed, label="GRI 3.0", linestyle="-", color="blue")
plt.legend()
plt.ylabel("Flame Speed [cm/sec]")
plt.xlabel("Equivalence Ratio")

# clean up
ck.done()

# plot results
if interactive:
    plt.show()
else:
```

(continues on next page)

(continued from previous page)

```
    plt.savefig("plot_flame_speed_threading.png", bbox_inches="tight")
```

### 22.12.4 Perform brute-force sensitivity analysis on multiple processors

The brutal force sensitivity analysis for ignition delay time shown in this example is based on the equation below:

$$S[i] = ln(IDT_1/IDT_2)/ln(perturb_factor^2)$$

$IDT_1$ is the ignition delay times when $A$ factor of reaction $i$ is multiplied by $perturb_factor$ $IDT_2$ is the ignition delay times when $A$ factor of reaction $i$ is multiplied by $1/perturb_factor$

*In literatures of combustion research, the $perturb_factor$ is usually 2.0.*

To conduct the brute-force A-factor sensitivity analysis, you will have to define the conditions in function `run_cases()`, then define the mechanism files, perturb factor, and number of processors in function `run_sens()`. Running sensitivity on multiple processors is a more efficient approach, especially when the mechanism is large. A known limit is that this script would skip reactions with negative $A$ factors. Therefore, the sensitivity of reactions in duplicate format with one or more negative $A$ factors may not be calculated correctly.

This example is created by Kuiwen Zhang.

#### Import PyChemkin package and start the logger

```
import multiprocessing as mp  # multi processors
import os
from pathlib import Path
import time

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color

# chemkin batch reactor models (transient)
from ansys.chemkin.core.batchreactors.batchreactor import (
    GivenVolumeBatchReactorEnergyConservation as BatchReactor,
)

# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

#### Run a list of cases

This function runs a list of $i$ cases. In each case, the $A$ factor of reaction $ids[i]$ will be multiplied by $factors[i]$.

```
def run_cases(work_path: str, chemfile: str, thermfile: str, ids: list, factors: list):
    """
```

(continues on next page)

(continued from previous page)

```
    Run a list of cases for sensitivity analysis.

    Parameters
    ----------
    work_path : str
        The working directory for the cases.
    chemfile : str
        The full path to the Chemkin gas chemistry file.
    thermfile : str
        The full path to the Chemkin thermodynamic data file.
    ids : list
        A list of reaction indices for which the A factor will be modified.
    factors : list
        A list of factors by which the A factor of the corresponding reaction
        will be multiplied.

    """
    # check working directory
    os.chdir(work_path)
    # set verbose mode
    ck.set_verbose(False)

    ######################################
    # Create a chemistry set for all cases
    # ====================================
    my_gas_mech = ck.Chemistry(label="GRI 3.0")
    my_gas_mech.chemfile = chemfile
    my_gas_mech.thermfile = thermfile

    ###################################
    # Pre-process the ``Chemistry Set``
    # =================================
    _ = my_gas_mech.preprocess()

    ########################################################################
    # Set up gas mixtures based on the species in this ``Chemistry Set``
    # ====================================================================
    # Use the *equivalence ratio method* so that you can easily set up
    # the premixed fuel-oxidizer mixture composition by assigning an
    # the *equivalence ratio* value. In this case, the fuel mixture consists
    # of methane, ethane, and propane as the simulated "natural gas".
    # The premixed air-fuel mixture has an equivalence ratio of 1.1.
    oxid = ck.Mixture(my_gas_mech)
    oxid.x = [("O2", 0.21), ("N2", 0.79)]
    oxid.temperature = 900
    oxid.pressure = ck.P_ATM

    fuel = ck.Mixture(my_gas_mech)
    fuel.x = [("C3H8", 0.1), ("CH4", 0.8), ("H2", 0.1)]
    fuel.temperature = oxid.temperature
    fuel.pressure = oxid.pressure
```

(continues on next page)

(continued from previous page)

```
    equi = 1.1

    mixture = ck.Mixture(my_gas_mech)
    mixture.pressure = oxid.pressure
    mixture.temperature = oxid.temperature
    products = ["CO2", "H2O", "N2"]
    add_frac = np.zeros(my_gas_mech.kk, dtype=np.double)
    # create the air-fuel mixture by using the equivalence ratio method
    ierror = mixture.x_by_equivalence_ratio(
        my_gas_mech, fuel.x, oxid.x, add_frac, products, equivalenceratio=equi
    )
    # check fuel-oxidizer mixture creation status
    if ierror != 0:
        print("Error: Failed to create the fuel-oxidizer mixture.")
        exit()

    ################################################################
    # Create the reactor object for ignition delay time calculations
    # ==============================================================
    # Use ``GivenVolumeBatchReactorEnergyConservation`` to instantiate a
    # *constant volume batch reactor that also includes the energy equation*. You
    # should use the ``mixture`` you just created.
    conv_bomb = BatchReactor(mixture, label="CONV")
    # show initial gas composition inside the reactor for verification
    conv_bomb.list_composition(mode="mole")

    # ########################################
    # Set up additional reactor model parameters
    # ========================================
    conv_bomb.volume = 10.0
    conv_bomb.pressure = 20.0 * ck.P_ATM
    # simulation end time [sec]
    conv_bomb.time = 0.1

    # turn ON adaptive solution saving
    conv_bomb.adaptive_solution_saving(mode=True, steps=20)
    # set ignition delay
    conv_bomb.set_ignition_delay(method="T_inflection")

    # set tolerances in tuple: (absolute tolerance, relative tolerance)
    conv_bomb.tolerances = (1.0e-10, 1.0e-8)

    ###################################
    # Change A factor and run the cases
    # =================================
    # Get original A factors
    afactor, beta, active_energy = my_gas_mech.get_reaction_parameters()
    # create sensitivity coefficient array
    idts = []
    # Create a file for sensitivity analysis results output
    with Path("sen_output.txt").open("w", encoding="utf-8") as f:
        f.write(f"cases to run = {len(ids)}, from {ids[0] + 1} to {ids[-1] + 1}\n")
```

(continues on next page)

(continued from previous page)

```
        f.flush()
        # loop over all reactions in the array
        for i in range(len(ids)):
            # Skip negative A factors
            if afactor[ids[i]] < 0:
                delaytime = conv_bomb.time
            else:
                afactor_new = afactor[ids[i]] * factors[i]
                # actual reaction index
                ireac = ids[i] + 1
                # update the A factor
                my_gas_mech.set_reaction_afactor(ireac, afactor_new)
                # run the reactor model
                conv_bomb.stop_output()
                runstatus = conv_bomb.run()
                if runstatus == 0:
                    # get ignition delay time
                    delaytime = conv_bomb.get_ignition_delay()
                    # restore the A factor
                    my_gas_mech.set_reaction_afactor(ireac, afactor[ids[i]])
                else:
                    # if get this, most likely the END time is too short
                    print(f"trouble finding ignition delay time for reaction {ireac}")
                    print(Color.RED + ">>> Run failed. <<<", end=Color.END)
                    # Set ignition delay time to simulation time limit
                    delaytime = conv_bomb.time
            idts.append(delaytime)
            # write reaction id and ignitino delay time to output file
            f.write(f"{ids[i] + 1}, {delaytime}\n")
            f.flush()
    # f.close()  # No need to close explicitly when using 'with' context
    # Return the ignition delay times
    return idts
```

### Run brutal force sensitivity analysis on multiple processors

```
def run_sens():
    """
    Run brutal force sensitivity analysis on multiple processors.

    This function sets up the mechanism, divides the tasks among multiple processors,
    and computes the sensitivity coefficients for the ignition delay times.
    """
    #
    current_dir = str(Path.cwd())
    num_processors = 6
    perturb = 1

    perturb_factor = (
        1.0 + perturb
    )  # A factor is multiplied and divided by 2 in sensitivity analysis
```

(continues on next page)

(continued from previous page)

```
    # set mechanism directory (the default Chemkin mechanism data directory)
    data_dir = Path(ck.ansys_dir) / "reaction" / "data"
    mechanism_dir = data_dir
    # create a chemistry set based on the diesel 14 components mechanism
    my_gas_mech = ck.Chemistry(label="GRI 3.0")
    # set mechanism input files
    # including the full file path is recommended
    my_gas_mech.chemfile = str(mechanism_dir / "grimech30_chem.inp")
    my_gas_mech.thermfile = str(mechanism_dir / "grimech30_thermo.dat")

    # set the start wall time to get the total simulation run time
    start_time = time.time()
    ierror = my_gas_mech.preprocess()
    if ierror == 0:
        print("mechanism information:")
        print(f"number of gas species = {my_gas_mech.kk:d}")
        print(f"number of gas reactions = {my_gas_mech.ii_gas:d}")
    else:
        # When a non-zero value is returned from the process,
        # check the text output files
        # chem.out, tran.out, or summary.out for potential error messages
        # about the mechanism data.
        print(f"Preprocessing error encountered. Code = {ierror:d}.")
        print(f"see the summary file {my_gas_mech.summaryfile} for details")
        exit()

    ids = np.arange(my_gas_mech.ii_gas)
    ids = np.repeat(ids, 2)
    factors = np.tile([perturb_factor, 1 / perturb_factor], my_gas_mech.ii_gas)
    base_num_task = len(ids) // num_processors  # basic number of tasks per core
    num_shift = len(ids) % num_processors  # number of cores taking +1 task

    args = []
    offset = 0
    # make a new directory for each array of cases and run the cases on
    # multiple processors
    for i in range(num_processors):
        work_path = Path(current_dir) / "RUN_" / str(i + 1).zfill(4)
        if not work_path.exists():
            work_path.mkdir(parents=True, exist_ok=True)
        if i < num_shift:
            div = base_num_task + 1
        else:
            div = base_num_task
        args.append(
            (
                work_path,
                my_gas_mech.chemfile,
                my_gas_mech.thermfile,
                ids[offset : offset + div],
                factors[offset : offset + div],
            )
```

(continues on next page)

(continued from previous page)

```
        )
        offset += div
    with mp.Pool(processes=num_processors) as pool:
        results = pool.starmap(run_cases, args)
    flat_results = np.concatenate(results)

    sens = (np.log(flat_results[0::2]) - np.log(flat_results[1::2])) / (
        np.log(perturb_factor * perturb_factor)
    )

    top = 10
    # rank the positive coefficients
    posindex = np.argpartition(sens, -top)[-top:]
    poscoeffs = sens[posindex]
    pos_sorted_index = np.argsort(poscoeffs)
    pos_sorted = poscoeffs[pos_sorted_index]

    # rank the negative coefficients
    neg_ig_sen = np.negative(sens)
    negindex = np.argpartition(neg_ig_sen, -top)[-top:]
    negcoeffs = sens[negindex[::-1]]
    neg_sorted_index = np.argsort(negcoeffs)
    neg_sorted = negcoeffs[neg_sorted_index]

    # print the top sensitivity coefficients
    print("positive sensitivity coefficients")
    for i in range(top):
        print(
            f"reaction {posindex[np.flip(pos_sorted_index)[i]] + 1}: "
            f"coefficient = {np.flip(pos_sorted)[i]}"
        )
    print()
    print("negative sensitivity coefficients")
    for i in range(top):
        print(
            f"reaction {negindex[neg_sorted_index[i]] + 1}: "
            f"coefficient = {neg_sorted[i]}"
        )

    # compute and report the total runtime (wall time)
    runtime = time.time() - start_time
    print(
        f"\ntotal simulation time: {runtime} [sec] over {my_gas_mech.ii_gas + 1} runs"
    )

    ########################################
    # Plot the ranked sensitivity coefficients
    # ========================================
    # Create plots to show the reactions whose A-factors have most positive
    # and negative influence on the ignition delay time.
    plt.rcParams.update({"figure.autolayout": True, "ytick.color": "blue"})
    plt.subplots(2, 1, sharex="col", figsize=(10, 5))
```

(continues on next page)

(continued from previous page)

```
    # convert reaction # from integers to strings
    rxnstring = []
    for i in range(top):
        # the array index starting from 0 so the actual reaction # = index + 1
        rxnid = posindex[pos_sorted_index[i]] + 1
        # add reaction index before reaction string
        rxnstring.append(str(rxnid) + ". " + my_gas_mech.get_gas_reaction_string(rxnid))
    # use horizontal bar chart
    plt.subplot(211)
    plt.barh(rxnstring, pos_sorted, color="orange", height=0.4)
    plt.axvline(x=0, c="gray", lw=1)
    # convert reaction # from integers to strings
    rxnstring.clear()
    for i in range(top):
        # the array index starting from 0 so the actual reaction # = index + 1
        rxnid = negindex[neg_sorted_index[i]] + 1
        rxnstring.append(str(rxnid) + ". " + my_gas_mech.get_gas_reaction_string(rxnid))
    plt.subplot(212)
    plt.barh(rxnstring, neg_sorted, color="orange", height=0.4)
    plt.axvline(x=0, c="gray", lw=1)
    plt.xlabel("Sensitivity Coefficients")
    plt.suptitle("Ignition Delay Time Sensitivity", fontsize=16)

    # plot results
    if interactive:
        plt.show()
    else:
        plt.savefig("plot_sensitivity_analysis_multiproc.png", bbox_inches="tight")
    return


if __name__ == "__main__":
    run_sens()
```

## 22.13 Advanced Applications

Examples in this section are to showcase the use of Pychemkin in more complicated applications. You can make improvements to a Pychemkin reactor model by adding new physical processes. Or, you can have Pychemkin interact with other Python applications to address complex problems.

### 22.13.1 Find the optimal operating parameters for a given SI engine

Finding the optimal solution is one of the prominent parts of the research and development activities. When it comes to *PyChemkin*, reaction mechanism (rate parameters) optimization, fuel surrogate blend optimization, and process/reactor parameters optimization are some of the common applications.

This example project describes the steps of integrating *PyChemkin* and *pymoo*, a well-known Python multi-objectives optimization package, to determine the optimal combination of the "start of combustion (SOC) timing" and the "exhaust gas recirculation (EGR) ratio" such that the engine would generate the highest power output with minimal nitric oxide (NO) emission and no engine knock. The engine power output in this example is measured by the indicated mean effective pressure (IMEP) between the intake valve close (IVC) and the exhaust valve open (EVO). The NO emission is indicated by the cylinder-averaged peak NO mass fraction during the same time period. The engine knock is defined

as the occurrence of auto-ignition of the "endgas" which is the gas mixture of the "unburned zone" of the SI engine model. The intensity of the engine knock in this example is represented by the peak value of the total thermicity of the endgas. When the peak thermicity value of a particular engine run goes above the preset threshold, the occurrence of engine knock is predicted. Consequently, the operating condition and the solution associated with the run will be discarded because of the violation of the no engine knock constraint enforced by the optimization process.

The multi-objective algorithm `NGSA-II` is employed to perform the optimization. The best pair of SOC and EGR ratio that satisfies the no engine knock constraint will be "determined" by applying a "decision-making" method from the *pymoo* package. The parameters and the solutions of cases around the Pareto frontier of the optimization process will be plotted in the "design space" and the "objective space", respectively, with the best parameter/solution marked.

Details of the Chemkin 0-D SI engine model are available at the "Ansys Product Help" site. Information about *pymoo* can be found online at pymoo.org.

### Import PyChemkin packages and start the logger

```
import datetime
import multiprocessing
import os
from pathlib import Path

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

# pymoo: multi-objective optimization in Python
from pymoo.algorithms.moo.nsga2 import NSGA2
from pymoo.core.problem import ElementwiseProblem
from pymoo.decomposition.asf import ASF
from pymoo.operators.sampling.lhs import LHS
from pymoo.optimize import minimize
from pymoo.parallelization.starmap import StarmapParallelization

import ansys.chemkin.core as ck  # Chemkin

# chemkin spark ignition (SI) engine model (transient)
from ansys.chemkin.core.engines.SI import SIengine
from ansys.chemkin.core.inlet import Stream  # external gaseous inlet
from ansys.chemkin.core.logger import logger
from ansys.chemkin.core.utilities import WorkingFolders
```

### Create an SI engine calculator class

Create a local class that wraps around the actual `SIengine` class to make the setup of the SI engine parameter study more convenient.

```
class SIengineCalculator:
    """SI engine calculator with fixed set up parameters."""

    def __init__(self, fresh_mixture: Stream):
        """Create an SI engine calculator."""
        """
        Create an SI engine calculator that instantiates an SI engine object with
```

(continues on next page)

(continued from previous page)

```
        the given fresh (unburnt) mixture condition and predefined engine
        parameters.

        Parameters
        ----------
            fresh_mixture: Mixture object
                the initial/fresh/unburnt condition
        """
        # instantiate an SI engine object
        # set up the run and working directory name
        self.name = "SI_engine"
        self.si_calculator = SIengine(fresh_mixture, self.name)
        # run status
        self.runstatus = -100
        # IMEP (objective #1)
        self.imep = 0.0
        # peak NO mass fraction (objective #2)
        self.max_no = 1.0
        # calculated maximum thermicity value in the unburned zone
        # [1/sec] (objective #3)
        self.max_sigma = 1.0e5
        # peak endgas temperature [K]
        self.max_temp = 0.0
        # set the required premixed flame model parameters
        self.set_engine_parameters()
        # set burn profile (variables/parameters)
        # the burn profile will be set right before each run during
        # the optimization process

    def run(self) -> int:
        """Run the SI engine calculation."""
        """
        Run individual SI engine calculation.

        Returns
        -------
            status: integer
                run status code
        """
        # reset status
        self.runstatus = -100
        # run the flame speed calculation
        self.runstatus = self.si_calculator.run()
        # extract the laminar flame speed from the solution
        if self.runstatus == 0:
            # postprocess the unburned zone solution
            unburnedzone = 1
            # burnedzone = 2
            # because the memory is shared, it must be done as soon as
            # the run is finished
            # get solution variables from the unburned zone
            ierr = self.si_calculator.process_engine_solution(zone_id=unburnedzone)
```

(continues on next page)

(continued from previous page)

```
            if ierr == 0:
                temperature = self.si_calculator.get_solution_variable_profile(
                    "temperature"
                )
                thermicity = self.si_calculator.get_solution_variable_profile(
                    "thermicity"
                )
                self.imep = self.si_calculator.get_engine_imep()
                self.max_temp = np.max(temperature)
                self.max_sigma = np.max(thermicity)
                # get the cylinder-averaged solution variables
                ierr = self.si_calculator.process_average_engine_solution()
                no_mass_frac = self.si_calculator.get_solution_variable_profile("no")
                self.max_no = np.max(no_mass_frac)
                del thermicity, temperature, no_mass_frac
                self.runstatus = ierr
            else:
                self.runstatus = ierr
        return self.runstatus

    def set_engine_parameters(self):
        """Set up basic engine parameters."""
        """
        Set up basic engine parameters related to key geometries and
        operating conditions.
        """
        # Set up basic engine parameters
        # These engine parameters are used to describe the cylinder volume during the
        # simulation. The ``starting_CA`` argument should be the crank angle
        # corresponding to the cylinder IVC. The ``ending_CA`` argument is typically
        # the EVO crank angle. cylinder bore diameter [cm]
        self.si_calculator.bore = 8.5
        # engine stroke [cm]
        self.si_calculator.stroke = 10.82
        # connecting rod length [cm]
        self.si_calculator.connecting_rod_length = 17.853
        # compression ratio [-]
        self.si_calculator.compression_ratio = 12
        # engine speed [RPM]
        self.si_calculator.rpm = 900
        # simulation start CA [degree]
        self.si_calculator.starting_ca = -120.2
        # simulation end CA [degree]
        self.si_calculator.ending_ca = 139.8
        # wall heat transfer parameters
        # Set up engine wall heat transfer model
        heattransferparameters = [0.15, 0.8, 0.0]
        # set cylinder wall temperature [K]
        t_wall = 434.0
        self.si_calculator.set_wall_heat_transfer(
            "dimensionless", heattransferparameters, t_wall
        )
```

(continues on next page)

(continued from previous page)

```
        # in-cylinder gas velocity correlation parameter (Woschni)
        # [<C11> <C12> <C2> <swirl ratio>]
        gv_parameters = [2.28, 0.318, 0.324, 0.0]
        self.si_calculator.set_gas_velocity_correlation(gv_parameters)
        # set piston head top surface area [cm2]
        self.si_calculator.set_piston_head_area(area=56.75)
        # set cylinder clearance surface area [cm2]
        self.si_calculator.set_cylinder_head_area(area=56.75)
        # other engine model parameters
        # set tolerances in tuple: (absolute tolerance, relative tolerance)
        self.si_calculator.tolerances = (1.0e-12, 1.0e-8)
        # turn on the force non-negative solutions option in the solver
        self.si_calculator.force_nonnegative = False
        # set minimum zonal mass [g]
        self.si_calculator.set_minimum_zone_mass(minmass=1.0e-5)
        # set the maximum solver time step
        self.si_calculator.max_ca_step = 0.05  # 0.1
        # set the number of crank angles between saving solution
        self.si_calculator.ca_step_for_saving_solution = 0.5
        # set the number of crank angles between printing solution
        self.si_calculator.ca_step_for_printing_solution = 10.0
        # turn ON adaptive solution saving
        self.si_calculator.adaptive_solution_saving(mode=True, steps=20)
        # set to detect auto-ignition
        # self.si_calculator.set_ignition_delay(method="T_inflection")
        # set species lower bound for equilibrium calculation
        self.si_calculator.set_burned_products_minimum_mole_fraction(1.0e-12)
        # suppress text output from the SI engine model
        self.si_calculator.stop_output()

    def set_burn_profile(
        self,
        start_combustion: float,
        burn_duration: float,
        wiebe_n: float,
        wiebe_b: float,
    ):
        """Set up the fuel burn rate profile."""
        """
        Set up the fuel burn rate profile using the Wiebe function.

        Parameters
        ----------
            start_combustion: double
                start of combustion (SOC) crank angle [CA]
            burn_duration: double
                burn duration in crank angle [CA]
            wiebe_n: double
                Wiebe function n parameter value
            wiebe_b: double
                Wiebe function b parameter value
        """
```

(continues on next page)

(continued from previous page)

```
        # start of combustion CA
        self.si_calculator.set_burn_timing(
            soc=start_combustion,
            duration=burn_duration,
        )
        self.si_calculator.wiebe_parameters(n=wiebe_n, b=wiebe_b)

    def get_solution_parameters(self) -> tuple[float, float, float]:
        """Engine performance parameters."""
        """
        Get the SI engine performance parameters to be optimized
        from the solution.

        Returns
        -------
            imep: double
                indicated mean effective pressure [bar]
            max_no: double
                peak cylinder-averaged NO mass fraction [-]
            max_sigma: double
                peak total thermicity of the eng gas [1/sec]
        """
        return self.imep, self.max_no, self.max_sigma
```

### Create an instance of the Chemistry Set

For PRF, the encrypted 14-component gasoline mechanism, `gasoline_14comp_WBencrypted.inp`, is used. The chemistry set is named `gasoline`.

```
def create_chemistry_set() -> ck.Chemistry:
    """Create and preprocess the gasoline chemistry set."""
    # Change to root of the project folder
    os.chdir(top_dir)
    # mechanism directory and files
    data_dir = Path(ck.ansys_dir) / "reaction" / "data"
    mechanism_dir = data_dir
    # create a Chemistry object for the gasoline mechanism
    my_gas_mech = ck.Chemistry(label="Gasoline")
    # set the mechanism input file for the Chemistry object
    my_gas_mech.chemfile = str(mechanism_dir / "gasoline_14comp_WBencrypt.inp")
    # preprocess the mechanism files
    _ = my_gas_mech.preprocess()
    return my_gas_mech


def setup_fresh_mixture(
    my_gas_mech: ck.Chemistry,
    fuel_compo: list[tuple[str, float]],
    phi: float,
    egr_rate: float,
) -> Stream:
    """Set up the chemistry set and create the unburned mixture."""
```

(continues on next page)

(continued from previous page)

```
    """
    Set up the chemistry set and create the fresh fuel-air mixture at
    the intake valve closing (IVC).

    Parameters
    ----------
        my_gas_mech: ck.Chemistry
            the chemistry set for the fuel-air mixture
        fuel_compo: list[tuple[str, float]]
            fuel species mole fractions
        phi: double
            equivalence ratio
        egr_rate: double
            EGR rate ratio

    Returns
    -------
        fresh: Mixture object
            fresh unburned mixture (initial charge)
    """

    ################################################
    # Set up the stoichiometric gasoline-air mixture
    # ==============================================
    # You must set up the stoichiometric gasoline-air mixture for the subsequent
    # SI engine calculations. Here the ``x_by_equivalence_ratio()`` method is used.
    # You create the ``fuel`` and the ``air`` mixtures first. You then define the
    # *complete combustion product species* and provide the *additives* composition
    # if applicable.
    # Finally, you simply set ``equivalenceratio=1`` to create the stoichiometric
    # gasoline-air mixture.
    #
    # For PRF 90 gasoline, the recipe is ``[("ic8h18", 0.9), ("nc7h16", 0.1)]``.
    #
    # .. note::
    #   The "surrogate blend utility" from the Chemkin "reaction workbench" can be
    #   used to create a surrogate that matches the properties (heating value,
    #   distillation curve, density, viscosity, etc.) of an actual fuel.
    #

    # create the fuel mixture
    fuelmixture = ck.Mixture(my_gas_mech)
    # set fuel composition
    fuelmixture.x = fuel_compo
    # setting pressure and temperature is not required in this case
    fuelmixture.pressure = 1.15 * ck.P_ATM
    # Update the fresh mixture temperature according to the EGR ratio
    # intake gas temperature [K]
    temp_intake = 310.0
    # engine exhaust gas temperature [K]
    temp_egr = 950.0
    # gas charge temperature [K]
```

(continues on next page)

(continued from previous page)

```
    fuelmixture.temperature = (1.0 - egr_rate) * temp_intake + egr_rate * temp_egr

    # create the oxidizer mixture: air
    air = ck.Mixture(my_gas_mech)
    air.x = [("o2", 0.21), ("n2", 0.79)]
    # setting pressure and temperature is not required in this case
    air.pressure = fuelmixture.pressure
    air.temperature = fuelmixture.temperature

    # products from the complete combustion of the fuel mixture and air
    products = ["co2", "h2o", "n2"]
    # species mole fractions of added/inert mixture.
    # You can also create an additives mixture here.
    add_frac = np.zeros(my_gas_mech.kk, dtype=np.double)  # no additives: all zeros

    # create the unburned fuel-air mixture
    fresh = ck.Mixture(my_gas_mech)

    # mean equivalence ratio
    equiv = phi
    _ = fresh.x_by_equivalence_ratio(
        my_gas_mech, fuelmixture.x, air.x, add_frac, products, equivalenceratio=equiv
    )

    ############################################################
    # Specify pressure and temperature of the fuel-air mixture
    # ============================================================
    # Since you are going to use ``fresh`` fuel-air mixture to instantiate
    # the engine object later, setting the mixture pressure and temperature
    # is equivalent to setting the initial temperature and pressure of the
    # engine cylinder.
    fresh.pressure = fuelmixture.pressure
    fresh.temperature = fuelmixture.temperature

    ##########################################
    # Add EGR to the fresh fuel-air mixture
    # ==========================================
    # Many engines have the configuration for exhaust gas recirculation (EGR). Chemkin
    # engine models let you add the EGR mixture to the fresh fuel-air mixture entering
    # the cylinder. If the engine you are modeling has EGR, you should have
    # the EGR ratio, which is generally the volume ratio of the EGR mixture and
    # the fresh fuel-air mixture.
    # However, because you know nothing about the composition of the exhaust gas,
    # you cannot simply combine these two mixtures. In this case, you use the
    # ``get_egr_mole_fraction()`` method to estimate the major components of
    # the exhaust gas from the combustion of the fresh fuel-air mixture. The
    # ``threshold=1.0e-8`` parameter tells the method to ignore any species with
    # a mole fraction below the threshold value. Once you have the EGR mixture
    # composition, use the ``x_by_equivalence_ratio()`` method a second time to
    # re-create the fuel-air mixture ``fresh`` with the original ``fuelmixture`` and
    # ``air`` mixtures, along with the EGR composition that you just got as the
    # *additives*.
```

(continues on next page)

(continued from previous page)

```
    egr_ratio = egr_rate
    # compute the EGR stream composition in mole fractions
    add_frac = fresh.get_egr_mole_fraction(egr_ratio, threshold=1.0e-8)
    # recreate the initial mixture with EGR
    ierror = fresh.x_by_equivalence_ratio(
        my_gas_mech,
        fuelmixture.x,
        air.x,
        add_frac,
        products,
        equivalenceratio=equiv,
        threshold=1.0e-8,
    )
    if ierror != 0:
        print("error...creating the initial fuel-oxidizer mixture.")
    return fresh
```

### Set up the optimization problem

Define the optimization problem. Set up the optimization parameters, variables, objectives, constraints, and the bounds of the variables.

Since the `_evaluate()` method perform one SI engine simulation at a time, the optimization problem is defined as an `ElementwiseProblem`. By implementing the SI engine model as an element-wise problem, the parallelization available in *pymoo* can be directly utilized.

There are two design parameters/variables in this project. The variable SOC is used to set up the mass burned fraction profile of the SI engine model. And the EGR ratio is applied to create the initial gas charge inside the engine cylinder. The objective and the constraint values come from the result of the SI engine simulation. The first objective is the negative value of IMEP, the second objective is the peak NO mass fraction in the cylinder, and the constraint value is the knock intensity as indicated by the total thermicity of the endgas.

In *pymoo*, the objectives are "minimized" so the objective of maximizing the IMEP must be reformulated into minimizing the negative value of IMEP, and the constraint must be written in the form of "g <= 0.0".

**Note**

See the *pymoo* documentation for details about how the optimization problem is set up and evaluated.

```
class SIEngineOptimization(ElementwiseProblem):
    """SI engine optimization object."""

    def __init__(
        self,
        numb_vars: int,
        numb_objs: int,
        fuel: list[tuple[str, float]],
        phi: float,
        **kwargs,
    ):
        """Set up the SI engine optimization."""
        """
```

(continues on next page)

(continued from previous page)

```
        Set up the SI engine optimization.
        The objective is to find the optimal fuel mass burn profile to
        maximize the engine indicated mean effective pressure (IMEP) while
        avoiding engine knock and minimizing the NO emission.
        The fuel mass burn profile is described by the anchor points:
        crank angles at 10%, 50%, and 90% mass burned.

        Parameters
        ----------
            numb_vars: integer
                number of variables
            numb_objs: integer
                number of objectives
            fuel: tuple (str, double)
                fuel species mole fractions
            phi: double
                equivalence ratio of the initial gas charge
        """
        # number of inequality constraints
        numb_constrs = 1
        # lower bounds for the variables
        lower_bounds = np.zeros(numb_vars, dtype=np.double)
        upper_bounds = np.zeros(numb_vars, dtype=np.double)
        lower_bounds[0] = -15.0
        lower_bounds[1] = 0.0
        # upper bounds for the variables
        upper_bounds[0] = 10.0
        upper_bounds[1] = 0.60
        # initialize the "Problem" object
        super().__init__(
            n_var=numb_vars,
            n_obj=numb_objs,
            n_ieq_constr=numb_constrs,
            n_eq_constr=0,
            xl=lower_bounds,
            xu=upper_bounds,
        )
        # number of evaluation calls
        self.call_count = 0
        self.fail_count = 0
        self.failed_cases: list[int] = []
        # initial gas charge parameters
        # fuel composition
        self.fuel = fuel
        # equivalence ratio
        self.phi = phi
        # create and preprocess the chemistry set
        self.my_gas_mech = create_chemistry_set()

    def _evaluate(self, x, out, *args, **kwargs):
        """Evaluate the objective function values."""
        """
```

(continues on next page)

(continued from previous page)

```
        Evaluate the objective function values with the given
        set of variable values.

        Parameters
        ----------
            x: list[float]
                values of the pymoo variables to be optimized.
            out: dict[str: list[float]]
               pymoo OUT dictionary containing evaluated function values and
               constrain values
        """
        # initialize "bad" return values to drive the search away from this state point
        f1 = 0.0
        f2 = 1.0
        #
        g1 = 5.0e4
        # variables
        # x[0]: start of combustion crank angle
        # x[1]: exhaust gas recirculation (EGR) rate
        # create and change to the working directory for this run
        name = "SI_engine_" + str(self.call_count)
        sub_work_folder = WorkingFolders(name, top_dir)
        # set up the unburned mixture of the SI engine at IVC
        init_mixture = setup_fresh_mixture(self.my_gas_mech, self.fuel, self.phi, x[1])
        # instantiate the SI engine
        engine_case = SIengineCalculator(init_mixture)
        # set the burned mass fraction profile
        # use two sets of Wiebe function parameters to describe
        # the burned mass fraction profile in the SI engine
        # profile #1: fast burn
        wiebe_n_egr = 1.8
        wiebe_b_egr = 8.5
        # profile #2: regular burn
        wiebe_n_fresh = 2.0
        wiebe_b_fresh = 8.0
        # EGR ratio
        r = x[1]
        r1 = 1.0 - r
        # burned mass profile as a function of EGR ratio
        wiebe_b = r * wiebe_b_egr + r1 * wiebe_b_fresh
        wiebe_n = r * wiebe_n_egr + r1 * wiebe_n_fresh
        burn_duration = 45.5  # degrees
        # set up the mass burned profile in the SI engine
        # the burn duration is fixed
        engine_case.set_burn_profile(x[0], burn_duration, wiebe_n, wiebe_b)
        # objects
        # f1: IMEP [bar] (to be maximized)
        # f2: peak cylinder-averaged NO mass fraction [-] (to be minimized)
        runstatus = engine_case.run()
        #
        if runstatus == 0:
            # get the results only when the simulation is successful
```

(continues on next page)

(continued from previous page)

```
            f1, f2, max_sigma = engine_case.get_solution_parameters()
            # compute the constraints
            # g1: minimal engine knock
            # max_sigma: peak endgas total thermicity (knock indicator) [1/sec]
            g1 = max_sigma - 5.0e4
        else:
            # tally the number of failed runs
            self.fail_count += 1
            self.failed_cases.append(self.call_count)
            # use bad values for the objectives and constraints
            f1 = -1.0e3  # a very low IMEP
            f2 = 1.0  # a high NO mass fraction
            g1 = 1.0e3  # a large constraint violation
        # clean up
        del engine_case
        sub_work_folder.done()
        #
        self.call_count += 1
        # return solution values to the optimizer
        # return negative solution value for maximization objective
        out["F"] = [-f1, f2]
        # return constraints
        out["G"] = [g1]
```

### Run the optimization

Select the optimization algorithm and the decision making method. Since this project has two objectives and one constraint, the multi-objective algorithm `NSGA-II` is selected as the optimization algorithm. A relatively small population size of 50 is used for this simple optimization problem.

The `StarmapParallelization()` method from *pymoo* is used to parallelize the SI engine simulation runs during the optimization process.

```
def run_optimization(
    numb_vars: int, numb_objs: int, fuel: list[tuple[str, float]], phi: float
):
    """Run the optimization process."""
    """
    Run the SI engine optimization process with pymoo.

    Parameters
     ----------
        numb_vars: integer
            number of variables
        numb_objs: integer
            number of objectives
        fuel: tuple (str, double)
            fuel species mole fractions
        phi: double
            equivalence ratio of the initial gas charge

    Returns
```

(continues on next page)

(continued from previous page)

```
    -------
        res: pymoo Result object
            pymoo minimization results and history
    """
    # create the "problem" to be optimized
    # set up the optimization algorithm
    # NSGA: Non-dominated Sorting Genetic Algorithm
    # LHS: Latin Hypercube Sampling
    algorithm = NSGA2(
        pop_size=50,
        sampling=LHS(),
        eliminate_duplications=True,
    )
    # set up the thermination condition
    # run the optimization
    run_parallel = True
    if run_parallel:
        # parallel mode
        # initialize the process pool and create the runner
        # number of available cpu cores
        numb_cores = max(os.cpu_count(), 1)
        numb_workers = numb_cores - 2
        numb_workers = 4  # max(numb_workers, 1)
        pool = multiprocessing.Pool(numb_workers)
        runner = StarmapParallelization(pool.starmap)
        # define the problem by passing the starmap interface of the thread pool
        problem = SIEngineOptimization(
            numb_vars=numb_vars,
            numb_objs=numb_objs,
            fuel=fuel,
            phi=phi,
            elementwise_runner=runner,
        )
    else:
        # serial mode
        problem = SIEngineOptimization(
            numb_vars=numb_vars, numb_objs=numb_objs, fuel=fuel, phi=phi
        )

    res = minimize(
        problem,
        algorithm,
        termination=("n_gen", 1),
        seed=11,
    )
    # run summary
    # execution time [sec]
    print(f"Total wall time: {datetime.timedelta(seconds=res.exec_time)}")
    # run statistics
    print(f"Failed runs: {problem.fail_count}/{problem.call_count}")
    if problem.fail_count > 0:
        print("Optimization outcome may not be reliable due to failed runs.")
```

(continues on next page)

(continued from previous page)

```
        print("failed cases")
        for n in problem.failed_cases:
            print(f"run # {n}")
    ############################
    # Post-process the solutions
    # ==========================
    # Parse the solutions from the optimization process and display them with
    # the best solution marked by a green cross mark. The variable values are stored
    # in the ``Result.X`` array and the corresponding objective values are in
    # the ``Result.F`` array. Because the objective of maximizing the "IMEP" has to be
    # converted to a minimization objective in *pymoo*, an alternative objective
    # of minimizing "-IMEP" is adopted. Consequently, the raw -IMEP objective solution
    # must be converted back to IMEP to make sense of the optimization and the
    # subsequent "decision-making" results.
    #
    # In this project, the decomposition function ``ASF()`` is applied to retrieve
    # the best solution. Weight factors are used to make the IMEP objective slightly
    # more important than the NO emission objective.

    # process the results
    x = res.X
    f = res.F
    # convert the objective result back to the original IMEP
    imep = -1.0 * f[:, 0]
    #
    approx_ideal = f.min(axis=0)
    approx_nadir = f.max(axis=0)
    # finding the optimal result
    # normalize the results
    diff = approx_ideal - approx_nadir
    if np.isclose(diff[0], 0.0, atol=1.0e-9) or np.isclose(diff[1], 0.0, atol=1.0e-9):
        nf = f - approx_ideal
    else:
        nf = (f - approx_ideal) / (approx_ideal - approx_nadir)
    # set the weight factors
    weight = np.zeros(n_objs, dtype=np.double)
    # IMEP weight factor
    weight[0] = 1.0 / 0.55
    # NO emission weight factor
    weight[1] = 1.0 / 0.45
    # find the best solution
    # use the Augmented Scalarization Function (ASF) method
    decomp = ASF()
    i = decomp.do(nf, weight).argmin()
    print("Best outcome according to ASF:")
    print(f"run # = {i}")
    print(f"Predicted IMEP = {imep[i]} [bar]")
    print(f"Predicted peak NO mass fraction = {f[i, 1]} [-]")
    print(f"Start of combustion = {x[i, 0]} [degree CA ATDC]")
    print(f"EGR ratio = {x[i, 1] * 100.0} % vol.")

    # plot the design space
```

(continues on next page)

(continued from previous page)

```
    xl, xu = problem.bounds()
    plt.figure(figsize=(7, 5))
    plt.title("Design Space")
    plt.scatter(x[:, 0], x[:, 1], s=30, facecolors="none", edgecolors="b")
    plt.scatter(x[i, 0], x[i, 1], color="green", marker="x", s=75, label="Best")
    plt.xlim(xl[0], xu[0])
    plt.ylim(xl[1], xu[1])
    plt.ylabel("EGR ratio [-]")
    plt.xlabel("Start of Combustion [degree CA]")
    plt.legend()
    if interactive:
        plt.show()
    else:
        plt.savefig("plot_SI_engine_optimization_designspace.png", bbox_inches="tight")

    # plot the objective space to show the Pareto frontier
    plt.figure(figsize=(7, 5))
    plt.scatter(imep, f[:, 1], s=30, facecolors="none", edgecolors="b")
    # note that because IMEP = -F[:, 0], approx_ideal[0] and approx_nadir[0]
    # must be modified accordingly
    ideal_imep = -1.0 * approx_ideal[0]
    nadir_imep = -1.0 * approx_nadir[0]
    plt.scatter(
        ideal_imep,
        approx_ideal[1],
        facecolors="none",
        edgecolors="red",
        marker="*",
        s=100,
        label="Ideal Point (Approx)",
    )
    plt.scatter(
        nadir_imep,
        approx_nadir[1],
        facecolors="none",
        edgecolors="black",
        marker="p",
        s=100,
        label="Nadir Point (Approx)",
    )
    plt.scatter(imep[i], f[i, 1], color="green", marker="x", s=75, label="Best")
    plt.title("Objective Space - Power & Emission")
    plt.xlabel("IMEP [bar]")
    plt.ylabel("Peak NO mass fraction [-]")
    plt.legend()
    if interactive:
        plt.show()
    else:
        plt.savefig("plot_SI_engine_optimization_objspace.png", bbox_inches="tight")
    #
    del imep, nf, xl, xu
    #
```

(continues on next page)

(continued from previous page)

```
    return res
```

### Set up the main driver of the optimization process

A main method for the optimization process is necessary especially with the intention to run the design point cases in parallel.

You can further post-process the results from the optimization process. Here the correlations between the design variables (SOC and EGR ratio) and the objectives (IMEP and NO emission) are explored. You can consult *pymoo* online API document to find out additional information offered by the `Result` object.

```
if __name__ == "__main__":
    # check working directory
    current_dir = str(Path.cwd())
    # create a separate working folder for the optimization runs
    top_level = WorkingFolders("SI_optimize", current_dir)
    top_dir = top_level.work
    top_level.done()
    #
    logger.debug("project root directory: " + top_dir)
    # set verbose mode
    ck.set_verbose(False)
    # set interactive mode for plotting the results
    # interactive = True: display plot
    # interactive = False: save plot as a PNG file
    interactive = False
    # properties of the initial gas charge
    # fuel composition
    # the "surrogate blend utility" from the Chemkin "reaction workbench" can be
    # used to create a surrogate that matches the properties (heating value,
    # distillation curve, density, viscocity, etc.) of an actual fuel
    fuel = [("ic8h18", 0.6), ("nc7h16", 0.0), ("mch", 0.1), ("c6h5c3h7", 0.3)]
    # equivalence ratio
    phi = 1.0
    # perform optimization
    # number of variables
    n_vars = 2
    # number of objectives
    n_objs = 2
    opt_results = run_optimization(n_vars, n_objs, fuel, phi)
    # process the optimization results
    x = opt_results.X
    f = opt_results.F
    imep = -1.0 * f[:, 0]
    # plot correlations between the variable/parameter and the objectives
    plt.figure(figsize=(7, 5))
    plt.scatter(x[:, 0], imep, s=30, facecolors="none", edgecolors="r")
    plt.title("Correlation - SOC & Power")
    plt.ylabel("IMEP [bar]")
    plt.xlabel("Start of combustion [CA]")
    if interactive:
        plt.show()
```

(continues on next page)

(continued from previous page)

```
    else:
        plt.savefig("plot_SI_engine_optimization_correlation1.png", bbox_inches="tight")
    #
    plt.figure(figsize=(7, 5))
    plt.scatter(x[:, 1], imep, s=30, facecolors="none", edgecolors="r")
    plt.title("Correlation - EGR & Power")
    plt.ylabel("IMEP [bar]")
    plt.xlabel("EGR ratio [-]")

    # clean up
    ck.done()

    if interactive:
        plt.show()
    else:
        plt.savefig("plot_SI_engine_optimization_correlation2.png", bbox_inches="tight")
```

### 22.13.2 A pseudo stochastic scheme to simulate a stratified-charged compression-ignition engine

For stratified-charge compression ignition (SCCI) engines, the multi-zone homogeneous charged compression ignition (HCCI) model can be applied to address the composition and temperature variations of the initial gas charge. The incylinder gas mixture at IVC is "grouped" to form several imaginary 'fluid material zones' based on the local gas temperature and composition. Note that the 'fluid material zone' does not have to form a single continuous volume of gas mass, rather it can be a collection of many small gas masses through out the cylinder.

The multi-zone HCCI engine model assumes that fluid flow would simply relocate, distort, and break down (into smaller volumes) a 'fluid material zone', and there is no mass or energy exchange between the zones. That is, the gas mass in a 'fluid zone' will remain in the same zone during the entirety of the simulation. The only interaction allowed between the 'fluid zones' is the pressure work that erases any pressure differences in the zones by changing the zonal volumes.

The pseudo-chastic SCCI (PSCCI) engine model described in this example assumes that the turbulence scalar mixing processes should act as the mechanism to facilitate the gas species and enthalpy exchange between different 'fluid material zones'. That is, the zonal masses remain the same but the gas species are allowed to move across the zone boundaries by turbulence diffusion. The turbulence diffusion effect is imitated by the stochastic micro mixing process. The micro mixing process will be performed at preset 'pause' crank angles. The PSCCI engine model will stop at every 'pause' crank angle to perform the micro mixing process and restart the simulation till the last 'pause' crank angle (which should be the EVO) is reached. The micro mixing sub-model of the PSCCI engine has three model parameters: the micro mixing time step size (delta_time), the characteristic turbulence scalar mixing time scale (tau), and the mixing model parameter (Cmix). The PSCCI engine model is termed 'pseudo-chastic' instead of 'stochastic' because it performs the micro mixing events less frequently than what is typically required for a stochastic simulation to reduce the simulation time. By increasing the number of the 'pause' crank angles, the PSCCI engine model should behave more closely to a 'true' stochastic model.

This example is focused on describing the process of setting up the micro mixing model. You can refer to the multi-zone HCCI engine example for the steps to set up the initial zonal properties of the PSCCI model. You can also compare the results from the PSCCI engine model in this example to the results from the multi-zone HCCI engine model to 'extract' the impacts of the micro mixing process.

```
```

### Import PyChemkin packages and start the logger

```
import copy
from pathlib import Path

import matplotlib.pyplot as plt  # plotting
import numpy as np  # number crunching

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color

# Chemkin homogeneous charge compression ignition (HCCI) engine model (transient)
from ansys.chemkin.core.engines.HCCI import HCCIengine
from ansys.chemkin.core.logger import logger
from ansys.chemkin.core.microprocess import MicroMixing
from ansys.chemkin.core.utilities import random_pick_integers

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
# set verbose mode
ck.set_verbose(True)
# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Create a chemistry set

The mechanism to load is the GRI 3.0 mechanism for methane combustion. This mechanism and its associated data files come with the standard Ansys Chemkin installation in the `/reaction/data` directory.

```
# set mechanism directory (the default Chemkin mechanism data directory)
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
mechanism_dir = data_dir
# create a chemistry set based on the GRI mechanism
MyGasMech = ck.Chemistry(label="GRI 3.0")
# set mechanism input files
# including the full file path is recommended
MyGasMech.chemfile = str(mechanism_dir / "grimech30_chem.inp")
MyGasMech.thermfile = str(mechanism_dir / "grimech30_thermo.dat")
MyGasMech.tranfile = str(mechanism_dir / "grimech30_transport.dat")
```

### Preprocess the chemistry set

```
# preprocess the mechanism files
_ = MyGasMech.preprocess()
```

### Set up the fuel-air mixture

You must set up the fuel-air mixture inside the engine cylinder right after the intake valve is closed. Here the `x_by_equivalence_ratio()` method is used. You create the `fuelmixture` and the `air` mixtures first. You then define the *complete combustion product species* and provide the *additives* composition if there is any. Finally, you set the `equivalenceratio` value to create the fuel-air mixture. In this case, the fuel mixture consists of methane, ethane, and propane as the simulated natural gas. Because HCCI engines generally run on lean fuel-air mixtures, the equivalence ratio is set to 0.8.

```
# create the fuel mixture
fuelmixture = ck.Mixture(MyGasMech)
# set fuel composition
fuelmixture.x = [("CH4", 0.9), ("C3H8", 0.05), ("C2H6", 0.05)]
# setting pressure and temperature is not required in this case
fuelmixture.pressure = 1.5 * ck.P_ATM
fuelmixture.temperature = 400.0
# create the oxidizer mixture: air
air = ck.Mixture(MyGasMech)
air.x = [("O2", 0.21), ("N2", 0.79)]
# setting pressure and temperature is not required in this case
air.pressure = 1.5 * ck.P_ATM
air.temperature = 400.0
# create the unburned fuel-air mixture
fresh = ck.Mixture(MyGasMech)
# products from the complete combustion of the fuel mixture and air
products = ["CO2", "H2O", "N2"]
# species mole fractions of added/inert mixture.
# can also create an additives mixture here
add_frac = np.zeros(MyGasMech.kk, dtype=np.double)  # no additives: all zeros
# mean equivalence ratio
equiv = 0.8
ierror = fresh.x_by_equivalence_ratio(
    MyGasMech, fuelmixture.x, air.x, add_frac, products, equivalenceratio=equiv
)
# check fuel-oxidizer mixture creation status
if ierror != 0:
    print("Error: Failed to create the fuel-oxidizer mixture.")
    exit()

# list the composition of the unburned fuel-air mixture
fresh.list_composition(mode="mole")
```

### Specify pressure and temperature of the fuel-air mixture

Since you are going to use the `fresh` fuel-air mixture to instantiate the engine object later, setting the mixture pressure and temperature is equivalent to setting the initial temperature and pressure of the engine cylinder.

```
fresh.temperature = 447.0
fresh.pressure = 1.065 * ck.P_ATM
```

### Add EGR to the fresh fuel-air mixture

Many engines have the configuration for exhaust gas recirculation (EGR). Chemkin engine models allow you to add the EGR mixture to the fresh fuel-air mixture entered the cylinder. If the engine you are modeling has EGR, you should have the EGR ratio, which is generally the volume ratio between the EGR mixture and the fresh fuel-air ratio. However, because you know nothing about the composition of the exhaust gas, you cannot simply combine these two mixtures. In this case, you use the `get_EGR_mole_fraction()` method to estimate the major components of the exhaust gas from the combustion of the fresh fuel-air mixture. The `threshold=1.0e-8` parameter tells the method to ignore any species with a mole fraction below the threshold value. Once you have the EGR mixture composition, use the `x_by_equivalence_ratio()` method a second time to re-create the `fresh` fuel-air mixture with the original `fuelmixture` and `air` mixtures along with the EGR composition you just got as the *"additives"*.

```
egr_ratio = 0.3
# compute the EGR stream composition in mole fractions
add_frac = fresh.get_egr_mole_fraction(egr_ratio, threshold=1.0e-8)
# recreate the initial mixture with EGR
ierror = fresh.x_by_equivalence_ratio(
    MyGasMech,
    fuelmixture.x,
    air.x,
    add_frac,
    products,
    equivalenceratio=equiv,
    threshold=1.0e-8,
)

# list the composition of the fuel+air+EGR mixture for verification
fresh.list_composition(mode="mole", bound=1.0e-8)
```

### Set up the HCCI engine reactor

Use the `HCCIengine()` method to create a multi-zone HCCI engine named `MyMZEngine` and make the new `fresh` mixture as the initial incylinder gas mixture at IVC. Set the `nzones` parameter to the number of zones in your multi-zone HCCI engine model.

```
# create a five-zone HCCI engine
numbzones = 5
MyMZEngine = HCCIengine(reactor_condition=fresh, nzones=numbzones)
# show initial gas composition inside the reactor
MyMZEngine.list_composition(mode="mole", bound=1.0e-8)
```

### Set basic engine parameters

Set the required engine parameters as shown in the following code. These engine parameters are used to describe the cylinder volume during the simulation. The `starting_ca` argument should be the crank angle corresponding to the cylinder IVC. The `ending_ca` is typically the EVC crank angle.

```
# cylinder bore diameter [cm]
MyMZEngine.bore = 12.065
# engine stroke [cm]
MyMZEngine.stroke = 14.005
# connecting rod length [cm]
MyMZEngine.connecting_rod_length = 26.0093
# compression ratio [-]
```

(continues on next page)

(continued from previous page)

```
MyMZEngine.compression_ratio = 16.5
# engine speed [RPM]
MyMZEngine.rpm = 1000

# set other parameters
# simulation start CA [degree]
MyMZEngine.starting_ca = -142.0
# simulation end CA [degree]
MyMZEngine.ending_ca = 116.0

# list the engine parameters
MyMZEngine.list_engine_parameters()
print(f"engine displacement volume {MyMZEngine.get_displacement_volume()} [cm3]")
print(f"engine clearance volume {MyMZEngine.get_clearance_volume()} [cm3]")
print(f"number of zone(s) = {MyMZEngine.get_number_of_zones()}")
```

### Set up the engine wall heat transfer model

By default, the engine cylinder is adiabatic. You must set up a wall heat transfer model to include the heat loss effects in your engine simulation. Chemkin support three widely used engine wall heat transfer models. The models and their parameters follow.

- `dimensionless`: [<a> <b> <c> <Twall>]
- `dimensional`: [<a> <b> <c> <Twall>]
- `hohenburg`: [<a> <b> <c> <d> <e> <Twall>]

There is also the in-cylinder gas velocity correlation (the Woschni correlation) that is associated with the engine wall heat transfer models. Here are the parameters of the Woschni correlation:

`[<C11> <C12> <C2> <swirl ratio>]`

You can also specify the surface areas of the piston head and the cylinder head for more precision heat transfer wall area. By default, both the piston head and the cylinder head surfaces are flat.

```
heattransferparameters = [0.035, 0.71, 0.0]
# set cylinder wall temperature [K]
t_wall = 400.0
MyMZEngine.set_wall_heat_transfer("dimensionless", heattransferparameters, t_wall)
# in-cylinder gas velocity correlation parameter (Woschni)
# [<C11> <C12> <C2> <swirl ratio>]
gv_parameters = [2.28, 0.308, 3.24, 0.0]
MyMZEngine.set_gas_velocity_correlation(gv_parameters)
# set piston head top surface area [cm2]
MyMZEngine.set_piston_head_area(area=124.75)
# set cylinder clearance surface area [cm2]
MyMZEngine.set_cylinder_head_area(area=123.5)
```

### Set zonal properties

By default, all zones in the multi-zone HCCI engine model have the same properties. You can artificially stratify the temperature and/or the equivalence ratio distribution in the cylinder at the IVC by utilizing the `set_zonal` methods of the `HCCI` object.

```
# initial zonal temperatures [K]
z_temperature = [447.5, 447.5, 447, 447, 447]
MyMZEngine.set_zonal_temperature(zonetemp=z_temperature)
# initial zonal volume fractions
z_volumefrac = [0.3, 0.25, 0.2, 0.2, 0.05]
MyMZEngine.set_zonal_volume_fraction(zonevol=z_volumefrac)
# initial wall heat transfer area fractions
z_htarea = [0.0, 0.15, 0.2, 0.25, 0.4]
MyMZEngine.set_zonal_heat_transfer_area_fraction(zonearea=z_htarea)
# initial zonal equivalence ratios
z_phi = [equiv, equiv, equiv, equiv, equiv]
MyMZEngine.set_zonal_equivalence_ratio(zonephi=z_phi)
# zonal EGR ratios
z_egrr = [0.3, 0.3, 0.3, 0.35, 0.35]
MyMZEngine.set_zonal_egr_ratio(zoneegr=z_egrr)
# set fuel "molar" composition
MyMZEngine.define_fuel_composition([("CH4", 0.9), ("C3H8", 0.05), ("C2H6", 0.05)])
# set oxidizer "molar' composition
MyMZEngine.define_oxid_composition([("O2", 0.21), ("N2", 0.79)])
# set products
MyMZEngine.define_product_composition(["CO2", "H2O", "N2"])
# set EGR composition in mole fractions
z_add = [add_frac, add_frac, add_frac, add_frac, add_frac]
MyMZEngine.define_additive_fractions(addfrac=z_add)
```

### Set output options

You can turn on the adaptive solution saving to resolve the steep variations in the solution profile. Here additional solution data points are saved for every 20 solver internal steps. You must include the `set_ignition_delay()` method for the engine model to report the ignition delay crank angle after the simulation is done. If `method="T_inflection"` is set, the reactor model treats the inflection points in the predicted gas temperature profile as the indication of an auto-ignition. You can choose a different auto-ignition definition.

**Note**

- Type `ansys.chemkin.show_ignition_definitions()` to get the list of all available ignition delay time definitions in Chemkin.
- The `set_ignition_delay()` method is required for the engine model to report the ignition delay time for each zone as well as the cylinder averaged ignition delay time derived from the cylinder averaged temperature profile.
- By default, time/crank angle intervals for both print and save solution are 1/100 of the simulation duration, which in this case is $dCA = (EVO - IVC)/100 = 2.58$. You can make the model report more frequently by using the `ca_step_for_saving_solution()` or the `ca_step_for_printing_solution()` method to set different interval values in crank angles.

```
# set the number of crank angles between saving solution
MyMZEngine.ca_step_for_saving_solution = 0.5
# set the number of crank angles between printing solution
MyMZEngine.ca_step_for_printing_solution = 10.0
# turn on adaptive solution saving
```

(continues on next page)

(continued from previous page)

```
MyMZEngine.adaptive_solution_saving(mode=True, steps=20)
# specify the ignition definitions
MyMZEngine.set_ignition_delay(method="T_inflection")
```

### Set solver controls

You can overwrite the default solver controls by using solver-related methods, such as those for tolerances.

```
# set tolerances in tuple: (absolute tolerance, relative tolerance)
MyMZEngine.tolerances = (1.0e-12, 1.0e-10)
# get solver parameters
atol, rtol = MyMZEngine.tolerances
print(f"Default absolute tolerance = {atol}.")
print(f"Default relative tolerance = {rtol}.")
# turn on the force non-negative solutions option in the solver
MyMZEngine.force_nonnegative = True
# show solver and output options
# show the number of crank angles between printing solution
print(
    f"Crank angles between solution printing: "
    f"{MyMZEngine.ca_step_for_printing_solution}"
)
# show other transient solver setup
print(f"Forced non-negative solution values: {MyMZEngine.force_nonnegative}")
```

### Display the added parameters (keywords)

Use the `showkeywordinputlines()` method to verify that the preceding parameters are correctly assigned to the engine model.

```
MyMZEngine.showkeywordinputlines()
```

### Set up the 'pseudo-cgastic' micro mixing model

The 'pseudo-chastic' micro mixing model option is turned on by setting the flag `do_micromixing` to 'True'. Then you should set the 'pause' points for performing the 'stochastic' micro mixing process. This can be achieved by using either the simulation intervals in crank angles or by specifying a series of 'pause' crank angles. In this example, a series of 'pause points' are defined by the `ca_stops` list. The second steps is to set the parameters of the micro mixing model. The micro mixing model currently available in the PSCCI engine model is the 'modified Curls model' which requires four parameters. The micro mixing time step size is the time during for the micro mixing process, `delta_time`. Typically, this time step size should be kept comparable to the solver time step size. Since Chemkin transient solver works differently from typical CFD solvers, the PSCCI engine model allows `delta_time` to be set independently of the time step size used by the solver. Subsequently the PSCCI engine model performs the micro mixing process less frequently and hence introduces larger bias and errors. In this example, `delta_time` is set to the crank angle interval between the 'pause points'. The second parameter is the number of statistical events/particles used in the stochastic process `numb_particles`. The number of particles is related to the 'statistical error' of the stochastic process. Typically 'statistical error' is inversely proportional to the square-root of the particle number. The third parameter for the micro mixing model is the 'scalar mixing time scale' `mixing_time_scale`. This parameter controls the intensity (or the scope) of the molecule-level mixing by flow turbulence, and its value should be set to the 'turbulence scalar mixing time scale' or the 'characteristic turbulence time scale' of the incylinder flow. The last parameter is the mixing model dependent parameters `c_mix`. For the 'modified Curls model', the default `c_mix` value is 1.0. This parameter controls the 'completeness' of the mixing of the particles. The higher the `c_mix` value, the

closer the particles resembling to each other after mixing. When `c_mix` is large enough, the particles are well-mixed and become identical after mixing.

When `do_micromixing` is set to 'True', the simulation will pause at each 'pause point'. Once the simulation is stopped, the micro mixing process is initiated by instantiating a micro mixing object `MicroMixing`. You will then specify the number of particles to be used in the micro mixing stochastic process by the `set_numb_particles()` method. You also have to extract the gas mixture properties of each 'fluid zone' from the multi-zone solution at the 'pause time' by using the `get_last_zone_mixtures()` method. You run the modified Curls micro mixing model by using the `modified_curls()` method with the model parameters and the zonal mixture objects obtained from the latest multi-zone simulation solution. Afterwards, use the `restart()` method with the updated zonal mixture properties from the micro mixing model to run the multi-zone simulation to the next 'pause point'. Repeat these steps till the simulation reaches the designated simulation end time (EVO).

**Note**

You can add more complexity to the model by including other processes during the pause. For example, you can imitate the ever-changing heat transfer area between the zones and the cylinder wall by randomly re-assigning the wall heat transfer area fractions of the zones. See the `random_wall_area` sections below for more details.

```
# set stopping/restarting CAs
# the last stop CA should be the actual simulation end time EVO
ca_stops = [-50.0, -20.0, -10.0, -5.0, 0.0, 5.0, 10.0, 20.0, 116.0]

# perform micro mixing process during restarts: set "do_micromixing = True"
do_micromixing = True
# do_micromixing = False
# total number of particles for the pseudo-chastic process
# the particles are shared among the zones in the HCCI engine
numb_particles = 400
# turbulence scalar mixing time scale [sec]
mixing_time_scale = 0.01
# mixing model parameter Cmix
mixing_model_parameter = 1.0

# randomly rearrange zonal wall heat transfer area: set "random_wall_area = True"
random_wall_area = False
if random_wall_area:
    # create a list for random pick
    area_list: list[int] = []
    for i in range(numbzones):
        area_list.append(i + 1)

# solution profile sets
crank_angle_data = []
pressure_data = []
temperature_data = []
volume_data = []
co_data = []
ch4_data = []
no_data = []

zone_temperature_data = []
zone_volume_data = []
```

(continues on next page)

(continued from previous page)

```
zone_co_data = []
```

### Run the simulation

Use the `run()` method to start the multi-zone HCCI engine simulation.

```
runcount = 0
#
for end_ca in ca_stops:
    # Run the simulation
    if runcount == 0:
        # simulation end CA [degree]
        MyMZEngine.ending_ca = end_ca
        runstatus = MyMZEngine.run()
        # check run status
        if runstatus != 0:
            # Run failed.
            print(Color.RED + ">>> Run failed. <<<", end=Color.END)
            exit()
        # Run succeeded.
        print(Color.GREEN + ">>> Run completed. <<<", end=Color.END)
    else:
        if random_wall_area:
            # re-assign the zonal wall heat transfer fractions
            picked, unpicked = random_pick_integers(numbzones, area_list)
            print(str(picked))
            this_area = copy.deepcopy(z_htarea)
            for i, a in enumerate(picked):
                this_area[i] = z_htarea[a - 1]
            print(str(this_area))
            MyMZEngine.set_zonal_heat_transfer_area_fraction(zonearea=this_area)
        if do_micromixing:
            # perform random micro mixing between particles
            # the mixing time step size is the time duration from the previous stop CA
            delta_time = MyMZEngine.get_time(
                MyMZEngine.ending_ca
            ) - MyMZEngine.get_time(MyMZEngine.starting_ca)
            # create a mixing instance
            zonemixing = MicroMixing()
            # set the total number of particles
            zonemixing.set_numb_particles(numb_particles)
            # get the zonal mixtures from the last multi-zone solution
            zonemixtures = MyMZEngine.get_last_zone_mixtures()
            # set up the particle properties based on the zonal mixtures
            zonemixing.set_particle_mixtures(zonemixtures)
            # perform micro-mixing of the particles to create mixed zonal mixtures
            # using the modified Curl's mixing model
            mixed_zones = zonemixing.modified_curls(
                delta_time,
                mixing_time_scale,
                mixing_model_parameter,
            )
```

(continues on next page)

(continued from previous page)

```
            # restart run(s)
            # use the mixed zonal mixtures to reset the simulation end CA [degree]
            runstatus = MyMZEngine.restart(end_ca=end_ca, new_mixtures=mixed_zones)
        else:
            # restart run(s)
            # reset the simulation end CA [degree]
            runstatus = MyMZEngine.restart(end_ca=end_ca)
        # check run status
        if runstatus != 0:
            # Run failed.
            print(Color.RED + ">>> Restart Run failed. <<<", end=Color.END)
            exit()
        # Run succeeded.
        print(Color.GREEN + ">>> Restart Run completed. <<<", end=Color.END)

    ######################################################
    # Postprocess the solution profiles in selected zone
    # ==================================================
    # The solution of the multi-zone HCCI engine model contains the results of
    # the individual zones plus the cylinder averaged results. This means that
    # if there are n zones in the multi-zone engine model, there are (n+1) solution
    # records: n zonal results and the cylinder averaged results.
    #
    # To process the result of the zone number :math:`j`\ , :math:`(1 \leq j \leq n)`\ ,
    # set the parameter value of ``zoneID`` to :math:`j` when you call the engine
    # postprocessor with the ``process_engine_solution()`` method. Otherwise, the
    # cylinder averaged results are postprocessed by default, that is, when the
    # ``zoneID`` parameter is omitted.
    #
    # .. note ::
    #   Because The ``process_engine_solution()`` method can process only one set of
    #   results at a time (one zonal result or the cylinder averaged result), you must
    #   postprocess the zones one by one to obtain all solution data of the multi-zone
    #   simulation.
    #
    thiszone = 1
    MyMZEngine.process_engine_solution(zone_id=thiszone)
    plottitle = "Zone " + str(thiszone) + " Solution"
    # get the number of solution time points
    solutionpoints = MyMZEngine.getnumbersolutionpoints()
    print(f"Number of solution points = {solutionpoints}.")
    # get the time profile
    timeprofile = MyMZEngine.get_solution_variable_profile("time")
    # convert time to crank angle
    ca_profile = np.zeros_like(timeprofile, dtype=np.double)
    count = 0
    for t in timeprofile:
        ca_profile[count] = MyMZEngine.get_ca(timeprofile[count])
        count += 1
    # get the cylinder pressure profile
    presprofile = MyMZEngine.get_solution_variable_profile("pressure")
    presprofile *= 1.0e-6
```

(continues on next page)

(continued from previous page)

```
    # create arrays for cylinder-averaged mixture temperature
    tempprofile = MyMZEngine.get_solution_variable_profile("temperature")
    # get the zonal volume profile
    volprofile = MyMZEngine.get_solution_variable_profile("volume")
    # get the zonal CO mass fraction profile
    co_profile = MyMZEngine.get_solution_variable_profile("CO")

    # post-process cylinder-averged solution
    # do NOT set the zoneID parameter
    MyMZEngine.process_average_engine_solution()
    # get the cylinder volume profile
    cylinder_volprofile = MyMZEngine.get_solution_variable_profile("volume")
    # create arrays for cylinder-averaged mixture temperature
    cylinder_tempprofile = MyMZEngine.get_solution_variable_profile("temperature")
    # get the cylinder-averaged CO mass fraction profile
    cylinder_co_profile = MyMZEngine.get_solution_variable_profile("CO")
    # get the cylinder-averaged CH4 mass fraction profile
    cylinder_ch4_profile = MyMZEngine.get_solution_variable_profile("CH4")
    # get the cylinder-averaged NO mass fraction profile
    cylinder_no_profile = MyMZEngine.get_solution_variable_profile("NO")

    # store solution profiles
    crank_angle_data.append(copy.deepcopy(ca_profile))
    pressure_data.append(copy.deepcopy(presprofile))
    volume_data.append(copy.deepcopy(cylinder_volprofile))
    temperature_data.append(copy.deepcopy(cylinder_tempprofile))
    co_data.append(copy.deepcopy(cylinder_co_profile))
    ch4_data.append(copy.deepcopy(cylinder_ch4_profile))
    no_data.append(copy.deepcopy(cylinder_no_profile))
    zone_volume_data.append(copy.deepcopy(volprofile))
    zone_temperature_data.append(copy.deepcopy(tempprofile))
    zone_co_data.append(copy.deepcopy(co_profile))
    # increase the run count
    runcount += 1
```

### Plot the engine solution profiles

Plot the zonal and the cylinder averaged profiles from the multi-zone HCCI engine simulation.

**Note**

You can get profiles of the thermodynamic and the transport properties by applying `Mixture` utility methods to the solution mixtures.

```
plt.subplots(2, 2, sharex="col", figsize=(12, 6))
plt.suptitle(plottitle, fontsize=16)
for i in range(len(crank_angle_data)):
    plt.subplot(221)
    plt.plot(crank_angle_data[i], pressure_data[i], "r-")
    plt.ylabel("Pressure [bar]")
    plt.subplot(222)
```

(continues on next page)

(continued from previous page)

```
    plt.plot(crank_angle_data[i], zone_volume_data[i], "b-")
    plt.plot(crank_angle_data[i], volume_data[i], "b--")
    plt.ylabel("Volume [cm3]")
    plt.legend(["Zone", "Cylinder"], loc="upper right")
    plt.subplot(223)
    plt.plot(crank_angle_data[i], zone_temperature_data[i], "g-")
    plt.plot(crank_angle_data[i], temperature_data[i], "g--")
    plt.xlabel("Crank Angle [degree]")
    plt.ylabel("Temperature [K]")
    plt.legend(["Zone", "Averaged"], loc="upper left")
    plt.subplot(224)
    plt.plot(crank_angle_data[i], zone_co_data[i], "m-")
    plt.plot(crank_angle_data[i], co_data[i], "m--")
    plt.xlabel("Crank Angle [degree]")
    plt.ylabel("CO mass fraction [-]")
    plt.legend(["Zone", "Averaged"], loc="upper left")

# clean up
ck.done()

# plot results
if interactive:
    plt.show()
else:
    plt.savefig("plot_pseudo_HCCI_engine.png", bbox_inches="tight")
```

### 22.13.3 Run chemkin applications in Python

PyChemkin does not support all chemkin functionalities and reactor models. However, it is still possible to run simulations of unsupported applications in Python.

This example describes the procedures to run any chemkin reactor model simulation in Python environment. By doing so, it opens up the possibility for interactions between chemkin applications and other applications under the Python environment. Thus, this example is primarily for Chemkin users (1) who know how to generate the application-specific input file and (2) who are familiar with the application keywords.

The entire process largely follows the same work flow for running any chemkin application from the command line. The first step is to identify the os platform and the correct location of the Ansys Chemkin installation. The second step is to set up and preprocess the `Chemistry Set`. The third step is to specify the application text input file, text output file, and the XML solution file. Then, create the batch script to run the simulation and post-process the solutions. The last step is to run the batch script created with the `subprocess.run()` method. The "final" solution data should be in MS EXCEL csv format stored in a number of "CKSoln_solution_xxx.csv" files. The original XML solution file should also be available.

The advantages of direct execution of the chemkin application (without using the PyChemkin methods) are

1. all chemkin applications are available;
2. access to all application options;
3. can run chemkin applications with custom-built user subroutines;
4. **complete set of solution (including the A-factor sensitivities) is created and** can be further post-processed by various utilities.

On the other hand, the disadvantages are

1. **problem setup is less interactive, you need to know how to manipulate the contents of** a text file to change options/parameters in Python;
2. additional steps are required to post-process the solution files.

This example employs the chemkin rotating-disk reactor model for silicon deposition from silane. You can easily modify this example for running other chemkin applications. For details of the chemkin applications and their inputs/options, please check out the "Theory" and the "Input" manuals at the Ansys Product Help website. Instructions of running chemkin applications from the command line and the "get solution" utilities can be found in the "Getting Started" and the "Visualization" manuals.

### Import PyChemkin packages and start the logger

```
from pathlib import Path
import platform
import subprocess
import time

import matplotlib.pyplot as plt
import pandas as pd

import ansys.chemkin.core as ck  # Chemkin
from ansys.chemkin.core import Color
from ansys.chemkin.core.chemistry import chemkin_bin_dir
from ansys.chemkin.core.logger import logger
from ansys.chemkin.core.utilities import copy_file, delete_files_by_extension

# set interactive mode for plotting the results
# interactive = True: display plot
# interactive = False: save plot as a PNG file
global interactive
interactive = True
```

### Establish the chemkin runtime environment

The batch script to establish the chemkin runtime environment is platform dependent. The first task is to find out the platform and the shell environment. Then you will run the environment setup script accordingly.

```
# set the application to be executed
# define the chemkin application
ck_app = "CKReactorRotatingDiskCVD"

# check platform
under_win = False
plat = platform.system()
if plat == "Windows":
    # Windows
    under_win = True
    # set the chemkin runtime environment batch script
    env_file = "run_chemkin_env_setup.bat"
    # define the chemkin application
    ck_app += ".exe"
```

(continues on next page)

(continued from previous page)

```
    # post-processer
    get_soln = "GetSolution.exe"
    soln_transpose = "CKSolnTranspose.exe"
elif plat == "Linux":
    # Linux
    # set the chemkin runtime environment for the /bash shell script
    env_file = "chemkin_setup.ksh"
    # post-processer
    get_soln = "GetSolution"
    soln_transpose = "CKSolnTranspose"
else:
    msg = [
        Color.RED,
        "unsupported platform",
        str(platform.system()),
        "\n",
        "PyChemkin does not support the current os.",
        Color.END,
    ]
    this_msg = Color.SPACE.join(msg)
    logger.critical(this_msg)
    exit()

# run the chemkin steup script to set up the chemkin runtime environment
# define the chemkin bin folder that holds all the executables
ckbin = chemkin_bin_dir()
# set the full path to the chemkin environment script
run_env_path = str(Path(ckbin) / env_file)
```

### Set up all directories required to run chemkin applications

Define the working directory `current_dir` used to run this Python script. Use the location of this Python source file `script_dir` to locate the data folder `example_mech_sata` containing the mechanism files and the application input files. And the data folder of the Ansys chemkin installation `data_dir` that stores common thermodynamic and the transport data files.

**Note**

This example uses mechanism files and input files from the 'examplesdata' folder, and assumes that this Python example file is located in the 'examplesadvanced' folder. Please verify the location and the availability of these files and modify the directories below to point to the correct files before running this example.

```
# get the working directory
current_dir_obj = Path.cwd()
current_dir = str(current_dir_obj)
logger.debug("working directory: " + current_dir)
# get the current py file location
try:
    script_dir_obj = Path(__file__).parent.resolve()
except NameError:
    script_dir_obj = current_dir_obj / ".." / "data"
```

(continues on next page)

(continued from previous page)

```
script_dir = str(script_dir_obj)
logger.debug("script directory: " + script_dir)
# use the relative path to find the location of the example mechanism files
example_mech_data = script_dir_obj / ".." / "data"
logger.debug("example input directory: " + str(example_mech_data))
data_dir = Path(ck.ansys_dir) / "reaction" / "data"
logger.debug("database directory: " + str(data_dir))
```

### Instantiate the Chemistry Set

Define the full path to the mechanism input files. This CVD example requires the gas mechanism, the surface mechanism, the thermodynamic data, and the transport data files. These files can be in different directories so make sure the correct directory path is used. For example, the mechanism files are in the 'examplesdata' folder, but the data files are in the 'data' folder under the Ansys Chemkin installation.

```
# set up all the mechanism input files for preprocessing
local_data_dir = "SICH4_CVD"
chem_input = str(example_mech_data / local_data_dir / "sih4_cvd_chem.inp")
surf_input = str(example_mech_data / local_data_dir / "sih4_cvd_surf.inp")
therm_input = str(data_dir / "therm.dat")
tran_input = str(data_dir / "tran.dat")
# create the Chemistry Set
MyCVDMech = ck.Chemistry(
    chem=chem_input,
    surf=surf_input,
    therm=therm_input,
    tran=tran_input,
    label="SiH4_CVD",
)
```

### Preprocess the surface chemistry set

Once the `Chemistry Set` is created, run the preprocessor. The *asc files will have their default names, for instance, 'chem.asc', and they will sit in the current working folder.

```
# preprocess the mechanism files
_ = MyCVDMech.preprocess()
```

### Prepare the application input files

Clean up the working folder by removing all application solution file. Copy the required input files from the 'example/data' folder to the working folder.

```
# delete the existing solution file *.zip
delete_files_by_extension(targetpath=current_dir, extensions=[".zip", ".dtd", ".bat"])
# copy the chemkindata.dtd from the data directory
copy_file(
    sourcepath=str(example_mech_data),
    targetpath=current_dir,
    filename="chemkindata.dtd",
)
```

(continues on next page)

(continued from previous page)

```
# define full path to the chemkin application
run_file = Path(ckbin) / ck_app
# check the application
if not run_file.is_file():
    print(f"Error...the Chemkin application {str(run_file)} does not exist.")
    print("   please make sure the path is correct...")
    exit()

# define the application text input and output files
app_input = "rotating_disk__sih4_cvd.inp"
app_output = "rotating_disk__sih4_cvd.out"
# define the full path to the text files
# the input file must be prepared before running the application
# the input file for this example is already prepared and is stored in the
# 'examples\data' folder
in_file_path = example_mech_data / local_data_dir / app_input
# check the input file
if not in_file_path.is_file():
    print(f"Error...the application input file {str(in_file_path)} cannot be found.")
    print("   please make sure the input file exists and the path is correct...")
    exit()
# the text output file will be saved in the current working folder during the simulation
out_file_path = str(current_dir_obj / app_output)

# XML solution file
xml_file = str(current_dir_obj / "XMLdata.zip")
```

### Create the run batch script to run the chemkin application

The Chemkin applications and the `GetSolution` utilities rely on the runtime environment established by the `run_env_path` script. Therefore, all the simulation runs and the post-processing task involving the `GetSolution` utilities must be included in the same batch script created below. In other words, you must establish the chemkin runtime environment in a batch script before running any chemkin applications. You can perform other non-chemkin related tasks after the `subprocess.run()` step.

```
if under_win:
    # Windows
    # define the run script name at the current working floder
    bat_file = current_dir_obj / "RUN_CKjob.bat"
    # create the batch script on the fly
    logger.debug("creating the batch script...")
    with bat_file.open(mode="w") as file:
        file.write("@REM Generated Batch Job Command File\n")
        file.write("REM get the runtime environment defined\n")
        file.write(f'@CALL "{run_env_path}"\n')
        file.write("@SET MKL_NUM_THREADS=1\n")
        file.write("@SET OMP_NUM_THREADS=1\n")
        file.write("@SET XERCES_DISABLE_DTD=1\n")
        file.write("REM RUN THE JOB AND CAPTURE EXIT STATUS\n")
        file.write(f"{ck_app} -i {in_file_path} -o {out_file_path} -x {xml_file}\n")
        file.write(r"@ECHO %errorlevel%" + "\n")
        file.write("@REM subprocess will terminate the script\n")
```

(continues on next page)

(continued from previous page)

```
        file.write("@REM when it detects an error during the simulation\n")
        file.write("@ECHO running 'GetSolution'...\n")
        file.write(f"{get_soln} {xml_file}\n")
        file.write(f"{soln_transpose}\n")
        file.write("exit\n")
        file.close()
    # use subprocess.run() to run the script just created
    # set the start wall time
    start_time = time.time()
    logger.debug("simulation started...")
    try:
        run_cmd = ["cmd", "/d", "/c", str(bat_file)]
        result = subprocess.run(
            run_cmd,
            capture_output=True,
            text=True,
            check=True,
            shell=False,
        )
        logger.debug("Command output:")
        logger.debug(result.stdout)
    except subprocess.CalledProcessError as e:
        logger.debug(
            f"Error executing command: {e.cmd} "
            f"with non-zero exit status {e.returncode}."
        )
        logger.debug(f"Stderr: {e.stderr}")
        exit()
    except subprocess.TimeoutExpired as e:
        logger.debug(f"Command timed out: {e.timeout}")
        exit()
    except FileNotFoundError as e:
        logger.debug(f"Command not found: {e}")
        exit()
    except OSError as e:
        logger.debug(f"OS error occurred: {e}")
        exit()
else:
    # Linux
    # define the run script name at the current working floder
    bat_file = current_dir_obj / "RUN_CKjob.sh"
    # create the bash script on the fly
    logger.debug("creating the batch script...")
    with bat_file.open(mode="w") as file:
        file.write("#!/bin/bash -v\n")
        file.write("### Generated Batch Job Script File\n")
        file.write("### get the runtime environment defined\n")
        file.write(f". {run_env_path}\n")
        file.write("export MKL_NUM_THREADS=1\n")
        file.write("export OMP_NUM_THREADS=1\n")
        file.write("export XERCES_DISABLE_DTD=1\n")
        file.write("### RUN THE JOB AND CAPTURE EXIT STATUS\n")
```

(continues on next page)

(continued from previous page)

```
        file.write(f"{ck_app} -i {in_file_path} -o {out_file_path} -x {xml_file}\n")
        file.write("### subprocess will terminate the script\n")
        file.write("### when it detects an error during the simulation\n")
        file.write(r"@echo $?" + "\n")
        file.write("@echo running 'GetSolution'...\n")
        file.write(f"{get_soln} {xml_file}\n")
        file.write(f"{soln_transpose}\n")
        file.write("exit\n")
        file.close()
    # use subprocess.run() to run the script just created
    # set the start wall time
    start_time = time.time()
    logger.debug("simulation started...")
    try:
        run_cmd = ["bash", str(bat_file)]
        result = subprocess.run(
            run_cmd,
            capture_output=True,
            text=True,
            check=True,
            shell=False,
        )
        logger.debug("Command output:")
        logger.debug(result.stdout)
    except subprocess.CalledProcessError as e:
        logger.debug(
            f"Error executing command: {e.cmd} "
            f"with non-zero exit status {e.returncode}."
        )
        logger.debug(f"Stderr: {e.stderr}")
        exit()
    except subprocess.TimeoutExpired as e:
        logger.debug(f"Command timed out: {e.timeout}")
        exit()
    except FileNotFoundError as e:
        logger.debug(f"Command not found: {e}")
        exit()
    except OSError as e:
        logger.debug(f"OS error occurred: {e}")
        exit()

# compute the total runtime
runtime = time.time() - start_time
logger.debug("simulation done...")
print(f"Total simulation duration: {runtime} [sec]\n")
```

### Post process the solution

Once the simulation is completed successfully, you have two options to post-process the solution:

1. you can use open the chemkin GUI and use the `visualizer` from the top banner. In this case, you can stop at this point. You only need the XML solution file located in the working folder.
2. you can use the chemkin `GetSolution` utilities to process the raw solution. The steps will be described below.

```
if interactive:
    user_input = input(
        ">>> Press [Enter] to continue, or type 'x' and "
        "press [Enter] to terminate the example: "
    )
    if user_input.lower() == "x":
        logger.debug("example terminated by user...")
        exit()
```

### Load and parse the solution

Load the solution from the csv files generated by the chemkin utilities `GetSolution` and `CKSolnTranspose`. You can find the raw solution file `xml_file` and processed solution files *.csv in the working folder. Use the `pandas.read_csv()` method to load the csv file in interest and extract the solution variables by their header names. Of course, if you are running this example in Windows, you can simply load the csv solution files into MS EXCEL.

**Note**

The `pandas` package is required to run this part of the example.

```
logger.debug("loading csv solution file...")
# read the solutions from the csv file for post-processing
csv_solution = "CKSoln_solution_no_1.csv"
# Read the CSV file into a DataFrame
soln = pd.read_csv(csv_solution)
logger.debug("solution loading done...")
# get the solution variable labels from the loaded data
soln_variables = list(soln.keys())
# get solution profiles along the centerline of the rotating CVD reactor
# get the solution index of the variables to be plotted
dist_id = -1
vel_id = -1
temp_id = -1
sih4_id = -1
si2h6_id = -1
for i, p in enumerate(soln_variables):
    if "Distance" in p:
        # distance from the substrate/disk [cm]
        dist_id = i
    elif "Axial_velocity" in p:
        # axial velocity [cm/sec]
        vel_id = i
    elif "Temperature" in p:
        # gas temperature [K]
        temp_id = i
    elif "fraction_SIH4_" in p:
        # SiH4 mole fraction
        sih4_id = i
    elif "fraction_H2_" in p:
        # H2 mole fraction
        h2_id = i
```

(continues on next page)

(continued from previous page)

```
# convert solution profiles into 1-D arrays
# distance from the surface along the reactor centerline [cm]
dist = soln[soln_variables[dist_id]]
# axial gas velocity [cm/sec]
ax_vel = soln[soln_variables[vel_id]]
# gas temperature [K]
temp = soln[soln_variables[temp_id]]
# SiH4 mole fraction [-]
mole_sih4 = soln[soln_variables[sih4_id]]
# H2 mole fraction [-]
mole_h2 = soln[soln_variables[h2_id]]
```

### Plot the solution profiles

Plot the species, the gas temperature, and the gas axial velocity profiles along the centerline of the CVD reactor.

```
plt.subplots(2, 2, sharey="row", figsize=(10, 8))
plt.suptitle("Rotating Disk CVD Reactor", fontsize=16)
plt.subplot(221)
plt.plot(temp, dist, "b-")
plt.xlabel("Gas Temperature [K]")
plt.ylabel("Distance From Surface [cm]")
plt.subplot(222)
plt.plot(-ax_vel, dist, "m-")
plt.xlabel("Gas Axial Velocity Towards Surface [cm/sec]")
plt.subplot(223)
plt.plot(mole_sih4, dist, "r-")
plt.xlabel("SiH4 Mole Fraction")
plt.ylabel("Distance From Surface [cm]")
plt.subplot(224)
plt.plot(mole_h2, dist, "g-")
plt.xlabel("H2 Mole Fraction")

# clean up run script
try:
    bat_file.unlink()
    logger.info(f"Deleted run script: {bat_file}")
except Exception as e:
    logger.warning(f"Failed to delete run script: {bat_file}. Error: {e}")
# check in license
ck.done()

# plot results
if interactive:
    plt.show()
else:
    plt.savefig("plot_cvd_profiles.png", bbox_inches="tight")
```

### 22.13.4 Run chemkin Java GUI in Python

GUI workflow for running Chemkin applications from Python.

This GUI wraps the high-level workflow used to launch Chemkin applications from the command line, which consists of the following steps: 1) generate a platform-specific run script, 2) execute the script with subprocess.

#### Import PyChemkin packages and start the logger

```
from pathlib import Path
import platform
import subprocess

from ansys.chemkin.core import Color
from ansys.chemkin.core.chemistry import chemkin_bin_dir
from ansys.chemkin.core.logger import logger
```

#### Establish the chemkin runtime environment

The batch script to establish the chemkin runtime environment is platform dependent. The first task is to find out the platform and the shell environment. Then you will run the environment setup script accordingly.

```
# Get local Ansys Chemkin installation bin folder
ckbin = Path(chemkin_bin_dir())

# Configure platform-dependent executable/env names.
under_win = platform.system() == "Windows"
if under_win:
    # Windows platform
    env_file = "run_chemkin_env_setup.bat"
    run_env_path = ckbin / env_file
    java_file = "run_chemkin.bat"
    run_java_path = ckbin / java_file
elif platform.system() == "Linux":
    # Linux platform
    env_file = "chemkin_setup.ksh"
    run_env_path = ckbin / env_file
    java_file = "run_chemkin.ksh"
    run_java_path = ckbin / java_file
else:
    raise RuntimeError(f"Unsupported platform: {platform.system()}")

# check files exist
if not run_env_path.is_file():
    raise FileNotFoundError(f"Chemkin setup script not found: {run_env_path}")
if not run_java_path.is_file():
    raise FileNotFoundError(f"Chemkin GUI launcher not found: {run_java_path}")

# Create run script.
if under_win:
    # Windows platform
    script_path = Path.cwd() / "RUN_CKjob_gui.bat"
```

(continues on next page)

(continued from previous page)

```
    with script_path.open("w", encoding="utf-8") as script:
        script.write("@REM Generated by run_chemkin_java_gui.py\n")
        script.write(f'@CALL "{run_env_path}"\n')
        script.write("@SET MKL_NUM_THREADS=1\n")
        script.write("@SET OMP_NUM_THREADS=1\n")
        script.write("@SET XERCES_DISABLE_DTD=1\n")
        script.write(f'@CALL "{run_java_path}"\n')
        script.write(r"@IF %ERRORLEVEL% NEQ 0 EXIT /B %ERRORLEVEL%")
        script.write("\n")
else:
    # Linux platform
    script_path = Path.cwd() / "RUN_CKjob_gui.sh"
    with script_path.open("w", encoding="utf-8") as script:
        script.write("#!/bin/bash\n")
        script.write("set -e\n")
        script.write(f'. "{run_env_path}"\n')
        script.write("export MKL_NUM_THREADS=1\n")
        script.write("export OMP_NUM_THREADS=1\n")
        script.write("export XERCES_DISABLE_DTD=1\n")
        script.write(f'. "{run_java_path}"\n')

logger.info(f"Created run script: {script_path}")

# Execute run script with subprocess.
try:
    if under_win:
        run_cmd = ["cmd", "/d", "/c", str(script_path)]
    else:
        run_cmd = ["bash", str(script_path)]

    result = subprocess.run(
        run_cmd,
        capture_output=True,
        text=True,
        check=True,
        shell=False,
    )
    # Log stderr if it exists.
    if result.stderr:
        msg = [
            Color.PURPLE,
            "--- stderr ---\n",
            Color.SPACEx6,
            result.stderr.rstrip(),
            Color.END,
        ]
        this_msg = Color.SPACE.join(msg)
        logger.error(this_msg)
except subprocess.CalledProcessError as e:
    if not e.returncode == 1:  # return code 1 is expected when user closes the GUI
        logger.error(f"Chemkin Java GUI failed with return code {e.returncode}")
        result = e  # capture the error output for logging
```

(continues on next page)

(continued from previous page)

```
        if result.stderr:
            msg = [
                Color.PURPLE,
                "process error\n",
                Color.SPACEx6,
                result.stderr.rstrip(),
                Color.END,
            ]
            this_msg = Color.SPACE.join(msg)
            logger.error(this_msg)

# clean up run script
try:
    script_path.unlink()
    logger.info(f"Deleted run script: {script_path}")
except Exception as e:
    logger.warning(f"Failed to delete run script: {script_path}. Error: {e}")
```

### 22.13.5 Run Chemkin Workbench GUI in Python

GUI workflow for running Chemkin Workbench from Python.

This GUI wraps the high-level workflow used to launch Chemkin Workbench from the command line, which consists of the following steps: 1) generate a platform-specific run script, 2) execute the script with subprocess.

```

```

#### Import PyChemkin packages and start the logger

```
from pathlib import Path
import platform
import subprocess

from ansys.chemkin.core import Color
from ansys.chemkin.core.chemistry import chemkin_bin_dir
from ansys.chemkin.core.logger import logger
```

#### Establish the chemkin runtime environment

The batch script to establish the chemkin runtime environment is platform dependent. The first task is to find out the platform and the shell environment. Then you will run the environment setup script accordingly.

```
# Get local Ansys Chemkin installation bin folder
ckbin = Path(chemkin_bin_dir())

# Configure platform-dependent executable/env names.
under_win = platform.system() == "Windows"
if under_win:
    # Windows platform
    env_file = "run_chemkin_env_setup.bat"
    run_env_path = ckbin / env_file
    java_file = "run_rdworkbench.bat"
```

(continues on next page)

(continued from previous page)

```
    run_java_path = ckbin / java_file

elif platform.system() == "Linux":
    # Linux platform
    env_file = "chemkin_setup.ksh"
    run_env_path = ckbin / env_file
    java_file = "run_rdworkbench.ksh"
    run_java_path = ckbin / java_file
else:
    raise RuntimeError(f"Unsupported platform: {platform.system()}")
# check files exist
if not run_env_path.is_file():
    raise FileNotFoundError(f"Chemkin setup script not found: {run_env_path}")
if not run_java_path.is_file():
    raise FileNotFoundError(f"Chemkin GUI launcher not found: {run_java_path}")

# Create run script.
if under_win:
    # Windows platform
    script_path = Path.cwd() / "RUN_CKjob_gui.bat"
    with script_path.open("w", encoding="utf-8") as script:
        script.write("@REM Generated by run_chemkin_workbench_gui.py\n")
        script.write(f'@CALL "{run_env_path}"\n')
        script.write("@SET MKL_NUM_THREADS=1\n")
        script.write("@SET OMP_NUM_THREADS=1\n")
        script.write("@SET XERCES_DISABLE_DTD=1\n")
        script.write(f'@CALL "{run_java_path}"\n')
        script.write(r"@IF %ERRORLEVEL% NEQ 0 EXIT /B %ERRORLEVEL%")
        script.write("\n")
else:
    # Linux platform
    script_path = Path.cwd() / "RUN_CKjob_gui.sh"
    with script_path.open("w", encoding="utf-8") as script:
        script.write("#!/bin/bash\n")
        script.write("set -e\n")
        script.write(f'. "{run_env_path}"\n')
        script.write("export MKL_NUM_THREADS=1\n")
        script.write("export OMP_NUM_THREADS=1\n")
        script.write("export XERCES_DISABLE_DTD=1\n")
        script.write(f'. "{run_java_path}"\n')

logger.info(f"Created run script: {script_path}")

# Execute run script with subprocess.
try:
    if under_win:
        run_cmd = ["cmd", "/d", "/c", str(script_path)]
    else:
        run_cmd = ["bash", str(script_path)]

    result = subprocess.run(
        run_cmd,
```

(continues on next page)

(continued from previous page)

```
        capture_output=True,
        text=True,
        check=True,
        shell=False,
    )
    # Log stderr if they exist.
    if result.stderr:
        msg = [
            Color.PURPLE,
            "--- stderr ---\n",
            Color.SPACEx6,
            result.stderr.rstrip(),
            Color.END,
        ]
        this_msg = Color.SPACE.join(msg)
        logger.error(this_msg)
except subprocess.CalledProcessError as e:
    if not e.returncode == 1:  # return code 1 is expected when user closes the GUI
        logger.error(f"Chemkin Java GUI failed with return code {e.returncode}")
        result = e  # capture the error output for logging
        if result.stderr:
            msg = [
                Color.PURPLE,
                "process error\n",
                Color.SPACEx6,
                result.stderr.rstrip(),
                Color.END,
            ]
            this_msg = Color.SPACE.join(msg)
            logger.error(this_msg)

# clean up run script
try:
    script_path.unlink()
    logger.info(f"Deleted run script: {script_path}")
except Exception as e:
    logger.warning(f"Failed to delete run script: {script_path}. Error: {e}")
```

## 22.14 Post-processing

PyChemkin has *post-process* utilities that can import the XML solution data file generated by selected Chemkin 1-D and 1.5-D reactor models. The examples in this section demonstrate how to use these utilities to import, visualize, and analyze the solution data from the (1-D steady-state) flame model, the (transient 0-D) multi-zone SI engine model, and the (pseudo 2-D) shear-layer channel flow reactor models.

### 22.14.1 Post-process Chemkin DI engine solution

PyChemkin has a post-process module `ChemkinSolutionImporter` that can used to import the XML solution data from selected Chemkin reactor models into Python. This example shows how use some of objects and utilities from the `ChemkinSolutionImporter`, the `SolutionData`, and the `SolutionGroup` modules to extract the solution variables from a transient multi-zone direct-injection (DI) engine simulation.

The Chemkin DI engine model discretizes the liquid fuel spray into multiple parcels (or zones) allowing for detailed tracking of fuel evaporation, mixing, and combustion within each zone. This examples shows how to extract the parcel-wise (zonal) information from the XML solution data. The "life-time" of the liquid fuel (n-heptane $nC_7H_{16}$) and its vapor are compared from two parcels representing the conditions at the tip and at the tail-end of the spray. It can be seen that as the atomized liquid fuel droplets are injected into the spray parcel, they mix with the hot air in the cylinder and start to evaporate leading to the formation of fuel vapor. Subsequently, the fuel vapor decomposes and participates in the combustion reactions within the spray parcel. The parcel at the spray tail-end exhibits similar process but both the liquid evaporation rate and the vapor consumption rate are higher due to the higher cylinder temperature and pressure.

In combination of other PyChemkin modules, such as `Chemistry` and `Mixture`, you can further analyze and visualize the simulation results.

**Note**

The *multi-zone DI engine model* is **not** available through the PyChemkin APIs.

```
from pathlib import Path

import matplotlib.pyplot as plt

from ansys.chemkin.core.chemkin_solution_utility import ChemkinSolutionImporter
from ansys.chemkin.core.logger import logger
from ansys.chemkin.core.utilities import (
    copy_file,
    delete_files_by_extension,
)

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
```

### Provide the Chemkin XML solution file

Find the data folder containing the XML solution file.

```
try:
    script_dir_obj = Path(__file__).parent.resolve()
except NameError:
    script_dir_obj = Path(current_dir)
script_dir = str(script_dir_obj)
# use the relative path to locate the XML solution data folder
local_data_folder = script_dir_obj / ".." / "data" / "1_D_solution"
print(f"solution data folder = {str(local_data_folder)}")
# set the xml solution data file
xml_solution_file = "XMLdata_ic_engine__DI_engine.zip"
xml_solution_path = local_data_folder / xml_solution_file
```

### Initialize the Chemkin solution importer

Initialize the Chemkin solution importer which will store all the raw solution data read from the XML solution file. Specify the working directory, clean up any existing solution files there before importing new solution data. Copy the XML solution file from the data directory to the working directory, and provide the full path of the XML file to the importer. The `ChemkinSolutionImporter` have a few options:

1. use `get_help()` method to get information about the Chemkin post-processor `GetSolution`
2. use `generate_preference_file()` method to create a preference file. You can make changes to the preference file `CKSolnList.txt` to customize the filters and the units before running the importer. Save a copy of the preference file to a different location to avoid it being deleted during the cleanup step.
3. Once the customized preference file is ready, use the `use_preference_file()` method to apply it before reading the XML solution file.

Then use the `import_solution()` and the `process_solution_data()` methods to import the XML solution file and to process the solution data for further processing.

```
# Instantiate the Chemkin solution importer object
ck_solution = ChemkinSolutionImporter()
# set the working directory
ck_solution.set_working_dir(current_dir)
# clean up the working directory before copying the new solution file
delete_files_by_extension(
    targetpath=current_dir, extensions=[".ckcsv", ".txt", ".xml", ".zip"]
)
# copy the solution file to the current working directory
copy_file(str(local_data_folder), current_dir, xml_solution_file)
# set the XML solution data file for post-processing
ck_solution.set_xml_data_file(str(Path(current_dir) / xml_solution_file))
# - optional step: generate the preference file from the solution data
# the preference file name is "CKSolnList.txt".
# ck_solution.generate_preference_file(str(Path(current_dir) / xml_solution_file))
#
# - optional step: edit the preference file before generating the solution data
# pause here if you want to edit the preference file before applying it
# Remember to save the edited preference file to a different location,
# so that it won't be deleted by the cleanup step.
#
# - optional step: apply the modified preference file
# copy it to the current working directory if it is stored elsewhere
# copy_file(str(local_data_folder), current_dir, "CKSolnList.txt")
# apply the preference file
# ck_solution.use_preference_file(str(Path(current_dir) / "CKSolnList.txt"))
#
# import solution data from the XML file
ck_solution.import_solution()
# import the solution data in text format for further analysis
ck_solution.process_solution_data()
```

### Post-process the imported solution data

Now you can get basic information about the solution data such as the number of solution groups, the number of solution points, the number of solution variables, etc. You use the methods provided by the `ChemKinSolution` object and/or the `SolutionGroup` object to manipulate the solution data for furrther analysis and plotting.

```
# get the list of solution group names from the imported solution data
solution_groups = ck_solution.get_solution_groups()
# print basic information about each solution group
for group in solution_groups:
    print(f"Solution Group: {group}")
```

(continues on next page)

(continued from previous page)

```
    print(f"number of solution points: {ck_solution.get_number_of_points()}")
    print(f"Number of gas species: {ck_solution.get_number_of_gas_species()}")
    # print(f"variables: {ck_solution.get_variable_labels(group)}")

# print(f"species symbols: {ck_solution.get_gas_species_symbols()}")

# Instantiate a solution group data object from the first solution group in the
# solution data.
grp_idx = 0
# get the name of the first solution group
this_group = solution_groups[grp_idx]
# get the solution group object for the first solution group
this_group_obj = ck_solution.get_group_object(this_group)
# define the zonal solutions to be plotted
zone_tag = ["Zone#1", "Zone#175"]
# define the independent variable
independent_var = "Crank_rotation_angle"
# get the array of the independent variable and its unit from the solution group object
distance_series = this_group_obj.get_variable_array_with_tag(
    independent_var, zone_tag[0]
)
distance = distance_series.to_numpy(dtype=float, copy=False)
distance_unit = this_group_obj.get_variable_unit_with_tag(independent_var, zone_tag[0])
# define the solution variable to be plotted
profile_1 = {}
profile_2 = {}
for zone_name in zone_tag:
    soln_var_1 = "Liquid_fuel_mass_nc7h16"
    soln_var_2 = "nc7h16"
    soln_profile_1_series = this_group_obj.get_variable_array_with_tag(
        soln_var_1, zone_name
    )
    soln_profile_1 = soln_profile_1_series.to_numpy(dtype=float, copy=False)
    if len(soln_profile_1) <= 0:
        print(f"Error...No data available for solution variable: {soln_var_1}")
        exit()
    profile_1[zone_name] = soln_profile_1

    soln_profile_2_series = this_group_obj.get_variable_array_with_tag(
        soln_var_2, zone_name
    )
    soln_profile_2 = soln_profile_2_series.to_numpy(dtype=float, copy=False)
    if len(soln_profile_2) <= 0:
        print(f"Error...No data available for solution variable: {soln_var_2}")
        exit()
    profile_2[zone_name] = soln_profile_2
    # same unit
    soln_unit_1 = this_group_obj.get_variable_unit_with_tag(soln_var_1, zone_name)
    soln_unit_2 = this_group_obj.get_variable_unit_with_tag(soln_var_2, zone_name)
```

### Plot the selected solution variable

plot solution profile

```
line_colors = ["b", "r"]
line_styles = ["-", "--"]
fig, ax1 = plt.subplots(figsize=(7.5, 5))
# Plot the primary data on the left y-axis
color1 = line_colors[0]
line_type = line_styles[0]
ax1.plot(
    distance,
    profile_1[zone_tag[0]],
    color=color1,
    linestyle=line_type,
    label=soln_var_1.replace("_", " "),
)
line_type = line_styles[1]
ax1.plot(
    distance,
    profile_1[zone_tag[1]],
    color=color1,
    linestyle=line_type,
    label=soln_var_1.replace("_", " "),
)
ax1.set_xlabel(f"{independent_var.replace('_', ' ')} {distance_unit}")
ax1.set_ylabel(
    f"{soln_var_1.replace('_', ' ')} {soln_unit_1.replace('_', ' ')}", color=color1
)
ax1.tick_params(axis="y", labelcolor=color1)
# Set x-axis limit
ax1.set_xlim(-30, 30)

# Create the secondary axes sharing the x-axis
ax2 = ax1.twinx()

color2 = line_colors[1]
line_type = line_styles[0]
ax2.plot(
    distance,
    profile_2[zone_tag[0]],
    color=color2,
    linestyle=line_type,
    label=soln_var_2.replace("_", " "),
)
line_type = line_styles[1]
ax2.plot(
    distance,
    profile_2[zone_tag[1]],
    color=color2,
    linestyle=line_type,
    label=soln_var_2.replace("_", " "),
)
ax2.set_xlabel(f"{independent_var.replace('_', ' ')} {distance_unit}")
```

(continues on next page)

(continued from previous page)

```
ax2.set_ylabel(
    f"{soln_var_2.replace('_', ' ')} {soln_unit_2.replace('_', ' ')}", color=color2
)
ax2.tick_params(axis="y", labelcolor=color2)
# Set x-axis limit
ax2.set_xlim(-30, 30)
#
plt.legend(
    [
        f"{zone_tag[0].replace('#', ' ')} : spray front",
        f"{zone_tag[1].replace('#', ' ')} : spray end",
    ],
    loc="best",
)
# plot results
plt.show()
# plt.savefig("plot_post_process_DI_engine.png", bbox_inches="tight")
```

### 22.14.2 Post-process Chemkin SI engine solution

PyChemkin has a post-process module `ChemkinSolutionImporter` that can used to import the XML solution data from selected Chemkin reactor models into Python. This example shows how use some of objects and utilities from the `ChemkinSolutionImporter`, the `SolutionData`, and the `SolutionGroup` modules to extract the solution variables from a transient multi-zone spark-ignition (SI) engine simulation.

This examples shows how to post-process the XML solution data generated by Chemkin multi-zone reactor models. The multi-zone solution contains extra tags to indicate the zone (label or number) of each data entry. This is important because it allows you to distinguish solution variables between different zones when analyzing the solution data.

In combination of other PyChemkin modules, such as `Chemistry` and `Mixture`, you can further analyze and visualize the simulation results.

```
from pathlib import Path

import matplotlib.pyplot as plt

from ansys.chemkin.core.chemkin_solution_utility import ChemkinSolutionImporter
from ansys.chemkin.core.logger import logger
from ansys.chemkin.core.utilities import (
    copy_file,
    delete_files_by_extension,
)

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
```

#### Provide the Chemkin XML solution file

Find the data folder containing the XML solution file.

```
try:
    script_dir_obj = Path(__file__).parent.resolve()
```

(continues on next page)

(continued from previous page)

```
except NameError:
    script_dir_obj = Path(current_dir)
script_dir = str(script_dir_obj)
# use the relative path to locate the XML solution data folder
local_data_folder = script_dir_obj / ".." / "data" / "1_D_solution"
print(f"solution data folder = {str(local_data_folder)}")
# set the xml solution data file
xml_solution_file = "XMLdata_ic_engine__SI_engine.zip"
xml_solution_path = local_data_folder / xml_solution_file
```

### Initialize the Chemkin solution importer

Initialize the Chemkin solution importer which will store all the raw solution data read from the XML solution file. Specify the working directory, clean up any existing solution files there before importing new solution data. Copy the XML solution file from the data directory to the working directory, and provide the full path of the XML file to the importer. The `ChemkinSolutionImporter` have a few options:

1. use `get_help()` method to get information about the Chemkin post-processor `GetSolution`
2. use `generate_preference_file()` method to create a preference file. You can make changes to the preference file `CKSolnList.txt` to customize the filters and the units before running the importer. Save a copy of the preference file to a different location to avoid it being deleted during the cleanup step.
3. Once the customized preference file is ready, use the `use_preference_file()` method to apply it before reading the XML solution file.

Then use the `import_solution()` and the `process_solution_data()` methods to import the XML solution file and to process the solution data for further processing.

```
# Instantiate the Chemkin solution importer object
ck_solution = ChemkinSolutionImporter()
# set the working directory
ck_solution.set_working_dir(current_dir)
# clean up the working directory before copying the new solution file
delete_files_by_extension(
    targetpath=current_dir, extensions=[".ckcsv", ".txt", ".xml", ".zip"]
)
# copy the solution file to the current working directory
copy_file(str(local_data_folder), current_dir, xml_solution_file)
# set the XML solution data file for post-processing
ck_solution.set_xml_data_file(str(Path(current_dir) / xml_solution_file))
# - optional step: generate the preference file from the solution data
# the preference file name is "CKSolnList.txt".
# ck_solution.generate_preference_file(str(Path(current_dir) / xml_solution_file))
#
# - optional step: edit the preference file before generating the solution data
# pause here if you want to edit the preference file before applying it
# Remember to save the edited preference file to a different location,
# so that it won't be deleted by the cleanup step.
#
# - optional step: apply the modified preference file
# copy it to the current working directory if it is stored elsewhere
# copy_file(str(local_data_folder), current_dir, "CKSolnList.txt")
# apply the preference file
```

(continues on next page)

(continued from previous page)

```
# ck_solution.use_preference_file(str(Path(current_dir) / "CKSolnList.txt"))
#
# import solution data from the XML file
ck_solution.import_solution()
# import the solution data in text format for further analysis
ck_solution.process_solution_data()
```

### Post-process the imported solution data

Now you can get basic information about the solution data such as the number of solution groups, the number of solution points, the number of solution variables, etc. You use the methods provided by the `ChemKinSolution` object and/or the `SolutionGroup` object to manipulate the solution data for furrther analysis and plotting.

```
# get the list of solution group names from the imported solution data
solution_groups = ck_solution.get_solution_groups()
# Instantiate a solution group data object from the first solution group in the
# solution data.
grp_idx = 0
# get the name of the first solution group
this_group = solution_groups[grp_idx]
# get the solution group object for the first solution group
this_group_obj = ck_solution.get_group_object(this_group)
# print basic information about each solution group
print(f"Solution Group: {this_group}")
print(f"number of solution points: {this_group_obj.number_of_points}")
print(f"Number of gas species: {this_group_obj.number_of_gas_species}")
# print(f"species symbols: {this_group_obj.gas_species_symbols}")
# for the SI engine solution, the methods with zone tag should be used
# zone tags
zone_tags = ["Unburned Zone", "Burned Zone", "Zone Avg"]
# define the independent variable
independent_var = "Crank_rotation_angle"
# get the array of the independent variable and its unit from the solution group object
distance_series = this_group_obj.get_variable_array_with_tag(
    independent_var, zone_tags[0]
)
distance = distance_series.to_numpy(dtype=float, copy=False)
distance_unit = this_group_obj.get_variable_unit_with_tag(independent_var, zone_tags[0])
# define the solution variable to be plotted
soln_var = "Temperature"
# soln_var = "co"
soln_profile = []
for zone in zone_tags:
    this_profile_series = this_group_obj.get_variable_array_with_tag(soln_var, zone)
    this_profile = this_profile_series.to_numpy(dtype=float, copy=False)
    if len(this_profile) <= 0:
        print(f"Error...No data available for solution variable: {soln_var}")
        exit()
    soln_profile.append(this_profile)
# same unit
soln_unit = this_group_obj.get_variable_unit_with_tag(soln_var, zone_tags[0])
```

#### Plot the selected solution variable

plot solution profile

```
curve_list = ["b--", "r-", "g-."]
for z, profile in enumerate(soln_profile):
    plt.plot(distance, profile, curve_list[z], label=f"{soln_var} {zone_tags[z]}")
plt.xlabel(f"{independent_var.replace('_', ' ')} {distance_unit}")
plt.ylabel(f"{soln_var} {soln_unit.replace('_', ' ')}")
plt.legend([f"{zone}" for zone in zone_tags])
# plot results
plt.show()
# plt.savefig("plot_post_process_SI_engine.png", bbox_inches="tight")
```

### 22.14.3 Post-process Chemkin shear-layer flow solution

PyChemkin has a post-process module `ChemkinSolutionImporter` that can used to import the XML solution data from selected Chemkin reactor models into Python. This example shows how use some of objects and utilities from the `ChemkinSolutionImporter`, the `SolutionData`, and the `SolutionGroup` modules to extract the solution variables from a planar shear-layer flow simulation.

The Chemkin shear-layer reactor models store the solutions in "slices" along the flow direction. This project extracts and processes the cross-flow profiles of an intermediate species ($Si_2Cl_6$) at different streamwise locations. These profiles illustrate how the species gets formed and released from the active surface and subsequently diffuses across the channel as the gas mixture travels along the length of the reactor.

In combination of other PyChemkin modules, such as `Chemistry` and `Mixture`, you can further analyze and visualize the simulation results.

**Note**

The Chemkin *shear-layer reactor models* are **not** available through the PyChemkin APIs.

```
from pathlib import Path

import matplotlib.pyplot as plt

from ansys.chemkin.core.chemkin_solution_utility import ChemkinSolutionImporter
from ansys.chemkin.core.logger import logger
from ansys.chemkin.core.utilities import (
    copy_file,
    delete_files_by_extension,
)

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
```

#### Provide the Chemkin XML solution file

Find the data folder containing the XML solution file.

```
try:
    script_dir_obj = Path(__file__).parent.resolve()
except NameError:
    script_dir_obj = Path(current_dir)
script_dir = str(script_dir_obj)
# use the relative path to locate the XML solution data folder
local_data_folder = script_dir_obj / ".." / "data" / "1_D_solution"
print(f"solution data folder = {str(local_data_folder)}")
# set the xml solution data file
xml_solution_file = "XMLdata_planar_shear_flow__tcs_cvd.zip"
xml_solution_path = local_data_folder / xml_solution_file
```

### Initialize the Chemkin solution importer

Initialize the Chemkin solution importer which will store all the raw solution data read from the XML solution file. Specify the working directory, clean up any existing solution files there before importing new solution data. Copy the XML solution file from the data directory to the working directory, and provide the full path of the XML file to the importer. The `ChemkinSolutionImporter` have a few options:

1. use `get_help()` method to get information about the Chemkin post-processor `GetSolution`
2. use `generate_preference_file()` method to create a preference file. You can make changes to the preference file `CKSolnList.txt` to customize the filters and the units before running the importer. Save a copy of the preference file to a different location to avoid it being deleted during the cleanup step.
3. Once the customized preference file is ready, use the `use_preference_file()` method to apply it before reading the XML solution file.

Then use the `import_solution()` and the `process_solution_data()` methods to import the XML solution file and to process the solution data for further processing.

```
# Instantiate the Chemkin solution importer object
ck_solution = ChemkinSolutionImporter()
# set the working directory
ck_solution.set_working_dir(current_dir)
# clean up the working directory before copying the new solution file
delete_files_by_extension(
    targetpath=current_dir, extensions=[".ckcsv", ".txt", ".xml", ".zip"]
)
# copy the solution file to the current working directory
copy_file(str(local_data_folder), current_dir, xml_solution_file)
# set the XML solution data file for post-processing
ck_solution.set_xml_data_file(str(Path(current_dir) / xml_solution_file))
# - optional step: generate the preference file from the solution data
# the preference file name is "CKSolnList.txt".
# ck_solution.generate_preference_file(str(Path(current_dir) / xml_solution_file))
#
# - optional step: edit the preference file before generating the solution data
# pause here if you want to edit the preference file before applying it
# Remember to save the edited preference file to a different location,
# so that it won't be deleted by the cleanup step.
#
# - optional step: apply the modified preference file
# copy it to the current working directory if it is stored elsewhere
# copy_file(str(local_data_folder), current_dir, "CKSolnList.txt")
```

(continues on next page)

(continued from previous page)

```
# apply the preference file
# ck_solution.use_preference_file(str(Path(current_dir) / "CKSolnList.txt"))
#
# import solution data from the XML file
ck_solution.import_solution()
# convert the solution data into group objects for further analysis
ck_solution.process_solution_data()
```

### Post-process the imported solution data

Now you can get basic information about the solution data such as the number of solution groups, the number of solution points, the number of solution variables, etc. You use the methods provided by the `ChemKinSolution` object and/or the `SolutionGroup` object to manipulate the solution data for furrther analysis and plotting.

```
# get the list of solution group names from the imported solution data
solution_groups = ck_solution.get_solution_groups()
# print basic information about each solution group
for group in solution_groups:
    print(f"Solution Group: {group}")
    print(f"number of solution points: {ck_solution.get_number_of_points()}")
    print(f"Number of gas species: {ck_solution.get_number_of_gas_species()}")

# print(f"variables: {ck_solution.get_variable_labels(solution_groups[0])}")
# print(f"species symbols: {ck_solution.get_gas_species_symbols()}")

# Instantiate a solution group data objects from a list of solution group names
# (corresponding to different distances from the entrance) in the solution data.
group_names = [
    "slice_5.000000",
    "slice_10.000000",
    "slice_20.000000",
    "slice_29.800000",
]
# curve attributes
curvelist = ["g", "b--", "r:", "m-."]
#
soln_x = {}
soln_y = {}
soln_unit = {}
# loop through the selected solution slices (groups)
for this_group in group_names:
    # get the solution group object for the selected solution group
    this_group_obj = ck_solution.get_group_object(this_group)
    # define the independent variable
    independent_var = "Height"
    # get the array of the independent variable and its unit from
    # the solution group object
    distance_series = this_group_obj.get_variable_array(independent_var)
    distance = distance_series.to_numpy(dtype=float, copy=False)
    distance_unit = this_group_obj.get_variable_unit(independent_var)
    # define the solution variable to be plotted
    # soln_var = "Temperature"
```

(continues on next page)

(continued from previous page)

```
    soln_var = "SI2CL6"
    # soln_var = "SICL4"
    soln_profile_series = this_group_obj.get_variable_array(soln_var)
    soln_profile = soln_profile_series.to_numpy(dtype=float, copy=False)
    if len(soln_profile) <= 0:
        print(f"Error...No data available for solution variable: {soln_var}")
        exit()
    soln_y[this_group] = distance
    soln_x[this_group] = soln_profile
    soln_unit[this_group] = this_group_obj.get_variable_unit(soln_var)
```

#### Plot the selected solution variable

plot solution profiles at different distances from the entrance.

```
for i, grp in enumerate(group_names):
    plt.plot(
        soln_x[grp], soln_y[grp], curvelist[i], label=grp.replace("slice_", "x = ")
    )
plt.ylabel(f"{independent_var} {distance_unit}")
y_label = f"{soln_var} {soln_unit[group_names[0]].replace('_', ' ')}"
plt.xlabel(y_label)
plt.legend([grp.replace("slice_", "x = ") for grp in group_names], loc="upper right")
plt.title("Profile Evolution along Channel")
# plot results
plt.show()
# plt.savefig("plot_post_process_cvd_flow.png", bbox_inches="tight")
```

### 22.14.4 Post-process Chemkin flame solution

PyChemkin has a post-process module `ChemkinSolutionImporter` that can used to import the XML solution data from selected Chemkin reactor models into Python. This example shows how use some of objects and utilities from the `ChemkinSolutionImporter`, the `SolutionData`, and the `SolutionGroup` modules to extract the solution variables from a 1-D steady-state flame simulation.

The original Chemkin project utilized the *Particle Tracking Module (PTM)* with the moments method to simulate the evolution of the size distribution of the soot particles in the flame. The PTM employs a surface-chemistry based mechanism to model the homogeneous nucleation of the soot particles from the precusor, in this case pyrene (A4), and the subsequent mass growth of the soot particles. The predicted profiles of the soot particle volume fraction and the gas precursor species are of particular interest in the analysis, and the subsequent visualization of these profiles is important for understanding the soot formation and transport processes in the flame. In this example, the particle size distributions are reconstructed from the moments values in the XML solution data, and the particle size distributions at three locations in the flame zone are visualized for comparisons.

In combination of other PyChemkin modules, such as `Chemistry` and `Mixture`, you can further analyze and visualize the simulation results.

**Note**

The *Particle Tracking Module (PTM)* is **not** available through the PyChemkin APIs.

```
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np

from ansys.chemkin.core.chemkin_solution_utility import ChemkinSolutionImporter
from ansys.chemkin.core.logger import logger
from ansys.chemkin.core.particles import PDF
from ansys.chemkin.core.utilities import (
    copy_file,
    delete_files_by_extension,
)

# check working directory
current_dir = str(Path.cwd())
logger.debug("working directory: " + current_dir)
```

### Provide the Chemkin XML solution file

Find the data folder containing the XML solution file.

```
try:
    script_dir_obj = Path(__file__).parent.resolve()
except NameError:
    script_dir_obj = Path(current_dir)
script_dir = str(script_dir_obj)
# use the relative path to locate the XML solution data folder
local_data_folder = script_dir_obj / ".." / "data" / "1_D_solution"
print(f"solution data folder = {str(local_data_folder)}")
# set the xml solution data file
xml_solution_file = "XMLdata_opposed-flow_flame__gas_soot_radiation.zip"
xml_solution_path = local_data_folder / xml_solution_file
```

### Initialize the Chemkin solution importer

Initialize the Chemkin solution importer which will store all the raw solution data read from the XML solution file. Specify the working directory, clean up any existing solution files there before importing new solution data. Copy the XML solution file from the data directory to the working directory, and provide the full path of the XML file to the importer. The `ChemkinSolutionImporter` have a few options:

1. use `get_help()` method to get information about the Chemkin post-processor `GetSolution`
2. use `generate_preference_file()` method to create a preference file. You can make changes to the preference file `CKSolnList.txt` to customize the filters and the units before running the importer. Save a copy of the preference file to a different location to avoid it being deleted during the cleanup step.
3. Once the customized preference file is ready, use the `use_preference_file()` method to apply it before reading the XML solution file.

Then use the `import_solution()` and the `process_solution_data()` methods to import the XML solution file and to process the solution data for further processing.

```
# Instantiate the Chemkin solution importer object
ck_solution = ChemkinSolutionImporter()
# set the working directory
```

(continues on next page)

(continued from previous page)

```
ck_solution.set_working_dir(current_dir)
# clean up the working directory before copying the new solution file
delete_files_by_extension(
    targetpath=current_dir, extensions=[".ckcsv", ".txt", ".xml", ".zip"]
)
# copy the solution file to the current working directory
copy_file(str(local_data_folder), current_dir, xml_solution_file)
# set the XML solution data file for post-processing
ck_solution.set_xml_data_file(str(Path(current_dir) / xml_solution_file))
# - optional step: generate the preference file from the solution data
# the preference file name is "CKSolnList.txt".
# ck_solution.generate_preference_file(str(Path(current_dir) / xml_solution_file))
#
# - optional step: edit the preference file before generating the solution data
# pause here if you want to edit the preference file before applying it
# Remember to save the edited preference file to a different location,
# so that it won't be deleted by the cleanup step.
#
# - optional step: apply the modified preference file
# copy it to the current working directory if it is stored elsewhere
# copy_file(str(local_data_folder), current_dir, "CKSolnList.txt")
# apply the preference file
# ck_solution.use_preference_file(str(Path(current_dir) / "CKSolnList.txt"))
#
# import solution data from the XML file
ck_solution.import_solution()
# convert the solution data into group objects for further analysis
ck_solution.process_solution_data()
```

### Post-process the imported solution data

Now you can get basic information about the solution data such as the number of solution groups, the number of solution points, the number of solution variables, etc. You use the methods provided by the `ChemKinSolution` object and/or the `SolutionGroup` object to manipulate the solution data for furrther analysis and plotting.

```
# get the list of solution group names from the imported solution data
solution_groups = ck_solution.get_solution_groups()
# print basic information about each solution group
for group in solution_groups:
    print(f"Solution Group: {group}")
    print(f"number of solution points: {ck_solution.get_number_of_points()}")
    print(f"Number of gas species: {ck_solution.get_number_of_gas_species()}")
    # print(f"variables: {ck_solution.get_variable_labels(group)}")

# print(f"species symbols: {ck_solution.get_gas_species_symbols()}")

# Instantiate a solution group data object from the first (and the only) solution group
# in the solution data.
grp_idx = 0
# get the name of the first solution group
this_group = solution_groups[grp_idx]
# get the solution group object for the first solution group
```

(continues on next page)

(continued from previous page)

```
this_group_obj = ck_solution.get_group_object(this_group)
# define the independent variable
independent_var = "Distance"
# get the array of the independent variable and its unit from the solution group object
distance_series = this_group_obj.get_variable_array(independent_var)
distance = distance_series.to_numpy(dtype=float, copy=False)
distance_unit = this_group_obj.get_variable_unit(independent_var)
# define the solution variable to be plotted
soln_var_1 = "Particle_volume_fraction"
soln_profile_1_series = this_group_obj.get_variable_array(soln_var_1)
soln_profile_1 = soln_profile_1_series.to_numpy(dtype=float, copy=False)
if len(soln_profile_1) <= 0:
    print(f"Error...No data available for solution variable: {soln_var_1}")
    exit()
soln_unit_1 = this_group_obj.get_variable_unit(soln_var_1)
# soln_var_2 = "Temperature"
# soln_var_2 = "Axial_velocity"
soln_var_2 = "A4"
soln_profile_2_series = this_group_obj.get_variable_array(soln_var_2)
soln_profile_2 = soln_profile_2_series.to_numpy(dtype=float, copy=False)
if len(soln_profile_2) <= 0:
    print(f"Error...No data available for solution variable: {soln_var_2}")
    exit()
soln_unit_2 = this_group_obj.get_variable_unit(soln_var_2)
```

### Plot the selected solution variable

plot solution profile

```
line_colors = ["b", "r"]
fig, (ax1, ax3) = plt.subplots(2, 1, figsize=(6, 8))
# Plot the primary data on the left y-axis
color1 = line_colors[0]
ax1.plot(distance, soln_profile_1, color=color1, label=soln_var_1.replace("_", " "))
ax1.set_xlabel(f"{independent_var} {distance_unit}")
ax1.set_ylabel(
    f"{soln_var_1.replace('_', ' ')} {soln_unit_1.replace('_', ' ')}", color=color1
)
ax1.tick_params(axis="y", labelcolor=color1)
ax1.set_title("Solution profiles between the nozzles")
# Create the secondary axes sharing the x-axis
ax2 = ax1.twinx()

color2 = line_colors[1]
ax2.plot(distance, soln_profile_2, color=color2, label=soln_var_2.replace("_", " "))
ax2.set_xlabel(f"{independent_var} {distance_unit}")
ax2.set_ylabel(
    f"{soln_var_2.replace('_', ' ')} {soln_unit_2.replace('_', ' ')}", color=color2
)
ax2.tick_params(axis="y", labelcolor=color2)
```

### Reconstruct the soot particle size distributions

Extract the moments values of the soot particle size distributions from the XML solution data. By assuming the particle size distribution follows the lognormal distribution, we can reconstruct it with only three size moments. Additional information about the properties of the soot particles are required for the reconstruction. These properties are the soot particle bulk density, the minimum particle inception size class, and the molecular mass of the core unit that makes up the particle, i.e., the carbon atom in the case of soot particles.

```
# reconstruct the particle size distributions from the moments values
# in the solution data
# get soot particle size distributions in the flame at specified locations
loc = [1.2, 1.3, 1.4]  # distance from the fuel inlet in [cm]
i_loc = []
# get moments from the solution data
m0_var = "0th_moment_of_particle_size_distribution"
m1_var = "1st_moment_of_particle_size_distribution"
m2_var = "2nd_moment_of_particle_size_distribution"
soln_m0 = this_group_obj.get_variable_array(m0_var).to_numpy(dtype=float, copy=False)
soln_m1 = this_group_obj.get_variable_array(m1_var).to_numpy(dtype=float, copy=False)
soln_m2 = this_group_obj.get_variable_array(m2_var).to_numpy(dtype=float, copy=False)
# set up size moments data for further analysis
m0 = []
m1 = []
m2 = []
for i, x in enumerate(loc):
    idx = np.searchsorted(distance, x)
    i_loc.append(idx)
    print(f"location of x= {x} is {idx}")
    m0.append(soln_m0[idx])
    m1.append(soln_m1[idx])
    m2.append(soln_m2[idx])
    print(f"m0: {m0[-1]}, m1: {m1[-1]}, m2: {m2[-1]}")
```

### Plot the selected solution variable

plot the particle size distributions at three locations in the flame zone.

```
# plot the particle size distributions
# minimum inception class (from the nucleation reactions)
class_min = 32.0
# bulk density of the particles [g/cm3]
soot_bulk_den = 1.8
# molecular mass of the core unit (atom or molecule)
# that makes up the core of the particle [g/mole]
soot_core_molar_mass = 12.0  # mass of one mole of carbon atom
# instantiate the population density function object for the soot particles
soot_pdf = PDF(core_wt=soot_core_molar_mass, bulk_density=soot_bulk_den)
# loop over the selected locations in the flame
for i, m0_val in enumerate(m0):
    m1_val = m1[i]
    m2_val = m2[i]
    # set up the moments of the particle size distribution function
    soot_pdf.set_moments(m0_val, m1_val, m2_val, class_min)
    # assume the size distribution follows the lognormal distribution
```

(continues on next page)

(continued from previous page)

```
    d, lognorm_pdf = soot_pdf.get_lognormal_pdf(
        samples=1000, limit=0.001, mode="volume"
    )
    ax3.plot(d, lognorm_pdf, label=f"x = {loc[i]} cm")

# convert the size class (number of the core in the particle) to volume
vol_min = soot_pdf.class_to_volume(class_min)
ax3.axvline(
    vol_min,
    color="red",
    linestyle="--",
    label=f"Minimum Volume = {vol_min * 1.0e4:.2e}",
)
ax3.set_title("Soot Particle Size Distributions in the flame zone")
ax3.set_xlabel("Primary particle volume (micron3)")
ax3.set_ylabel("Number density (1/cm3)")
ax3.legend()
# need to zoom in to see the differences
plt.tight_layout()
plt.show()
# plt.savefig("plot_post_process_flame.png", bbox_inches="tight")
```

CHAPTER

# TWENTYTHREE

# CONTRIBUTE

Thank you for your interest in contributing to PyChemkin. Contributions for making the project better can include fixing bugs, adding new features, and improving the documentation.

Overall guidance on contributing to a PyAnsys library appears in the Contributing topic in the *PyAnsys developer's guide*. Ensure that you are thoroughly familiar with this guide before attempting to contribute to PyChemkin.

The following contribution information is specific to PyChemkin.

- **Clone the repository.**
  To clone and install the latest PyChemkin release in development mode, run these commands:

  ```
  git clone https://github.com/ansys/pychemkin/
  cd pychemkin
  python -m pip install --upgrade pip
  pip install -e .
  ```

- **Follow the code style.**
  PyChemkin follows the PEP 8 standard as described in PEP 8 in the *PyAnsys developer's guide* and uses `pre-commit` for style checking.

  To ensure your code meets minimum code styling standards, run these commands:

  ```
  pip install pre-commit
  pre-commit run --all-files
  ```

  You can also install this as a pre-commit hook by running this command:

  ```
  pre-commit install
  ```

- **Run the tests.**
  Prior to running the tests, you must run this command to install the test dependencies:

  ```
  pip install - .tests
  ```

  To run the tests, navigate to the root directory of the repository and run this command:

  ```
  pytest
  ```

- **Build the documentation.**
  Prior to building the documentation, you must run this command to install the documentation dependencies:

  ```
  pip install -e .doc
  ```

  To build the documentation, run the following commands:

```
cd doc
```

- On Linux:

```
make html
```

- On Windows:

```
./make.bat html
```

The documentation is built in the `docs/_build/html` directory.

CHAPTER

TWENTYFOUR

# ARTIFACTS

Download the latest version of the artifacts from the release page: https://pypi.org/project/ansys-chemkin-core/#files

CHAPTER

TWENTYFIVE

# RELEASE NOTES

This document contains the release notes for the PyChemkin project.

# PYTHON MODULE INDEX

## a

ansys.chemkin.core, ??
ansys.chemkin.core.chemistry, ??
ansys.chemkin.core.chemkin_solution_utility,
    ??
ansys.chemkin.core.chemkin_wrapper, ??
ansys.chemkin.core.color, ??
ansys.chemkin.core.constants, ??
ansys.chemkin.core.flame, ??
ansys.chemkin.core.grid, ??
ansys.chemkin.core.hybridreactornetwork, ??
ansys.chemkin.core.info, ??
ansys.chemkin.core.inlet, ??
ansys.chemkin.core.keyword, ??
ansys.chemkin.core.logger, ??
ansys.chemkin.core.microprocess, ??
ansys.chemkin.core.mixture, ??
ansys.chemkin.core.particles, ??
ansys.chemkin.core.profile, ??
ansys.chemkin.core.reactormodel, ??
ansys.chemkin.core.realgaseos, ??
ansys.chemkin.core.steadystatesolver, ??
ansys.chemkin.core.surface, ??
ansys.chemkin.core.surface_chemistry, ??
ansys.chemkin.core.surface_chemistry_controller,
    ??
ansys.chemkin.core.surface_components, ??
ansys.chemkin.core.utilities, ??
ansys.chemkin.core.validation, ??